

VICTOR BARCELOS LACERDA

RENAN OSÓRIO RIOS

PROGRAMAÇÃO PARA LEIGOS COM APP INVENTOR

UM GUIA PARA QUEM SE INTERESSA EM
CRIAR APLICATIVOS DE FORMA PRÁTICA



Edifes

Programação para Leigos com App Inventor

Um guia para quem se interessa em criar
aplicativos de forma prática

VICTOR BARCELOS LACERDA
RENAN OSÓRIO RIOS

Programação para Leigos com App Inventor

Um guia para quem se interessa em criar
aplicativos de forma prática

Giovany Frossard Teixeira
Revisão técnica



Edifes

Vitória, 2023



Edifes

Editora do Instituto Federal de Educação,
Ciência e Tecnologia do Espírito Santo
R. Barão de Mauá, nº 30 – Jucutuquara
29040-689 – Vitória – ES
www.edifes.ifes.edu.br | editora@ifes.edu.br

Reitor: Jadir José Pela

Pró-Reitor de Administração e Orçamento: Lezi José Ferreira

Pró-Reitor de Desenvolvimento Institucional: Luciano de Oliveira Toledo

Pró-Reitora de Ensino: Adriana Pionttkovsky Barcellos

Pró-Reitor de Extensão: Lodovico Ortlieb Faria

Pró-Reitor de Pesquisa e Pós-Graduação: André Romero da Silva

Coordenador da Edifes: Adonai José Lacruz

Conselho Editorial

Aldo Rezende * Aline Freitas da Silva de Carvalho * Aparecida de Fátima Madella de Oliveira * Eduardo Fausto Kuster Cid * Felipe Zamborlini Saiter * Gabriel Domingos Carvalho * Jamille Locatelli * Marcio de Souza Bolzan * Mariella Berger Andrade * Ricardo Ramos Costa * Rosana Vilarim da Silva * Rossanna dos Santos Santana Rubim * Viviane Bessa Lopes Alvarenga.

Produção editorial

Projeto Gráfico: Assessoria de Comunicação Social do Ifes

Revisão de texto: Thaís Rosário da Silveira (Edifes)

Diagramação e epub: Higor Ferraço (Edifes)

Capa: Higor Ferraço (Edifes)

Dados Internacionais de Catalogação na Publicação (CIP)

L131p Lacerda, Victor Barcelos.

Programação para leigos com App Inventor [recurso eletrônico] : um guia para quem se interessa em criar aplicativos de forma prática / Victor Barcelos Lacerda, Renan Osório Rios. – Vitória, ES : Edifes, 2023.

1 recurso on-line : ePub ; 342 p. ; il. col.

ISBN: 978-85-8263-775-3 (e-book).

1. App Inventor (Recurso eletrônico). 2. Desenvolvimento ágil de software. 3. Aplicativos móveis – Desenvolvimento. I. Rios, Renan Osório. II. Título.

CDD 22 – 005.1

Biblioteca responsável: Rossanna dos Santos Santana Rubim – CRB6- ES 403

DOI: 10.36524/9788582637548

Este livro foi avaliado e recomendado para publicação por pareceristas *ad hoc*.
Esta obra está licenciada com uma Licença Atribuição-NãoComercial-SemDerivações 4.0



SUMÁRIO

Capítulo 1: Introdução ao App Inventor	11
1.1. Plataforma	13
1.1.1. Embasamento.....	14
1.1.2. Objetivo	18
1.1.3. Metodologia	19
1.2. Desenvolvedores	26
1.2.1. App Inventor.....	26
1.2.2. Android.....	27
1.2.3. Você e suas ideias são bem-vindas!	28
1.3. Sobre a Plataforma App Inventor	31
1.3.1. Versões.....	31
1.4. Sites Relacionados	36
1.4.1. Plataforma Scratch.....	36
1.4.2. Fórum Oficial App Inventor	38
1.4.3. Site Oficial Android.....	40
1.5. Apresentação da Interface	41
1.5.1. Sessão de Projetos	41
1.5.2. Sessão de Design	42
1.5.3. Sessão de Blocos e Mecânica.....	45
1.5.4. Barra Superior	49
1.6. Atividades de fixação!	54
1.7. Encerramento do capítulo.....	55
 Capítulo 2: Desenvolvimento do Primeiro Aplicativo	 56
2.1. Apresentação do Aplicativo a ser Desenvolvido	56
2.1.1. Introdução.....	56
2.1.2. Público-Alvo.....	57
2.1.3. Ideia.....	58
2.2. Explicações das Funcionalidades.....	59
2.2.1. Detalhamento e Subdivisão do Problema.....	59
2.2.2. Pensamento Computacional.....	60
2.3. Apresentação da Ferramenta	63
2.3.1. Categorias de Componentes.....	64

2.3.2. Avaliação de Componentes.....	67
2.4. Desenvolvimento passo a passo.....	69
2.4.1. Introdução às Telas.....	69
2.4.2. Manejo de Componentes.....	73
2.4.3. Primeiros Passos.....	73
2.4.4. Tratando da Lógica e da Mecânica	77
2.4.4.1. Estudando os Componentes como Categorias.....	77
2.4.4.2. Organizando Blocos.....	78
2.4.5. Calculadora de IMC.....	82
2.4.5.1. Lógica de Programação	85
2.4.5.1.1 Variáveis.....	86
2.4.5.1.2 Constantes	87
2.4.5.1.3 Sequencialidade de Instruções	88
2.4.5.1.4 Analisando Condições.....	88
2.4.5.1.5 Estendendo Comparações.....	89
2.4.5.1.6 Repetindo Códigos.....	90
2.4.5.1.7 Funções e Procedimentos.....	91
2.4.5.1.8 Modificador de Estrutura de Bloco	96
2.4.5.1.9 Retomando o desenvolvimento.....	96
2.4.5.1.10 Comentário sobre Orientação a Objetos.....	98
2.4.5.1.11 Finalizando a Primeira Utilidade.....	100
2.4.5.2. Vendo os Resultados	101
2.4.6. Seleção de Cores.....	103
2.4.6.1. Criando a interface do seletor.....	104
2.4.6.2. Criando a lógica do seletor	106
2.4.6.2.1 Estudando Vetores	106
2.4.6.2.2 Estruturas de Dados	109
2.4.6.3. Organizando os blocos.....	112
2.4.7. Autenticador	115
2.4.7.1. Uso Consciente de Telas e Parte Visual.....	115
2.4.7.2. Edição dos blocos do Autenticador.....	118
2.4.8. Calculadora de Idade	122
2.4.8.1. Introduzindo o componente EscolheData.....	123
2.4.8.2. Aspecto Visual.....	123
2.4.8.3. Lógica da Utilidade de Calcular Idade.....	124
2.4.8.4. Uso do Temporizador	125

2.4.8.5. Cálculo Básico.....	126
2.4.8.6. Processando e Cobrindo Possibilidades com SENÃO-SE	126
2.4.8.7. Finalizando	127
2.4.9. Bloco de Notas.....	128
2.4.9.1. Aspecto Visual.....	129
2.4.9.2. Cuidando da Lógica das Notas.....	130
2.4.10. Relógio.....	143
2.4.10.1. Aspecto Visual dos Relógios	144
2.4.10.2. Editando os blocos do Relógio	148
2.4.10.2.1 O tempo é feito de Instantes	152
2.4.10.2.2 Contagem regressiva para a interação.....	154
2.4.10.2.3 Desbloqueando o processamento da função.....	156
2.4.10.2.4 Imprecisão do Disparo do Temporizador.....	158
2.4.10.2.5 Prosseguindo com o desenvolvimento	160
2.4.10.2.6 O Componente Som e o Componente Tocador	161
2.4.10.2.7 Formatos de Arquivo	164
2.4.10.2.8 Cronômetro como último serviço	168
2.5. Compartilhando um Aplicativo!.....	175
2.6. Atividades de Fixação!	178
2.7. Encerramento do capítulo	182

Capítulo 3: Segundo Aplicativo, agora com

Foco em Acessibilidade	184
3.1. Estudo de Componentes	184
3.2. Dentre as Atualizações, um Problema	186
3.3. Apresentação do Aplicativo a ser Desenvolvido	188
3.3.1. Introdução.....	188
3.3.2. Público-Alvo.....	189
3.3.3. Ideia.....	189
3.3.4. Definição da Situação.....	190
3.4. Mas antes de tudo, tem que ser acessível!.....	192
3.4.1. Tecnologias Assistivas.....	194
3.4.2. Reinventando a Roda?.....	199
3.4.3. Acessibilidade no Android.....	201
3.4.4. Ferramentas de Avaliação	202
3.4.5. Usuários com visão não-debilitada.....	204

3.5. Começando o Desenvolvimento!	205
3.5.1. Pilha de funções	215
3.5.2. Pilha de Telas.....	216
3.5.3. Acessibilidade agora mesmo	218
3.5.4. Tela de Configuração do Aplicativo.....	222
3.5.4.1. Composição da Tela.....	223
3.5.4.2. Partindo para o código-fonte	224
3.5.4.3. Inicialização da Tela.....	225
3.5.4.4. Gravação de Configurações.....	229
3.5.5. A Tela Principal do Jogo	233
3.5.5.1. Composição da Tela.....	233
3.5.5.2. A Ideia Geral	237
3.5.5.3. A Estrutura do Arquivo.....	238
3.5.5.4. Um Castelo de Blocos Sempre Começa por Unidades.....	239
3.5.5.5. Garantindo o Resto da Partida.....	261
3.5.6. A tela de Resultados e Criação de Problemas.....	277
3.5.6.1. O Exibidor de Resultados	277
3.5.6.1.1 Blocos.....	279
3.5.6.1.2 Qualquer Componente	280
3.5.6.1.3 O Bloco Componente.....	281
3.5.6.1.4 As funções de adaptação.....	282
3.5.6.1.5 A Identificação de Tarefa.....	283
3.5.6.1.6 Trabalhando em Anunciar Vencedores	283
3.5.6.2. O Criador de Problemas.....	287
3.5.6.2.1 Opção de Área.....	287
3.5.6.2.2 Criador de Problemas Matemáticos.....	288
3.5.6.2.3 Criador de Problemas de Português.....	289
3.5.6.2.4 Botões de Ação.....	289
3.5.6.2.5 Visualizador de Texto Bruto.....	290
3.5.6.2.6 Código-fonte	291
3.6. Publicação do Aplicativo na Galeria App Inventor	309
3.7. Atividades de Fixação!.....	310
Capítulo 4: Alcançando o Mundo Inteiro!	313
4.1. Alcançar o Mundo tem um Custo.....	313
4.1.1. Considerações Iniciais.....	313

4.1.2. Cadastrando um Aplicativo na Loja.....	316
4.2. Fornecendo Informações do App.....	317
4.3. Enviando um pacote APK.....	320
4.4. O Mundo é Nosso!.....	323
4.5. Atividades de Fixação	323
Gabarito de atividades	324
Capítulo 1	324
Capítulo 2	326
Capítulo 3	330
Capítulo 4	334
Referências.....	335

APRESENTAÇÃO

Talvez você já tenha visto como, em algum momento, muitas pessoas passam o dia inteiro (ou quase isso), lidando com alguma tela digital onde são mostradas informações e imagens. Informação é algo muito precioso hoje em dia.

As pessoas lidam com essas telas inserindo, buscando, recuperando, trabalhando, detalhando, simplificando, obtendo e repassando informações uma quantidade absurda de vezes por dia e, às vezes, elas nem percebem isso! O fato é que informação está em todos os lugares. Organizações são fontes de informações, plantas, animais, atmosfera, pessoas, e até mesmo informações são fontes de outras informações!

Os computadores se espalharam pelo mundo como fogo em combustível derramado. Em nossas mãos temos smartphones que executam aplicativos que nos ajudam na comunicação diária e nos fluxos de informação com os quais lidamos todos os dias. Quase que o mundo todo está conectado e unido por essa gigante rede de informação!

Pense que você pode se ajudar a gerenciar os seus próprios fluxos de informação e causar um impacto real nas suas ações cotidianas: isso pode ser feito por meio de um aplicativo. Para além disso, imagine que esse aplicativo possa ser usado por muito mais pessoas e ajudá-las de alguma forma. Agora imagine que quem criou esse aplicativo foi você. Sim! Você pode criar aplicativos!

Não pense que é muito complexo, difícil, a complexidade de um aplicativo depende exclusivamente de você! Você não precisa fazer coisas complexas se não quiser. Além disso, o processo de criação de soluções digitais para problemas do dia a dia fica cada vez mais fácil a cada ano, mês ou semana! Um exemplo disso é a plataforma App Inventor que é o foco deste livro!

Por meio dos conhecimentos transmitidos em cada parágrafo deste livro, tentaremos transmitir o que você precisa saber para construir aplicativos que sejam minimamente funcionais e ainda trabalharemos as bases essenciais para o crescimento de seu conhecimento na área de desenvolvimento de soluções digitais! Tentaremos explicar conceitos de forma simples mas sem descartar a riqueza de detalhes.

Você é capaz de desenvolver ótimos aplicativos e eles podem causar grandes impactos na vida das pessoas. Então embarque conosco nesta viagem de exploração do mundo computacional!

CAPÍTULO 1: INTRODUÇÃO AO APP INVENTOR

No mundo atual, é comum ter uma lista de tarefas do dia a dia, um calendário, um relógio sempre à disposição, informações dos estabelecimentos ao redor, mensagens de última hora, notícias boas, ruins e alarmantes, tudo isso na palma da mão. É certo que vivemos numa era onde informação é valiosa, e vale tanto quanto dinheiro!

As informações podem se originar de diferentes áreas, têm destinos diferentes, origens diferentes, complexidade variante e são produzidas tanto por pessoas como por organizações e sistemas. As pessoas nas cidades (e até no meio rural) vivem conectadas, transmitindo e recebendo informações dos mais variados tipos, seja pela televisão, por rádio, por computadores, por outros dispositivos através da **internet**¹, e assim por diante. Esse fluxo frenético de dados e informações que trafegam pelo nosso cotidiano é, em parte, canalizado por **dispositivos eletrônicos**².

Um dos dispositivos eletrônicos que se popularizou bastante foi o **smartphone**³. Capaz de deixar o ambiente computacional mais enxuto

1 Rede mundial de computadores com a qual bilhões de dispositivos se conectam e se comunicam através de uma grande e complexa infraestrutura de dispositivos voltados às telecomunicações.

2 São máquinas capazes de utilizar energia como força motora e, ainda assim, realizar processamento sobre algum tipo de informação.

3 Smartphone é um aparelho digital, um computador em miniatura, geralmente do tamanho de uma mão que possui alguns botões e uma tela (geralmente sensível a toque). Uma evolução dos telefones celulares.

e simplificado, ele se tornou exemplo de praticidade e facilidade de aprendizado. Os fabricantes trabalham para fazer sistemas amigáveis ao usuário final, isso tem se mostrado promissor visto que a quantidade de smartphones comercializados e a quantidade de novos modelos ao longo dos semestres não param de crescer.

Nesses dispositivos, assim como em um computador comum, existem **programas**⁴ que realizam as tarefas desejadas por seus usuários. Esses programas passaram a ser chamados de “aplicativos”. Os aplicativos são programas construídos de forma que sejam capazes de ser executados nesses aparelhos de configurações mais modestas.

Os aplicativos se tornaram uma forma de empresas continuarem a se manter no mercado, oferecendo conectividade com seus serviços na internet a partir de uma **plataforma móvel**⁵ mais simples, com preços mais acessíveis e mais amigável ao **usuário final**⁶. Pessoas físicas também veem nos aplicativos uma chance de automatizar suas tarefas, ou pelos menos algum jeito de prover auxílio a elas. É uma área que tende a crescer muito já que o número de smartphones comercializados é bem maior do que o de computadores de mesa tradicionais.

Segundo o **website**⁷ StatCounter e suas pesquisas (<http://gs.statcounter.com/os-market-share>), a soma dos dispositivos usando sistemas operacionais para dispositivos móveis (que acessam a internet) é maior que a soma dos mesmos para computadores de mesa segundo os resultados de Fevereiro de 2019, a produção de aplicativos se mostra situada

4 Uma descrição de um grupo de tarefas ou ações que o computador ou dispositivo deve realizar.

5 Dá-se o nome de “plataforma móvel” ao conjunto de dispositivos portáteis que englobam aparelhos como os smartphones.

6 Usuário final é a pessoa no fim da cadeia produtiva, aquela que faz uso do aparelho, o consumidor, dono do dispositivo ou usuário que tem acesso a ele para realizar as ações que precisa a fim de satisfazer suas necessidades.

7 Uma página informativa (ou não) que pode ser acessada através da internet. Conhecido também apenas como “site”.

em meio a um mercado em plena expansão. O resultado das pesquisas pode ser visto na figura 1.



Figura 1: Uso de dispositivos por plataforma/sistema operacional.

(Fonte: StatCounter)

“No levantamento de número de downloads, mercados emergentes como Brasil, Indonésia e Índia foram os principais responsáveis pelo recorde da Google Play.” diz o portal de notícias CanalTech (<https://canaltech.com.br/mercado/mercado-de-aplicativos-bate-recorde-de-downloads-e-receita-no-fim-de-2017-107384/>) com relação aos recordes de receita e **downloads**⁸ vencidos pelo mercado de aplicativos na virada de 2017 para 2018, “Chegou a US\$ 17 bilhões e cresceu 20% em relação ao ano anterior” continua o portal sobre a receita gerada pelas plataformas Android e iOS.

Além disso, o fato é: aplicativos são feitos por alguém, se eles são feitos por pessoas, isso significa que você também pode aprender a fazer! No primeiro tópico deste capítulo apresentaremos o App Inventor.

1.1. Plataforma

Com a filosofia de que “Todos podem fazer aplicativos que impressionem o mundo”, o App Inventor foi desenvolvido pelo **Instituto de Tecnologia de Massachussets**⁹ (MIT, na sigla em inglês) nos Estados

⁸ Download é um termo usado para indicar a transferência (ou cópia) de um conteúdo que está na internet para o dispositivo que o usuário está usando para acessá-la (também conhecido popularmente como “Baixar”).

⁹ O MIT é uma instituição de ensino norte-americana situada nos Estados Unidos da América que tem forte atuação no campo de desenvolvimento da tecnologia e computação.

Unidos. Mas antes de nos aprofundarmos nos conceitos da plataforma, vejamos alguns tópicos de embasamento.

1.1.1. Embasamento

Os smartphones (ou telefones inteligentes) superaram seus antecessores (**telefones celulares**¹⁰) e passaram a fazer muito mais do que apenas ligar, mandar mensagens, armazenar contatos, e manter registro de tudo o que foi feito. Eles são capazes de executar jogos, reproduzir vídeos e fotografar em alta resolução, auxiliar processos de aprendizagem em instituições de ensino, manter o dia do usuário cronometrado e organizado, processar arquivos técnicos usados no trabalho e realizam várias outras tarefas. São, de fato, um minicomputador potente que cabe na palma da mão. Veja a diferença na figura 2.



Figura 2: Smartphones são computadores de bolso que cabem na palma da mão.
(Fonte: Do Autor)

Esse mundo da tecnologia, onde os protagonistas são os dispositivos móveis, acabou por criar um ambiente onde várias empresas e pessoas individuais podem se dedicar a criar suas próprias soluções

10 Pequenos dispositivos eletrônicos que, geralmente, cabem no bolso e na palma da mão capazes de realizar ações de um telefone como ligar para outro celular, manter uma agenda de contatos, um calendário, reproduzir sons, mostrar ou tirar fotos e mais algumas funções.

para **problemas computacionais**¹¹, entretanto, usando menos recursos o que os disponíveis em computadores convencionais, os chamados **desktops**¹². Esse ambiente é conhecido como plataforma **Mobile**¹³.

Como dito, smartphones são computadores, e por isso, herdam várias de suas características. Veja algumas das similaridades, também ilustradas na figura 3:

- Uma unidade central de processamento, ou processador (CPU)¹⁴;
- Uma memória principal de alta velocidade e volátil onde ficam os processos sendo executados no momento (RAM)¹⁵;
- Uma memória secundária mais lenta e não-volátil onde são armazenados **dados de longa duração**¹⁶ (também conhecida como “memória interna” em smartphones);
- Capacidade de anexar outros dispositivos de armazenamento de longa duração ao dispositivo (como, por exemplo, um **Cartão SD**¹⁷).

11 Problemas que podem ser solucionados de forma (semi) automatizada onde o processamento de informação é a peça central da resolução do problema.

12 Computadores de mesa, geralmente compostos por um monitor para exibir imagens e vídeo, um gabinete onde ficam os componentes-chave para o processamento de dados, um teclado por onde o usuário pode controlar o comportamento do computador e repassar textos a ele, e, por fim, um mouse, por onde o usuário pode controlar uma pequena seta (ou cursor) exibida no monitor para indicar tarefas ao computador.

13 Os termos e expressões “plataforma móvel” e “plataforma mobile” são equivalentes.

14 Chip de computação que possui minúsculos e inúmeros componentes elétricos capazes de controlar o fluxo da eletricidade de forma ordenada para representar dados assim como realizar a interação e manipulação entre eles.

15 Uma pequena placa de chips onde existem diversos “compartimentos” que são capazes de armazenar energia a fim de representar dados que serão repassados ao processador. O acesso aos compartimentos ocorre de forma aleatória

16 Acessórios de diferentes tipos que são capazes de guardar dados de forma persistente, de forma que não se percam com facilidade assim que são desconectados de seu aparelho utilizador. Incluem CD's, DVD's, pendrives, disquetes, fitas e discos magnéticos, cartões de memória etc.

17 Um tipo de armazenamento de dados de longa duração, cartões SD são pequenos (do tamanho de uma unha) e comportam uma quantidade muito grande de dados, compreendendo capacidades de armazenamento de meio bilhão (512 megabytes) de letras até aproximadamente 137 bilhões (128 gigabytes) de letras em arquivos de texto comum.

- **Barramentos e circuitos**¹⁸ que interliguem todos os dispositivos.

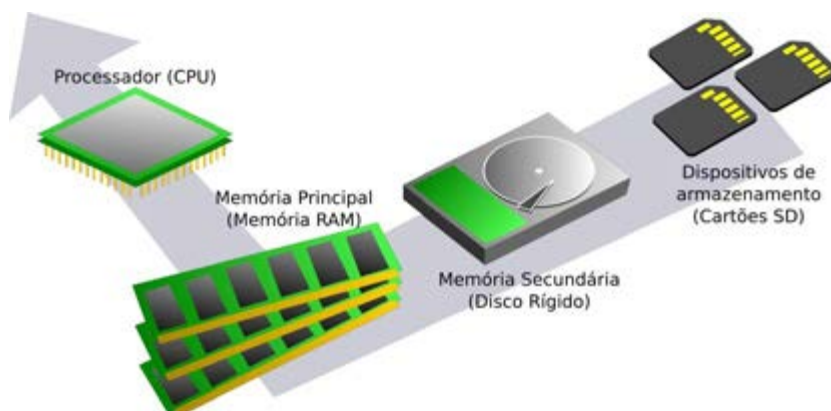


Figura 3: Principais componentes de todos os computadores.

(Fonte: Do Autor)

Quando dizemos que uma memória é volátil, isso quer dizer que os dados armazenados nela são perdidos quando o fluxo de energia acaba. Algumas memórias são de longa duração justamente por serem não voláteis.

Para que os **componentes físicos**¹⁹ do aparelho possam ser operados pelos **aplicativos**²⁰, é preciso haver um intermediário entre os dois. É nesse momento que um tipo especial de sistema entra em cena. O Sistema Operacional (SO) é um software mais robusto, e de caráter muito técnico, que visa criar uma **abstração**²¹ entre os aplicativos/

¹⁸ Conexões condutoras de eletricidade que servem para alimentar dispositivos elétricos ligados nelas.

¹⁹ Conhecidas como peças mecânicas e elétricas, ou pelo termo “hardware”.

²⁰ Aplicativos não são peças físicas, mas são instruções armazenadas em algum tipo de memória já citado que, ao serem lidas pelo processador, alteram o comportamento das peças. Aplicativos, programas e sistemas que rodam sobre o hardware são conhecidos como “software”.

²¹ Uma abstração é um conceito mais elaborado que não depende (ou não depende apenas) de

programas e os componentes físicos do aparelho: um aplicativo de fotos não precisa saber como operar a câmera, um aplicativo de música não precisa saber como operar o alto-falante, um aplicativo de mensagens não precisa saber como operar módulos de rede.

Os aplicativos apenas requisitam certas funções do aparelho ao SO (com instruções limitadas) e ele se encarrega de operar os componentes segundo as especificações que possui em si e em programas especiais chamados **drivers**²².

Ninguém de fato usa um sistema operacional, mas os aplicativos que o usuário usa precisam dele, porque essa é a função dele, servir aos aplicativos, assim como manter o sistema coerentemente operante, lidando com erros e possibilitando que usuário e aplicativo se comuniquem. Se não existissem sistemas operacionais, cada aplicativo ou programa teria que possuir uma versão para cada modelo de computador, dispositivo ou smartphone existente, e isso é completamente inviável.

O Sistema Operacional também tem capacidades interessantes e que muitas vezes não nos imaginamos vivendo sem. Por exemplo, ele pode ser **multitarefa**²³ e, por isso, somos capazes de usar vários programas ou aplicativos “ao mesmo tempo”. O que de fato acontece, é que o SO processa um aplicativo de cada vez em cada ciclo de processamento, mas isso acontece de forma tão rápida que temos a impressão de que tudo acontece simultaneamente.

Então, ao usar um aplicativo, há uma hierarquia de elementos envolvidos que muitas vezes passa despercebida aos olhos do usuário final (conforme ilustra a figura 4 após a listagem):

O Aplicativo está no topo, fazendo a comunicação direta com o Usuário;

conceitos concretos. É uma representação de algo puramente intelectual. No contexto aplicado, também tem o sentido de redução ou simplificação da realidade.

22 Software especializado na manipulação direta do hardware.

23 Capacidade que faz com que o SO se aproveite da velocidade de processamento do computador para realizar diversas tarefas de forma cíclica e parcial, dando a ilusão de que todas ocorrem ao mesmo tempo.

O Sistema Operacional recolhe as requisições do Aplicativo;
 Os Drivers são operados pelo Sistema Operacional, com o objetivo de concretizar as requisições do aplicativo;
 Os Componentes Físicos são operados pelos Drivers a partir de mecanismos internos do smartphone.



Figura 4: Camadas hierárquicas de um sistema computacional geral.
 (Fonte: Do Autor)

É importante ter esses conceitos em mente quando for preciso desenvolver aplicativos avançados com necessidades muito específicas em relação ao sistema computacional em geral. Mas para esse livro, a parte que receberá mais enfoque é o topo da hierarquia onde o App Inventor atua, a camada dos aplicativos.

1.1.2. Objetivo

O App Inventor possui enfoque na plataforma Android, o SO para dispositivos móveis mais popular do mundo. É um sistema operacional desenvolvido pela empresa **Google**²⁴ e pela Open Handset Alliance

²⁴ Empresa norte-americana de tecnologia sediada nos Estados Unidos da América, conhecida pelo motor de buscas na internet mais popular do mundo (<https://www.google.com>) que leva o

(abordaremos isso mais tarde) sendo da categoria de código aberto, onde qualquer pessoa pode obter o código que forma o sistema, modificá-lo e redistribuí-lo.

O App Inventor tem como objetivo facilitar a criação de aplicativos, de uma forma que qualquer pessoa possa aprender conceitos básicos de **programação**²⁵ e ainda possa compartilhar seus trabalhos por meio dos métodos de distribuição desejados. Ele fornece várias ferramentas para o usuário abstraindo vários conceitos técnicos para que o usuário que está desenvolvendo o aplicativo não tenha que se preocupar com eles.

1.1.3. Metodologia

No contexto simples, um computador é uma ferramenta capaz de processar ou computar algo, como dados, números etc. Evoluindo esse conceito para caracterizar a máquina com a qual lidamos no dia a dia, um computador, assim como apontado pelo Portal Educação (<https://www.portaleducacao.com.br/conteudo/artigos/informatica/afinal-o-que-e-um-computador/41539>) nada mais é do que uma máquina que integra diversas peças e ferramentas enquanto executa instruções, mas, embora pareça simples, esse conceito é muito amplo e, por si só, já abre um leque de possibilidades e conceitos muito mais avançados que computadores podem englobar e trabalhar sobre.

A um conjunto de instruções se dá o nome de programa (ou “rotina”, “software”, ou mesmo “aplicativo” como em nosso caso). Esse programa é uma série de instruções **sequenciais**²⁶ conforme aponta o portal Significados (<https://www.significados.com.br/software/>). Esse programa é lido pelo processador do computador e dependendo da instrução lida, vários mecanismos são ativados ou desativados fazendo

mesmo nome da empresa.

25 Técnica empregada para concretizar uma solução de problema computacional em um programa, sistema ou aplicativo.

26 Instruções que seguem uma sequência, denotam linearidade, são executadas uma após a outra.

com que dados possam ser adquiridos, computados, armazenados, relatados ou mesmo apagados.

No passado, não existiam computadores pessoais como os que conhecemos hoje, não havia sequer o conceito de smartphones. Os computadores eram gigantes, precisavam de energia equivalente à quantidade necessária para manter uma cidade, **esquentavam muito**²⁷, quebravam com muita facilidade e serviam apenas para uma ou para pouquíssimas operações, conforme relatado pelo portal Hardware.com.br (<https://www.hardware.com.br/guias/historia-informatica/eniac.html>). Um exemplo desse tipo de computador foi o ENIAC, construído para lidar de forma rápida com cálculos de artilharia durante a segunda guerra mundial.

Com a evolução da tecnologia, os computadores passaram por um processo de **miniaturização**²⁸. E isso permitiu concentrar mais poder de processamento em menos espaço físico. Entretanto, a função deles continuou a mesma.

É importante notar que computadores não aprendem o que supostamente deveriam fazer por si mesmos (a não ser que sejam dotados de **inteligência artificial**²⁹), eles precisam que alguém os ensine. Mas como você ensinaria em português uma receita de bolo a um alemão que só fala sua língua nativa? Computadores seguem a mesma lógica. Por serem máquinas elétricas e mecânicas, eles possuem vários circuitos por onde correntes elétricas são conduzidas. A linguagem usada pelos computadores é o código binário, cujo alfabeto só possui dois caracteres:

- Zero (ausência de energia);

27 Segundo as leis da física, o fluxo de eletricidade (ao encontrar resistências num circuito elétrico) se dispersa sendo transformado em calor. Esse princípio é o que faz com que chuveiros e torradeiras elétricos esquentem a água e torrem o pão. Porém, ele também se aplica a computadores, onde seus efeitos são indesejados.

28 Processo de empregar novas técnicas de fabricação para concentrar mais componentes de um produto em um espaço menor e tornar o produto mais eficiente e menos exigente quanto ao uso de recursos.

29 Sistemas que, quando expostos a certas situações, podem aprender como agir perante elas, e assim, determinar as melhores soluções para eventuais problemas envolvendo tais situações.

- Um (presença de energia).

Porém, é extremamente complicado para um ser humano conseguir ensinar algo a um computador usando apenas estes dois caracteres. Com o intuito de facilitar essa tarefa e deixar essa comunicação mais legível aos humanos, surgiram as linguagens de programação.

“Linguagem de Programação é uma linguagem escrita e formal que especifica um conjunto de instruções e regras usadas para gerar programas (software) [...]” informa o blog Universidade da Tecnologia (<https://universidadetecnologia.com.br/o-que-e-linguagem-de-programacao/>). Linguagens de Programação são idiomas que um computador pode falar, e que são mais diversos em alfabeto do que o código binário.

Assim, fica mais fácil para uma pessoa transmitir suas instruções ao computador. No início, surgiu uma linguagem especial, chamada de linguagem de montagem (por ser muito técnica e muito próxima do código binário) conhecida por “Assembly”. Escrever códigos em Assembly necessitava de conhecimentos avançados em computação e qualquer erro no programa poderia causar danos e **avarar**³⁰ o aparelho em que era executado.

Com o tempo, surgiram mais linguagens de programação com finalidades específicas (como aplicações em meio comercial ou científico, por exemplo) e ficou gradualmente mais fácil se comunicar com o computador. Essa distância da língua mais “orgânica” ou “humana”, chama-se de “nível de abstração”. Quanto mais próximo do entendimento humano, mais abstrata (alto nível), quanto mais distante, menos abstrata (baixo nível). Essa diferença é vista na figura 5.

30 Por lidar diretamente com o hardware, um erro no código poderia realmente estragar o processador por endereçar um fluxo de eletricidade a alguma parte sensível do chip. Esse efeito poderia, inclusive, repercutir para outras peças do computador.

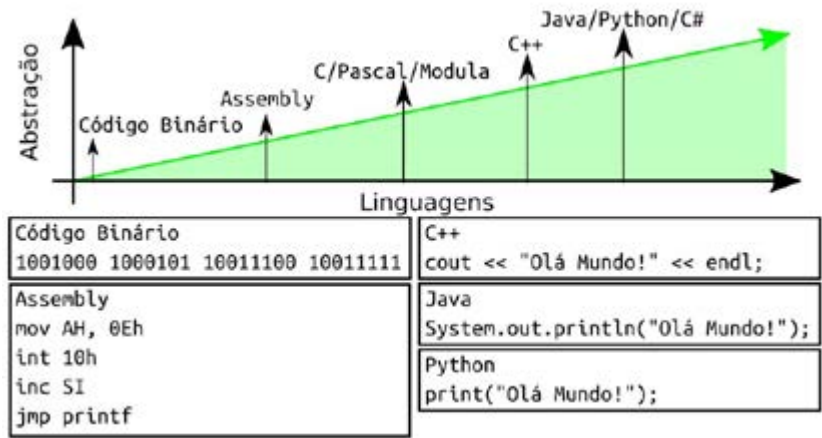


Figura 5: Exemplos de linguagens de programação segundo o nível crescente de abstrações.

(Fonte: Do Autor)

Primeiramente, as linguagens de programação precisavam ser traduzidas para o computador com o uso de programas especiais chamados “**compiladores**”³¹. Sua função é obter o código escrito por um programador, analisar suas falhas morfológicas e sintáticas (assim como em qualquer idioma) e converter este código para seu correspondente em código binário. Assim, um programa executável é gerado pelo compilador, adaptado e otimizado ao sistema que está sendo usado. Esse programa é independente de seu compilador e geralmente é dotado de alta performance.

Mas linguagens de programação **continuam evoluindo**³², e para algumas delas, a necessidade de compilação foi anulada, este é o caso das linguagens interpretadas. A figura do compilador é substituída pela do interpretador, um programa especial que obtém o código do programador e faz a tradução e as requisições ao SO em **tempo real**³³. Um

31 Isso envolve a conversão do código de alto nível para código de máquina (binário), de baixo nível. Esse processo é feito pelo compilador, um programa que constrói outros programas.

32 Assim como idiomas humanos (como o Latim), algumas linguagens de programação também são consideradas mortas ou obsoletas (como a linguagem Cobol).

33 Interpretadores funcionam como tradutores. O nativo diz uma mensagem, o tradutor realiza

programa de código interpretável é dependente de seu interpretador e costuma ser mais lento, porém, independe da plataforma em que é executado. A diferença entre linguagens compiladas e interpretadas por ser vista na figura 6.

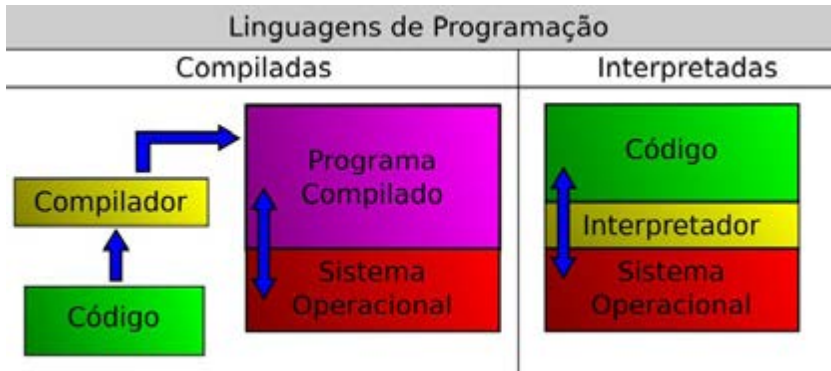


Figura 6: Principal diferença entre linguagens compiladas e interpretadas.
(Fonte: Do Autor)

O Sistema Operacional Android usa Java para seus aplicativos. Java é uma linguagem interpretada criada pela empresa **Sun Microsystems**³⁴. O código é obtido do programador e convertido em um código otimizado chamado bytecode (para ganhar velocidade durante o processamento) e, então, enviado para execução em seu interpretador, a **Máquina Virtual Java**³⁵.

Porém, a linguagem Java é **codificada**³⁶ através de texto, e isso é realmente pouco intuitivo/atrativo para quem deseja iniciar estudos em programação. Percebendo essa necessidade de tornar o ensino de programação algo mais suave e a necessidade de criar uma transição

a conversão entre os idiomas, e só então repassa a mensagem ao estrangeiro em seu idioma. Essa conversão leva tempo e reduz o desempenho.

34 Empresa de tecnologia norte-americana sediada nos Estados Unidos da América.

35 Em computação, há um conceito conhecido como “virtualização” onde é possível simular um sistema ou ambiente computacional dentro de outro sistema ou ambiente. A Máquina Virtual Java se apropria desse conceito para interpretar seus programas.

36 Codificar é outro termo usado com o mesmo significado de programar.

agradável entre a linguística humana e a programação em si, a plataforma App Inventor aposta bastante em elementos visuais e uma forma interativa/intuitiva de programar.

Nisso, a plataforma possui uma linguagem em que cada instrução é um **bloco**³⁷, e cada especificação é um **anexo em forma de blocos**³⁸. Os blocos são montados de forma sequencial da mesma forma que a programação convencional. A **interface**³⁹ simplificada ajuda o usuário a observar e memorizar detalhes, separando visualmente por cores cada categoria de instrução e especificação. Abordaremos isso de forma mais profunda nos próximos capítulos.

Sendo assim, o usuário inicialmente obtém uma percepção descontraída da ferramenta, se sentindo à vontade para testar as possibilidades e se divertir com os resultados, sem se preocupar muito com **detalhes sintáticos**⁴⁰ da linguagem. Ao fim, o código escrito pelo usuário (ou melhor, montado com os blocos), é traduzido para Java, envelopado em um **pacote de software**⁴¹ contendo o programa e disponibilizado para que o usuário possa testá-lo em seu smartphone.

Mas apenas ter o pacote pode não ser suficiente. O Android possui uma loja oficial de aplicativos, discutiremos como o Android (<https://www.android.com/>), o App Inventor (<http://ai2.appinventor.mit>.

37 Por exemplo, se for preciso executar a instrução “imprimir mensagem na tela”, essa instrução seria representada graficamente como um pequeno bloco de montar colorido.

38 Na instrução de exemplo “imprimir mensagem na tela”, precisamos de uma mensagem (como “Bom dia!” ou “O programa/aplicativo começou!”). Essa mensagem seria uma especificação em formato de bloco com um encaixe para a instrução que necessita dela.

39 Interface, de forma geral, é a forma com que duas partes podem se comunicar. No caso, os botões, menus, seções e telas apresentados pelo App Inventor formam uma interface de comunicação com o usuário (esse tipo de interface é comumente conhecido como “interface gráfica”, ou GUI para o termo em inglês “Graphical User Interface”), mas também existem interfaces de texto, comando e diversos outros tipos.

40 A sintaxe de uma linguagem estabelece como os termos usados por ela devem ser ordenados a fim de formar um significado coerente. Ao dizer “O carro andar bom uns partido nadas” não foi possível passar uma ideia coerente pois a mensagem possui palavras que existem e estão corretas, entretanto não seguem corretamente a sintaxe da língua portuguesa. A sintaxe de uma linguagem de programação é geralmente uma parte difícil de se memorizar.

41 Para que possa ser instalado, um aplicativo tem todos os seus recursos e códigos reunidos em um único arquivo chamado pacote. No caso do SO Android, o pacote atende pela sigla APK.

edu/?locale=pt_BR) e a Google Play (<https://play.google.com/>) se unem no próximo tópico.

Integração

Ao terminar um programa, você pode mantê-lo só para você, passar para uma pessoa ou grupos específicos, ou distribuir para o público em geral. Há várias formas de distribuir um software. Antigamente isso poderia ser feito passando o programa por **disquetes**⁴², com o tempo passou a ser feito com CD's, DVD's, pen drives e, atualmente, o método mais popular é através da internet.

Uma forma menos popular de distribuir software é repassando o **código-fonte**⁴³ do programa para que a outra pessoa seja capaz de compilá-lo ou interpretá-lo. Porém, isso traz à tona uma questão importante, uma vez que a pessoa tem acesso a seu código, ela pode alterá-lo. É nesse cenário que entram as licenças de software.

Licenças⁴⁴ estabelecem o que pode ou não ser feito sobre o uso do programa ou seu código. Programas de código fechado estão sob os termos de **licenças proprietárias**⁴⁵ e a modificação do código (mesmo por **engenharia reversa**⁴⁶), está vedada. Programas de código aberto estão sob os termos de licenças livres que permitem às pessoas modificar, usar e alterar o software da forma como quiserem.

Softwares proprietários podem ser comercializados, softwares livres são **gratuitos**⁴⁷, existem várias licenças sob as quais você pode licenciar o seu programa. Algumas delas exigem custos de registro,

42 Pequeno dispositivo (do tamanho de uma mão, ou de sua palma) achatado e quadrado com baixa capacidade de armazenamento. Arquivos maiores que a capacidade de um disquete costumavam ser divididos em várias partes, cada uma em um disquete.

43 Código-fonte inclui todo o código escrito que forma o programa antes de sua compilação.

44 Contratos ou Termos de Uso (com valor legal/jurídico) incorporados ao software com os quais o usuário concorda explicitamente durante sua instalação (ou implicitamente através do uso).

45 Essas licenças são usadas geralmente por softwares de empresas para uso comercial.

46 Engenharia Reversa é uma técnica onde algo é reconstruído do zero a partir da observação de sua forma, comportamento ou funcionamento.

47 Porém sua distribuição pode ser cobrada.

outras podem ser feitas de forma gratuita como no caso da licença livre **GPL**⁴⁸.

Mas de uma forma ou outra, seu programa chegará ao seu público-alvo. Percebendo a integração entre Google Play e Android, a plataforma App Inventor não possui um sistema de contas de usuário próprio. Assim como vários outros sites que dão a opção de não realizar login com uma conta própria (e sim com a conta de um outro site, geralmente redes sociais), o App Inventor permite fazer login apenas com a conta Google do usuário.

Essa integração se deve ao fato de os aplicativos construídos, ao serem enviados para compartilhamento e distribuição na Google Play, estarão **automaticamente vinculados à conta de usuário**⁴⁹. Com isso, informações referentes a testes e qualidade de desenvolvimento podem ser obtidas de outros usuários.

1.2. Desenvolvedores

1.2.1. App Inventor

Os desenvolvedores do App Inventor, mantido pelo MIT, tem a promessa de ajudar as pessoas a descobrirem que podem criar aplicativos poderosos e com funções completas que podem fazer a diferença na vida real, inclusive crianças. A forma interativa e intuitiva com que o fluxo de trabalho no App Inventor acontece, permite ao utilizador aproveitar sua criatividade para focar mais em funcionalidades e menos em detalhes técnicos.

“O App Inventor do MIT é um ambiente intuitivo de programação visual que permite que todos – até mesmo crianças – possam construir aplicativos totalmente funcionais para smartphones e tablets. Aqueles que são novos ao App Inventor podem fazer um simples primeiro apli-

48 A GPL (“General Public License” ou “Licença Pública Geral” em tradução livre) é uma das licenças de software livre mais conhecidas mundialmente, é mantida pelo Projeto GNU.

49 Entretanto, para realizar envios de aplicativos para a Google Play, é preciso pagar uma taxa para aprovação de uma conta de desenvolvedor.

cativo pronto e funcionando em menos de 30 minutos. E além disso, nossa ferramenta baseada em blocos facilita a criação de aplicativos complexos e de alto impacto em um tempo significativamente menor do que em ambientes de programação tradicionais.” (tradução nossa) afirma a página “Sobre nós” no site do App Inventor (<http://appinventor.mit.edu/explore/about-us.html>).

No site da plataforma (http://ai2.appinventor.mit.edu/?locale=pt_BR) podem ser encontrados vários **links**⁵⁰ para informações a respeito dos desenvolvedores (inclusive uma lista com os membros da equipe), a missão do App Inventor, atualizações periódicas da plataforma, uma lista com citações de projetos que ajudaram a causar um grande impacto em várias áreas, mesmo as fundamentais como saúde pública e educação.

1.2.2. Android

O Android (<https://www.android.com/>) foi desenvolvido pela Android Inc que mais tarde foi adquirida pela Google e pela Open Handset Alliance, uma aliança de várias empresas do ramo de tecnologia que tinham como foco fazer um SO de dispositivos móveis que fosse altamente **personalizável e atualizável**⁵¹.

A promessa foi cumprida, com o lançamento de sucessivas versões, atualmente o Android ainda está sob propriedade da Google e da Open Handset Alliance, sendo o SO para dispositivos móveis mais popular do mundo.

O Android foi construído tendo como base um **núcleo de sistema**⁵² (kernel) Linux, mas que foi adaptado para ser executado em uma **arqui-**

50 Dentre vários significados, link, nesse contexto, se refere a um texto especial que, quando clicado, leva o usuário a visualizar outro tipo de página da internet ou conteúdo.

51 Personalizar e Atualizar são características de softwares que permitem ao usuário, respectivamente, mudar a aparência e comportamento (das funcionalidades oferecidas ao usuário), e aplicar mudanças no núcleo do funcionamento do software.

52 Um núcleo de sistema, ou kernel, é uma peça fundamental em qualquer sistema operacional, sua função é reger como cada componente do conjunto computacional deve ser (des)alocado e coordenar como a comunicação entre elas ocorrem.

tetura de processadores⁵³ (ARM) diferente da usada em desktops (x86). No esquema de funcionamento do Android está incluída a máquina virtual Java responsável por executar todas as aplicações com as quais o usuário tem contato, de forma transparente e imparcial, de modo que todas os aplicativos são encarados e tratados de forma igual pelo SO.

1.2.3. Você e suas ideias são bem-vindas!

Adquirindo os aprendizados, conceitos e técnicas deste livro, você estará apto para criar seus próprios aplicativos, e isso o fará um desenvolvedor também! Mas antes, é preciso discursar sobre alguns aspectos importantes que antecedem ao desenvolvimento em si.

Quando você decide viajar para um lugar distante onde não conhece ninguém, é imprescindível que seja feito um planejamento. Em momentos de aperto, não ter uma mão amiga para prestar ajuda pode significar um momento de dificuldade garantido! Quando se trata de desenvolvimento de software, planejamento também é algo muito importante. Saber milimetricamente o que seu programa deve fazer facilita em muito o processo na hora de programar.

Há várias formas de organizar as funcionalidades de seu programa:

- Definir claramente o que ele deve fazer (Objetivo);
- Definir as ferramentas necessárias (Componentes do programa);
- Mapear o fluxo de informações e de uso do programa (Passo-a-passo);

Entender o problema é fundamental para entender como resolvê-lo, esses são métodos básicos para manter o mínimo de organização ao desenvolver um programa. Conforme seus programas aumentarem

⁵³ Processadores de computadores têm modelos diferentes, mas, acima disso, são feitos em arquiteturas diferentes. A forma de funcionamento muda de acordo com a arquitetura. Por esse motivo, programas compilados para arquiteturas x86 não são compatíveis com processadores de arquitetura ARM. Por esse e outros motivos, não é possível executar programas Desktop na plataforma Mobile.

em complexidade e tamanho, seria interessante pesquisar por formas mais avançadas de gerenciar projetos como por meio de:

Levantamento de requisitos⁵⁴ (por questionários e entrevistas);

Diagramação do fluxo de informação e de estados do programa (por diagramas **UML**⁵⁵);

Documentação de cada funcionalidade (para manter o programa compreensível a quem der continuidade ao seu desenvolvimento).

Vale ressaltar que essa parte do processo de desenvolvimento ajuda a definir o que o programa deve fazer, e não como fazer (excluindo todos os detalhes inerentes à programação). Saber destes detalhes ajudam os desenvolvedores a não se perderem no processo. Muitas pessoas que começam a programação antes de entenderem plenamente o problema, acabam se perdendo em meio aos códigos e tendo que recomençar o projeto do zero a fim de readquirirem o entendimento do próprio programa.

Após a fase de planejamento e da compreensão do problema, o programa pode partir para a fase de programação. Um planejamento bem-feito permite que o programa seja implementado independente da linguagem utilizada, desde que ela seja capaz de atender a todas as funcionalidades necessárias ao funcionamento do programa.

É importantíssimo lembrar que quando se trata de programar, problemas grandes devem ser quebrados em pequenos problemas que, quando solucionados individualmente, são capazes de solucionar o todo – esse pensamento é bem ilustrado pela expressão “Dividir para conquistar”. Além disso, a solução individual dos problemas deve seguir um pensamento lógico baseado no fluxo de informação.

54 Em computação, “Análise de Sistemas” ou “Engenharia de Requisitos” é uma forma de levantar informações sobre todas as funcionalidades e necessidades que o sistema deve atender para que seu objetivo seja alcançado com êxito.

55 Como antigamente havia várias formas de modelar e organizar os objetivos de um sistema, equipes de desenvolvimento adotavam suas próprias soluções. Mas isso gerava problemas porque quando havia troca de membros da equipe, havia também problemas para que os novos membros entendessem a modelagem do sistema. Para acabar com tais problemas, surgiu a Unified Modeling Language (UML) com especificações padronizadas de como sistemas devem ser descritos e modelados por meio de vários documentos e formatos de diagramas.

Durante a programação e os testes de seu aplicativo, perceberá que instruções que não foram suficientemente especificadas poderão gerar comportamento não planejado, erros fatais, travamentos e lentidão no fluxo de processamento. São indicativos de erros e falhas no código do programa devido à má manipulação de estruturas de código ou de informações.

Esses erros, chamados popularmente de “bugs” (em referência aos grandes e antigos computadores que queimavam quando insetos entravam em seus circuitos), não são percebidos pelos compiladores e interpretadores, e cabe ao programador/desenvolvedor analisar o código e perceber onde e como eles ocorrem. Trata-se de erros lógicos, em que a solução foi escrita corretamente, mas seu funcionamento não acontece como o esperado.

Ao terminar seu programa, pode levá-lo para a fase de testes finais e depois para a distribuição, porém, se depois de disponibilizá-lo forem percebidos bugs, ele pode ser submetido novamente ao processo de planejamento e programação. É interessante manter cópias do programa a cada vez que ele for ressubmetido a esses processos.

Usar um esquema de **versionamento**⁵⁶ (organizando cada produto final por códigos) e fazer uma lista com todas as correções e funcionalidades que foram adicionadas (conhecido como “**changelog**”⁵⁷, ou registro de mudanças) ao programa são artifícios usados para localizar, tanto usuários quanto desenvolvedores, no que se refere ao uso do programa na linha do tempo de seu desenvolvimento.

Usar um programa sabendo que ele é capaz de executar a função X e não tê-la em sua cópia, pode indicar que sua versão está desatualizada (ou que ela está atualizada e esta função foi removida em uma versão prévia).

56 Versionamento é uma técnica de desenvolvimento de sistemas que consiste em nomear, de forma crescente, uma cópia do programa com uma sequência de números a cada vez que uma funcionalidade (ou um conjunto delas) é adicionada, corrigida ou removida.

57 Literalmente “Registro de mudanças” é um documento que lista todas as mudanças feitas num software em ordem cronológica e por versionamento. O App Inventor também possui um changelog, ele está disponível no seguinte link: <http://appinventor.mit.edu/ai2/ReleaseNotes.html>

1.3. Sobre a Plataforma App Inventor

O App Inventor passou por atualizações e renovações ao longo dos anos, e por isso, houve uma mudança de logotipo. Em seus primórdios, a plataforma se baseava nos princípios de outra plataforma, a Scratch, que, por sua vez foi inspirada pela linguagem de programação Logo.

1.3.1. Versões

A linguagem de programação interpretada Logo consistia em dar comandos simples a um **cursor**⁵⁸ (ou uma tartaruga, posteriormente) para que ele se movesse na direção desejada, sendo assim, um rastro do caminho percorrido era deixado na tela do dispositivo, assim, isso poderia ser usado para formar desenhos ou escrever palavras ou frases na tela, assim como a teoria apontada por Bruno de Oliveira Schneider da Universidade Federal de Lavras (<http://algol.dcc.ufla.br/~bruno/wxlogo/docs/oqueue.html>).

Surgida em 1967, foi muito usada para introduzir leigos, em qualquer faixa etária, ao ensino de programação. Os comandos e instruções eram curtos, o estudante poderia captar a essência de programação com os conceitos de funções, repetição, parâmetros/argumentos, variáveis etc.

Com base nessa lógica de restringir o ambiente de programação para o ensinamento a leigos (principalmente focado no público infanto-juvenil), surgiu em 2007, mas vinha sendo desenvolvida desde 2003, a linguagem de Programação Scratch que também foi desenvolvida pelo MIT, segundo informações da Wiki Oficial Scratch (<https://en.scratch-wiki.info/wiki/Scratch>).

58 Um exemplo muito comum de cursor é o ponteiro, ou seta, que representa o mouse. Trata-se de um elemento que indica a localização de uma informação, na maioria das vezes é possível interagir com o cursor. No caso citado, o cursor da linguagem Logo era capaz de se mover pela tela e deixar rastros.

Scratch possui um esquema de programação baseado em blocos e **orientado a eventos**⁵⁹, onde os comandos são montados sequencialmente, recebem suas especificações e, então, são executados de acordo com o evento associado a ele, como, por exemplo: se tal botão é pressionado, tal conjunto de instruções deve ser executado.

Ao perceber que o ideal defendido pela linguagem Scratch poderia ser potencializado, outros desenvolvedores do MIT se uniram e idealizaram o App Inventor, uma plataforma online onde pessoas poderiam criar aplicativos com funcionalidades pré-definidas cujos comportamentos são definidos pelo esquema de blocos e orientação a eventos que foram herdados da Scratch.

O projeto começou tendo por base os esforços de mesmo propósito da Google. A plataforma já havia sido lançada em Dezembro de 2010 pela Google, mas foi repassada ao MIT e reformulada por esse até ser, finalmente, relançada ao público geral em Janeiro de 2012. Uma **postagem**⁶⁰ no **blog**⁶¹ do time de desenvolvedores relata o início dos trabalhos em 2011 (<http://appinventor.mit.edu/explore/content/development-starts-mit.html>). A primeira versão do App Inventor tinha o seguinte logotipo mostrado na figura 7.



Figura 7: Primeiro logotipo da plataforma App Inventor.

59 Orientação a Eventos é uma forma de programação onde os procedimentos programados são ativados apenas se um evento acontecer. Quando um usuário ou outro sistema realizam um certo evento ou ação, dizemos que eles fazem um “disparo”. Esse disparo é percebido pelo sistema em questão e, então, o procedimento é colocado em prática. Um exemplo pode ser os semáforos de trânsito: o motorista para no cruzamento quando o semáforo está no vermelho. O cronômetro interno do semáforo conta o tempo de mudança, e, quando esse tempo se esgota, ele dispara o evento de mudar a cor. O semáforo realiza a troca do fluxo de energia e a luz verde acende.

60 Um conteúdo lançado na internet, por meio de algum site, tendo uma seção ou página dedicada para si no mesmo. Também é conhecido corriqueiramente como “post”.

61 Um site na internet caracterizado por trazer informações periódicas a respeito de algum assunto central, geralmente de maneira informal.

(Fonte: App Inventor)

A peça de quebra-cabeça significava a facilidade do processo de programação e a associava a uma brincadeira que incita o raciocínio lógico ao montar uma imagem por encaixar diversas peças com pinos e entradas, assim como o funcionamento do esquema de blocos usado na plataforma. O símbolo verde é um fragmento da logo oficial do Sistema Operacional Android, fazendo referência a uma das finalidades da plataforma: desenvolver aplicativos para o sistema. Segue o logotipo do Android na figura 8.



Figura 8: Logotipo do Sistema Operacional Android

(Fonte: Android)

A plataforma foi um grande sucesso com um pico de 30.000 usuários ativos semanalmente em 2014. O App Inventor “Classic” (como ficou conhecida a primeira versão) foi capaz de receber 1,7 milhão de usuários que, juntos, produziram cerca de quase quatro milhões de aplicativos.

Mas por volta de 2011, a equipe de desenvolvimento já percebia a necessidade de melhora, de trocar o sistema. Com a vinda do App Inventor 2, a necessidade de baixar um **complemento Java**⁶² para fazer a plataforma funcionar foi eliminada, além disso, as seguintes mudanças foram feitas:

- Totalmente online, não há necessidade de baixar qualquer software extra para usar a plataforma;

62 Como a tecnologia para implementar certas funcionalidades à plataforma não estavam muito desenvolvidas na época, para que o navegador do usuário fosse capaz de mostrar a interface da plataforma era necessário baixar e instalar um complemento Java nativamente no computador.

- Possibilidade de criar telas extras para o aplicativo, independente da sessão em que o usuário está no momento;
- Maior facilidade na seleção e organização dos blocos;
- Novos blocos foram incluídos (e até substituíram outros que se tornaram obsoletos);
- Adição de caixas de combinação (anulando blocos de função idêntica, porém de efeito diferente);
- Adição de modificadores de forma de bloco;
- Adição de diferenciação de variáveis entre variáveis globais e locais;
- Lançamento de um emulador Android oficial da plataforma para Windows, Mac e Linux;
- Adição de um Bloco para seleção de Cores;

Quando o App Inventor 2 foi introduzido, houve uma demora para os usuários **migrarem**⁶³. Uma ferramenta para realizar a conversão dos projetos do Classic para o 2 foi disponibilizada pelos desenvolvedores aos usuários, assim, os projetos **não seriam perdidos**⁶⁴.

Vários problemas de segurança relacionados à Java surgiram na época e isso tornava a migração um tanto quanto alarmante. Porém alguns usuários ainda se mostravam resistentes. Perto de seu fim, o App Inventor Classic ainda servia a uma base de 3.000 usuários ativos semanalmente. Entretanto, em Julho de 2015 o serviço foi definitivamente encerrado. O documento oficial de despedida da plataforma está no site (<http://appinventor.mit.edu/explore/blogs/hal/2015/07/mit-app.html>).

Assim que foi imposto que o software Java auxiliar não deveria mais ser usado, alguns membros da equipe começaram a trabalhar em uma solução nativa da plataforma para desenhar os gráficos dos blocos na tela. Visto que a performance de aplicações em Javascript (uma linguagem interpretada que funciona em navegadores web, sem

63 Ato de mudar, de uma plataforma (ou sistema) para outra, os projetos criados.

64 Quando uma nova versão de um sistema é concebida, algumas funcionalidades podem deixar de existir. Sendo assim, projetos que dependiam de tais funcionalidades podem ficar inoperantes na nova versão do sistema. A isso dá-se o nome de “Quebra de Compatibilidade”.

vínculos com a linguagem Java) tinha aumentado consideravelmente naquela época, o pequeno projeto se tornou passível de sucesso e logo foi incorporado à linha principal de desenvolvimento do App Inventor 2.

Com o impulso de ser parte oficial da plataforma, a solução gráfica ganhou mais destaque e logo evoluiu para uma plataforma independente de **desenvolvimento geral**⁶⁵ com blocos. Ela foi chamada Blockly. Atualmente ela é usada tanto no App Inventor quanto em outros sites (como o Code.org em <https://code.org/>). Com um sistema novo, o logotipo foi redesenhado, agora de uma forma mais vívida, como se segue na figura 9.



Figura 9: Logotipo da segunda versão da plataforma App Inventor

(Fonte: App Inventor)

Criada com a técnica de flat design (com cores sólidas e constantes, de paleta bem definida), o logotipo do App Inventor 2 mostra a nova mascote da plataforma, uma pequena abelha. Um post sobre a nova logo pode ser encontrado no site oficial (<http://appinventor.mit.edu/explore/blogs/karen/2017/08/about.html>), nele são destacados os seguintes pontos:

- Abelhas fazem do mundo um lugar melhor;

⁶⁵ Uma solução de desenvolvimento geral é um sistema (ou um subsistema não operante por si mesmo) que é anexado a um sistema maior, mas não se restringe a ele, podendo ser usado por inúmeros sistemas diferentes que o adaptam ao seu funcionamento para solucionar diversos problemas.

- Abelhas trabalham individualmente e em grupo, sempre melhorando suas comunidades;
- Abelhas são criadoras e construtoras;
- Abelhas são comprometidas com seus alvos e objetivos;
- Assim como o trabalho das abelhas (polinização) ajuda a manter o planeta saudável, o potencial que a plataforma oferece a seus usuários faz com que eles possam impactar positivamente a realidade e o mundo em que vivem.

Além disso, há no post pequenas considerações sobre meio ambiente, nutrição e saúde pública, processo criativo e marketing. A forma com a qual a nova logo foi escolhida também é relatada no texto.

1.4. Sites Relacionados

A plataforma App Inventor, como apresentado, está relacionada a outras iniciativas. Para a abordagem deste livro, consideraremos as duas maiores influências: Android e Scratch. Esses são os pontos de partida da plataforma em questão e serão explicados com mais detalhes adiante.

1.4.1. Plataforma Scratch

A Linguagem de programação Scratch (para mais informação, consulte <https://scratch.mit.edu/>) tem como premissa ensinar a crianças e jovens conceitos de programação de forma divertida, intuitiva e descontraída. Se trata de um sistema online (com programa nativo para desktops) que manipula imagens, sons e efeitos numa determinada área da tela cujos elementos nela posicionados podem ser afetados pelos blocos associados a eles e montados pelo usuário de acordo com suas necessidades.

Com um grande acervo de jogos, animações, e aplicações interativas publicados online, a Scratch se provou fiel em cumprir seu objetivo, por esse motivo ela foi usada como ponto inicial no ensino de programação em instituições de ensino, desde escolas fundamentais a faculdades.

Ao usar a plataforma Scratch pela primeira vez, o usuário se depara com um conjunto de ferramentas conforme mostrado na figura 10:

1. Botões de início e fim de execução;
2. Painel de Sprites (imagens) e ferramentas de edição de suas propriedades;
3. Ferramentas de **Upload**⁶⁶ de mídia;
4. Área do editor de blocos;
5. Coletânea de blocos editáveis;
6. Área de execução do projeto.



Figura 10: Sessões da interface de usuário da plataforma Scratch.

(Fonte: Scratch)

Outras ferramentas como, por exemplo, editor de imagem e som, botão de tela cheia e campo de texto com título de projeto podem ser encontradas na plataforma. Além de poder editar, salvar e publicar o projeto totalmente online, a plataforma disponibiliza a opção de salvar o projeto no computador (baixar), caso o usuário deseje usar o programa nativo.

Antes de usar a plataforma App Inventor, poderia ser de proveito usar Scratch. Como desenvolver aplicativos envolve conceitos obrigato-

⁶⁶ Upload é um termo inverso ao de Download, significando uma transferência de um arquivo ou conteúdo do dispositivo do usuário para a internet.

riamente um pouco mais técnicos para quem começou a estudar programação, aprender um tanto da lógica de programação com Scratch pode ajudar no desenvolvimento posterior de habilidades na hora de programar.

1.4.2. Fórum Oficial App Inventor

Em parte dos casos, softwares disponibilizados ao público geral adquirem uma quantidade de usuários que fazem uso regular dele. Essa base de usuários muitas vezes não se forma tendo pleno conhecimento das funções do software com a qual está envolvida, e isso resulta em usuários com dúvidas sobre como tal programa, aplicativo, sistema ou plataforma funciona. Então, essa base de usuários se une de forma organizada e forma uma “**comunidade**”⁶⁷.

Comunidades geralmente fornecem um meio de comunicação pela qual os usuários podem conversar, debater, perguntar, responder, sugerir ou criticar sobre assuntos que dizem respeito ao software adotado por ela. Esse é o caso do App Inventor, uma comunidade de usuários se formou e, como a plataforma tinha documentação escassa, logo usuários começaram a ter dúvidas.

Se a dúvida de um usuário não é respondida pelos tutoriais disponibilizados, ele pode recorrer à ajuda de outros usuários por meio de um fórum online. Um fórum online é uma modalidade de sites da internet em que as pessoas podem se cadastrar, definir o tema do fórum, categorizar assuntos, criar tópicos sob o domínio de algum assunto e responder a esses **tópicos com postagens**⁶⁸.

Ao entrar na página inicial do Fórum Oficial do App Inventor (<https://groups.google.com/forum/#!forum/mitappinventortest>), o usuário se deparará com uma página de um grupo do Google. A

67 Grupo de usuários de uma determinada plataforma, sistema, solução, software, aplicativo, programa, jogo etc. que se une para ações e discussões relacionadas ao produto a que está vinculada.

68 Como exemplo, se um usuário encontrar um bug ou falha em um software, ele pode acessar o fórum da comunidade, localizar o subfórum (assunto) relacionado ao desenvolvimento (ou solução de bugs) e criar um tópico sobre o bug encontrado, assim, ele descreveria a falha com uma postagem e outros usuários responderiam a ele com outras postagens.

maioria das categorias pode ser vista logo na entrada da página, deixando mais rápida a seleção do assunto. Como o site é de domínio do Google, o usuário pode entrar no fórum usando sua conta, assim como é feito no App Inventor por natureza. A interface do fórum é representada na figura 11.

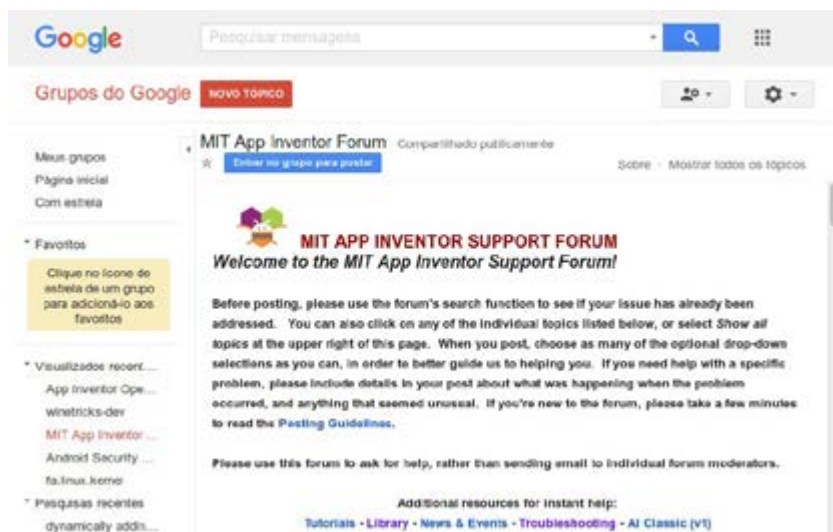


Figura 11: Fórum oficial da plataforma App Inventor.

(Fonte: App Inventor e Google)

1.4.3. Site Oficial Android

Plataforma-alvo do App Inventor, o projeto Android possui um site oficial onde podem ser encontradas diversas informações. O site, por si só, se subdivide em sessões chamadas de “subdomínios”⁶⁹. O subdomínio principal “www” (<https://www.android.com/>) contém

⁶⁹ Um site é um domínio na internet, um nome (como “google.com”) associado a um endereço de rede, também conhecido como IP. Adicionalmente, dentro desse domínio, pode haver subdomínios que são seções de um mesmo domínio (exemplos podem ser “mail.google.com”, “drive.google.com” e “myaccount.google.com”).

informações mais gerais, como, por exemplo, os tipos de tecnologia em que o Sistema Operacional Android é capaz de operar, aplicativos famosos que estão disponíveis para instalação, sugestões de dispositivos que já vêm com o sistema instalado e notícias acerca do tema.

No subdomínio “developer” (<https://developer.android.com/>), são apresentadas informações referentes ao desenvolvimento de aplicativos, assim como informações referentes às características da plataforma. Além disso, o desenvolvedor pode encontrar textos a respeito das diretrizes de qualidade do Google para aplicativos feitos para Android, Interfaces de Programação, serviços nativos do SO, dados sobre a mais recente versão lançada do sistema, Android Studio (o ambiente oficial para desenvolvimento de aplicativos Android), documentação oficial da plataforma, e notícias gerais (tanto dos desenvolvedores quando de mídias externas)

Por fim, no subdomínio “source” (<https://source.android.com/>), são encontradas informações referentes ao **Projeto Android de Código Aberto**⁷⁰, com os arquivos de código fonte, discussões de bugs/erros, dados de compilação, detalhes sobre o kernel do SO e instruções de como compilar e embutir este kernel em dispositivos.

Em adição aos pontos citados, o site possui um link para o blog oficial, onde o usuário pode encontrar informações acerca dos últimos acontecimentos envolvendo o Android, as atualizações disponibilizadas para a plataforma, notícias de novos produtos, e notícias envolvendo o meio empresarial em que o Android se encontra.

1.5. Apresentação da Interface

Agora que já apresentamos vários aspectos teóricos, históricos e técnicos acerca de dispositivos, plataformas, sistemas operacionais, formas de desenvolvimento, programação, empresas etc., chegou a hora de apresentar a plataforma-alvo deste livro. Por ser destinado a um público não muito preocupado com questões técnicas, o App Inventor

70 Ou AOSP, da sigla em inglês para “Android Open Source Project”.

apresenta uma interface de usuário bastante amigável, simplificada e com uma curva de aprendizado suave.

Tal estratégia em muito se alinha com a proposta de Aprendizagem Significativa de Ausubel (2003) pois facilita a interação entre o que o aluno já sabe e o que ele deseja aprender, principalmente pelo viés lúdico e interativo da ferramenta.

1.5.1. Sessão de Projetos

Assim que o usuário adentrar a plataforma sob a autenticação de sua conta Google, ele será recepcionado com uma página que possui alguns botões e uma área de lista, conforme as seguintes listagens ilustradas, em seguida, pela figura 12:

- Iniciar Novo Projeto (Cria um projeto zerado para o usuário);
- Apagar Projeto (Remove o projeto selecionado da lista e o apaga);
- Publicar na galeria (Expõe o aplicativo, geralmente pronto na galeria interna do App Inventor).

Na lista são apresentados todos os projetos do usuário em desenvolvimento. Em formato de tabela, informações do projeto são apresentadas junto à opção de entrar no modo de edição.

- Caixa de seleção (Para aplicar as ações de antes a um ou mais projetos);
- Nome (Nome/Título de um projeto em específico criado pelo desenvolvedor);
- Data de criação (Data e hora em que o projeto foi criado);
- Data de última modificação (Data de quando o projeto foi modificado e salvo pela última vez);
- Estado de publicação (Informa se o projeto foi publicado ou não).

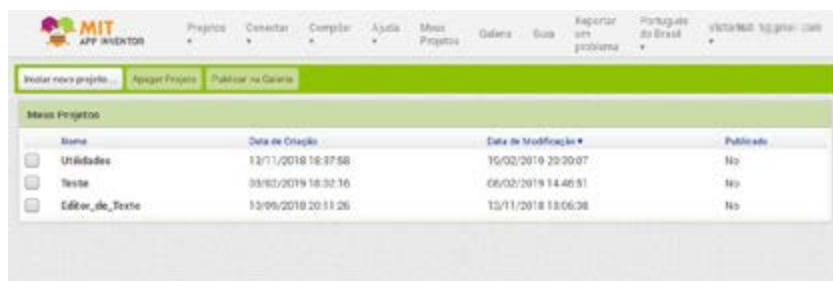


Figura 12: Sessão de projetos do App Inventor.

(Fonte: App Inventor)

Para começar, vamos criar um projeto de teste. Clique em Iniciar Novo Projeto. Uma pequena janela se abrirá com dois botões (Cancelar para desistir de criar e OK para confirmar criação) e uma caixa de texto onde o usuário deve informar o nome do projeto. Dê o nome que desejar e clique em OK.

Seguindo o comportamento padrão o projeto foi adicionado na lista e é aberto para edição. Agora o usuário se depara com outra sessão da plataforma, com mais opções e funcionalidades. Esta é a Sessão de Design.

1.5.2. Sessão de Design

A Sessão de Design é onde as coisas ficam visualmente divertidas, aqui é onde o desenvolvedor define a aparência do aplicativo, os elementos que vão a compor e quais serão suas configurações. A sessão é dividida em quatro partes, cada uma com uma finalidade específica. A representação desta sessão é mostrada na figura 13.



Figura 13: Partes da sessão de design do App Inventor.

(Fonte: App Inventor)

A parte 1 é a de coletânea de componentes ou paleta de componentes, onde estão reunidos todos os **elementos operacionais**⁷¹ que o aplicativo pode ter, de acordo com o acervo da plataforma App Inventor. Os componentes são organizados em forma de lista, subdivididos em categorias segundo suas áreas de atuação (ou funcionamento) e possuem breves descrições sobre suas capacidades e objetivos como dicas de uso. Esses componentes possuem propriedades funcionais específicas de cada um com as quais o desenvolvedor pode interagir para criar o comportamento desejado.

Na parte 2 temos o visualizador, uma tela de smartphone Android simulada para que o desenvolvedor possa incluir e selecionar (para edição) os componentes desejados na tela do aplicativo. Essa tela **não é um emulador**⁷² (programa que age de forma a imitar o processamento de um sistema), ela serve apenas para posicionamento e visualização.

71 O App Inventor agrupa as principais funcionalidades existentes (como botões, deslizadores, sensores, campos de texto etc.) e que podem ser usadas no ambiente Android para que o desenvolvedor possa selecioná-las facilmente.

72 Embora possa parecer que, ao clicar em um elemento no visualizador vá mostrar certa interatividade, isso não acontece, o único papel do visualizador é posicionar e esquematizar componentes na tela (ou inclui-los para uso no aplicativo). Para que o desenvolvedor possa testar seu aplicativo,

Já a parte 3 possui o quadro de componentes adicionados, todos os componentes escolhidos e inseridos no visualizador estarão visíveis em forma de lista (com hierarquia em **formato árvore**⁷³) nessa parte da sessão. Aqui, o desenvolvedor pode rearranjar a hierarquia, renomear elementos para que tenham uma **identificação única**⁷⁴, ou excluí-los quando não forem mais desejados.

Logo abaixo deste quadro, há um pequeno aditivo para que o desenvolvedor seja capaz de enviar mídias (como imagens e sons) que **serão usadas por componentes do aplicativo**⁷⁵.

E, por fim, na parte 4 está o painel de **propriedades**⁷⁶, onde os elementos selecionados (tanto no visualizador, quanto no quadro de componentes) podem ter suas propriedades ajustadas para se posicionarem e ocuparem melhor a tela. Configurações de texto, cores e afins também podem ser encontradas nesta parte da sessão. A quantidade de propriedades editáveis varia de componente para componente.

Mas limpar o carro não garante que ele ande bem, é preciso operar uma série de mecanismos para dar a partida, acelerar, trocar a marcha, frear, manobrar, estacionar etc. A essência de um aplicativo não é sua parte visual, mas sim sua mecânica, a forma como ele funciona e realiza o serviço a que se propõe. Veremos, a seguir, informações a respeito do editor de blocos e como ele fornece ferramentas tanto para interferir e personalizar o comportamento dos componentes quanto para agilizar seu processo de criação e montagem dos próprios blocos.

é preciso compilá-lo, baixá-lo em seu dispositivo Android e instalá-lo. Usar um emulador de Android é uma alternativa. Um emulador é um programa que tenta simular o hardware e o software de um sistema a fim de executá-lo dentro de um outro sistema.

73 Uma hierarquia em formato de árvore é uma forma de organizar elementos onde um elemento pode conter outros dependendo dos critérios usados para o agrupamento. Um exemplo são as pastas de um sistema de arquivos, uma pasta pode conter arquivos e mais pastas assim como cada uma destas pastas também podem conter pastas e arquivos.

74 Componentes com nomes iguais não são permitidos, seus nomes devem ser únicos porque sistemas computacionais geralmente não trabalham com dados que podem causar ambiguidade e, se esta é admitida, o sistema fica propenso a erros.

75 Um componente que serve para mostrar uma imagem na tela precisa que esta imagem tenha sido enviada para a plataforma antes de poder usá-la efetivamente.

76 Propriedades, nesse contexto, também podem ser chamadas de configurações.

1.5.3. Sessão de Blocos e Mecânica

Ao abrir a sessão de blocos, o desenvolvedor se depara com uma área em branco, alguns ícones e uma lista em formato árvore. Bem, vamos por partes, focaremos primeiro na lista. Ela possui três categorias: Internos, Tela (Screen1), e Qualquer Componente.

Os elementos dispostos na lista são agrupados nessas categorias e cada um deles possui blocos referentes ao seu funcionamento. Note que na área em branco onde os blocos podem ser editados, há uma série de pontos. Esses pontos servem para alinhar os blocos de forma que eles não fiquem muito desorganizados conforme o projeto cresce e muito mais blocos sejam adicionados. Segue sua representação na figura 14.



Figura 14: Sessão de blocos e mecânica do App Inventor, os blocos são identificados por cores e categorias.

(Fonte: App Inventor)

Na categoria Internos, estão blocos referentes às instruções e especificações primitivas da linguagem de blocos. Elas são subdivididas em ainda outras categorias que são:

- Controle (interferências no fluxo de processamento);
- Lógica (operações lógicas e comparações);
- Matemática (operações e funções matemáticas);
- Texto (manipuladores de sequências de caracteres);

- Listas (operações com listas de dados);
- Cores (especificações de cores pré-definidas ou compostas por cores primárias);
- Variáveis (controle de variáveis em memória);
- Procedimentos (onde o desenvolvedor pode definir suas próprias rotinas de instruções).

Na categoria Tela, o desenvolvedor pode ver os componentes que foram incluídos na tela da Sessão de Design e selecionar os blocos referentes a eles. Pode-se notar que cada um tem funções e blocos específicos para si, assim como modificadores de informações internas. Pode-se, ainda, notar que alguns componentes possuem blocos especiais que são executados quando o usuário interage com eles na tela. Esses blocos são chamados de eventos e funcionam como o “puxar do gatilho” para iniciar o processamento de uma certa funcionalidade.

A seguir, na categoria Qualquer Componente, o desenvolvedor pode alterar configurações de todos os componentes de um mesmo tipo, isso pode se provar útil em situações onde um mesmo elemento interativo é repetido diversas vezes (como uma lista de reprodução em um aplicativo de música, em que cada música é um botão em uma lista).

Ao lado da lista de blocos, há uma grande área em branco, essa é a área de blocos e sequenciação. É nesta área que ficam todos os blocos, onde o desenvolvedor pode colocá-los, combiná-los e adaptá-los. Ao clicar com o botão direito do mouse, conforme apresentado na Figura 15, um menu de contexto é mostrado com várias opções, como, por exemplo:

- Desfazer (a última ação realizada);
- Refazer (a última ação desfeita);
- Baixar blocos como imagens (como se fosse uma foto);
- Minimizar e maximizar blocos (para focar em outros mais importantes e diminuir a poluição visual da área de blocos);
- Deletar todos os blocos;
- Organizar blocos (horizontalmente, verticalmente ou por categoria);
- E operações de mochila (explicadas adiante).



Figura 15: Menu de contexto geral da sessão de blocos mostrando várias opções (Fonte: App Inventor)

Perceba que é por esse menu de contexto que podemos intercam-
biar o uso ou não da grade (pontilhado) de fundo e se os blocos
adicionados se alinharão a ela. Além disso, note que boa parte da plata-
forma App Inventor está traduzida para o português brasileiro, mas
alguns aspectos, opções, menus etc. ainda estão em inglês. Isso acon-
tece porque o trabalho de tradução ainda está ocorrendo.

No lado direito da área de blocos estão os **controles de zoom**⁷⁷,
com elas, é possível aumentar/diminuir o zoom ou centralizar a visão
do código. Adicionalmente, pode-se mudar a **área de foco**⁷⁸ por clicar e
arrastar⁷⁹ na direção desejada qualquer parte em branco da área de
blocos.

Quando se clica com o botão direito do mouse em cima de algum
bloco, conforme Figura 16, o menu de contexto é alterado. Agora, nele

⁷⁷ Zoom é um termo usado para descrever a aproximação ou afastamento da visão de um deter-
minado objeto. Quanto maior o zoom, maior o objeto e vice-versa.

⁷⁸ Uma área de foco é a área da tela que mostra o que o usuário pode ver no momento, se houver
algo fora dela, provavelmente ela pode ser ajustada para mostrar os elementos externos ao usuário.

⁷⁹ Se refere à ação de manter o botão do mouse pressionado, como um clique prolongado, o
clique é finalizado quando o cursor do mouse alcança a posição necessária.

podem ser encontradas as opções: duplicar, maximizar/minimizar, adicionar comentário, ativar/desativar, adicionar à mochila, deletar o bloco e todos os seus relacionados internos, opções de ajuda e executar (só funciona se o App Inventor estiver conectado ao dispositivo Android que testará o aplicativo).

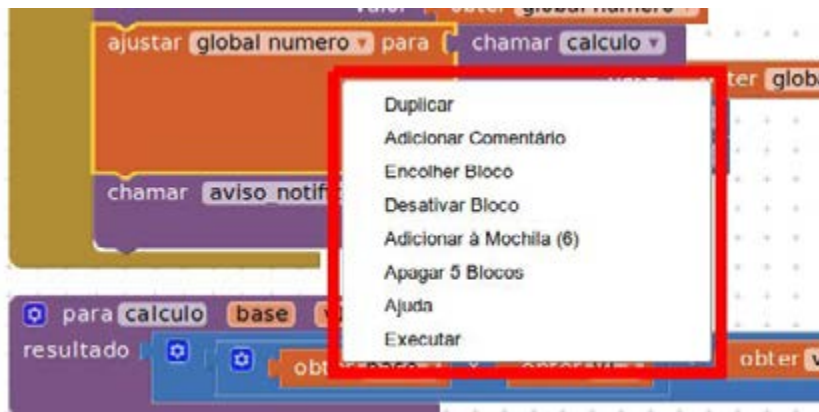


Figura 16: Menu de contexto dedicado a blocos.

(Fonte: App Inventor)

Ainda nessa sessão, pode ser percebido um ícone de mochila. Às vezes, usar a ferramenta de copiar e colar pode não ser algo muito ágil, assim como duplicar blocos, para isso existe a mochila. Se trata de uma área de transferência gerenciável, é possível adicionar blocos (ou estruturas de blocos já prontas) que podem ser acessadas facilmente com a mochila, sem entrar em menus de contexto. Agora, para remover um item da mochila, é preciso clicar com o botão direito do mouse e escolher a opção de remover.

Por fim, além de poder excluir elementos usando os menus de contexto, o desenvolvedor pode simplesmente clicar e arrastar blocos para o ícone de lixeira. Quando apenas um bloco é excluído, não há pedido de confirmação de operação, coisa que ocorre quando uma estrutura de blocos é excluída.

1.5.4. Barra Superior

Além da sessão de design e da de blocos, o App Inventor possui uma barra superior com diversos botões e opções que complementam as funcionalidades da plataforma. Ela é dividida em duas partes: as opções relacionadas ao projeto em edição no momento e acessos a outras partes da plataforma.

- A respeito das opções relacionadas ao projeto, o primeiro menu (Projetos) fornece as seguintes opções:
- Meus projetos (onde é possível ver novamente a lista de projetos ativos do desenvolvedor);
- Iniciar novo projeto (onde o desenvolvedor pode criar um projeto do zero);
- Opções de importação (fazer transferência de arquivo do computador, ou **repositório**⁸⁰, para a plataforma e reabrir o projeto);
- Apagar projeto (apagar todos os dados, mídias e blocos do projeto na plataforma);
- Salvar projeto (opção para salvar o projeto manualmente);
- Salvar projeto como (salva o projeto atual como um projeto independente e separado);
- Salvar ponto de **controle**⁸¹ ou ponto de checagem (mesma funcionamento que o “Salvar projeto como”, porém com um nome gerado automaticamente);
- Opções de exportação (salvar um ou todos os projetos no computador, baixar projeto localmente);
- Opções de conjunto de chaves (operações referentes a **criptografia**⁸² e protocolos de segurança).

80 Um repositório é uma forma de organização de recursos de software onde podem ser encontrados seus códigos e executáveis binários. São mais populares em sistemas operacionais baseados em Linux.

81 Usado principalmente quando se quer apenas marcar um ponto em que o aplicativo é estável e o desenvolvedor passa para uma próxima versão, que é instável. Um aplicativo ou software instável quer dizer que ele está propenso a falhas e erros fatais. Um checkpoint, ponto de controle ou ponto de checagem é como marcar um ponto seguro na linha do tempo do desenvolvimento.

82 Criptografia é uma área do conhecimento que se dedica a estudar como informações podem ser embaralhadas e revertidas ao estado original. Isso é feito de forma sistemática e deve permitir

Em seguida, há um menu Conectar para que o desenvolvedor conecte seu dispositivo Android à plataforma para testar seu aplicativo agilmente. Essa conexão pode ser feita das seguintes formas:

- Por rede **Wi-fi**⁸³ (em conjunto com os aplicativos oficiais App Inventor para esse fim);
- Emulador (um programa para computador que imita o funcionamento do sistema Android);

USB⁸⁴ (repassar ao dispositivo o aplicativo por meio do cabo USB);

Em adição, há opções para reiniciar as conexões (caso haja algum problema). Esses menus e opções podem ser verificados na figura 17.

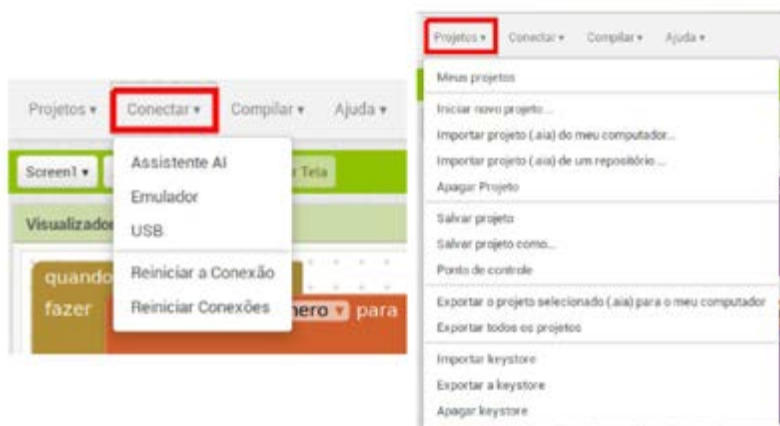


Figura 17: Menus “Conectar” e “Projetos” da barra superior.

(Fonte: App Inventor)

Depois, há o menu de Compilar. Este menu dá opções para que o desenvolvedor concretize seu projeto, compilando os blocos em códigos

que, ao computar a informação tendo uma chave como base, tal informação voltará ao seu estado original.

83 Wi-Fi ou “Wireless Fidelity” é um tipo de transmissão de dados via redes de computadores onde as informações trafegam pelo ar em ondas de rádio.

84 Universal Serial Bus (USB) é um padrão de cabeamento para conexão de dispositivos como smartphones, impressoras, computadores, caixas de som, pen drives etc.

e os códigos em um pacote de instalação que, posteriormente, pode ser executado num aparelho ou emulador Android. São duas opções:

- Compilar aplicativo e fornecer código **QR**⁸⁵ para download;
- Compilar aplicativo e realizar download diretamente para o computador.

Nota: durante o desenvolvimento deste livro, a plataforma oferecia quatro opções, duas delas eram as citadas, e as outras duas eram feitas especificamente para aplicativos visando a plataforma Android 8. Isso se deve ao fato de que pelo avanço da tecnologia e questões de segurança, a empresa Google decidiu que todos os aplicativos admitidos na Google Play devem ser compilados para serem compatíveis com a geração mais nova do sistema Android. Embora isso garanta que os aparelhos mais novos tenham uma gama maior de aplicativos disponíveis, isso **quebra a compatibilidade**⁸⁶ de tais aplicativos com versões Android mais antigas. Portanto, versões do sistema lançadas antes da 4.0 (Ice Cream Sandwich) estão propensas a não receber mais aplicativos pela Google Play. Novos aplicativos têm de seguir esta regra a partir de primeiro de Agosto de 2018 e a partir de primeiro de Novembro, todos os novos aplicativos, e atualizações dos já publicados, devem ser compatíveis com a geração mais nova. As opções de publicar aplicativo para Google Play seguem essa conformidade compilando aplicativos no **Kit de Desenvolvimento de Software 26**⁸⁷, enquanto que as opções de compilação para arquivo mantém o uso do Kit 7, garantindo a compatibilidade com versões Android mais antigas. A

85 Código QR é um código de barras bidimensional que pode ser interpretado e lido pela maioria dos smartphones atuais. Podem guardar números, textos, dados binários e links da internet.

86 Em software, quando dizemos que há uma quebra de compatibilidade, significa dizer que um software não é mais capaz de rodar em um ambiente de software (plataforma) porque, para ser executado, depende de elementos que não existem neste ambiente. Há também quebra de compatibilidade entre versões de um mesmo software, enquanto uma versão mais nova de um programa pode editar um arquivo no formato, por exemplo, ABCD, uma versão anterior pode editar apenas formatos ABC (sendo que ABCD é a evolução do ABC).

87 A plataforma Android disponibiliza oficialmente para seus desenvolvedores um Kit de Desenvolvimento de Software com todas as funcionalidades que um aplicativo pode tirar proveito. Essas funcionalidades incluem tanto ferramentas e componentes incorporáveis aos aplicativos, quanto funcionalidades pertencentes ao SO.

plataforma App Inventor declarou que vai suportar essas versões enquanto puder. Informações sobre o fim do suporte Google para versões antigas do Android, como as versões 2.1 e anteriores, podem ser encontradas no blog de desenvolvedores Android (<https://android-developers.blogspot.com/2017/06/ending-support-for-android-market-on.html>). Informações sobre as opções de compilação extras podem ser encontradas na nota oficial da plataforma (<http://appinv.us/xnot1>).

Após o menu de Compilação, há o menu de Ajuda, também apresentado na figura 18, em que são encontradas opções que fornecem informações ao usuário e desenvolvedor sobre aspectos da plataforma, artigos de ajuda, uma parte da documentação etc.

- Sobre: informações sobre a versão da plataforma, data de última compilação (da plataforma em si), versão do aplicativo de teste rápido de aplicativos, link para as notas de lançamento da versão e documentação legal acerca dos termos de uso e privacidade.
- Biblioteca: informações sobre primeiros passos com o App Inventor, informações para pessoas que pretendem ensinar com ou sobre a plataforma, artigos de suporte e resolução de problemas, tópicos da documentação oficial, assim como outros tópicos.
- Extensões: funcionalidades não oficiais desenvolvidas por terceiros que podem ser incorporadas aos aplicativos em desenvolvimento.
- Tutoriais: explicações intuitivas sobre algumas das funcionalidades oficiais disponíveis para incorporação aos aplicativos.
- Solução de problemas: uma coletânea de problemas comuns resolvidos com os quais as pessoas podem se deparar ao usar a plataforma.
- Fóruns: link para o fórum oficial da comunidade.
- Reportar problema: se o problema encontrado por uma pessoa não for reconhecido pela comunidade ou pelos desenvolvedores oficiais da plataforma, o usuário pode reportá-lo aqui.

- Informação do Assistente: aplicativo de conexão ágil para execução de aplicativos feitos com App Inventor. São disponibilizados um link para download do aplicativo e qual é a versão do aplicativo.
- Mostrar Tela de Início: exibe o quadro de informações inicial da plataforma com o código da versão em execução, uma informação rápida a respeito do lançamento e um link para as notas de lançamento na íntegra.



Figura 18: Menus “Compilar” e “Ajuda”.

(Fonte: App Inventor)

Na outra parte da barra superior, existem botões de acesso rápido com funcionalidade repetida de opções da outra parte. Os outros botões são:

- Galeria: alguns aplicativos em destaque na galeria interna da plataforma.
- Seletor de Idioma: um menu de opções para traduzir o App Inventor em vários idiomas.
- Menu de perfil e conta:
 - Perfil: editar um perfil de usuário simples e interno da plataforma;
 - Sair: realiza saída da plataforma de forma segura, desconectando a conta Google.

1.6. Atividades de fixação!

1. Se você sente que não conseguiu absorver muito do conteúdo que foi trabalhado, veja as próximas perguntas para tentar fixar o conteúdo!
2. O que são computadores?
3. Qual é o código mais básico e primitivo entendida por computadores e qual é o seu alfabeto?
4. Quais componentes todos os computadores compartilham?
5. O que é um Sistema Operacional (SO)?
6. Um usuário que trabalha num escritório usa programas para lidar com mensagens eletrônicas, planilhas, cartas, documentos etc., usuários domésticos usam programas para ver vídeos, acessar a internet, ouvir músicas etc., um programa usa outro programa para conseguir a hora atual. Qual é o conceito de um programa?
7. O que impede que um aplicativo possa diretamente danificar o hardware?
8. O que são linguagens de programação?
9. Qual é a função de um programador?
10. Quanto mais alta a abstração de uma linguagem de programação, maior sua dificuldade de ser entendida por humanos?
11. Qual é a diferença entre uma linguagem compilada e outra interpretada?
12. Como é possível que um site de informações localizado no outro lado do mundo seja acessado a partir de nosso país?
13. Smartphones são a evolução de celulares convencionais em que sentido?
14. Qual foi a iniciativa voltada para programação na plataforma móvel lançada pelo MIT com o intuito de facilitar a programação e qual é a sua filosofia guia?
15. Qual é a melhor forma de lidar com um grande problema que denota certos padrões?
16. Por meio de que técnica um software pode ser organizado a cada atualização?

17. Como Scratch, App Inventor, Android e Google Play estão relacionados?
18. O que pode ser feito na Sessão de Projetos do App Inventor?
19. O que pode ser feito na Sessão de Design do App Inventor?
20. O que pode ser feito na Sessão de Blocos do App Inventor?

1.7. Encerramento do capítulo

Agora que a plataforma App Inventor foi introduzida, alguns conceitos básicos foram trabalhados, assim como informações históricas, é hora de “pôr a mão na massa”. Nos próximos capítulos, serão apresentadas ideias de aplicativos que podem usar várias funcionalidades disponibilizadas pela plataforma. Os princípios do uso das ferramentas e sua adaptação para atenderem a princípios de acessibilidade também serão abordados.

Os conceitos de programação aqui apresentados serão muito úteis, caso deseje se aprofundar nos estudos de programação, usando ferramentas mais robustas e que necessitam de mais conhecimentos.

Capítulo 2: Desenvolvimento do Primeiro Aplicativo

Finalmente chegamos no primeiro passo ao mundo dos desenvolvedores, nesse capítulo você aprenderá a usar e manipular componentes disponibilizados pela plataforma em estudo. Se prepare para uma avalanche de informação pois a coletânea de componentes não é pequena, muito menos cada uma das funções de seus integrantes.

Mas calma! Não é agora que começaremos a mexer em blocos e/ou suas especificações, conectar instruções e tudo mais. Antes disso acontecer, precisamos ter um embasamento teórico para entender o que temos que fazer e como temos que fazer! Portanto, siga com a leitura, apresentaremos os princípios que você precisa saber antes de entrar em cena com tudo o que tem!

2.1. Apresentação do Aplicativo a ser Desenvolvido

2.1.1. Introdução

Antes de botar a mão na massa e programar efetivamente, precisamos nos concentrar num detalhe inicial, antes de fazermos, precisamos saber o que fazer. Nos primeiros tópicos deste capítulo vamos apenas definir o plano-piloto que nos guiará durante o desenvolvimento. É essencial ter um objetivo claro em mente para não se perder. Então, como esse será o primeiro aplicativo, precisa ser algo relativamente fácil, simples, porém, rico em conteúdo.

Essas características se devem ao fato de, ao iniciar o estudo em alguma área, não é aconselhado começar com tópicos avançados ou demasiadamente complicados, isso é um grande fator desmotivante e pode lhe impedir de prosseguir nos estudos. Ficar frustrado com os resultados iniciais muitas vezes frustra como um todo a perspectiva pessoal do aprendizado e incentiva o sentimento de desistência.

Portanto, faça as coisas em seu próprio passo, lembre-se que o objetivo da programação é tornar as coisas fáceis, inclusive tornar fácil o próprio processo de tornar as coisas fáceis (por isso temos adventos como o App Inventor). Se o seu aprendizado for um projeto pessoal, leve em conta o seguinte pensamento: em vez de se perguntar “Como posso me manter motivado até o fim do projeto?”, pergunte-se “Como fazer um projeto de forma a acabar antes de me desmotivar?”.

Nessa mesma lógica, se tiver outras ideias de aplicativos além da que será apresentada, sinta-se à vontade para colocá-las em prática, mas inicialmente, procure resolver problemas pequenos.

2.1.2. Público-Alvo

Se você deseja fazer um aplicativo que afete um determinado grupo de usuários marcado por uma característica em comum, esse passa a ser seu **público-alvo**⁸⁸. Nem todas as pessoas usarão seu aplicativo e nem tudo o que elas precisam fazer será realizado por ele, por isso é bom ter um público-alvo bem definido e focar o desenvolvimento do aplicativo e suas funcionalidades com base nas necessidades dessas pessoas.

O público-alvo não precisa ser necessariamente um grupo de usuários, se o desenvolvedor programar um aplicativo para auxiliá-lo em uma coisa intrínseca de sua rotina (e nada além disso), seu público-alvo é apenas uma pessoa, ele mesmo.

⁸⁸ Grupo de pessoas dotado de certas características em comum que podem ser afetadas por um produto, serviço, mensagem ou ação.

Focar em um certo grupo pode ajudar a popularizar um aplicativo. De certo modo, quando os usuários do público-alvo pretendido ficam satisfeitos com o resultado do aplicativo, o projeto se mostra um sucesso e pode ser compartilhado mais facilmente de pessoa para pessoa, aumentando a base de usuários. Isso abre uma oportunidade para aumentar a área de atuação ou enfoque do aplicativo, fazendo que seja capaz de agradar e servir a mais usuários por adicionar funcionalidades.

2.1.3. Ideia

Sendo assim, chegamos à nossa proposta inicial, nosso primeiro aplicativo será um pequeno centro de utilidades para treinar o uso da plataforma. As utilidades presentes no aplicativo serão:

1. Uma calculadora de Índice de Massa Corporal (IMC)
2. Uma calculadora de Idade
3. Um seletor de Cores
4. Um simples bloco de Notas
5. Um exemplo de Autenticador
6. Um relógio com funcionalidade de cronômetro e contagem regressiva

Poderíamos ter mais funcionalidades em nosso aplicativo, porém tentaremos mantê-las simples e sob controle. Agora que já traçamos os objetivos é necessário fazer um levantamento dos componentes que serão necessários para a construção de nosso aplicativo. Tanto no caso do IMC quanto da idade, precisaremos da entrada de dados numéricos, então sabemos que precisaremos obter dados em formato texto, precisaremos da conversão deste texto para números e também da realização de cálculos matemáticos propriamente ditos, porém o último envolve datas.

No caso do autenticador, precisaremos lidar com senhas, no bloco de notas precisaremos lidar com texto permanente e no relógio, cronô-

metro e contagem regressiva, precisaremos de um componente capaz de gerar eventos disparados pelo relógio do sistema.

2.2. Explicações das Funcionalidades

Agora que já temos uma ideia superficial do propósito do projeto e um simples panorama do que precisa ser feito, faremos considerações mais profundas a respeito das funcionalidades apresentadas e no que elas consistem.

2.2.1. Detalhamento e Subdivisão do Problema

Nosso aplicativo de utilidades gerais foi subdividido em 6 problemas menores e independentes uns dos outros. Nos pormenores, cada um deles terá um objetivo específico. Cada um dos objetivos é especificado na tabela 1.

Tabela 1: Subdivisões do aplicativo de Utilidades Gerais.

Subdivisão	Objetivo
Calculadora de IMC	Calcular o IMC de uma pessoa com base na entrada de dois dados numéricos (altura e peso).
Calculadora de Idade	Calcular quantos anos uma pessoa viveu até o presente momento com base em sua data de nascimento.
Seletor de Cores	Formar uma cor que será mostrada na tela de acordo com o nível de intensidade das três cores primárias aditivas (vermelho, verde e azul).
Bloco de Notas	Um bloco de notas simples com capacidade de armazenamento limitada, porém que permanece no aplicativo mesmo que ele seja fechado e reaberto.
Autenticador	Registrar momentaneamente uma senha criada pelo usuário e usá-la para comparar com uma nova senha fornecida posteriormente para checar a igualdade entre elas.
Relógio	Uma parte do aplicativo que se subdivide em três: relógio com hora e data atuais registrados no SO, contagem regressiva baseada num dado número inteiro de segundos e um cronômetro controlado por botões.

(Fonte: Do Autor)

2.2.2. Pensamento Computacional

Agora que todas as funções do programa foram devidamente explicadas, é preciso ressaltar um ponto: o que estamos fazendo se chama de “pensamento computacional”. Dividir um problema em partes menores (dividir para conquistar), reconhecer que uma mesma funcionalidade pode ser usada para causar efeitos diferentes em situações diferentes (buscar padrões no problema) e decidir quais aspectos são ou não relevantes à resolução do problema são características desse tipo de pensamento.

Durante o desenvolvimento de aplicativos, busca-se desenvolver a capacidade de identificar padrões. Procure resolver problemas diferentes usando as mesmas estruturas de código adaptadas ao caso em que estão sendo aplicadas. **Reutilizar**⁸⁹ um mesmo trecho de código mantém o projeto todo mais enxuto e gerenciável.

Mas, além de se preocupar com a qualidade e legibilidade do código, é preciso se preocupar com seu fluxo de processamento. É preciso mapeá-lo cuidadosamente para que o máximo de falhas e inconsistências sejam evitadas.

Quando alguém lhe pede para ir até a padaria e comprar um sanduíche, quais são os passos que você deve seguir? Sair, comprar e voltar? Isso é ótimo para explicar praticamente como satisfazer esse pedido. Porém, imagine que você estivesse programando um robô para fazer tal coisa, sair, comprar e voltar passa a ser o seu objetivo. O robô não sabe nada sobre a vida, o universo e tudo mais. Sair, sair de onde? Comprar, comprar o que? Voltar, voltar pra onde e como? Agora podemos perceber como as coisas são um pouco mais complicadas do que apenas três passos.

Veja como esse pedido pode ser mais bem especificado:

1. Adquirir dinheiro para a compra com o requisitante;

⁸⁹ Reutilizar pedaços de código é algo muito valorizado hoje em dia. Os desenvolvedores sentem uma queda muito grande de produtividade quando repetem códigos iguais ou parecidos que poderiam perfeitamente ser isolados como uma solução de desenvolvimento geral e usados em todos os lugares onde essa solução fosse necessária.

1. 1. Identificar o requisitante;
1. 2. Obter o dinheiro;
1. 3. Guardar o dinheiro consigo em um compartimento seguro;
1. 4. Perguntar a quantidade máxima do gasto e memorizar;
2. Sair da localidade atual para a rua mais próxima;
 2. 1. Checar se já não está na rua;
 2. 1.1 Pular para o passo 2.4;
 2. 2. Se não estiver;
 2. 2.1 Calcular a distância entre a localidade atual e as possíveis ruas;
 2. 2.2 Analisar obstáculos;
 2. 2.3 Traçar caminho até a rua mais próxima;
 2. 3. Percorrer caminho planejado até a rua pretendida;
 2. 4. Traçar e percorrer caminho entre as ruas e a localização objetivo;
 2. 4.1 Se a localização objetivo for a padaria, pular para passo 3;
 2. 4.2 Se a localização objetivo for o local de origem, pular para o passo 6;
3. Entrar na padaria;
 3. 1. Analisar as prateleiras e estufas;
 3. 2. Identificar o sanduíche e checar seu preço;
 3. 2.2 1 Se não houver sanduíche;
 3. 2.2 2 Procurar por um produto semelhante de preço parecido com base no valor que se tem e no limite de gasto do valor.
 3. 2.2 3 Se não houver nenhum produto que possa ser comprado com o dinheiro que se tem, pular para o passo 5;
 3. 3. Memorizar o preço do produto;
4. Se dirigir até a fila da padaria;
 4. 1. Identificar filas e computar quantas pessoas há em cada uma;
 4. 2. Entrar na fila com menos pessoas;
 4. 3. Realizar o pagamento do sanduíche;
 4. 4. Checar o valor do troco;
 4. 4.1 Se o valor do troco não condizer com a diferença do valor pago com o valor recebido, realizar pedido de recontagem;

4. 5. Sair da fila;
5. Sair da padaria e pular para o passo 2;
6. Identificar o requisitante;
 6. 1. Se houver produto, entregar o produto ao requisitante;
 6. 1.1 Se o produto não for o sanduíche pedido, justificar a escolha com o fato da falta de sanduíches;
 6. 2. Se não houver produto, justificar sobre os altos preços;
 6. 3. Devolver o valor que se tem;

O que acabamos de fazer pode ser chamado de passo a passo ou **algoritmo**⁹⁰. Porém ainda pode-se perceber que, para um robô, esse passo a passo não quer dizer nada, e mesmo que significasse algo, seria extremamente vago e ambíguo. Qual é o conceito de rua? Como fazer cálculo de proximidade? Máquinas não trabalham com nossas palavras assim como abordamos no primeiro capítulo.

A respeito deste último ponto, não denotar ambiguidade é algo fundamental no pensamento computacional. Se essa for permitida, erros de processamento de informação podem ser imensamente ampliados e causar erros fatais com muito mais frequência. Tanto sistemas de usuário final doméstico como usuários finais de sistemas de tempo real não querem que seus computadores, programas e sistemas operacionais quebrem com facilidade ou gerem informações que possam ser facilmente mal-entendidas.

Analisar riscos, possibilidades, erros, formas de processamento, desempenho, caminhos e processar todas essas coisas faz parte do pensamento computacional. É uma habilidade que pode ser usada em várias áreas. Lidar com sistemas (de várias naturezas), substituir partes deles sem levar todo resto ao chão ou causar falhas e, além disso, fazer com que seja mais fácil realizar modificações (implicando em uma melhor **manutenibilidade**⁹¹) e gerar novas implementações de uma

90 Sequência finita de passos a serem executados que são escritos claramente de forma estruturada e não denotam ambiguidade.

91 Capacidade de um software de ser submetido a manutenções, consertos, atualizações e migrações etc.

funcionalidade (implicando em uma maior **escalabilidade**⁹²) são preocupações das técnicas de **encapsulamento**⁹³ que também integram o pensamento computacional.

Desenvolver pensamento computacional pode até parecer complicado mas é um exercício constante e não precisa começar de forma difícil. Estamos trabalhando num ambiente isolado onde erros não causam prejuízos graves. Sinta-se livre para analisar quais são as melhores soluções para manter seus aplicativos funcionais, eficazes, estáveis e que, ao mesmo tempo, sejam fáceis de realizar manutenção e tenham código legível.

2.3. Apresentação da Ferramenta

No App Inventor, percebemos que há diversas ferramentas organizadas na tela, um grande número delas, por sinal. Vamos usar este tópico para nos familiarizar com essa interface. No primeiro lugar da lista de componentes disponíveis está a categoria de Interface de Usuário, foquemos nela para estudo. A categoria é ilustrada pela figura 19.

92 Capacidade do sistema de se manter operacional, coerente e efetivo em suas tarefas mesmo que passe por crescimento gradual, tanto em funcionalidades quanto em quantidade e intensidade de uso.

93 Capacidade de tornar um elemento de sistema plenamente isolado, podendo se usado como uma solução generalizada.

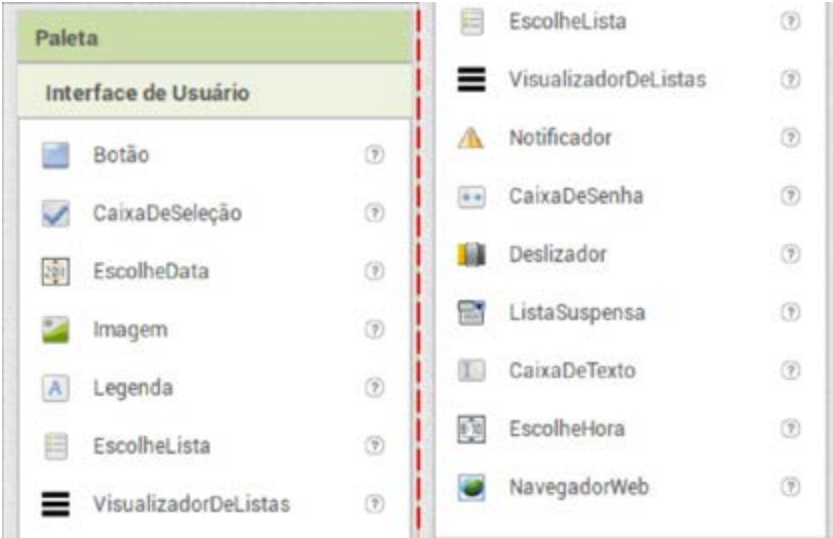


Figura 19: Categoria de componentes para Interface de Usuário.
(Fonte: App Inventor)

2.3.1. Categorias de Componentes

A primeira categoria “Interface de usuário” oferece uma série de componentes comumente usados como ferramentas básicas de interação com o usuário para entrada e saída de dados. Vamos listá-los aqui e explicá-los por meio da tabela 2.

Tabela 2: Componentes da categoria Interface de Usuário.

Componente	Função
Botão	Mostra na tela um botão clicável.
Caixa de Checagem	Mostra uma pequena caixa clicável que possui os estados ligado/desligado.
Escolha de Data	Botão que abre uma pequena janela onde o usuário pode escolher uma data.
Imagem	Recipiente gráfico onde uma imagem pode ser mostrada na tela.
Legenda/Texto	Recipiente gráfico que mostra um texto na tela.

Lista de Escolhas	Um botão que, quando pressionado, mostra uma lista de opções selecionáveis (em uma tela temporária criada apenas para isso).
Visualizador de Lista	Um recipiente que permite agrupar várias opções (como se fossem botões) num mesmo componente.
Notificador	Oferece diversas formas de notificar um usuário, gerar entradas no registro do SO Android ou obter dados do usuário.
Caixa de Senha	Um campo de texto digitável onde o usuário pode digitar sua senha de forma que os caracteres não sejam mostrados na tela.
Deslizador	Uma barra que pode ser deslizada pelo clique/toque para registrar um valor numa escala gradual.
Lista Suspensa	Um botão que, quando clicado, mostra uma pequena lista abaixo de si com opções disponíveis para escolha.
Caixa de Texto	Um campo de texto digitável.
Escolha de Hora	Um botão que mostra uma pequena janela em que o usuário pode escolher um horário.
Navegador Web	Um componente que serve como um mininavegador web para ações pontuais e simples envolvendo sites na internet.

(Fonte: App Inventor)

Conforme vimos por nessa listagem, existem vários componentes básicos de interação com o usuário nessa categoria. Podemos até, de fato, fazer uma calculadora usando apenas os itens citados. Entretanto, é importante ressaltar que, ao lado de cada item da categoria, se faz presente um pequeno ícone de interrogação, esse pequeno botão prove ajuda mais aprofundada sobre a função do referido componente ao desenvolvedor. Ao fim da nota de ajuda, pode ser encontrado um link para uma página com mais informações sobre o componente (todas em inglês até o momento).

Depois da categoria de interface de usuário, temos a categoria de “Organização”, aqui são disponibilizados componentes que atuam na organização e posicionamento de elementos na tela. Eles podem estar arranjados verticalmente, horizontalmente ou em formato de tabela e os componentes desta categoria têm a capacidade de comportar elementos dentro de si, mesmo que ultrapassem o limite da tela, utili-

zando-se da técnica de **rolagem**⁹⁴. Podemos ver a categoria “Organização” na figura 20.

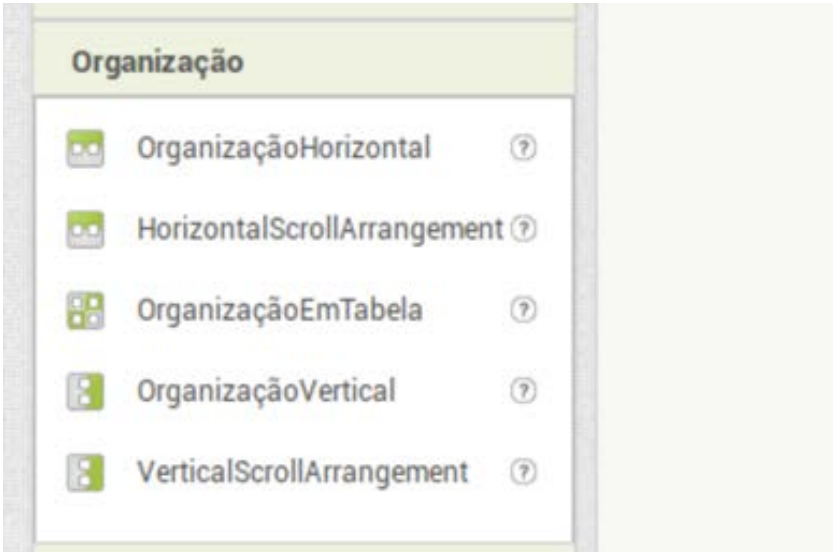


Figura 20: Categoria de componentes de organização.
(Fonte: App Inventor)

Quanto à especificação dos componentes, esta é descrita na tabela 3.

Tabela 3: Componentes da categoria Organização.

Componente	Função
Organização Horizontal	Dispõe graficamente outros componentes dentro de si ordenados da esquerda para a direita.
Organização Horizontal com Rolagem	Mesma função de Organização Horizontal porém com a capacidade de ser deslizável.

94 Quando um elemento é maior do que o tamanho da tela, um aplicativo pode usar a técnica de rolagem para mostrá-lo ao usuário, isso quer dizer que o programa mostrará barras e botões que controlam qual parte do conteúdo pode ser vista no momento. Navegadores de internet são um ótimo exemplo, ao usar um navegador e fazer uma pesquisa na internet cujo conteúdo é muito extenso, o navegador faz com que o usuário tenha que rolar a página para baixo ou para cima a fim de poder ver o resto do conteúdo.

Organização Vertical	Dispõe graficamente outros componentes dentro de si ordenados de cima para baixo.
Organização Vertical com Rolagem	Mesma função de Organização Vertical porém com a capacidade de ser deslizável.
Organização em Tabela	Organiza conteúdos em forma de tabela de acordo com um dado número de tabelas e linhas.

(Fonte: App Inventor)

As outras categorias pertencem a áreas de atuação e funcionamento um pouco mais avançadas, então discutiremos posteriormente apenas alguns elementos que precisaremos para completar a funcionalidade do aplicativo.

2.3.2. Avaliação de Componentes

Como discutido, temos seis problemas centrais a serem resolvidos, precisamos disponibilizar um aplicativo completo. Para tanto, separamos certas capacidades do aplicativo e discursamos sobre elas individualmente. Em seguida, vimos listas de componentes oferecidos pela plataforma de desenvolvimento. Agora precisamos avaliar esses componentes e fazer uma pergunta: como esses componentes resolverão nossos subproblemas?

De início, pensemos sobre a calculadora de IMC: nós precisamos que o usuário informe dois valores numéricos que, depois de informados, são calculados e entregam o índice de massa corporal do usuário. A entrada de dados é feita pelo teclado virtual do celular, isso significa que precisamos de um componente capaz de lidar com informação do tipo texto, um componente que o usuário possa tocar, ative o teclado virtual e receba a informação. Esse componente é exatamente a **“Caixa de Texto”**⁹⁵ (também conhecido como campo de texto).

Sendo assim, apresentaríamos duas caixas de texto descrevendo o tipo de dado que precisamos. Entretanto, para descrever o tipo de dado

95 Além dos botões, a caixa de texto é um dos componentes mais usados para entrada de dados em sistemas informatizados. Elas estão presentes em formulários, no discador de números do telefone, nos questionários com respostas discursivas, nas páginas de autenticação de usuários das redes sociais, nos aplicativos de mensagens instantâneas etc.

que precisamos, temos que usar outro componente cuja função é mostrar texto na tela, esse componente é o “Legenda”, também conhecido como “Label”.

Assim que os dados fossem fornecidos, o usuário teria um que pressionar um botão na tela que dispararia o evento que coletaria os dados dos caixas de texto, avaliaria a validade dos dados e, se tudo estiver certo, finalmente calcular o IMC. O resultado seria exibido em uma legenda logo abaixo do botão.

Ligando cada um dos componentes, trocando informações entre eles usando os blocos e avaliações dos resultados obtidos, somos capazes de construir uma estrutura funcional que se interconecta e trabalha junta para solucionar os subproblemas propostos.

Essa união de componentes resulta no processamento de dados que o usuário dá entrada no aplicativo. Sendo assim, a informação, obtida pelos **periféricos**⁹⁶ de entrada e pelo sistema operacional, trafega pelos componentes internos do aplicativo e é manipulada por eles. Resultando numa saída de dados, seja por tela, alto-falantes ou memória do smartphone.

Mas é importante ressaltar que há componentes que não são visíveis graficamente no aplicativo, isso quer dizer que, se forem incluídos, eles não aparecerão na tela. Eles não precisam fazer interação com usuário, estão ali para serem usados por outros componentes. Esse é o caso do componente de Arquivo.

O componente de Arquivo serve para conectar o aplicativo ao sistema de arquivos do smartphone e do SO. Um componente o chama, requisitando um certo arquivo, e ele vai atrás desse arquivo trazendo seu resultado, se conseguir encontrá-lo. Em nenhum momento o usuário precisa interagir com ele, essa necessidade é suprida pelos componentes com representação gráfica, onde o usuário pode dizer qual arquivo deseja acessar.

96 Equipamentos não-intrínsecos do conjunto computacional processador, placa-mãe e memória principal. Exemplos dele são câmeras, mouses, teclados, impressoras, digitalizadores etc.

Por falar nessa questão de não precisar interagir, é importante notar também que todos os componentes seguem o princípio da caixa-preta, todos eles são funcionalidades já prontas que só estão esperando para ser usadas. Nem o desenvolvedor e nem o usuário precisam saber como eles fazem o que fazem, apenas precisam saber o que fazem, quando e como usá-los.

Da mesma forma o usuário não precisa saber como seu aplicativo faz algo, mas sim o que ele faz e como usá-lo. O que interessa não é o meio, e sim o fim. Se um mesmo trabalho puder ser feito de forma igual ou semelhante por duas ferramentas diferentes, a escolha da ferramenta vira algo quase que supérfluo.

É claro que não usaremos apenas legendas, campos de texto, e botões, mas, por agora, isso é suficiente para começar. Os demais componentes serão apresentados e explicados no decorrer de nosso desenvolvimento.

2.4. Desenvolvimento passo a passo

Enfim, chegamos ao desenvolvimento. Mas antes falemos um pouco mais de teoria! Calma, estava apenas brincando! Agora sim vamos escolher nossos componentes e posicioná-los na tela, editando suas ações e configurações visuais. Veremos, enfim, nossos planos ganhando vida pela primeira vez. Olhos atentos à coletânea de componentes.

2.4.1. Introdução às Telas

Primeiro de tudo, precisamos saber que o aplicativo é dividido em Telas (também chamadas de “Atividade” ou “Activity”), isso se deve ao fato de os componentes posicionados em uma tela não poderem ser criados e/ou excluídos em **tempo de execução**⁹⁷ (pelo menos, não pelo

⁹⁷ Quando o aplicativo consegue realizar por si só operações que normalmente são indicadas manualmente pelo programador, dizemos que essa é uma operação dinâmica e realizada em tempo de execução. Um ótimo exemplo é o uso de memória do dispositivo. Quando um aplicativo não pede

App Inventor). Telas se fazem necessárias quando um certo conjunto de componentes já está na tela e precisamos mostrar outro conjunto (por exemplo, um botão que abre um menu que ocupa a tela toda, o botão fica em uma tela, e quando clicado, chama outra tela que possui apenas o menu). A figura 21 ilustra o uso de telas.

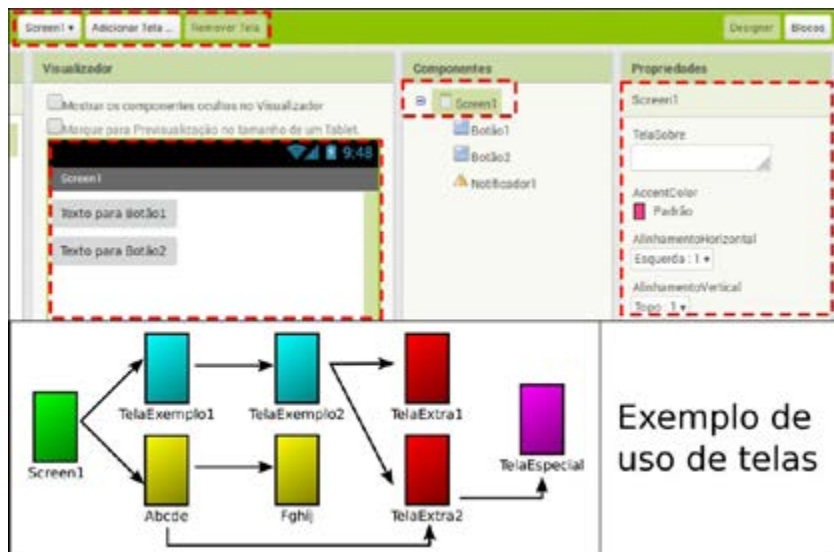


Figura 21: Menu de manipulação de telas, suas propriedades, forma de componente e conexão entre telas.

(Fonte: App Inventor)

Conforme mostrado na figura 21, podemos perceber que a tela “Screen1” possui dois botões como componentes integrantes e um notificador. Suponhamos que esses botões devam ser a única coisa nessa tela, se quiséssemos mostrar o conteúdo decorrente do ato de pressioná-los, precisaríamos recorrer a outra tela ou usar componentes de organização que ficassem alternando sua visibilidade.

mais memória RAM ao SO, dizemos que ele funciona com memória reservada de forma estática, mas quando ele realiza esse pedido, essa operação passa a não ser indicada pelo programador, e sim pela lógica indicada por ele. Essa dependência apenas da lógica para executar a operação a torna dinâmica.

Excluindo a ideia dos organizadores, faz-se saber que uma tela é capaz de abrir outra conforme apontado no fim da figura 21. Mas é importante ressaltar que cada tela tem seu próprio conjunto de blocos e componentes, são coisas separadas. Por exemplo, os componentes da tela “TelaExemplo1” não podem ser vistos na tela “TelaExtra2” e, como consequência, os blocos referentes aos componentes dentro delas não podem ser vistos ou acessados a partir da outra. Essa regra vale para todas as telas.

A tela vazia apresentada pelo App Inventor (já mencionada “Screen1”) também é um componente que serve para comportar todos os outros, sendo, portanto, obrigatória (um aplicativo precisa de, pelo menos, uma tela para existir, **mesmo que ela não seja visível ao usuário**⁹⁸). Com isso, ao tentar selecioná-la, suas características podem ser editadas na janela ou lista de propriedades. As propriedades da tela incluem diversas configurações, dentre elas, as que são apresentadas na tabela 4.

Tabela 4: Propriedades da tela principal do aplicativo.

Propriedade	Efeito
Sobre a tela	Um breve comentário feito pelo desenvolvedor (geralmente uma descrição das funções da tela e seu objetivo). Essa descrição pode ser vista pelo usuário se ele usar um botão de menu (próprio do SO) enquanto estiver visualizando a tela.
AccentColor	Em versões mais novas do Android, alguns campos de texto possuem animações com linhas destacadas e textos deslizantes. Quando alguns componentes estão em foco, essa cor é usada para destacá-los.
Alinhamento vertical e horizontal	Essas opções definem como o conteúdo da tela deve ser alinhado. Horizontalmente, há como centralizar e alinhar à esquerda ou à direita. Verticalmente, há como posicionar os componentes no topo, meio ou fundo da tela.

98 Alguns aplicativos não são pensados para interagir com o usuário, mas sim com outros aplicativos, por causa disso eles não precisam de uma tela ou representação gráfica, assim como os componentes invisíveis do App Inventor.

Nome do App	Aqui o desenvolvedor define qual é o nome do App, aquele que aparecerá com o ícone ao instalar o App. Se for deixado em branco, assume o nome do projeto que consta no App Inventor.
Cor de fundo	Define qual será a cor de fundo da tela.
Imagem de Fundo	Em vez de usar uma cor como fundo de tela, permite usar uma imagem. Se a imagem tiver transparência, ainda poderá ser notada a cor de fundo escolhida para a tela. Note, entretanto, que dependendo da orientação da tela, a imagem pode ficar muito espichada.
Animações de Abertura e Fechamento	Permitem escolher de que forma o fechamento e a abertura de uma tela ocorrerão: se será por leve desvanecimento, por zoom, por deslizamento etc.
Orientação da Tela	Permite configurar de que forma a tela se comportará de acordo com o movimento do smartphone. É possível travar a tela em posição vertical ou horizontal (com o smartphone em pé ou deitado) ou deixar que se adapte.
Rolável	Uma caixa de checagem. Se a caixa estiver selecionada, permitirá que a tela tenha conteúdos de forma que ultrapassem o tamanho físico vertical da tela ao mesmo tempo que o usuário ainda pode acessá-los. Senão, os componentes só poderão ser vistos a área inicial coberta pela tela física do dispositivo.
Mostrar Barra de Status	Também conhecida como Barra de Notificações, define se ela deve ser mostrada ou escondida pelo aplicativo (geralmente usada para exibição em tela cheia).
Título Visível	Define se o título da tela em questão deve ser mostrado no topo logo abaixo da barra de notificações.
Versão de Código	Um campo especial para especificar qual é a versão do aplicativo (um número natural que deve ser somado em 1 a cada versão).
Nome da Versão	É possível usar um esquema de versionamento próprio (ou mesmo um já existente, como o “Semantic Versioning” (https://semver.org/)) para nomear versões de um aplicativo, porém, pode-se usar versionamento com nomes ou códigos com letras também.

(Fonte: App Inventor)

Algumas opções só estão disponíveis na Tela 1 e sua configuração repercute em todas as outras, elas são: Nome do App, Versão de Código e Nome da Versão. Adicionalmente, opções relacionadas a cores geralmente têm uma lista de cores pré-definidas, mas junto à lista, há uma

opção para formar uma cor personalizada, como: AccentColor e Cor de Fundo.

2.4.2. Manejo de Componentes

Os componentes que precisamos podem ser incluídos no aplicativo por clicar e arrastá-los da lista à esquerda para visualizador no centro da visão do App Inventor. Com isso, o componente é embutido e fica pronto para uso. Ao selecioná-lo no **visualizador**⁹⁹, a janela de propriedades muda para que ele possa ser editado.

Se for difícil selecionar um componente escolhido através do visualizador (ou se o componente não tiver representação gráfica), podemos recorrer à janela de componentes adicionados, lá estão todos os componentes que estão em uso no momento.

2.4.3. Primeiros Passos

Agora que sabemos esses conceitos básicos, vamos pensar em como faremos a tela principal de nosso aplicativo. Primeiro precisamos saber qual utilidade o usuário deseja acessar, temos seis escolhas disponíveis além de um botão para sair do aplicativo. Portanto, podemos criar sete botões e nomeá-los de acordo com o que representam. Essa é a tela principal e fica responsável por essas ações.

O botão “Cálculo de IMC” chamaria uma outra tela que conteria os campos de texto e o botão para o cálculo de IMC. “Calculadora de Idade” levaria à sua respectiva tela com as devidas funcionalidades e assim por diante. No fim da lista de botões, haveria um botão chamado “Sair” responsável por fechar o aplicativo.

Então vamos começar, selecione o componente “Botão” na coleção de componentes e arraste-o até o visualizador, você verá que um botão agora está visível no nosso aplicativo, mas ele precisa ser modificado para melhor se adaptar à tela. Pode-se perceber que quando o

⁹⁹ Componentes sem representação gráfica são mostrados ligeiramente abaixo da representação da tela do smartphone, eles também podem ser selecionados por ali.

botão é selecionado, aparecem outras opções na parte de Propriedades. Lá podem ser encontradas duas opções de Largura e Altura. Ao clicar em cada uma delas, podem ser vistas quatro escolhas, conforme a tabela 5.

Tabela 5: Opções comuns às propriedades de Largura e Altura.

Opção	Efeito
Automático	O tamanho do componente é decidido automaticamente de acordo com o espaço ocupado por seu conteúdo.
Preencher principal	Ocupa todo o espaço que o componente de maior nível hierárquico imediato ocupa.
Pontos/Pixels	Define, por meio de um número, a quantidade de pixels que um componente ocupará na tela.
Porcentagem	Define por meio de uma porcentagem do espaço disponível qual será o tamanho de um componente na tela.

(Fonte: App Inventor)

Para ilustrar o processo de clicar e arrastar o componente de botão, posicioná-lo na tela, verificar sua adição na lista de componentes da tela, observar a mudança do quadro de Propriedades e alterar as configurações de Altura e Largura, verifique a figura 22.

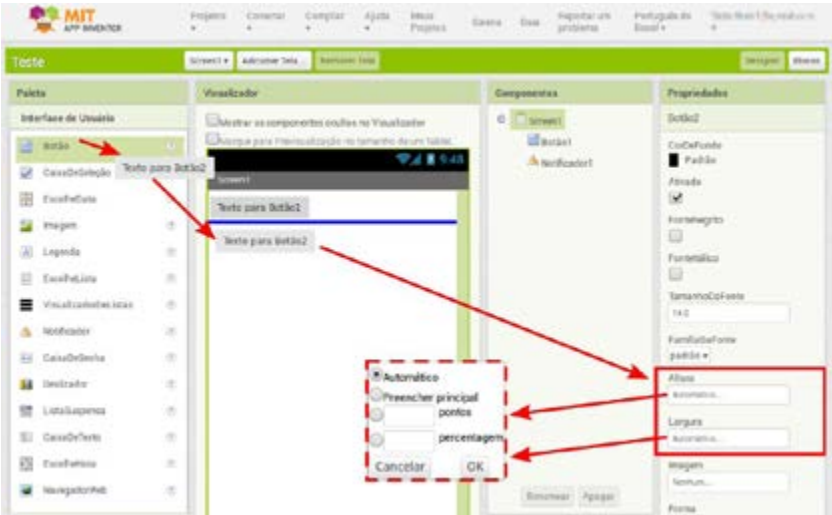


Figura 22: Controles de altura e largura de componentes com representação gráfica.
(Fonte: Autor e App Inventor)

Caso o termo tenha ficado obscuro, “pixel” é uma pequena unidade de representação gráfica em computadores. Um pixel é um pequeno quadrado na tela de um computador ou smartphone que assume uma cor. Uma foto digital é formada de pixels, até milhões ou bilhões deles (ou mais). Tente ampliar uma imagem no seu celular até os mínimos detalhes, se ele não suavizar a imagem, você será capaz de ver vários quadradinhos coloridos, esses são os pixels. Pixel Art é um gênero artístico atual que se utiliza deles (em formato ampliado), segue um exemplo na figura 23.

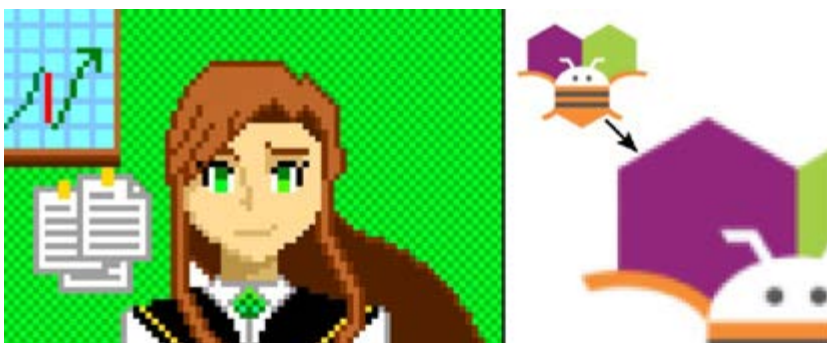


Figura 23: Imagens são compostas por pixels, pequenos quadrados que assumem uma cor.

(Fonte: Autor e App Inventor)

Depois de escolher o modo de dimensionamento do botão que lhe convier, você pode editar qual texto aparece nele. A configuração que permite isso está na janela de Propriedades sob o nome de “Texto”. Por enquanto não vamos editar as funcionalidades dos botões, então aqui explicamos outras de suas propriedades conforme os dados da tabela 6.

Tabela 6: Algumas propriedades do componente Botão.

Propriedade	Efeito
Cor de Fundo	Define a cor do botão.
Fonte Negrito	Define se o texto do botão é escrito em negrito.
Fonte Itálico	Define se o texto do botão é escrito em itálico.

Ativado	Define se o botão realiza alguma ação quando pressionado, se a caixa não estiver marcada, o botão não realiza ações ao ser pressionado.
Tamanho da Fonte	Define o tamanho em que o texto do botão é escrito.
Família da Fonte	Define a família da fonte em que o texto do botão é escrito. As opções são com serifa, sem serifa, mono espaçada e padrão. Fontes com serifa possuem detalhes em suas pontas, fontes sem serifa são mais simples e não possuem detalhes nas pontas. Já fonte mono espaçadas definem que todos os caracteres devem ocupar exatamente o mesmo espaço horizontalmente, sendo assim, um exemplo seria que a letra "i" ocuparia o mesmo espaço que a letra "m".
Imagem	Define se, em vez de uma cor, o botão deve ter uma imagem de fundo.
Forma	Define a forma do botão em relação ao seu contorno.
Alinhamento do Texto	Define a posição do texto no espaço delimitado pelo botão.
Cor do Texto	Define a cor em que o texto do botão é escrito.
Visível	Define se o botão deve ser mostrado na tela.

(Fonte: App Inventor)

Algumas configurações de componentes não estão restritas apenas a eles, elas podem ser vistas também em outros. Por exemplo, as configurações Fonte Negrito e Fonte Itálico estão presentes tanto no componente de Campo de Texto como no componente de Botão e no de Legenda. Isso se deve ao fato de os três lidarem com textos de alguma forma.

Tendo isso esclarecido, é preciso repetir o processo mais seis vezes para que o usuário possa acessar todas as telas e também sair do aplicativo. Antes de passarmos para a edição de blocos de interatividade, é preciso esclarecer que é possível renomear componentes incluídos no aplicativo, essa opção está no fim da janela de componentes adicionados. Dar nomes significativos podem ajudar você a codificar mais rápido por saber com qual componente específico está lidando. Segue o botão de renomear na figura 24 (é preciso estar com o componente pretendido selecionado para renomeá-lo).

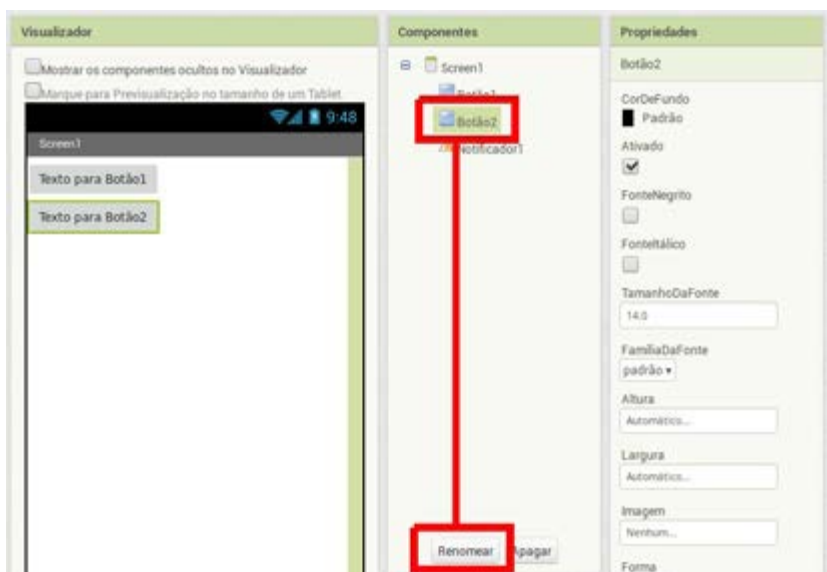


Figura 24: Componentes incluídos podem ser renomeados. .
(Fonte: App Inventor)

2.4.4. Tratando da Lógica e da Mecânica

Agora vamos ter uma demonstração de como podemos tornar esses botões interativos. Troque da sessão de design para a sessão de blocos. Nela você pode observar que há a lista de blocos que já discutimos. O que nos interessa por agora são os componentes que estão no fim dessa lista. Ao incluir os botões no aplicativo por meio do modo de design, é notável que eles também ficam visíveis no fim da lista. Isso acontece porque esses componentes também se tornam categorias de blocos.

2.4.4.1. Estudando os Componentes como Categorias

Essas categorias novas possuem blocos referentes a cada funcionalidade dos componentes específicos que representam. Nelas podem ser achados blocos de instruções, blocos de eventos e **blocos de atributos**¹⁰⁰.

¹⁰⁰ Um bloco de atributo garante acesso a uma característica do componente. Por exemplo, se

Geralmente, blocos de eventos são representados na cor amarela, eles são os blocos que destravam o processamento dos blocos incluídos dentro de si mesmos. Os blocos roxos significam funções (ou “Procedimentos”), que abordaremos mais adiante. Os blocos verde-claro são blocos de acesso a atributos e os blocos verde-escuro são blocos de mudança de atributos.

Sendo assim, ao selecionar o botão de sair, por exemplo, na lista de categorias, podemos ver um conjunto de blocos relacionados a ele (o botão de sair) e, principalmente, aqueles que são disparados por eventos. Precisamos indicar que, quando esse botão for pressionado, o aplicativo seja fechado.

2.4.4.2. Organizando Blocos

Escolha o bloco de evento correspondente ao clique. Alguns botões possuem **caixas de combinação**¹⁰¹, nelas você pode escolher o **componente**¹⁰² afetado, no nosso caso, é o botão de Saída. Escolhendo o botão a ser afetado pelo evento no bloco faz com que os blocos subsequentes sejam executados toda vez que o botão de saída for clicado ou pressionado. Agora, dentro do bloco de evento, precisamos de algo que faça o aplicativo fechar.

Para tal ação, na categoria de Controle, há um bloco chamado **“Fechar aplicação”**¹⁰³. Esse bloco pode ser anexado a um bloco anterior, mas repare que não possui uma alça com a qual os outros blocos podem se ligar. Ele possui essa forma porque, quando o aplicativo é fechado, a tela a que ele pertence é deixada para trás, seu processamento é encerrado e o aplicativo é fechado.

colocarmos o texto de um botão como “Botão 1” na sessão de design, podemos mudar seu texto ou usá-lo no código por meio dos blocos de atributo.

101 Um botão especial que, quando clicado, mostra uma série de opções suspensas (como uma lista). Após clicar na opção, ela geralmente é mantida e se torna efetiva.

102 Ou atributos nos blocos referentes aos mesmos.

103 O bloco diz “aplicação” porque é uma tradução ao pé da letra. “App” é uma abreviação de “Application” no inglês e, quando traduzido para o português significa aplicação. Portanto aplicação e aplicativo são a mesma coisa em nosso contexto de estudos.

Blocos que possuem um encaixe à esquerda são especificações de outros blocos, um exemplo disso são os blocos de operações matemáticas, a maioria deles não age por si só, mas sim como um complemento de outros blocos. Agora, blocos com uma entrada para encaixe à direita são blocos que necessitam de complementos para funcionar, um exemplo é o bloco de controle “Se/Então”, ele possui uma alça interior delimitando um **escopo**¹⁰⁴ e uma entrada à direita, essa entrada é onde blocos de especificação podem ser anexados. Abordaremos esse bloco mais tarde. A figura 25 ilustra os blocos discutidos.

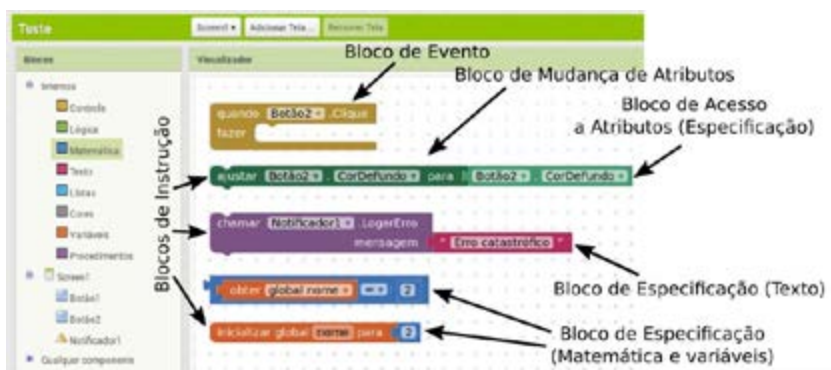


Figura 25: Distinção entre blocos de diferentes tipos, cores e formas.
(Fonte: App Inventor)

Agora que concluímos essa parte do código, ele deve ficar parecido com o resultado da figura 26.



Figura 26: Evento de clique em botão com um bloco de fechar aplicativo.
(Fonte: App Inventor)

¹⁰⁴ Escopo quer dizer uma área de atuação. No contexto atual, significa um conjunto de blocos que é envolvido por um bloco maior.

Isso garante que quando o usuário pressionar o botão Saída, um evento será disparado por esse botão chamado “Clique”, nosso código interceptará esse sinal e executará os blocos que estabelecemos dentro do bloco de evento, sendo assim, o bloco “Fechar aplicação” será executado e o aplicativo será fechado.

Os outros botões passarão por um funcionamento parecido, então, vamos começar com o botão que nos levará à calculadora de IMC. Precisamos que, ao pressionar o botão, o usuário não feche o aplicativo, mas sim abra uma outra tela com a devida calculadora. Isso é feito usando o mesmo bloco de evento para clique que usamos para o botão de saída.

Então, faça o seguinte, na lista de componentes na sessão de blocos, clique no botão responsável pela **calculadora de IMC**¹⁰⁵ e escolha o seu bloco de evento para cliques. Pronto, temos nosso interceptador de evento para esse botão. Agora, dentro desse bloco precisamos de um bloco que abra uma tela diferente da que estamos, para isso, abra a categoria de blocos de controle.

Ao encontrar o bloco “Abrir outra tela” você perceberá que ele tem uma entrada para encaixe à direita. Isso significa que ele precisa de um complemento para saber qual tela abrir. Entretanto, só temos uma tela no momento. Para criar outra tela, precisamos olhar na barra onde estão os botões que fazem a troca entre as sessões de blocos e design, na outra extremidade podemos ver o título do projeto e ao lado três botões: o primeiro é uma caixa de combinação com as telas já criadas, o segundo é para criar uma tela (botão Adicionar Tela...) e o terceiro é para excluir a tela que estamos editando no momento.

Apenas como uma curiosidade, a tela padrão de todos os projetos se chama “**Screen1**”¹⁰⁶, ela não pode ser renomeada e nem apagada, ela

105 Note que se o bloco para evento de clique do botão Saída for escolhido novamente, o App Inventor poderá acusar um erro. Isso se dá porque esse botão já tem uma estrutura de código para lidar com esse evento e se o bloco de evento for repetido, o código se torna ambíguo. O processador do smartphone não terá nenhum critério para saber qual deles deve ser executado.

106 Literalmente “Tela 1”.

é a raiz de todo o aplicativo. As demais telas podem ser apagadas e renomeadas.

Então vá em frente e crie uma tela chamada IMC. Não precisamos editá-la agora, o App Inventor a abrirá automaticamente, então volte para a caixa de combinação e selecione a Screen1. Na sessão de blocos, adicione o bloco “Abrir outra tela” ao bloco de eventos do botão da calculadora de IMC. Agora entre na categoria Texto e pegue o bloco de especificação com uma caixa de texto entre duas aspas. Junte ele ao bloco de “Abrir outra tela” e escreva o nome da tela que abriremos exatamente igual a quando você escreveu ao criá-la. O resultado deve ser parecido com o da figura 27.



Figura 27: Bloco de evento associado a um bloco de abrir outra tela especificada por um bloco de texto com seu nome.

(Fonte: App Inventor)

Note que, embora os dois blocos tenham escrito “IMC”, o bloco de eventos se refere ao botão que leva à tela e o bloco de especificação diz qual tela deve ser aberta. Por conveniência, ambos têm nomes iguais.

Antes de prosseguir para a edição da tela IMC, vamos observar um pouco os blocos relacionados à tela em si, porque ela também é um componente muito importante. Na lista de categorias de blocos, você pode perceber que o componente da tela também está presente. Clicando sobre ele temos vários blocos, mas vamos destacar só alguns na tabela 7, os de eventos.

Tabela 7: Alguns blocos de evento do componente Tela e seus objetivos.

Bloco	Objetivo
Voltar Pressionado	Esse bloco intercepta um evento quando o botão de voltar físico ou virtual do smartphone é pressionado.

Inicializar	Esse bloco intercepta um evento toda vez que o aplicativo abre a tela em questão.
Orientação da Tela Mudou	Esse bloco intercepta um evento toda vez que o usuário gira o smartphone para paisagem ou retrato.
Outra Tela Foi Fechada	Esse bloco intercepta um evento toda vez que o usuário retorna à tela que abriu a tela que acabou de ser fechada.

(Fonte: App Inventor)

Apenas como curiosidade, “Paisagem” e “Retrato” são modos de descrever como a tela do smartphone está posicionada: paisagem também é conhecida como “tela deitada” de forma que ela seja maior horizontalmente e, retrato também é conhecida como “tela em pé” de forma que ela seja maior verticalmente.

2.4.5. Calculadora de IMC

Agora já garantimos o acesso a uma tela e o acesso à saída do aplicativo, foquemos em criar o código da calculadora de IMC por agora. Já sabemos como modificar a aparência de um botão e como interceptar um evento de clique. Isso por agora é suficiente. Agora introduziremos quatro novos componentes: campo de texto, legenda, organização vertical e organização horizontal.

Se por acaso não se lembrar de qual é a fórmula do IMC, segundo o portal Minha Vida (<https://www.minhavidade.com.br/alimentacao/tudo-sobre/32159-imc>), ela está representada na figura 28.

$$IMC = \frac{\text{Peso}}{\text{Altura}^2}$$

Figura 28: Fórmula para o cálculo de Índice de Massa Corporal (IMC).
(Fonte: Portal Minha Vida)

Mas é importante lembrar que computadores não trabalham com fórmulas assim. Se fôssemos converter essa fórmula para um

formato interpretável por computadores poderíamos ter o seguinte código: “ $IMC = (peso / potencia(altura, 2))$,”

O objetivo de nossa tela é aplicar essa fórmula nos dados fornecidos pelo usuário e dar-lhe o resultado. Há uma prévia na figura 29 de como nossa tela deve se parecer.



Figura 29: Aspecto visual e componentes da tela de IMC.

(Fonte: App Inventor)

Para início de conversa, vamos analisar quais são os elementos envolvidos nessa tela. De início temos uma legenda com o título “Cálculo de IMC”. Legendas são elementos que não servem para interagir diretamente com o usuário, por isso elas não possuem blocos de eventos, sua função é unicamente mostrar texto. Em relação às configurações dos botões, as legendas têm as seguintes novas configurações: “HTML Format” e “Tem Margens”.

Tem margens é uma configuração às bordas da representação gráfica de seu componente, não é possível verificar seu efeito no designer conforme descrito na documentação oficial (<http://ai2.appinventor.mit.edu/reference/components/userinterface.html#Label>).

Já a propriedade HTML Format se refere à capacidade de formatar o texto mostrado com base em uma linguagem de marcação usada em alta escala na internet, a linguagem HTML (esta opção é para quem deseja misturar várias formatações numa mesma legenda). O conteúdo

e a formatação se misturam na propriedade Texto, veja um exemplo: “Isto” é um texto formatado em <i>HTML</i>”

O resultado seria (“” e “”¹⁰⁷ indicam texto em negrito, “<i>” e “</i>” indicam texto em itálico): “**Isto** é um texto formatado em HTML”.

Prosseguindo, temos um componente de Organização Vertical e dentro dele, dois componentes de Organização Horizontal, cada um deles contendo um Campo de Texto com uma Legenda. Ambos servem para posicionar elementos na tela de acordo com um certo padrão. No vertical, os componentes são posicionados um embaixo do outro. No Horizontal, os componentes são posicionados uns ao lado dos outros. Ambos os componentes têm uma variante cada em que é possível fazer rolagem como foi citado.

Assim como o componente de legenda, esses componentes de organização não lidam com interação direta com o usuário e, por isso, não têm blocos de eventos. Verifique suas propriedades conforme apontado na tabela 8.

Tabela 8: Algumas propriedades dos componentes Organização Horizontal/ Vertical.

Propriedade	Efeito
Alinhamento horizontal	Define como o conteúdo deve ser posicionado horizontalmente. As escolhas são alinhamento à esquerda, alinhamento à direita ou alinhamento ao centro.
Alinhamento vertical	Define como o conteúdo deve ser posicionado verticalmente. As escolhas são alinhamento ao topo, alinhamento ao fundo (ou à base) e alinhamento ao centro.
Imagem	Define se uma imagem deve ser mostrada como fundo do componente.

(Fonte: App Inventor)

107 A maioria das etiquetas HTML, ou mais popularmente conhecidas por “tags”, requer uma tag de abertura e outra de fechamento, “” indica o início de um texto em negrito (B de “Bold” que em inglês significa negrito), “” indica o fim do

As demais propriedades já foram estudadas, então passemos adiante. O último componente que merece nosso destaque por enquanto é o campo de texto. Esse sim possui interação direta com o usuário e, portanto, possui blocos de eventos e blocos de procedimentos. Eles são indicados e detalhados na tabela 9.

Tabela 9: Eventos e procedimentos do componente Campo de Texto.

Evento	Objetivo
Recebeu Foco	Intercepta um evento toda vez que o usuário pressiona o campo de texto para digitar algo (ou transfere o foco por meio de um teclado físico).
Perdeu Foco	Intercepta um evento toda vez que o usuário foca ou pressiona qualquer coisa que faça a edição do campo de texto ser interrompida.
Procedimento	Efeito
Ocultar Teclado	Força o fechamento do teclado virtual do smartphone.
Pedir Foco	Habilitar a edição do campo de texto automaticamente, mas não ativa o teclado automaticamente.

(Fonte: App Inventor)

Agora que observamos de perto esses componentes é hora de programar e conectar todos os blocos pra completar nossas funcionalidades. Mas antes disso vejamos um pouco de lógica de programação.

2.4.5.1. Lógica de Programação

Recorda-se de quando estávamos programando o botão que levou à tela da calculadora de IMC? Recorda-se de como ele funcionava? O usuário pressionava o botão e ele lançava um evento, esse evento era interceptado por um bloco de eventos que captava justamente um clique e, a partir dali, executava todos os blocos dentro de si que estavam anexados a ele por meio de uma alça. A parte no interior do bloco de eventos será nosso foco agora.

2.4.5.1.1 Variáveis

Certamente, o leitor deve ter percebido que a palavra “variáveis” foi dita algumas vezes no decorrer deste livro, mas o que de fato é uma variável? De forma resumida, uma variável é um espaço limitado na memória principal do computador alienada a um número que o identifica. Esse espaço de memória é capaz de armazenar um certo dado.

Assim como o nome sugere, o valor que é armazenado nesse espaço de memória pode ser mudado com o passar do tempo. O valor pode ser acessado toda vez que o programador invoca o nome da variável no código, por exemplo: digamos que exista em nosso código a variável “entrada”, essa variável conterà um número que informaremos via teclado e depois será multiplicado por dois.

Assim que informarmos o número, a variável armazenará o valor em seu espaço de memória e esse valor poderá ser usado toda vez que chamarmos a variável “entrada”. Portanto, se codificarmos: *entrada = entrada * 2;*

... O valor da variável “entrada” seria multiplicado por dois e seu resultado seria armazenado no **mesmo espaço de memória**¹⁰⁸. Se informássemos o valor de “entrada” como sendo cinco, a linha de código acima seria correspondente a “*entrada = 5 * 2;*”. Note, porém, que não substituímos o “entrada” por cinco antes do sinal de igual porque, ali, ele não representa um valor, mas sim o espaço de memória que armazenará o resultado. Esse processo não é uma equação, se chama “atribuição”.

A atribuição não significa que ambos os termos devem ser iguais, mas sim que o primeiro termo deve armazenar o resultado do segundo. Em outras palavras, o resultado do segundo termo será “atribuído” ao primeiro.

¹⁰⁸ Perceba que, ao armazenar o valor no mesmo espaço de memória, sobrescreveremos o valor anterior. Ele será perdido para que o novo possa ser armazenado. Mas se esse não for o comportamento desejado, pode-se usar uma outra variável para armazenar o valor antigo, ou mesmo o resultado.

As variáveis estão disponíveis no App Inventor sob a categoria de mesmo nome na sessão de blocos. Ainda dentro do tópico de variáveis, existem duas modalidades delas, mas deixaremos esse tópico para mais tarde quando discutirmos a respeito de funções. Note na figura 30 que elas podem ser acessadas tanto através de sua categoria na lista de categorias de blocos quanto passando o cursor do mouse através de um bloco de definição.



Figura 30: Blocos de criação, acesso e manipulação de variáveis.

(Fonte: App Inventor)

2.4.5.1.2 Constantes

Ao contrário das variáveis, também existem os espaços de memória que guardam valores imutáveis, são as chamadas “constantes”. Seu valor só pode ser definido no tempo em que a programação está sendo realizada, qualquer tentativa de edição no valor de uma constante em tempo de execução é bloqueada e ignorada.

Constantes são utilizadas principalmente para armazenar dados que serão sempre os mesmos e não podem ser alterados de forma alguma a fim de não ter que digitá-los manualmente. Por exemplo: é muito mais difícil (e suscetível a erros) digitar 3.14159 do que designar esse valor a uma constante e nomeá-la como “PI”.

2.4.5.1.3 Sequencialidade de Instruções

Nos computadores, as instruções são executadas uma após a outra. Mesmo que pareça que tudo é feito ao mesmo tempo, isso é apenas uma ilusão criada pela rapidez do processamento do computador. No App Inventor podemos ver isso através das alças dos blocos. Um bloco é executado e, quando seu processamento termina, a alça que o liga ao próximo é acessada, ela alcança o próximo bloco esse pode ser executado na sequência. Na programação a ordem das instruções faz diferença.

Mas digamos que, em nossa tela de IMC, uma mensagem de erro será mostrada se o usuário tentar informar letras ou outros símbolos em vez de números. Para um computador, letras não podem ser interpretadas como números da mesma forma que as tratamos em equações, é preciso fazer códigos para que o computador as entenda assim. Do contrário, unir letras e números numa operação matemática direta pode causar erros de processamento e **comportamento indefinido**¹⁰⁹.

2.4.5.1.4 Analisando Condições

Com a finalidade de evitar esse tipo de problema, podemos avaliar os dados fornecidos e checar se consistem em números. Caso sejam números, prosseguimos nossas operações, caso não sejam, interrompemos a operação e exibimos a mensagem de erro. As instruções “SE” e “SENÃO”¹¹⁰ são usadas nesses casos.

109 Se um computador não é programado para interpretar um certo tipo de situação em todos os seus casos e exceções mas o programador faz com que o computador lide com tal situação, ele cai em comportamento indefinido, ou seja, ele tentará lidar com o processamento da forma como tem especificado em si mesmo, mesmo que essa não seja a forma correta. Um exemplo é quando um programador cria uma variável numérica sem dar um valor inicial a ela e a usa numa operação matemática. Alguns compiladores poderiam colocar o valor zero, nulo ou mesmo indefinido para tal variável. O número zero pode ser usado em operações matemáticas (com algumas exceções, o que causaria ainda mais erros), o valor indefinido não. Portanto, o programador quase nunca tem como saber quais efeitos uma instrução mal especificada repercutirá.

110 É importante notar que para usar o comando “Senão” é obrigatória a existência de um “Se” que o preceda imediatamente (não pode haver nenhuma instrução entre um Se e um Senão) embora o

A instrução SE faz uma avaliação, por exemplo: “Tal valor é um número?”, se sim, os comandos delimitados dentro da instrução SE são executados. Do contrário o fluxo de processamento é desviado para a instrução SENÃO, onde são executados os comandos que informariam o acontecimento do erro. Esses comandos “SE” e “SENÃO” estão presentes no App Inventor em forma de blocos sob a categoria de blocos de controle.

É importante lembrar que a instrução SE só pode aceitar comparações que resultem em valores lógicos: verdadeiro ou falso. O bloco de comparação na categoria de **Lógica**¹¹¹ é capaz de fazer isso, igualmente ao bloco de comparação na categoria **Matemática**¹¹², porém este último é feito para lidar com números, então possui mais opções.

2.4.5.1.5 Estendendo Comparações

Se mais de uma condição precisar ser testada no mesmo bloco SE, podemos nos utilizar das instruções E e OU. Elas servem para estender o poder do SE unindo pares de comparações. Esses blocos também geram um resultado lógico dependendo das condições agrupadas por eles.

O bloco E dá um resultado de valor verdadeiro **APENAS se as duas condições testadas por ele forem verdadeiras**¹¹³. O bloco OU dá o resultado verdadeiro se **PELO MENOS uma das condições for verdadeira**¹¹⁴. Veja isso representado na **tabela 10**¹¹⁵ segundo informações do portal InfoEscola (<https://www.infoescola.com/matematica/conectivos-logicos/>).

Se possa existir sem o Senão.

111 Este bloco só é capaz de checar a igualdade e a diferença entre valores lógicos.

112 Este bloco, por lidar com números, pode checar se um valor é igual, menor, maior, maior ou igual, menor ou igual, ou diferente. O resultado é verdadeiro ou falso.

113 Se uma das condições, ou mesmo as duas forem falsas, o bloco retorna o valor falso como resultado.

114 Se uma das condições, ou mesmo as duas forem verdadeiras, o resultado é verdadeiro, se ambas forem falsas, falso.

115 A fim de resolver operações lógicas, existem tabelas especiais como a apresentada para exemplificar os operadores E e OU, elas são comumente chamadas de “Tabelas Verdade”.

Tabela 10: Explicação prática dos operadores lógicos E e OU.

Condição 1	Comparador ou Operação	Condição 2	Resultado
Verdadeiro	E	Verdadeiro	Verdadeiro
Verdadeiro	E	Falso	Falso
Falso	E	Verdadeiro	Falso
Falso	E	Falso	Falso
Verdadeiro	OU	Verdadeiro	Verdadeiro
Verdadeiro	OU	Falso	Verdadeiro
Falso	OU	Verdadeiro	Verdadeiro
Falso	OU	Falso	Falso

(Fonte: InfoEscola)

2.4.5.1.6 Repetindo Códigos

Mas se for pedido que, por algum motivo, sejam mostradas na tela quinhentas vezes a mesma mensagem, você usaria 500 blocos? Certamente que essa parece uma tarefa exaustiva e pouco recompensadora. Para esse caso em que é preciso repetir instruções, temos instruções especiais na programação. Elas são chamadas de **“Estruturas de Repetição”**¹¹⁶.

Exemplifiquemos o parágrafo anterior dizendo que cada uma das mensagens deve ter um número de 1 a 500. Poderíamos usar uma variável (começando com o valor 1) para guardar esse número, **incrementá-lo**¹¹⁷ a cada repetição, usar seu valor diretamente como mensagem na tela e ao mesmo tempo usar seu valor na **condição de parada**¹¹⁸ da estrutura de repetição.

Nesse caso, a condição de parada seria “número menor ou igual a 500”, sendo assim, o **laço**¹¹⁹ continuaria enquanto nossa variável

116 Comandos de programação especiais cuja função é repetir o código contido dentro de si.
117 Somar uma unidade ao valor da variável. O contrário de incrementar é decrementar e significa subtrair uma unidade do valor da variável.
118 Condição que, enquanto é atendida (verdadeira), mantém a estrutura de código se repetindo. Para isso, as comparações lógicas usadas são as mesmas usadas na instrução SE.
119 Termo usado para designar uma repetição, também pode ser chamado pelo estrangeirismo “loop”.

“número” fosse menor ou igual a quinhentos. Quando o valor da variável fosse maior que 500, a estrutura de repetição seria interrompida e o programa continuaria seu fluxo normal.

Porém, note que o código que muda o valor da nossa variável deve estar dentro da estrutura de repetição. Se esse código não estivesse dentro do laço, o valor seria sempre um. Consequentemente, a estrutura de repetição continuaria sendo **executada infinitamente**¹²⁰ porque um sempre será menor que quinhentos.

2.4.5.1.7 Funções e Procedimentos

Por fim, chegamos no tópico das funções, também chamadas sub-rotinas, subprogramas ou procedimentos como acontece no caso do App Inventor. Funções são códigos normais como quaisquer outros. O objetivo principal das funções é **modularizar**¹²¹ um código geral.

Veja por meio do exemplo, na figura 31, que um código que pode ficar muito mais enxuto quando fazemos uso de funções.

120 Geralmente, manter um laço infinito num programa simples significa que ele ficará travado porque todo o fluxo de processamento está no laço em si. Porém em programas onde a nossa programação e a interface gráfica são tratadas por mecanismos diferentes, nem sempre esse travamento ocorre, porém o laço infinito se torna um desperdício constante do tempo de processamento do computador levando a uma gradual aparente lentidão.

121 No nosso contexto, modularizar significa separar os códigos referentes a ações diferentes em grupos bem definidos. Por exemplo, os códigos que realizam operações matemáticas teriam suas funções como “adição”, “subtração”, “multiplicação”, “divisão”, “potenciação” e “raiz quadrada” separadas umas das outras, sendo assim, os cálculos não seriam chamados diretamente no código principal, mas o código principal apenas chamaria suas funções matemáticas deixando o código mais enxuto.

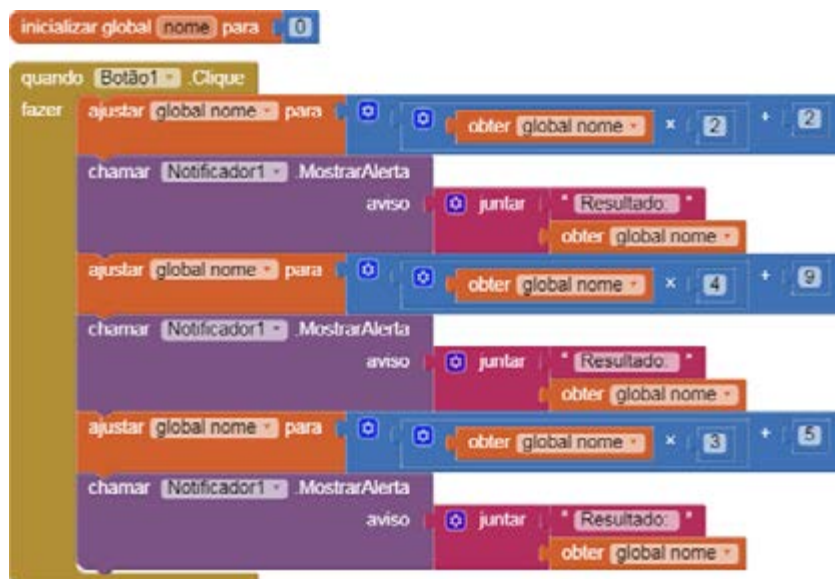


Figura 31: Um código com instruções que seguem um padrão, mas repetidas manualmente.

(Fonte: App Inventor)

Esse é nosso código inicial. Seu objetivo é: quando o “Botão1” for pressionado, uma variável global chamada “número” de valor zero será envolvida numa operação matemática e mostrada na tela três vezes com o resultado das operações realizadas sob diferentes valores.

Podemos perceber que há conjuntos de blocos que se repetem. O problema de escrever códigos repetidos à mão é que, em algum momento, por desatenção, o programador pode cometer erros em algum código. Pelo fato de serem todas estruturas iguais ou, no mínimo, semelhantes, fica mais difícil encontrar em qual parte do código o erro ocorreu dado que o mesmo trecho de código é repetido várias vezes.

Para impedir que isso aconteça, podemos extrair a parte imutável de um código e isolá-la numa função, a parte variável, ou seja, os valores com os quais ela vai trabalhar, continuam no código principal. Pode ser que você tenha notado que o nome “função” tem a ver com as funções matemáticas, aquelas como “f(x)” ou “g(x)”. A filosofia é praticamente a

mesma. Agora, se um erro acontecer, sabemos que os devemos nos concentrar na função e não tanto ao longo do código!

Enquanto na matemática as funções nos dão fórmulas prontas para serem aplicadas em diferentes valores, as funções na programação oferecem estruturas de código prontas para ser aplicadas em diferentes variáveis! A principal diferença é que agora, em vez de apenas usá-las, nós também as criaremos. Veja o código anterior agora fazendo uso de funções na figura 32.

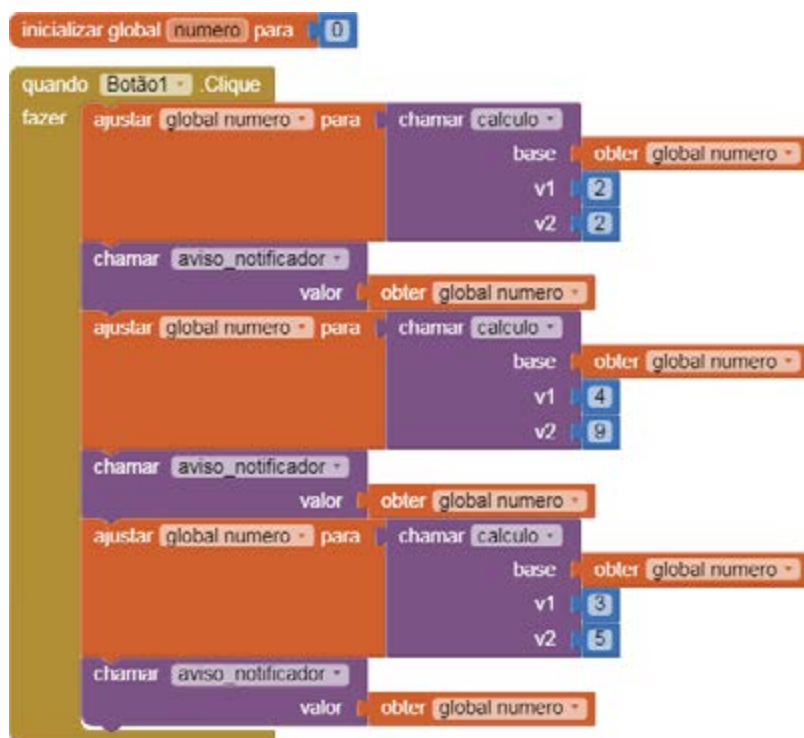


Figura 32: Código genérico fazendo uso de funções/procedimentos.

(Fonte: App Inventor)

Repare que a fórmula matemática à qual aplicávamos o processamento da variável “número” desapareceu do código, em seu lugar chamamos uma função chamada “calculo” e passamos para ela o valor da variável e os demais números usados na fórmula. Além disso, o

bloco que mandava a mensagem para o Notificador mostrá-la na tela desapareceu, em seu lugar chamamos apenas a função “aviso_notificador”. Veja onde estes códigos estão agora conforme a figura 33.

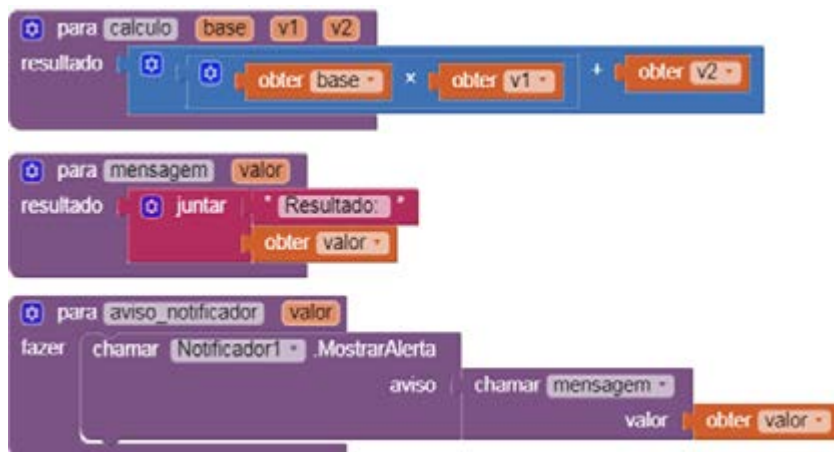


Figura 33: Código repetido manualmente escrito apenas uma vez e isolado dentro de um procedimento

(Fonte: App Inventor)

A fórmula matemática está na função cálculo. Perceba que ela não recebe diretamente os valores que precisamos na fórmula, ela usa suas próprias variáveis de cabeçalho para isso. Essas variáveis são chamadas de **parâmetros**¹²². Perceba que, por serem variáveis que atuam apenas dentro do procedimento e não têm uma declaração explícita, elas só podem ser acessadas passando o cursor do mouse sobre seu campo de nome no cabeçalho do procedimento.

Perceba que as variáveis de cabeçalho de uma função não podem ser usadas fora do escopo dela. Lembra-se do tópico de variáveis onde dissemos que discutiríamos os dois tipos de variáveis mais à frente? Essa hora chegou, existem dois tipos de variáveis: as globais e as locais,

¹²² Para evitar confusão, as variáveis que recebem valores no cabeçalho de uma função são chamadas de parâmetros, os valores passados para essas variáveis são chamados de argumentos. Fazer uso de parâmetros diminui a dependência do procedimento quanto às variáveis globais e o torna mais autossuficiente. Imagine usar um procedimento que depende de uma variável global em um código que não tem e não pode ter esta variável, seria uma catástrofe.

variáveis globais podem ser usadas em todos os lugares do código, variáveis locais só podem ser usadas no escopo a que pertencem. Parâmetros, por exemplo, são variáveis locais. Veja a diferenciação na figura 34.



Figura 34: Erro ao usar uma variável local fora de seu escopo de criação.
(Fonte: App Inventor)

Vale ressaltar também que podemos chamar uma função dentro de outra função, a “aviso_notificador” chama a função de “MostrarAlerta” do Notificador usando como argumento o resultado da função “mensagem” que tem como papel realizar a junção do texto com o resultado da variável calculada.

No App Inventor existem dois tipos de procedimentos ou funções, as simples e as que retornam um resultado, repare que, na figura 33, duas delas são do tipo que retorna um resultado e uma é um procedimento simples. A diferença está no início do escopo, o procedimento simples possui uma alça interna para anexar blocos. O procedimento com retorno possui seu início como uma entrada de encaixe como se pedisse um bloco de especificação. Na verdade essa entrada é por onde o programador especifica o valor a ser retornado para o procedimento que chamou a presente função.

Apenas como curiosidade, é possível que uma função chame a ela mesma, essa técnica se chama “recursividade”. Isso pode dar a ideia errada de que a função vai chamar a ela mesma infinitamente e nunca

retornará ao código principal, mas isso acontece apenas se a recursividade for mal planejada. Essa técnica é usada principalmente quando as estruturas de repetição não são o melhor meio de lidar com um problema que requer uma solução repetitiva.

2.4.5.1.8 Modificador de Estrutura de Bloco

Por fim, vamos destacar um último elemento dos blocos, alguns deles possuem um pequeno ícone de engrenagem branca sobre um fundo azul. Esses botões servem para modificar a forma do bloco. Por exemplo, o botão de engrenagem no bloco de um procedimento dá ao programador a opção de adicionar parâmetros no cabeçalho dele, assim como a possibilidade de editar seus nomes. No bloco de junção de textos, o mesmo botão oferece a capacidade de aumentar o número de entradas para encaixe à direita. Nos blocos matemáticos como multiplicação e adição, o mesmo botão altera a forma do bloco para que mais números possam ser encaixados no mesmo bloco.

2.4.5.1.9 Retomando o desenvolvimento

Agora que já sabemos a maioria dos conceitos básicos envolvendo o App Inventor, podemos prosseguir com o desenvolvimento de nosso aplicativo. A mecânica de nossa tela de IMC é dada pelo conjunto de blocos mostrado na figura 35.



Figura 35: Código implementando a funcionalidade da tela de IMC.
(Fonte: App Inventor)

Temos dois blocos de eventos em nosso código, um que detecta quando o botão de voltar é pressionado para fechar a tela e outro para

detectar se o botão de calcular foi pressionado. Quando este último é pressionado, nossa mecânica principal é disparada. Perceba que a análise de dados se faz presente.

O primeiro bloco a ser executado é uma instrução SE. Por meio de sua especificação podemos perceber que ele lida com duas condições ao mesmo tempo:

1. O valor do atributo Texto do componente Campo de Texto “Peso” pode ser interpretado como um número?
2. O valor do atributo Texto do componente Campo de Texto “Altura” pode ser interpretado como um número?

É importante lembrar que o bloco “é número?” por si só já retorna um valor lógico verdadeiro ou falso para o conteúdo que ele avalia. O fato de compararmos esse valor com o próprio valor lógico verdadeiro é apenas para ilustrar a situação: o bloco “é número?” checa se o valor do atributo Texto do componente Peso pode ser lido como um número válido, se sim ele retorna verdadeiro, senão, falso. O que fazemos depois é checar se a resposta do bloco é igual a “verdadeiro” mesmo que isso não seja realmente necessário neste caso.

O uso do bloco E permite que unamos essas duas condições e só aceitemos continuar o processamento se ambas forem verdadeiras, no caso, se os valores nos Campos de Texto forem números válidos. Dado o resultado como verdadeiro, prosseguimos para o próximo bloco onde ajustamos o texto da Legenda “Resultado” para uma **junção do texto “Seu IMC é” com o resultado do cálculo matemático**¹²³ dado pela fórmula do IMC usando os valores nos Campos de Texto.

Entretanto, se uma das nossas condições for falsa, ou mesmo as duas forem falsas, desviamos o fluxo do processamento para a área do

¹²³ Perceba que o App Inventor pode facilmente encarar um número como um texto, mas o contrário nem sempre acontece, afinal não há como transformar “ACB#\$\$” em um número de forma coerente.

SENÃO onde informamos que o valor da Legenda “Resultado” deve ser o aviso “Informe números válidos para o cálculo!”.

2.4.5.1.10 Comentário sobre Orientação a Objetos

Repare que não podemos usar o bloco de atributo que possui apenas o nome do componente, mas sim o atributo Texto deste. Isso se dá porque o bloco de atributo que só possui o nome do componente se refere a ele em si (como um todo), seus códigos internos, espaço de memória e tudo mais. Isso engloba o atributo Texto que é onde estão devidamente armazenados os dados numéricos informados pelo usuário, os dados que precisamos. Porém não há nenhuma indicação de que é esse conteúdo que queremos acessar, e sim o todo, portanto o código fica ambíguo. Veja este conceito aplicado na figura 36.

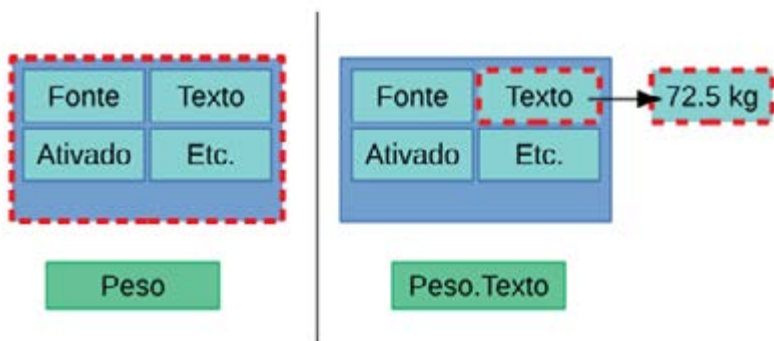


Figura 36: Diferença entre usar o bloco de objeto (esquerda) e bloco de atributo (direita).
(Fonte: Do Autor)

Cabe lembrar que a linguagem em blocos apresentada pela plataforma App Inventor não é orientada a objetos, mas sim estruturada e orientada a eventos. Entretanto, isso não a impediu de replicar alguns pontos de aspecto minimamente semelhantes com a programação orientada a objetos, seja pela aparência dos blocos de acesso/ajuste de atributos ou pelo fato de poder haver um conjunto de procedimentos comuns a todos os componentes de um mesmo tipo (mas que funcionam diferente dependendo do componente incluído na tela que os executa).

O fato de alguns procedimentos de componentes permitirem a escolha do componente a usar o procedimento também (como no caso do Temporizador) lembra um pouco a orientação a objetos (métodos). Em suma, nesse tipo de programação, os objetos de estudo do mundo real e até mesmo **elementos abstratos**¹²⁴ são convertidos em **classes**¹²⁵. A partir dessas classes são construídos **objetos**¹²⁶ que podem ser utilizados pelo sistema.

Se a linguagem da plataforma fosse orientada a objetos, poderíamos dizer que o componente Caixa de Texto é apenas uma classe, mas a partir do momento que o incluímos no aplicativo, sua classe é usada para criar um objeto. Esse objeto é a Caixa de Texto que nomeamos como “Peso”, o componente que aparece em nossa tela, podemos tocar, digitar, recuperar o texto digitado e tudo mais. Esse objeto teria seus atributos e **métodos**¹²⁷ separados do contexto global do aplicativo.

Para exemplificar melhor, um exemplo muito usado para explicar os conceitos e diferenças entre classe e objeto é o da planta de engenharia e da casa. Uma planta específica como uma casa deve ser feita, a casa é construída a partir destas especificações. Entretanto, uma planta não pode ser habitada, não se pode usar o banheiro de uma planta, nem montar a mobília nos cômodos. Ela é só uma base conceitual para a construção da casa. Assim que a casa é construída, todas essas ações se tornam possíveis. Nesse exemplo, a planta representa a classe e a casa representa o objeto.

124 Por exemplo, alunos e professores são pessoas, objetos de estudo que possuem existência concreta, porém uma matrícula é um conceito abstrato representado geralmente por um código ou número que envolve a relação de uma pessoa com uma instituição, não é possível tocar, carregar, polir, arremessar, quebrar ou fazer qualquer ação física com uma matrícula.

125 Uma coletânea de variáveis e procedimentos de programação referentes às características e ações de um certo objeto de estudo presente no mundo real que serão relevantes ao funcionamento do sistema. É apenas uma base conceitual a partir da qual são formados os objetos, afinal, não há como pedir a um computador que crie um certo objeto se esse não souber como criá-lo.

126 A concretização de uma classe, um objeto é, de forma geral, a teoria de uma classe posta em prática. O objeto tem a mesma estrutura da classe e pode ser usado como uma variável que contém suas próprias variáveis internas e funções concernentes ao tratamento das variáveis que possui.

127 Na orientação a objetos, procedimentos pertencentes a uma classe ou objeto são chamados de métodos.

A Orientação a Objetos adiciona à **programação estruturada**¹²⁸ o conceito de classes e objetos de onde partem vários outros conceitos mais técnicos como, por exemplo, **herança**¹²⁹, **polimorfismo**¹³⁰ e **encapsulamento**¹³¹ (nenhum destes conceitos é implementado pela linguagem da plataforma App Inventor). Isso deixa mais fácil a representação de objetos de estudo mais complexos na programação, mais do que as variáveis sozinhas poderiam representar. Mesmo que esse seja um tema amplo demais para abordarmos nessa discussão, vale ressaltar que a orientação a objetos é um paradigma de programação em constante ascensão.

Todavia, antes de terminarmos este comentário, podemos afirmar que a plataforma online DevMedia possui muitos artigos/conteúdos focados no público desenvolvedor/programador e parte de seu acervo é focado em linguagens de programação orientadas a objetos. Se desejar saber mais sobre este assunto, recomendamos o artigo “Os 4 pilares da Programação Orientada a Objetos” (<https://www.devmedia.com.br/os-4-pilares-da-programacao-orientada-a-objetos/9264>).

2.4.5.1.11 Finalizando a Primeira Utilidade

Por fim selecione o bloco de evento “VoltarPressionado” da tela da calculadora de IMC e adicione ele ao código, dentro dele, coloque um bloco de controle que faça a tela ser fechada. Fazendo isso, garantimos que, ao apertar o botão de voltar do smartphone, o usuário será capaz de ver a tela sendo fechada e voltar à tela anterior. Não se esqueça de colocar esse evento para as próximas utilidades do aplicativo!

128 Tipo de programação em que o código é executado de forma sequencial e uniforme seguindo as especificações de laços, funções, condicionais etc.

129 Uma técnica onde uma classe pode herdar características da outra evitando que as similaridades tenham que ser escritas manualmente de novo pelo desenvolvedor, isso aumenta a reusabilidade do código, deixando-o enxuto.

130 Uma técnica que permite que procedimentos herdados de uma classe possam ter seu escopo reescrito na classe derivada mas mantendo seu nome e, opcionalmente, parâmetros.

131 Técnica onde o acesso a atributos e procedimentos de uma classe é liberado ou restringido devido à possibilidade de uso indiscreto deles por parte do código externo à classe.

2.4.5.2. Vendo os Resultados

Agora que terminamos de programar nossa primeira utilidade, você deve estar com ansiedade para ver o resultado. Para que possamos testar nosso aplicativo, podemos seguir por três caminhos: fazer um teste em tempo real, fazer um teste em ambiente simulado ou fazer um teste em ambiente real.

Para poder fazer um teste em tempo real, é preciso ter um smartphone Android conectado à mesma rede do computador executando o App Inventor. Além disso, é preciso ter o aplicativo do Companion instalado no smartphone. Este aplicativo pode ser baixado tanto pela Google Play (<https://play.google.com/store/apps/details?id=edu.mit.appinventor.aicompanion3>) quanto diretamente pelo site da plataforma (<http://appinv.us/companion>).

Após instalado, precisamos acessar o menu “Conectar” na parte superior do App Inventor. Sendo assim, teremos várias opções e uma delas se refere ao uso do Companion (“Assistente AI”). Escolhendo esta opção, o App Inventor concederá um código de acesso (via texto ou código QR).

Esse código pode ser informado ao Companion (executando no smartphone) por meio de duas formas: a primeira é por meio da digitação direta do código textual no campo de texto fornecido, a segunda, é por meio da câmera do smartphone que tentará identificar o código por meio da interpretação do código QR. Após informar ou detectar o código, o App Inventor transferirá todos os códigos e recursos do App para o smartphone.

Quando o processo acabar, o aplicativo será executado no smartphone e o usuário poderá fazer seu teste em tempo real. Além disso, é importante notar que, ao trocar de telas, o aplicativo pode demonstrar lentidão ou pequenos travamentos. Isso acontece porque, ao mesmo tempo que o aplicativo está sendo executado, o computador tentará mostrar os códigos da tela que está sendo executada.

As outras duas formas de testar o aplicativo dizem respeito ao uso de um emulador do sistema Android fornecido pela plataforma e pela instalação direta do App no smartphone pelo uso de seu pacote compilado final. Como a forma de instalação e configuração do emulador de Android varia de plataforma para plataforma, sempre recorra aos tutoriais oficiais (<http://appinventor.mit.edu/explore/ai2/setup-emulator.html>) para obter as informações corretas e mais recentes sobre o processo.

Para instalar o aplicativo diretamente, precisamos compilá-lo. Há dois modos de compilação, um para download direto do App e outro por código QR. A primeira forma funciona pelo seguinte: a plataforma compilará o aplicativo gerando um pacote APK contendo o App, este pacote será baixado diretamente para o computador, entretanto, lembre-se que pacotes APK não podem ser instalados pelo sistema operacional do PC (a menos que ele tenha sido preparado especificamente para isso).

Tendo o pacote baixado no PC, precisaremos passá-lo para o smartphone. Isso pode ser feito via rede ou via cabo USB, escolha o que for mais rotineiro, viável e/ou fácil de executar. Por fim, é preciso abrir o arquivo no smartphone (por meio do gerenciador de arquivos) com o instalador de pacotes do SO Android. Sendo assim, o App será instalado.

Com o modo de compilação para código QR, o pacote APK é baixado diretamente no smartphone, consequentemente facilitando o processo. Entretanto, para ambos os modos, antes de instalar o pacote APK, talvez seja necessário mudar uma configuração do sistema.

Por padrão, smartphones Android bloqueiam a instalação de pacotes APK que não sejam fornecidos pela loja oficial de aplicativos da Google. Essa é uma medida de segurança tomada para evitar que os usuários instalem, de forma desavisada, Apps maliciosos que não têm uma procedência conhecida e segura.

Em versões Android mais antigas (2.3), a opção que queremos se encontrava no submenu Aplicações com o nome de “Fontes Desconhecidas”. Em versões recentes (6.0), a opção já se encontrava no submenu

Segurança, com o mesmo nome. Pode ser que isso mude em versões mais novas, portanto é preciso sempre recorrer às informações da versão do Android que estiver sendo usada no momento.

Com o término da instalação do App, ele se juntará à coleção de aplicativos instalados e pode ser executado a partir do menu com todos os aplicativos. Após testar a funcionalidade que criamos, devemos prosseguir com a criação de nosso App!

2.4.6. Seleção de Cores

Agora que nossa calculadora de IMC está pronta, faremos um seletor de cores para entender como deslizadores funcionam, como trabalhar com listas e como lidar com a formação de cores no App Inventor. Primeiramente, entendamos como as cores digitais funcionam. Existem três formas principais de representar cores em um computador: **RGB**¹³², **HSL**¹³³ e **CYMK**¹³⁴.

O App Inventor se utiliza do padrão RGB para criar suas cores. Nesse padrão as cores primárias vermelho, verde e azul podem ter um valor que vai de zero a 255 representando a intensidade da cor. Veja alguns exemplos de seletores na figura 37.

132 O sistema básico de representação de cores onde as cores primárias são vermelho, verde e azul (Red, Green, Blue). Todas as cores são formadas dependendo da intensidade de cada uma das primárias. Por exemplo, a máxima intensidade de azul com a máxima intensidade de verde resulta na cor ciano. A total falta de intensidade das cores primárias resulta no preto, enquanto que a presença total delas resulta no branco. Esse sistema é altamente usado para representação gráfica em computadores e telas digitais.

133 Uma outra forma de representar o sistema RGB, porém o usuário pode escolher uma cor como se estivesse vendo um arco-íris e então ajustar o nível de brilho e intensidade da cor para poder formar outras a partir da escolhida.

134 Um sistema de cores inverso ao RGB onde as cores primárias são justamente as junções das primárias deste. Ciano, magenta, amarelo e preto são as cores que formam esse sistema amplamente usado para impressões em papel e mídias físicas não-digitais.

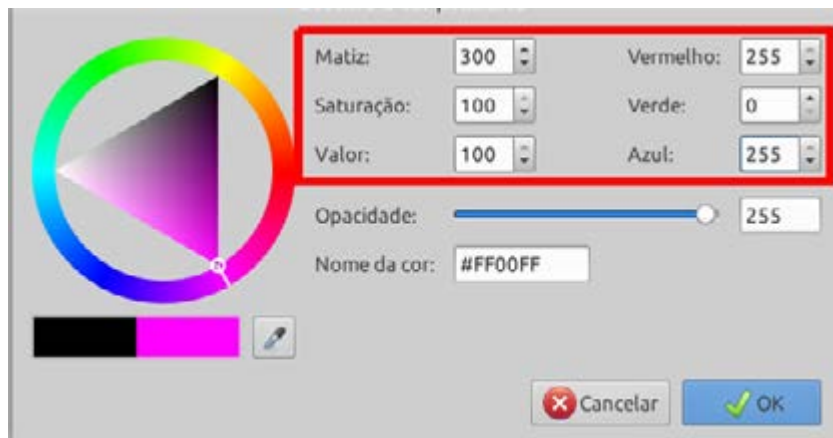


Figura 37: Exemplo de um seletor de cores HSL e RGB.

(Fonte: Editor de Imagens Pinta)

A figura 37 mostra um seletor de cores de um programa editor de imagens (Pinta). Os dois sistemas (HSL e RGB) estão separados em colunas. Tendo o conhecimento de como o RGB funciona, precisamos colocá-lo em prática em nossa tela. Portanto nosso objetivo é: extrair três valores numéricos do usuário que podem variar de zero a 255, indicando a intensidade de nossas cores primárias. Vá em frente e crie uma tela para que possamos fazer isso.

2.4.6.1. Criando a interface do seletor

Nessa nova tela, usaremos três deslizadores para extrair os dados numéricos do usuário e, para que não haja confusão, precisamos indicar qual deslizador está associado a qual cor específica. Por fim, precisaremos de um componente que seja capaz de ter sua cor alterada para mostrar o resultado final ao usuário, como efeito da junção das três intensidades de vermelho, verde e azul. A tela deve se parecer com o que é retratado na figura 38.

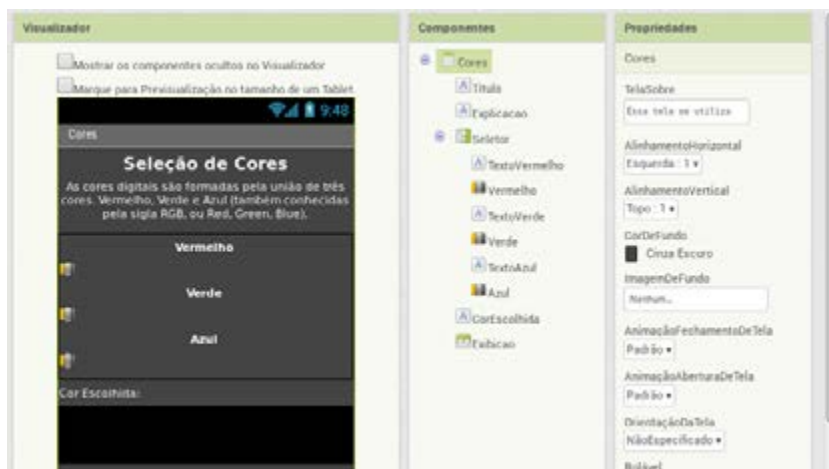


Figura 38: Aspecto visual da tela do seletor de cores.

(Fonte: App Inventor)

Na figura 38 podemos ver, inicialmente, duas Legendas, uma mostrando o título da tela e outra mostrando uma breve descrição do que ela representa. Em seguida temos uma Organização Vertical para agrupar nossos componentes: três pares de uma Legenda e um Deslizador, cada um para uma cor. Logo abaixo, temos uma Legenda (informando a cor resultante) e uma Organização Horizontal que usaremos apenas para mostrar a cor formada pelos deslizadores na propriedade Cor de Fundo. Agora que organizamos a parte gráfica da tela, vamos partir para a lógica.

Deslizadores trabalham com valores decimais (racionais) portanto precisaremos arredondar seus valores (números inteiros) antes de criarmos as cores. Também podemos configurar o valor mínimo e o valor máximo dos deslizadores na sessão de design. Portanto, precisamos estabelecer que esses valores sejam, respectivamente, zero e 255. Há ainda uma opção na janela de propriedades do deslizador que permite indicar qual deve ser o valor inicial dele. Há, ainda, como configurar as cores do deslizador, a parte à esquerda indica o valor do deslizador no momento, a parte à direita indica quanto falta para o fim

do intervalo permitido (como se fosse uma reta numérica da matemática).

2.4.6.2. Criando a lógica do seletor

Antes de tudo, precisamos entender um conceito específico da forma com a qual o App Inventor lida com cores. Ele faz isso através de “**listas**”¹³⁵. Mas antes, entremos num tópico que nos ajudará melhor a entender o conceito de lista.

2.4.6.2.1 Estudando Vetores

Em outras linguagens de programação, é comum ver que apenas variáveis sozinhas podem ser uma forma muito pobre de lidar com vários dados do mesmo tipo. Por exemplo, consideremos um sistema simples de uma escola para registrar notas de alunos onde cada aluno tem sua nota.

O que poderíamos fazer? Criar uma variável para a nota de cada aluno? Certamente que poderíamos fazer isso, entretanto isso é muito pouco prático. Não seria simples armazenar as notas e também não haveria forma simples de fazer operações que precisem de todas as notas, isso porque seria preciso usar o nome de todas as variáveis. Agora imagine uma linha de código usando trinta ou quarenta variáveis (ou mesmo 400 para representar os alunos da escola inteira), isso tornaria o código muito extenso e o programador estaria plenamente sujeito a erros.

Pensando em casos assim, foi concebido o conceito de **vetor**¹³⁶. Para ilustrar o conceito, imagine que uma variável comum é um carro.

135 Listas são estruturas de dados que são usadas por várias linguagens de programação. Entretanto, no App Inventor, o conceito de listas e vetores se confunde. Elas servem para estender o poder das variáveis para guardar mais informação sob uma mesma variável dividida em espaços de memória diferentes indicados por um índice. Imagine um trem, a locomotiva e os vagões são todos os espaços de memória, mas o trem todo é uma variável só. Cada vagão é referente a um índice diferente desta variável onde pode ser armazenado um dado.

136 Uma variável especial de certo tipo de dado que possui um nome e um indicador numérico

Um carro pode carregar uma certa quantidade de carga dentro de si, mas se ele ficar lotado, não há onde colocar a carga extra. Essa é a variável simples, capaz de guardar somente um dado. Se preenchemos uma variável com uma nota de aluno de uma sala de trinta, onde armazenaríamos os outros 29? Se guardarmos nela mesma, a nota anterior será sobrescrita.

Estendendo o poder da variável, entra em ação o vetor. Ele se assemelha a um trem de tamanho fixo especificado por quem o comanda. Um trem simples é composto de uma locomotiva e vários vagões onde cada um pode carregar uma certa quantidade de carga (assim como um carro). Sendo assim, se um vagão já está preenchido, pode-se preencher o próximo. Vetores são assim, eles possuem um nome e um **índice**¹³⁷ que aponta para um “vagão” ou um espaço de memória que pode ser usado como uma variável simples.

Sendo assim, por exemplo, ficaria mais fácil realizar um cálculo de média. Poderíamos ter uma variável para acumular o valor de todas as notas numa soma, essa soma seria realizada dentro de uma estrutura de repetição, que percorreria o vetor inteiro, incrementando o valor do índice (que estaria em uma variável específica) a cada repetição.

Se o limite do vetor fosse cinco, então a estrutura de repetição seria repetida cinco vezes e, a cada vez, a variável usada como índice seria incrementada para acessar o valor da nota que está armazenada na próxima posição do vetor. Ao fim teríamos a soma de todas as notas e só precisaríamos dividir pelo número de alunos da classe (que já conseguimos com a variável que guarda o número de repetições). Veja a comparação das variáveis com o vetor na figura 39.

que pode chegar até um valor máximo especificado pelo programador.

137 Número natural que indica as posições de memória associadas à variável que dá nome ao vetor.

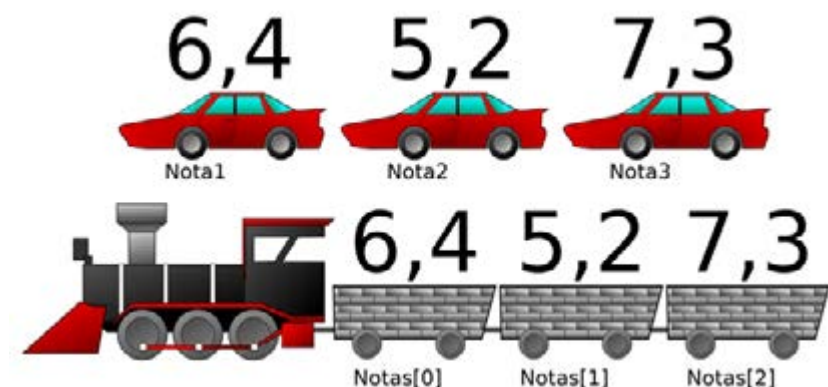


Figura 39: Diferença entre um vetor e variáveis isoladas.

(Fonte: Do Autor)

Repare que, no caso do vetor, os espaços de memória estão aliados ao mesmo nome de variável, mas são referenciados pelo número do índice que leva a dados de mesmo tipo, mas com valores diferentes. Isso garante mais flexibilidade no código quanto ao armazenamento de dados.

Nomes de variáveis não são manipuláveis, mas números são. O índice é um número, logo podemos acessar diferentes variáveis sob um mesmo nome, mas em espaços de memória diferentes, apenas indicando a posição de memória que precisamos por meio do índice. Essa é a ideia geral dos vetores.

Porém, lembre-se que estudamos que vetores têm tamanho fixo. Se o programador especificar que um vetor terá vinte posições, ele terá vinte posições, nada mais, nada menos, isso não é ajustável no tempo de execução. Embora eles nos tragam a facilidade de armazenar e acessar diferentes valores sob o mesmo nome de variáveis, o tamanho do vetor não é maleável. Expandindo conceitos e modos de funcionamento, surgem outras estruturas de dados.

2.4.6.2 Estruturas de Dados

Estruturas de dados são objetos dotados da capacidade de guardar dados na programação. Algumas são homogêneas, feitas para suportar apenas um tipo de dado (como os vetores e **matrizes**¹³⁸), e outras heterogêneas, para **suportar vários**¹³⁹. Porém, é notável que algumas delas têm a capacidade de se expandir automaticamente conforme o conteúdo é adicionado. Em seu âmago, essas estruturas usam código para alocação dinâmica de memória e isso lhes confere essa habilidade de ter um tamanho variável e maleável.

Algumas das estruturas de dados expansíveis **não possuem um indicador numérico**¹⁴⁰ intrínseco a elas igual os que foram apresentados no estudo de vetores. Em vez disso, elas possuem um indicador de início (e talvez um de fim), o primeiro elemento se liga no indicador de início e cada um dos elementos presentes na estrutura é capaz de “apontar” para os próximos elementos conforme eles são adicionados na estrutura. Existem várias estruturas de dados que **seguem este formato**¹⁴¹, as principais são:

1. Filas, usando padrão **FIFO**¹⁴².
2. Pilhas, usando padrão **LIFO**¹⁴³.

138 Estruturas de dados semelhantes aos vetores, porém utilizando-se de dois ou mais índices simultâneos. Matrizes são estruturas de dados multidimensionais onde cada índice corresponde a uma posição em um “eixo” e a interseção deles leva a um endereço de memória.

139 Alguns exemplos são as classes (e seus objetos) das linguagens Java e C++, objetos da linguagem JavaScript, “structs” da linguagem C e tabelas da linguagem de programação brasileira Lua.

140 Mesmo assim, o programador pode implementar códigos e mecanismos que permitam o acesso via índices numéricos, um exemplo seria pelo uso de estruturas de repetição que percorreriam a estrutura de dados até encontrar a posição pedida por meio do índice.

141 Entretanto, nada impede que estas estruturas sejam implementadas sob uma base de vetor, ainda tendo uma limitação fixa de armazenamento. Além disso, o programador pode criar código capaz de alocar vetores secundários com mais espaço à medida que for preciso mais armazenamento.

142 Sigla para “First In, First Out”, ou “O primeiro que entra é o primeiro que sai”. Os elementos que entram numa fila devem sair dela exatamente na ordem em que entraram.

143 Sigla para “Last In, First Out”, ou “O que entrou por último é o primeiro a sair”. Os elementos que entram numa pilha devem sair dela na ordem inversa à que entraram.

3. **Árvores**¹⁴⁴, usando distribuição binária.
4. **Listas**¹⁴⁵.

Essas estruturas precisam lidar com um tipo abstrato de dados (TAD). Segundo Celes Filho (2004), “Nos casos em que um módulo define um novo tipo de dado e o conjunto de operações para manipular dados desse tipo, dizemos que o módulo representa um tipo abstrato de dados (TAD)”. Este TAD é capaz de guardar o dado de nosso interesse e ainda apontar coerentemente para os próximos elementos que também são TAD’s.

Para lidar com o trabalho de gerenciar as informações necessárias ao funcionamento coerente da estrutura de dados, podemos definir um conjunto de funções que lidam com isso e limitar o uso direto de nosso TAD por meio deste conjunto.

Sendo assim, futuros desenvolvedores precisariam se preocupar em apenas entender para quê servem e como usar nosso conjunto de funções, não precisariam aprender como manipular nossa estrutura de dados diretamente ou como as funções foram implementadas.

Não estudaremos essas estruturas de dados da listagem anterior pois cada uma tem uma forma específica de ser implementada e isso foge ao escopo do livro, mas é um tópico importante a se considerar se planeja seguir seus estudos com uma linguagem de programação mais robusta. Das estruturas apresentadas, a que mais nos interessa agora são as listas.

144 Os elementos entram numa árvore por um ponto em comum e são organizados tendo em mente se seu identificador é maior ou menor ao desse ponto em comum. Se for menor, o ele é guardado num espaço específico, senão, é guardado em outro. Por conseguinte, os elementos que entrarem depois terão que avaliar se são maiores ou menores do que alguns elementos que já foram adicionados. Árvores são amplamente usadas quando é necessário que a velocidade de busca de uma informação seja maior quando comparada à de outras estruturas de dados.

145 Tanto pilhas como filas podem ser encaradas como listas. A diferença é que, em pilhas, só é possível acessar ou retirar um elemento que entrou por último na estrutura, em filas, só é possível acessar ou retirar o que entrou primeiro. Em listas, qualquer elemento pode ser acessado e retirado ou incluído a qualquer momento em qualquer posição.

Listas são uma estrutura de dados bem versátil podendo ser lidas apenas em um sentido (do início ao fim, quando **simplesmente encadeadas**¹⁴⁶) ou nos dois sentidos (do início ao fim e vice-versa, quando **duplicamente encadeadas**¹⁴⁷). Elas podem se expandir conforme o conteúdo é adicionado e qualquer elemento pode ser acessado, removido ou alterado em qualquer posição e a qualquer momento (ou não, se forem implementadas sobre um vetor).

As listas do App Inventor misturam o que há de melhor em vetores com o conceito de lista. Listas convencionais não possuem um índice único que indica a posição de memória como vimos em vetores, os elementos são indicados diretamente por outros elementos através do endereço de memória em que se encontram.

Porém, no App Inventor, listas possuem ambas as capacidades de serem ordenadas por um índice (como visto em vetores) e de serem expansíveis (como visto nas listas convencionais). Para maior agilidade no acesso aos dados, a lista pode ter sido implementada sobre um vetor aliado a código de redimensionamento automático sob demanda.

Entretanto, note que não temos blocos de atributo para fazer acesso a uma lista. Em vez disso, temos uma categoria completa de blocos (“Listas”) para lidar com uma variável que carrega uma lista como valor armazenado. Note que cada bloco tem formatos diferentes que requerem argumentos para seus parâmetros e tem ações bem definidas no contexto do uso de listas.

Esses blocos caracterizam o conjunto de operações apontado por Celes Filho (2004) e escondem de nós como a lista funciona internamente. Sendo assim, só precisamos contextualizar os blocos apresentados em nosso problema para idealizar uma solução e programá-la efetivamente.

Note que, para quase todos os blocos que tratam listas, temos que fornecer uma variável que contenha a lista em si. Isso deixa explícito que o TAD “lista” define uma estrutura de dados (com os dados em si)

146 Cada elemento indica apenas a posição de memória do próximo elemento.

147 Cada elemento indica tanto a posição de memória do próximo elemento quanto a do anterior.

e está armazenado em uma variável, mas não podemos interagir com esta estrutura diretamente. Em vez disso, precisamos usar os blocos da categoria Listas. A junção das operações com a estrutura formam o TAD.

Para lidar com cores, o App Inventor se utiliza do sistema RGB onde o valor da intensidade das cores é guardado numa lista. Seguindo a ordem, o primeiro elemento da lista representa a cor vermelha, o segundo representa a verde e o último representa o azul. Apenas como curiosidade, enquanto as linguagens de programação derivadas da linguagem C começam vetores com índice em zero, o App Inventor inicia suas listas com o índice 1.

2.4.6.3. Organizando os blocos

Agora que temos os fundamentos para entender o que acontecerá em nossa mecânica, podemos prosseguir com o desenvolvimento do aplicativo. Primeiro precisamos conseguir os valores numéricos dos deslizadores, veja na figura 40 como podemos fazer isso.

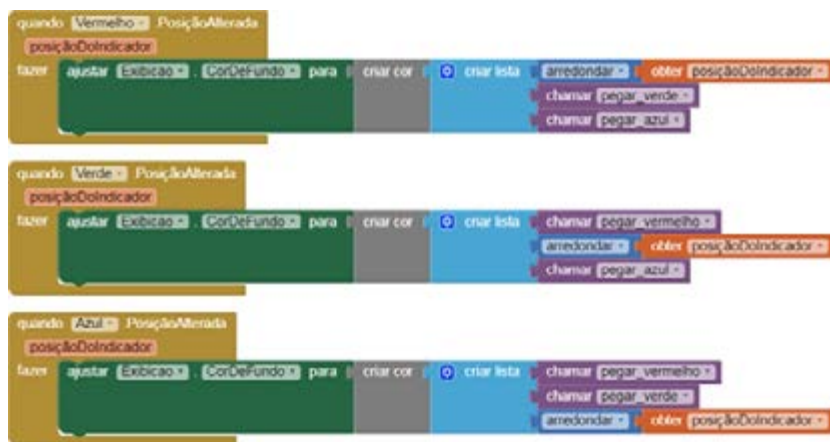


Figura 40: Como obter o valor numérico dos deslizadores por eventos.
(Fonte: App Inventor)

Perceba na figura 40 que temos três blocos de eventos, cada um deles detecta a mudança no valor de um deslizador. Assim que ele tem

sua posição alterada, o evento é lançado e podemos executar nosso código. Perceba também que alguns blocos de eventos podem agir de forma semelhante a blocos de procedimentos, permitindo que sejam passados argumentos por meio de parâmetros (porém não são editáveis).

No caso, o argumento passado pelo parâmetro é o valor numérico do deslizador que está dentro do intervalo que definimos na sessão de design. É importante notar que cada bloco de evento cuidará de um número isolado, e não dos três. Então estudaremos apenas o caso do vermelho já que as três cores têm o mesmo funcionamento.

Primeiro dizemos que alteraremos o atributo “Cor de Fundo” do componente “Exibição” que é uma Organização Horizontal que estamos usando apenas para exibir uma cor. O bloco de alteração de atributo precisa de uma especificação, mas não uma especificação qualquer, precisamos fornecer uma cor.

Embora o App Inventor ofereça cores prontas, esse não é nosso objetivo, então precisamos criar uma cor personalizada. Com isso é preciso entrar na categoria de blocos “Cores” e escolher o bloco “Criar cor”. Por padrão, esse bloco já vem com um bloco para a criação de uma lista, por sua vez, este último vem com três blocos de números para que possamos especificar a intensidade de vermelho, verde e azul. Porém esses blocos numéricos não serão de utilidade para nós já que quem fornecerá os valores será o usuário através dos deslizadores.

Assim que anexamos os blocos de criar cor e de criar lista ao código, precisamos especificar de onde os valores numéricos virão. Para isso, no caso do vermelho, o primeiro valor da lista é dado como sendo o arredondamento (bloco da categoria de matemática) do valor passado como argumento pelo parâmetro “posiçãoDoIndicador”.

Se o usuário posicionou o indicador do deslizador no meio do intervalo, teríamos 127,5 como resultado, no entanto não é de costume usar números racionais para formação de cores, apenas números naturais, por isso usamos um arredondamento.

Prosseguindo com nosso estudo, temos tanto o azul como o verde sendo dados como os resultados numéricos de dois procedimentos

chamados “pegar_verde” e “pegar_azul”, eles são usados porque enquanto o usuário muda uma cor, precisamos preservar as outras para formar a cor resultado. O funcionamento destes procedimentos pode ser visto na figura 41.



Figura 41: Procedimentos que preservam as cores não modificadas pelos eventos.
(Fonte: App Inventor)

Note que usamos o tipo de procedimento que admite retorno de valores para a função que chamou nossos procedimentos, isso porque precisamos do valor das cores e, ainda assim, manter o código modularizado. Perceba na figura que o valor que devolvemos é extraído da lista conforme o App Inventor segue o padrão RGB.

Seguindo essa especificação, pegamos a cor do componente Organização Horizontal que chamamos de “Exibicao” e repartimos ela, isso resulta numa lista. Essa lista é entregue como especificação do bloco “selecionar item da lista”. Mas esse bloco precisa de outra especificação, o índice do elemento que precisamos, assim como estudamos no conceito de vetores. Então fornecemos o índice de acordo com o padrão RGB.

Depois de fornecermos esses dois elementos, a lista e o índice, o bloco achará o elemento e o retornará para o bloco que o chamou, no caso, nosso procedimento. O procedimento retornará o valor da cor

requerida para o formador da nova cor dentro do referido bloco de eventos.

Como o bloco de criar lista já vai ter o valor arredondado do deslizador da cor modificada pelo usuário, unimos essa informação com os valores de cores restantes e formamos uma nova lista que será usada como cor do “Exibicao”. Assim, concluímos nosso seletor de cores!

2.4.7. Autenticador

Agora aprenderemos a fazer um autenticador que pede que o usuário registre uma senha qualquer e depois requiere a mesma senha para autenticá-lo. Com isso temos que introduzir um componente novo, o “Campo de Senha”. Esse componente é exatamente um Campo de Texto comum, mas com a capacidade de transformar o que é digitado em pequenos **pontos ou bolinhas**¹⁴⁸, independente do caractere. As propriedades deste componente são as mesmas do Campo de Texto, exceto o fato de não possuir as propriedades “Somente Números” e “Multi Linha”.

2.4.7.1. Uso Consciente de Telas e Parte Visual

Para não fazer muito uso de telas, usaremos a Organizações Verticais aliadas à propriedade “Visível”. Uma delas será chamada de “Definicao” e a outra “Validacao”, quando estivermos requisitando uma senha para registro, mostraremos apenas a Definicao, quando requisitarmos a senha para autenticação, mostraremos apenas a Validacao.

Dentro da Definicao haverá uma legenda pedindo ao usuário que registre uma senha, um campo de texto para capturar a senha digitada no teclado e um botão que disparará o evento de registro. Dentro da Validacao haverá uma legenda pedindo ao usuário que informe a senha para autenticação, um campo de senha para recolher a senha, um botão

¹⁴⁸ Essa é uma medida de segurança para evitar que outras pessoas possam ver qual é a senha de um determinado usuário.

para disparar o evento de comparação, um botão para disparar o evento de redefinição de senha e, por fim, uma legenda que mostra se a autenticação foi ou não bem-sucedida.

Poderíamos deixar essa tela ainda mais econômica se usássemos apenas duas legendas, um campo de senha e um botão e tratássemos a diferença entre definição e validação de senha ser tratada inteiramente pelo código, entretanto vamos ressaltar nesse tópico a importância de simular múltiplas telas.

Simular telas se faz preciso porque o App Inventor é limitado, o navegador que acessa a plataforma também é, e os dispositivos Android também são. Mesmo que sejam potentes, todos os computadores têm limites. Segundo a documentação oficial do App Inventor, adicionar muitas telas a um aplicativo pode ser perigoso porque causa lentidão na plataforma, tanto na sessão de design quanto no editor de blocos.

Se um desenvolvedor continuar adicionando telas a torto e a direito, isso aumentará o uso de recursos necessários para executar o App Inventor de forma drástica e isso pode levar a casos extremos onde o desenvolvedor não poderá nem mesmo alterar o projeto de volta ao estado de quando funcionava. Dados esses problemas, a plataforma não seria capaz nem mesmo de compilar o aplicativo e gerar o pacote APK. O projeto estaria perdido sem poder ser modificado.

Mais informações sobre esse tema podem ser encontradas no site do App Inventor (<http://ai2.appinventor.mit.edu/reference/other/manyscreens.html>), mas é de suma importância informar que é preciso fazer uso consciente das telas e enxugar o aplicativo o suficiente para que tenha o mínimo possível delas.

O aviso na página afirma que, geralmente, até 10 telas o App Inventor ainda apresenta um bom desempenho, além disso os problemas de instabilidade começam a aparecer e nada pode ser garantido. Para nossos estudos, nesse aplicativo de Utilidades, usamos 7 delas e ainda fazemos uso das técnicas de simulação de telas por alternar a visibilidade dos componentes.

Quando um componente fica invisível ele deixa seu espaço na tela livre para outros componentes. Podemos usar isso a nosso favor.

Quando o usuário apertar o botão para registrar sua senha, tornaremos o componente Definicao invisível e o Validacao se tornará visível (ocupando o espaço que era do Definicao). Sendo assim, veja como ficaria nossa tela visualmente na figura 42.

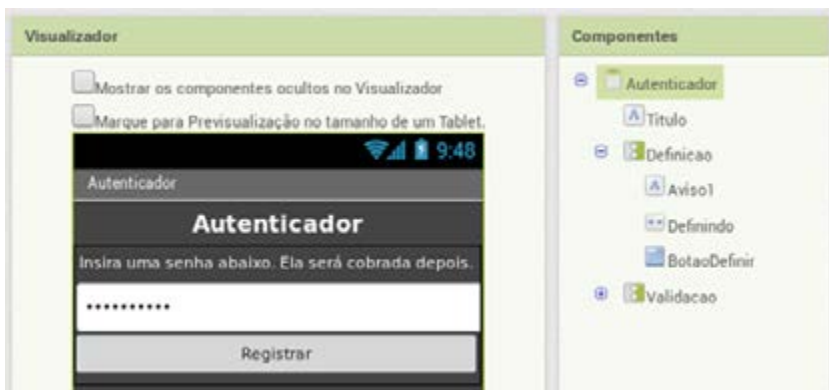


Figura 42: Aspecto visual da tela do Autenticador.

(Fonte: App Inventor)

Antes de prosseguirmos, as configurações que colocamos na sessão de design são as primeiras que aparecem no aplicativo assim que o colocamos para executar. Entretanto, você já deve ter percebido que várias dessas configurações podem ser mudadas durante o tempo de execução pelos blocos que montamos.

Para que o aplicativo se comporte da forma esperada, precisamos configurar a Validacao como invisível na sessão de design e deixar a Definicao como visível. A mudança de visibilidade será feita pelo código. Além disso, é preciso ressaltar que o App Inventor possui uma ferramenta no visualizador voltada especialmente para esses cenários.

Quando se lida com mudança de visibilidade, é comum que o desenvolvedor comece a mudar a configuração o tempo todo para saber com o que está lidando e também saber se o resultado visual está agradável. Acontece que ficar mudando a configuração o tempo todo pode levar o desenvolvedor a se esquecer de colocar a configuração na forma referente à execução do aplicativo quando nenhuma alteração foi feita pelo código.

Para esses casos, acima do visualizador há uma caixa de checagem clicável que permite mostrar todos os componentes ocultos sem que seja necessário mexer na configuração. Junto à opção de mostrar todos os componentes ocultos, é apresentada também uma opção para que o visualizador seja ajustado para o tamanho de um tablet para que o desenvolvedor possa saber se o design ficaria bom numa tela de resolução maior, essas opções podem ser vistas na figura 43.



Figura 43: Opções do Visualizador para componentes invisíveis e mudança de tamanho de tela.

(Fonte: App Inventor)

A legenda Resultado não possui texto, por isso não aparece na figura, mas está localizada ligeiramente abaixo do componente Botao-Redefinir, dentro da organização Validacao. Agora que já sabemos o que devemos fazer, partiremos para a edição de blocos de nosso Autenticador.

2.4.7.2. Edição dos blocos do Autenticador

Primeiro de tudo, precisamos garantir que os componentes estejam com a visibilidade certa quando executarmos essa tela do aplicativo. Quando entramos nesse capítulo do livro, fizemos uma introdução ao conceito das telas. Dentro dessa introdução, ressaltamos alguns eventos que um componente de tela pode disparar, dentre eles há o evento “Inicializar”.

Usaremos o inicializar para configurar a visibilidade dos componentes de Organização Vertical. Mas perceba que, basicamente, quando um é mostrado, o outro é escondido, então podemos modularizar isso criando duas funções, uma chamaremos de “estado_inicial” e a outra “estado_final”, uma esconderia o Validacao e outra esconderia o Definicao respectivamente. Veja como os blocos ficariam na figura 44.



Figura 44: Procedimentos que mudam a visibilidade das organizações conforme o estado atual da tela.

(Fonte: App Inventor)

Depois de assegurar que a mudança de visibilidade será feita, o usuário verá o campo de senha para definição de sua senha. Ao inserir a senha, ele pressionará o botão para registrá-la, com isso um evento de clique será disparado pelo botão e o interceptaremos com o seguinte código mostrado na figura 45.



Figura 45: O botão de definir senha dispara a mudança de estado, a limpeza do campo de texto para definição da senha e o armazenamento dela.

(Fonte: App Inventor)

Perceba na figura que, primeiro, ajustamos uma variável global (inicializada como texto vazio) para guardar o valor do texto do campo de senha. Em seguida, limpamos o valor do texto do campo de senha da Definicao para evitar que a senha continue nele quando o usuário apertar o botão de redefinição. Após realizarmos estas mudanças, fazemos a mudança de estado com a função “estado_final”. A Organização Vertical Definicao some e a Validacao rouba a cena.

Agora requisitamos o usuário a mesma senha registrada para autenticá-lo, então, assim que a senha for informada, o usuário tentará colocar a senha sucessivas vezes para ver se consegue se lembrar dela corretamente. Após inseri-la, o usuário pressiona o botão de validar, é aí que precisamos fazer uma comparação com o bloco SE. Se a senha informada for igual à registrada, então mudamos o texto da legenda de resultado para “**LOGADO**”¹⁴⁹ e aplicamos a cor verde ao texto usando uma das cores pré-definidas pelo App Inventor.

Se a senha não for igual à registrada pelo usuário mais cedo, então temos que definir o texto da legenda de resultado para “**NÃO LOGADO**”¹⁵⁰, e então aplicamos uma cor vermelha ao texto para conferir um aspecto de ênfase. Depois disso limpamos o valor do texto do campo de senha da Validacao para que o usuário possa entrar com

¹⁴⁹ “Logado”, “login”, “logon” e outros são termos que significam que, num contexto geral, uma permissão de acesso foi concedida.

¹⁵⁰ O termo antônimo a login ou logon é “logout” ou “logoff”.

outra senha, independente da anterior estar certa ou errada. Repare como fica o código na figura 46.



Figura 46: O botão de validar dispara um evento que resulta na autenticação ou proibição de acesso do usuário.

(Fonte: App Inventor)

Note que, na instrução SE, não usamos um comparador matemático ou lógico, para comparar textos existe um bloco próprio para essa atividade, ele se encontra na categoria de blocos “Texto”. Para checar se um texto é igual a outro, é preciso ver se cada caractere é exatamente igual, o tamanho do texto também precisa ser igual, assim como a ordem dos caracteres.

Agora que garantimos que o usuário possa registrar uma senha, inseri-la para autenticação, ver o resultado da comparação, precisamos programar o botão de redefinir a senha para que ele volte a ver o componente Definicao. Podemos fazer isso com o código mostrado na figura 47.



Figura 47: O botão de redefinir limpa os dados armazenados e troca o estado para inicial.

(Fonte: App Inventor)

Nesse evento de clique do botão de redefinir, nós limpamos o valor da variável global que guarda a senha. Em seguida nós estabelecemos que o texto da legenda de resultado será limpo para que quando a Validação for mostrada novamente, não apareça o resultado da última autenticação. Por fim, modificamos as configurações de visibilidade por chamar a função de “estado_inicial”. No fim, nossa tela fica do jeito que estava quando foi inicialmente aberta. Com isso concluímos o autenticador.

2.4.8. Calculadora de Idade

Caminhando para a próxima utilidade, aprenderemos a usar um novo componente, o EscolheData. Esse componente é uma maneira uniforme de conseguir uma informação de data do usuário. Ele é um botão que, quando pressionado, simplesmente mostra um pequeno **popup**¹⁵¹ onde, geralmente, são apresentados três **spinners**¹⁵², um para o dia, um para o mês e outro para o ano. Veja um exemplo na figura 48.



Figura 48: Exemplo de uso de spinners no popup do componente EscolheData.
(Fonte: Android)

151 Uma pequena janela ou quadro que aparece sobre as demais quando o usuário interage com um objeto.

152 Um campo de texto especial com dois botões integrados ao canto, onde é possível incrementar ou decrementar um valor.

2.4.8.1. Introduzindo o componente EscolheData

Como não usaremos o componente de botão, temos que lidar com outro evento. Constan na tabela 11 os eventos que podem ser lançados pelo componente de escolher data.

Tabela 11: Eventos destacados do componente EscolheData.

Evento	Causa
DepoisDeAjustarData	Acontece toda vez que o usuário termina de escolher sua data dentro do popup quando pressiona o botão de confirmar.
ToqueParaBaixo	Acontece toda vez que o usuário toca o botão que abrirá o popup.
ToqueParaCima	Acontece toda vez que o usuário deixar de tocar o botão que abrirá o popup.

(Fonte: App Inventor)

O evento que usaremos é o “DepoisDeAjustarData”. Como nosso objetivo é calcular por quantos anos o usuário está vivo (levando em consideração anos, meses e dias), pediremos sua data de nascimento. Quando ele confirmar a data, iniciaremos o processamento de nosso código. Mas é importante ressaltar que os dados da data não são fornecidos como argumentos pelos parâmetros do evento, eles são atributos do componente, por isso precisaremos usar os blocos de acesso a atributos.

2.4.8.2. Aspecto Visual

Podemos ver na figura 49 como nossa utilidade deve se parecer, ela é bastante simples visualmente, mas a lógica não acompanha esse atributo. Mantenha-se atento aos detalhes que se seguem nos tópicos adiante.



Figura 49: Aspecto visual da calculadora de idade.

(Fonte: App Inventor)

2.4.8.3. Lógica da Utilidade de Calcular Idade

Assim que o evento for disparado, a primeira coisa que precisamos fazer é obter a data de nascimento, veja na figura 50 como podemos fazer isso por meio de um procedimento que acessa os atributos de nosso EscolheData e os armazena em três variáveis globais.



Figura 50: A obtenção dos dados do EscolheData é feita por acesso aos atributos.

(Fonte: App Inventor)

Depois de armazenarmos esses dados, precisamos conseguir a data atual do sistema operacional Android, para que possamos fazer isso, vamos precisar de um novo componente, o Temporizador. É possível perceber que ele não está presente nas categorias de componentes que

estudamos até agora (Interface de Usuário e Organização), ele se encontra na categoria de Sensores (porque é sensível à mudança do tempo).

2.4.8.4. Uso do Temporizador

O temporizador é um componente que não interage diretamente com o usuário, por isso não possui representação gráfica, note que, ao ser incluído ele é mostrado diretamente abaixo do visualizador. Seu modo de funcionamento é o seguinte: um temporizador manipula instantes. Instantes são compostos por ano, mês, dia, segundo, centésimos de segundo, milésimo de segundo e outros. Abordaremos mais sobre o temporizador quando fizermos a última utilidade.

O que mais importa para a resolução de nosso problema é usar o bloco de procedimento “Agora” do Temporizador. Esse bloco devolve um instante com os dados temporais do momento em que é executado. Assim que conseguimos seus valores, temos que armazená-los em outras variáveis para usá-los no código. Veja os procedimentos, o temporizador no visualizador e sua posição na categoria de Sensores na figura 51.

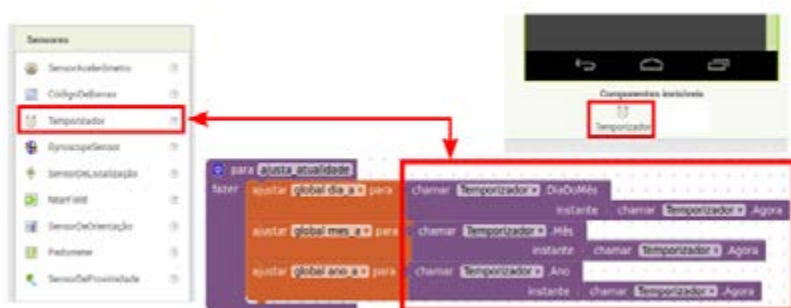


Figura 51: Uso primário do temporizador para conseguir dados de tempo atual e sua representação no visualizador.

(Fonte: App Inventor)

2.4.8.5. Cálculo Básico

A função “ajusta_atualidade” foi chamada logo após a “ajusta_nascimento”, sendo assim, podemos seguir com a explicação. Dentro do bloco de evento que fizemos inicializamos uma variável local para lidar com a idade da pessoa. Esse valor será computado pela função “idade” que receberá dois argumentos via parâmetros, o ano do nascimento e o ano atual.

Dentro da função idade há um cálculo matemático que consiste em subtrair o ano de nascimento do ano atual, resultando na diferença entre os dois anos, que seria a idade da pessoa. Esse valor é devolvido e armazenado em nossa variável local de idade (mas esse cálculo assume que a pessoa já fez aniversário). Dentro do escopo onde a variável é definida, iniciamos uma avaliação para saber se a pessoa fez ou não aniversário.

2.4.8.6. Processando e Cobrindo Possibilidades com SENÃO-SE

Dentro de um bloco SE, comparamos se o mês atual é menor que o mês do nascimento (na escala de 1 a 12). Se for, a pessoa não fez aniversário ainda, então decrementamos a variável. Porém, perceba que se a pessoa estiver no mês de seu aniversário, o aplicativo informará o tempo de vida que ela teria como se já tivesse feito aniversário, independente do dia.

Note que, no entanto, se recorrermos simplesmente ao SENÃO, a ação tomada valerá tanto para um dia menor que o aniversário quanto para qualquer data depois do aniversário. Para lidar com esse caso em que precisamos de apenas um caso contido no restante dos cenários compreendidos pelo SENÃO, temos que modificar a estrutura do bloco SE usando o pequeno botão de engrenagem azul e adicionar um caso “SENÃO-SE”.

Essa adição cria um SE secundário que age sobre alguns dos resultados não admitidos pelo SE primário. A adição SENÃO-SE é

equivalente a incluir um bloco SE dentro de um SENÃO, serve para deixar o código mais enxuto e fácil de ler. Com essa extensão podemos fazer a condicional para verificar se o mês atual é o mesmo do aniversário.

Agora podemos adicionar nossa comparação de dias pela adição de um bloco SE no escopo do SENÃO-SE. Se o dia atual for menor que o dia do aniversário, então quer dizer que a pessoa ainda não fez aniversário, por isso temos que decrementar o valor da variável de idade.

Qualquer data além do aniversário não causará uma entrada nos blocos condicionais que resultam num decremento, logo o valor da variável fica intacto já que a pessoa já fez aniversário. Veja como essas condicionais são arranjadas na figura 52.

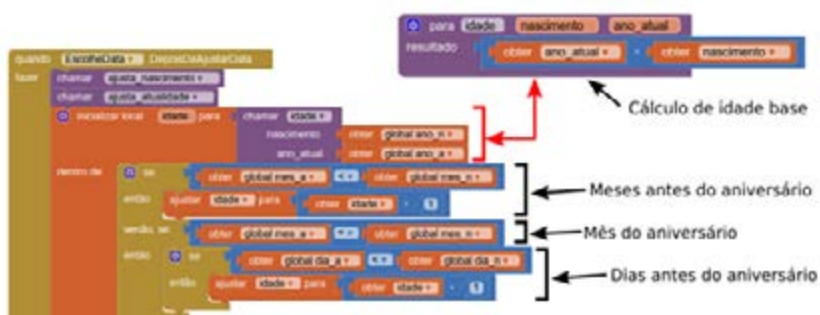


Figura 52: Critérios usados para determinar se o aniversário aconteceu.
(Fonte: App Inventor)

2.4.8.7. Finalizando

Agora que terminamos de determinar se o usuário já fez o aniversário e aplicamos os devidos decrementos, precisamos mostrar esse resultado na tela. Fazemos isso por chamar a função “ajusta_resultado”. Sua única responsabilidade é mudar o valor do texto de uma legenda informando o resultado do processamento da data.

Para validar o dado precisamos de uma condicional que veja se o valor é um número natural, afinal, o cálculo em nossa função “idade” admite valores negativos. Caso o valor da idade do usuário seja nega-

tivo mudamos o texto da legenda para “VOCÊ VOLTOU NO TEMPO??”, caso seja um número válido, ou seja, maior que zero, mostramos ele como o texto da legenda. Esse processo pode ser verificado na figura 53.

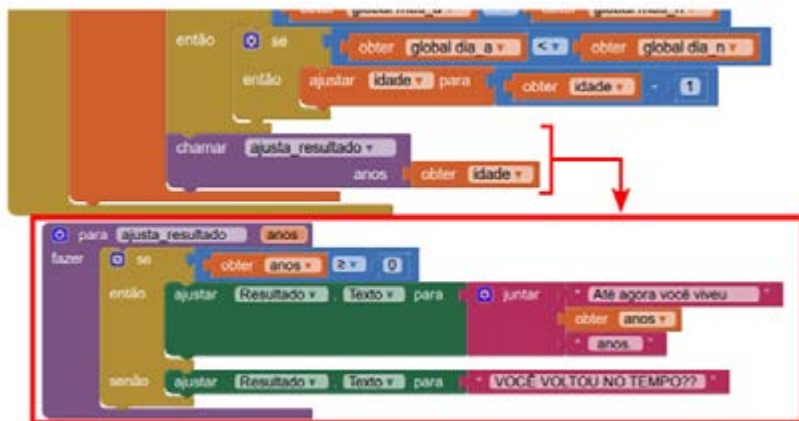


Figura 53: Procedimento que filtra valores de idade negativos.

(Fonte: App Inventor)

Feito isso, chegamos à conclusão de nossa calculadora de idade, sintase à vontade para tentar algo mais avançado como computar quantos meses e dias o usuário viveu desde o último aniversário!

2.4.9. Bloco de Notas

Chegamos a um ponto crucial de nosso aplicativo, essa é a utilidade que usa um dos componentes mais valiosos para nossos desenvolvimentos futuros. Você percebeu que, até agora, nenhuma das nossas utilidades continuavam com os dados nos componentes depois que as fechávamos e abríamos novamente? Isso se deve ao fato de que não fizemos uso de nenhuma técnica de armazenamento de dados permanente. Mudaremos isso a partir dessa utilidade.

Quando uma pessoa usa um bloco de notas para escrever uma nota, ela espera encontrar as notas no mesmo estado que as deixou ao terminar de editar o texto quando abrir o aplicativo novamente. Isso é exatamente o que faremos. Para essa utilidade, usaremos botões,

legendas, uma organização horizontal com rolagem, um campo de texto e, o mais importante componente para essa lição, um mini “**banco de dados**”¹⁵³, assim como um componente de conexão com arquivo.

Como o código dessa utilidade é consideravelmente extenso, vamos tentar abordá-lo de forma breve mantendo a riqueza dos detalhes importantes.

2.4.9.1. Aspecto Visual

Pode-se verificar na figura 54 como nossa utilidade deve se parecer.

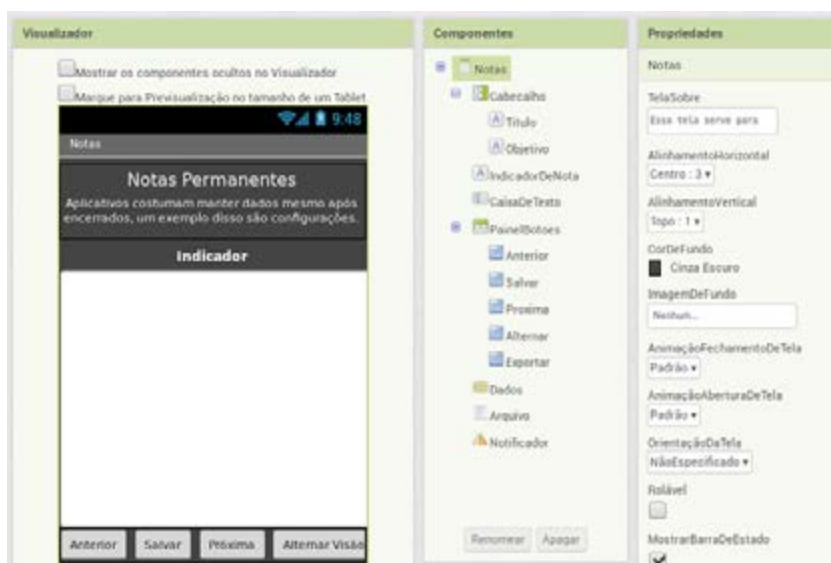


Figura 54: Aspecto visual da tela do bloco de notas.

(Fonte: App Inventor)

Como podemos ver na figura 54, primeiro temos uma legenda com o título da utilidade, depois uma pequena explicação sobre armazena-

¹⁵³ Bancos de dados são arquivos que possuem dados de diversos tipos organizados em tabelas que podem ter ou não relação entre si. Esses arquivos estruturados são manipulados por Sistemas de Gestão de Bancos de Dados (SGBD). O banco de dados mencionado no App Inventor não é bem um banco de dados, está mais para um depósito de variáveis que devem ser guardadas de forma persistente.

mento persistente e um caso de aplicação deste. A seguir temos outra legenda onde está escrito “Indicador”, mas essa legenda servirá para indicar qual das notas ele está editando no momento (nos limitaremos a 10 notas, mas sintam-se à vontade para modificar a mecânica para que seja possível guardar mais do que esse limite).

Como o objeto mais importante para o usuário, temos o campo de texto onde podem ser escritas as notas propriamente ditas. Abaixo do campo de texto se encontra a organização horizontal rolável que contém os botões que são capazes de:

1. Avançar para a próxima nota.
2. Voltar à nota anterior.
3. Salvar a nota atual.
4. Alternar a visibilidade das legendas na tela para que o usuário tenha mais espaço para ver o teclado e o campo de texto ao mesmo tempo (é muito útil quando a tela é pequena e o usuário gosta de digitar com o smartphone em modo paisagem).
5. Exportar o conteúdo da nota para um arquivo de texto no cartão SD do smartphone.

Repare na figura 54 que não somos capazes de enxergar o botão de exportação. Isso se dá porque ele está fora da área visível do visualizador, mas, ainda assim, ele se encontra após o botão de Alternar Visão. Quando executarmos o aplicativo, ao deslizarmos os botões, poderemos ver o botão de exportação. Esse é um pequeno aspecto problemático de trabalhar com organizações roláveis no App Inventor, o visualizador não permite seu deslize.

2.4.9.2. Cuidando da Lógica das Notas

Assim que abrirmos a tela desta utilidade, há uma série de coisas que precisam ser feitas para que ela se apresente completamente funcional ao usuário. Primeiro de tudo precisamos desativar todos os botões, e a caixa de texto também, para evitar que o usuário cause bugs

na tela (fazemos isso pelo procedimento “desativar_tudo”), ela não é totalmente utilizável à primeira vista (pode-se deixar os botões desativados por padrão na sessão de design também). Fazemos essas ações por meio do evento “Inicializar” presente no componente Tela.

Depois de desativarmos os botões, precisamos preparar a tela para receber as notas. Aqui as coisas podem ficar um pouco mais complicadas, mas chamamos um procedimento chamado “**inicializar**”¹⁵⁴ para criar os espaços de registro dentro do componente de banco de dados que usaremos. A propósito, o componente se chama “TinyDB” e se encontra na categoria de Armazenamento.

Assim que chamamos a função “inicializar” a primeira coisa que ela faz é chamar nosso banco de dados por meio do procedimento “ObterValor”. Esse procedimento precisa de dois argumentos em seus parâmetros: um nome de rótulo e um valor que será devolvido caso esse rótulo não exista no registro do banco. Em nosso caso, o rótulo se chama “iniciado”, usaremos esse rótulo para que o código possa saber se o espaço de registro das nossas notas já foi criado.

Se comprovarmos que o rótulo não existe comparando o valor devolvido pelo TinyDB com o valor lógico falso, então temos que chamar a função “criar_notas”. Essa função tem o objetivo de criar os espaços de registro de cada nota dentro do banco, na prática ela está declarando que certos rótulos existem para que possamos usá-los seguramente quando a tela estiver operacional. Veja na figura 55 como ficou a comparação do bloco SE.

154 Em programação, inicialização quer dizer ações que são tomadas quando uma coisa acontece pela primeira vez. No caso de variáveis, inicialização significa assimilar um valor inicial a ela no momento em que é declarada. No App Inventor isso pode ser visto tanto no bloco de declaração de variável local quanto no de variável global pelo fato desses blocos pedirem uma especificação de valor.

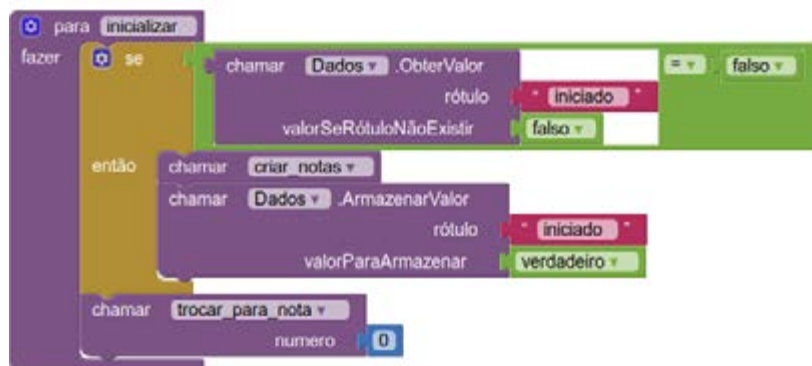


Figura 55: O procedimento “inicializar” detecta se o espaço para registro das notas já foi criado no TinyDB.

(Fonte: App Inventor)

Dentro de seu escopo, a função “criar_notas” define uma variável local chamada “contador”. Lembra de quando estudamos estruturas de repetição? Essa é a hora de colocá-las na prática. Precisamos abrir registros para dez notas e, com certeza, não seria vantajoso declarar a existência de cada um deles um por um. Sendo assim, percebemos que queremos fazer uma ação padronizada repetidas vezes, então vale mais apenas usar uma estrutura de repetição.

Dentro do escopo definido pela variável local se encontra uma estrutura de repetição chamada “ENQUANTO”¹⁵⁵ (esse bloco pode ser encontrado na categoria “Controle”), por sua vez, dentro do bloco ENQUANTO existe um procedimento do TinyDB chamado “ArmazenarValor”. Esse procedimento é parecido com o ObterValor, porém ele faz a operação contrária, em vez de ler, ele escreve um valor (presente no segundo parâmetro) dentro do banco e o associa a um certo rótulo que definimos no primeiro parâmetro.

O TinyDB só tem uma propriedade que pode ser editada pela Sessão de Design, ela se chama “Namespace”, caso você trabalhe com muitos registros, talvez seja necessário separá-los por categorias. Isso

¹⁵⁵ Essa estrutura de repetição está presente na maioria das linguagens de programação sendo chamada também de “while” conforme o termo em inglês.

pode ser feito por meio do Namespace. Se um TinyDB estiver usando um determinado Namespace, os registros serão lidos e escritos tendo como base aquele Namespace.

Vários objetos do componente TinyDB podem acessar um mesmo Namespace, conseguindo acesso aos mesmos conjuntos de registros. Mas é de suma importância que você tenha cuidado ao realizar operações simultâneas, isso pode gerar **corridas por dados**¹⁵⁶.

Prosseguindo com nosso bloco de notas, como não queremos que ele crie um registro e fique **sobrescrevendo ele até o fim do laço**¹⁵⁷, temos que adicionar algo na criação do rótulo que seja capaz de diferenciá-los uns dos outros, algo que mude a cada repetição.

Esse elemento diferenciador pode ser exatamente o contador de nossa estrutura de repetição. O contador é uma variável numérica que terá seu valor incrementado a cada repetição até atingir o mesmo valor da variável global “limite_notas”, sendo essa a nossa condição de parada que encerrará o laço. Para unir o nome do rótulo com a parte numérica variável, podemos usar o bloco de juntar texto tendo como argumentos a parte textual (opcional) e depois o valor do contador.

Note que o valor inicial do contador é zero, o valor da variável global limite_notas é dez, e a condição de parada é que o valor do contador não seja menor que a variável global limite_notas (enquanto ele for menor, o laço se manterá em repetição). Mas lembre-se de que, ao fim de cada repetição, o valor do contador precisa ser incrementado! Se isso não acontecer, o laço continuará sendo executado eternamente e poderá travar seu aplicativo quando ele estiver sendo executado. Veja a seguir na figura 56 como ficou nossa função “criar_notas”.

156 “Corrida por Dados”, “Condição de Corrida”, “Condição de Concorrência” ou “Data Races” são situações inesperadas em que um mesmo recurso de armazenamento é usado por mais de uma estrutura de código, onde uma fica refém da possibilidade de haver ou não modificação no conteúdo do armazenamento por parte da primeira estrutura que conseguir o acesso. Essas situações podem levar a comportamento inesperado, principalmente quando as estruturas de código não seguem alguma forma de sincronia.

157 Isso faria com que, ao fim do laço, só tivéssemos um registro criado.



Figura 56: O procedimento “criar_notas” cria os espaços de registro para as notas no TinyDB.

(Fonte: App Inventor)

Antes de prosseguirmos é importante informar que, para redefinir (ou zerar) todo o conteúdo do TinyDB, é preciso usar seu procedimento chamado “ApagarTudo”.

Feito isso, a função “criar_notas” atinge seu objetivo e voltamos ao bloco SE da função “inicializar”. Seguindo o caminho, agora precisamos informar ao nosso aplicativo (quando ele for aberto no futuro) que os registros para as notas já foram criados e ele não precisará fazer isso novamente. Então usamos mais uma vez o procedimento ArmazenarValor, mas agora, modificando o valor do rótulo “iniciado” para verdadeiro, revise a Figura 55 para compreender melhor essa explicação.

Com isso, a última coisa que nosso procedimento “inicializar” faz, depois de sair do bloco de SE, é chamar a função “trocar_para_nota” onde fornecemos o número da nota que queremos acessar e visualizar para edição no campo de texto. Antes de tudo, a função “trocar_para_nota” realiza uma comparação, verificando se o número da nota pedida está dentro dos limites permitidos, em nosso caso o limite vai de zero a dez.

Se o número da nota estiver dentro do limite, o bloco SE executa uma mudança do texto do campo de texto principal inserindo nele o texto armazenado pela referida nota no TinyDB. Se, por algum motivo,

o TinyDB não tiver o registro da nota pedida, a caixa de texto mostrará a mensagem “ERRO INTERNO”.

Depois de carregar o texto da nota no campo de texto, temos que mudar o número da nota na variável global “nota_atual”. Usaremos essa variável em outra parte do código. Então, ativamos todos os botões do painel inferior com a função “ativar_tudo” para que usuário possa criar suas notas. Assim que os botões são ativados, chamamos a função “avaliar_numero_nota” cujo objetivo é ver se a nota atual é a primeira ou a última da sequência.

Caso a nota atual seja a primeira da sequência, então temos que desativar o botão de nota anterior porque não há notas anteriores à primeira, caso seja a última, precisamos desativar o botão de próxima nota. Essas ações são desempenhadas pelas funções “primeira_nota” e “ultima_nota” como se segue na figura 57.



Figura 57: Procedimentos que bloqueiam a ação de certos botões para impedir ações fora do limite do aplicativo.

(Fonte: App Inventor)

Essas medidas servem para evitar que o usuário seja capaz de ultrapassar nosso limite e acabe resultando em erros no aplicativo como, por exemplo, a tentativa de acesso a um registro inexistente no TinyDB. Veja a função “trocar_para_nota” na figura 58.

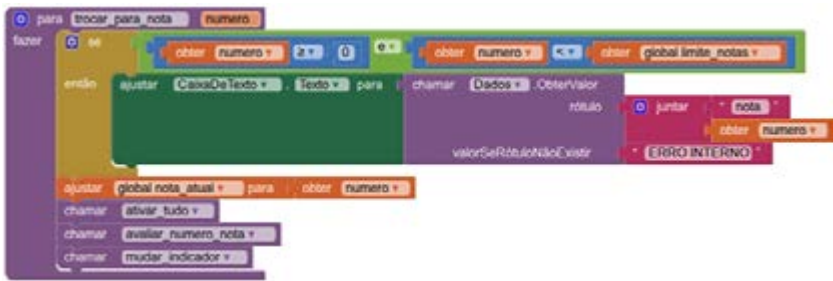


Figura 58: O procedimento “trocar_para_nota” verifica o número da nota e a carrega na caixa de texto se estiver no intervalo permitido.
(Fonte: App Inventor)

Por fim, a função “trocar_para_nota” executa uma última função chamada “mudar_indicador” cuja função é apenas mudar o texto da legenda que vimos mais cedo (cujo texto era “Indicador”). O texto mostrado nessa legenda deve informar ao usuário qual é a nota atual que está sendo editada. Veja na figura 59 como a “trocar_para_nota” se relaciona com as demais funções chamadas por ela.

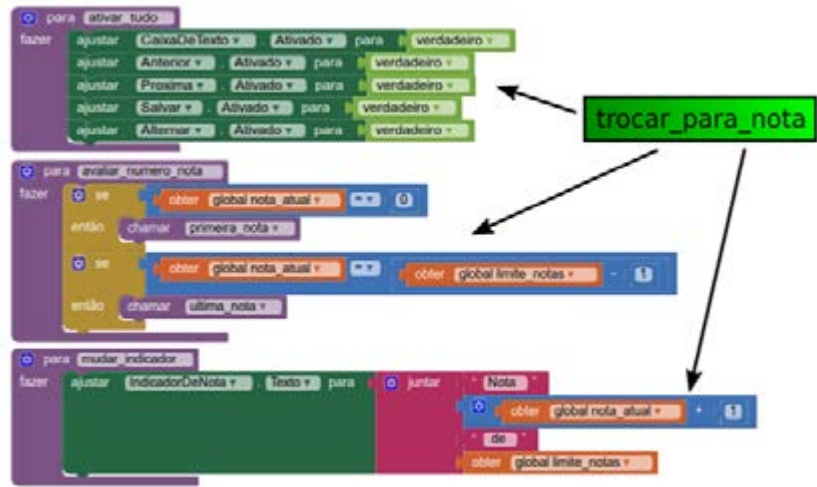


Figura 59: Relacionamento da função “trocar_para_nota” com outros procedimentos de regulação de funcionamento.
(Fonte: App Inventor)

Assim que todas essas ações forem tomadas, o usuário estará livre para edição e restringiremos nossas ações a lidar com eventos e o processamento de dados decorrente desse. Começemos pelos botões de próximo ou anterior. Quando o usuário os pressiona, precisamos trocar de nota. Isso é feito, mais detalhadamente, quando detectamos o evento de clique nestes botões.

Ao detectar o evento, a única coisa que precisamos fazer é chamar a função “trocar_para_nota” com o número da nota que precisamos. Em nosso caso, o valor da nota que queremos seria dado pelo decremento do valor da variável global “nota_atual” para o botão de nota anterior e o incremento do valor dela para o botão de próxima nota. Apenas isso precisa ser feito porque todas as ações referentes à troca da nota estão unificadas no procedimento “trocar_para_nota” (e subdivididas entre suas chamadas de procedimentos internos), o que facilita e agiliza nossa codificação.

Veja na figura 60 como ficaria o código que trataria o evento em ambos os botões de próxima nota e nota anterior.

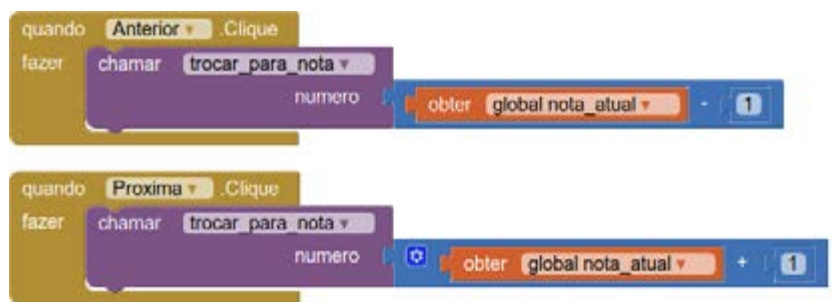


Figura 60: Eventos de botões que levam às notas anteriores ou próximas.

(Fonte: App Inventor)

Lembra-se de quando falamos do botão de “Alternar Visão”? Sua única função é trocar o estado de visibilidade das legendas de título e subtítulo da utilidade para que o usuário tenha mais espaço na tela para enxergar o que está digitando. Sendo assim a caixa de texto poderia se estender verticalmente pela tela devido à ausência desses conteúdos. Veja na figura 61 o código para o evento.



Figura 61: O botão de alternar visão torna o cabeçalho invisível, cedendo espaço à caixa de texto. O bloco “não” inverte um valor lógico: verdadeiro vira falso e vice-versa. (Fonte: App Inventor)

Tendo tratado da edição, controle de componentes e da recuperação dos dados, chegamos nos pontos cruciais dessa utilidade, quando o usuário pressionar o botão de salvar, a mágica ocorrerá. A função deste botão é gravar em nosso TinyDB o texto digitado na caixa de texto principal. O que precisamos fazer é interceptar o evento, como de praxe, e simplesmente chamar o procedimento “ArmazenarValor” de nosso banco para que ele guarde o valor do atributo Texto do componente Caixa de Texto sob o domínio do rótulo que se refere à nota editada no momento.

Repare que, entretanto, é preciso usar exatamente o mesmo padrão de rótulo usado nas funções “criar_notas” e “trocar_para_notas”, caso contrário, criaremos novos rótulos que nunca serão acessados por nosso aplicativo. Além disso, é preciso notar que o aplicativo não informa que o usuário fez alterações na nota desde o último salvamento, se ele sair da nota editada, sem pressionar o botão de salvar, o efeito desta ação será a perda dos dados inseridos no campo de texto (sinta-se à vontade para resolver esse problema!).

Assim que guardássemos o valor do texto da caixa, teríamos que avisar ao usuário que a nota foi salva. Fariamos isso por chamar o procedimento “MostrarAlerta” do componente Notificador apenas enviando o texto “Nota salva.” como argumento. No código, fizemos isso colocando esse procedimento dentro de outro chamado “notificar” já que usaremos ele para o último evento que trataremos nessa utilidade. Sendo assim, ambas as funções teriam uma ação centralizada em um procedimento só.

Caso, futuramente, precisemos que, junto de todas as notificações, sejam tomadas outras ações que sejam semelhantes numa avaliação caso a caso, podemos estender o código da função notificar para atender

a esta demanda. Isso implica nos princípios de manutenibilidade e escalabilidade que comentamos no início deste capítulo.

Além de tudo o que foi discutido, finalmente oferecemos ao usuário a capacidade de exportar uma nota para um arquivo no cartão de memória no celular. A ação tomada é praticamente a mesma, entretanto, em vez de utilizar o componente do TinyDB, finalmente usaremos o componente de Arquivo. Como abordaremos só um caso de uso deste componente, a tabela 12 mostra mais informações sobre ele.

Tabela 12: Eventos e Procedimentos do componente Arquivo.

Evento	Causa
DepoisDeSalvarArquivo	Término do salvamento do conteúdo no arquivo.
RecebeuTexto	A chamada do procedimento LerDe. O conteúdo do arquivo é entregue como argumento pelo parâmetro “texto”.
Procedimento	Efeito
AcrescentarAoArquivo	Mantém os conteúdos já gravados no arquivo e adiciona o novo conteúdo ao seu fim. Necessita do texto a ser gravado.
Apagar	Simplesmente exclui um arquivo do sistema de arquivos do Android.
LerDe	Copia os conteúdos do arquivo especificado para a memória principal do smartphone para poder ser usada no aplicativo. Essa função por si só não devolve nenhum valor, ela realiza a cópia do conteúdo do arquivo e posteriormente dispara o evento “RecebeuTexto” onde o parâmetro “texto” recebe o conteúdo do arquivo.
SalvarArquivo	Diferente do procedimento AcrescentarAoArquivo, esse procedimento cria um arquivo (ou sobrescreve um arquivo de mesmo nome) para guardar apenas o texto que recebe em seus parâmetros.

(Fonte: App Inventor)

O procedimento que mais nos interessa é o “SalvarArquivo”. Já temos todo o conteúdo do arquivo exposto na caixa de texto, o usuário edita o que precisa, adiciona o que quer, apaga o que não julga mais necessário, mas o que importa é que todo o conteúdo que o arquivo precisa ter está ali. Nada além. Por isso apenas fornecemos ao procedimento o texto da caixa e o nome do arquivo.

Agora entramos em um detalhe um tanto técnico. Sistemas de arquivos atuam organizando, rotulando, mensurando, gravando, lendo, alterando e apagando arquivos. Porém, como diz a primeira característica, o sistema de arquivos organiza arquivos. Quando você olha para seu computador de mesa ou acessa algum tipo de gerenciador de arquivos no smartphone, já percebeu que, por vezes os arquivos estão dentro de “pastas”¹⁵⁸? Veja na figura 62 um exemplo disso.

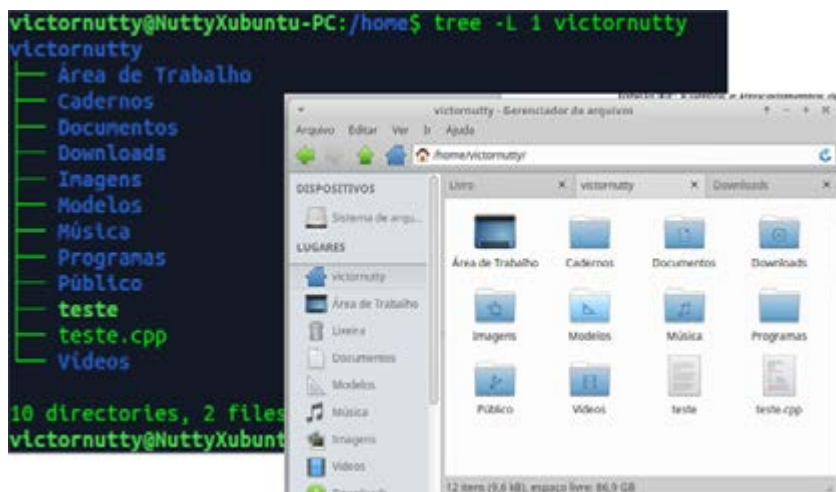


Figura 62: Estrutura de diretórios/pastas de um sistema operacional.
(Fonte: Xubuntu)

Perceba na figura que os arquivos e pastas estão dentro de uma pasta em comum chamada “victornutty” e parte dessas pastas internas também têm arquivos dentro de si, e assim por diante. Agora perceba que na janela do gerenciador de arquivos é possível ver uma barra de endereços onde está escrito “/home/victornutty”. Isso é um “caminho”¹⁵⁹.

158 Pastas são arquivos especiais cuja função é conter arquivos e outras pastas dentro de si. A nomenclatura oficial é “diretórios”, mas o termo pasta se popularizou devido ao uso ícones de pequenas pastas para representá-los.

159 Caminhos são textos que indicam uma sequência hierárquica de diretórios necessárias para se acessar um arquivo. Em sistemas Windows, o nome de cada diretório é separado por contrabarras (“\”), enquanto que em sistemas derivados do Unix, como o Linux, diretórios são separados por

Isso quer dizer que se quiséssemos abrir o arquivo “teste.cpp”, teríamos que especificar ao programa editor todo o caminho para se chegar ao arquivo começando do primeiro diretório e passando por todos os outros até chegar ao pretendido, somente aí informariamos o nome do arquivo para edição, o resultado seria algo como: “/home/victornutty/teste.cpp”.

Entretanto, por se tratar de um procedimento técnico que não precisa ser entendido pelo usuário final, isso é abstraído pelos desenvolvedores usando janelas, ícones, botões, caixas de texto etc., para que o usuário tenha uma noção visual de onde está indicando um arquivo para abertura ou salvando o mesmo.

Acontece que, como estamos desenvolvendo um aplicativo, temos que lidar com esses detalhes. A documentação do App Inventor faz algumas declarações e ressalvas quanto ao uso do componente Arquivo (<http://ai2.appinventor.mit.edu/reference/components/storage.html#File>).

Todos os procedimentos do componente Arquivo necessitam do nome de arquivo para leitura ou gravação, mas um esquema de caminhos existe para diferenciar se o arquivo a ser gravado será armazenado no cartão SD ou irá para a pasta interna do sistema relacionada ao aplicativo. Veja na tabela 13 como usar o esquema.

Tabela 13: Aplicação de nomes de arquivo e caminhos no componente Arquivo.

Nome de arquivo fornecido	Efeito
meu_arquivo.txt	O arquivo “meu_arquivo.txt” será guardado dentro da pasta interna do Android associada ao aplicativo.
/meu_arquivo.txt	A barra inicial informa ao componente Arquivo para gravar o arquivo “meu_arquivo.txt” no diretório “/sdcard” (cartão SD) do Android.
/pasta_generica/meu_arquivo.txt	Repare que o nome do arquivo é precedido por um caminho “/pasta_generica/” informando que o arquivo “meu_arquivo.txt” deve ser gravado dentro da pasta “pasta_generica” que está dentro da pasta “/sdcard” (Cartão SD) do Android.

(Fonte: App Inventor)

Assim que indicamos o nome do arquivo, adicionamos uma barra ao seu início para informar ao componente que queremos gravá-lo no cartão SD. O componente realizará a gravação e o usuário poderá conferir o arquivo usando seu gerenciador de arquivos. Veja como fica o código na figura 63 a respeito do botão de Salvar, Exportar para Arquivo e a função “notificar”.



Figura 63: Relações entre os botões, seus eventos e componentes usados nos processos de salvar e exportar notas.

(Fonte: App Inventor)

Antes de encerrar essa utilidade e partir para a última, é preciso ressaltar que o App Inventor faz algumas declarações quando ao uso do cartão SD do smartphone (<https://appinventor.mit.edu/explore/ai2/concepts/images-and-sounds.html>). “O App Inventor [...] não possui (ainda) uma maneira de listar os arquivos no cartão SD. Você terá que usar outros aplicativos, ou o gerenciador de arquivos do Android para fazer isso.” (tradução livre) informa a plataforma em sua documentação.

Usar o armazenamento do cartão SD pode ser vantajoso por dispor de maiores capacidades, porém ele está sujeito à simples remoção pelo usuário a qualquer momento. Se seu aplicativo depender criticamente de dados guardados nele, isso pode significar que seu aplicativo não

será funcional (ao menos, não plenamente) em um smartphone sem cartão SD.

Além disso, pode-se perceber que nessa utilidade, não demos opção para que o usuário pudesse escolher a pasta para onde a nota seria exportada, e nem oferecemos uma maneira de o usuário definir qual nome o arquivo teria. A primeira situação é explicada pela incapacidade do App Inventor de listar, de alguma forma, os arquivos e pastas presentes no cartão SD. O usuário teria que digitar por extenso o caminho e o nome de arquivo desejado.

Entretanto, lidar com caminhos e nomes de arquivos tem que ser algo exato, muitos caracteres não podem ser usados em nomes de arquivos porque, caso fossem usados, corromperiam o sistema de arquivos. Imagine usar barras num nome de arquivo como, por exemplo, no caso de uma data, o resultado seria: **“/sdcard/pastaqualquer/arquivo_28/03/2019.txt”**¹⁶⁰. Claramente isso resultaria em erros. Dar essa liberdade ao usuário final também concede a ele a possibilidade de causar danos ao sistema de arquivos.

Com isso encerramos o desenvolvimento da penúltima utilidade e seguimos para o desenvolvimento da última utilidade de nosso aplicativo. Continue firme nos estudos!

2.4.10. Relógio

Chegamos à última utilidade de nosso aplicativo. Nela, aprenderemos a lidar com temporizadores e eventos de tempo, assim como seus procedimentos, como lidar com instantes, durações, limitações do componente, o perigo de laços infinitos etc. Como foi explicado, nessa utilidade, ofereceremos três serviços relacionados ao tempo: um relógio, uma contagem regressiva e um cronômetro.

¹⁶⁰ Ao ser interpretado pelo sistema de arquivos esse caminho seria entendido como “acesse o arquivo 2019.txt que está dentro da pasta 03, dentro da pasta arquivo_28, dentro da pasta pastaqualquer, dentro da pasta sdcard” em vez de “acesse o arquivo arquivo_28/03/2019.txt dentro da pasta pastaqualquer, dentro da pasta sdcard”.

Para começar, é importante dizer que em vez de usar três telas para cada um destes serviços, vamos respeitar as limitações técnicas do App Inventor e simular três telas em uma por meio do intercâmbio das configurações de visibilidade dos componentes.

2.4.10.1. Aspecto Visual dos Relógios

De início, vamos inserir duas legendas como de costume, uma para o título da utilidade e outra para o subtítulo com uma explicação do que é proposto na tela. A seguir utilizaremos um componente novo chamado **“Lista Suspensa”**¹⁶¹. Esse componente se trata de um botão que, quando é pressionado pelo usuário, exhibe uma lista de itens em que ele pode escolher uma opção.

Após escolher essa opção, o botão a mantém como opção ativa. É por meio desse componente que o usuário decidirá qual serviço vai usar. Vejamos na tabela 14 algumas das propriedades deste componente.

Tabela 14: Propriedades do botão ListaSuspensa.

Propriedade	Efeito
CadeiaDeElementos	Define as opções mostradas pelo botão assim que pressionado, cada opção é separada da outra por uma vírgula e um espaço.
Largura	Define a largura da representação gráfica do botão.
Mensagem	Define uma mensagem de título que aparecerá junto à lista de opções.
Seleção	Define qual opção aparecerá selecionada assim que o botão for exibido na tela (o item precisa estar escrito exatamente igual a seu correspondente na propriedade CadeiaDeElementos).
Visível	Controla se o componente é ou não mostrado na tela.

(Fonte: App Inventor)

Assim que o usuário escolher o serviço a ser usado, os componentes de nossos três serviços devem ser ocultados ou mostrados

161 Esse componente também é conhecido como “Combo Box” na nomenclatura em inglês e, também, por “Caixa de Combinação” em português.

dependendo da escolha. Para tornar isso mais simples, usaremos três componentes de Organização Vertical, uma vez que ao aplicar a (in) visibilidade em um componente, o efeito é repercutido em todos os componentes englobados por ele.

Por padrão o relógio será o serviço mostrado na tela inicialmente, então a contagem regressiva e o cronômetro devem ser escondidos. Para que possamos vê-los a todo momento durante a edição, usaremos a opção acima do visualizador para mostrar todos os componentes independentes da configuração de visibilidade.

Começando pelo relógio simples, precisaremos nos envolver apenas com três legendas, o relógio não faz uma interação de entrada com o usuário, apenas realiza saída de dados informando as horas atuais. Por isso, veja como ele deve se parecer na figura 64.



Figura 64: Serviço de relógio simples da tela de relógios.

(Fonte: App Inventor)

Perceba na figura que nosso relógio mostra os textos “Hora e Data”, “Hora” e “Data”. Isso ficará assim apenas no editor para que possamos

saber com que tipo de dado a legenda lida. A primeira é apenas o título do serviço que o usuário está usando, “Hora” é onde colocaremos os valores de hora do temporizador convertidos em texto para a legenda e “Data” é onde colocaremos os valores de data fornecidos pelo mesmo componente.

A seguir, temos o cronômetro, ele é formado de duas legendas e dois botões. Das legendas, uma mostra o título do serviço e a outra mostra o tempo passado que foi contabilizado pelo cronômetro. Logo depois temos uma organização horizontal para alinhar os dois botões lado a lado, botões esses que são responsáveis por (o primeiro) iniciar e pausar/continuar a contagem do cronômetro e (o segundo), redefinir o tempo contado para iniciar uma nova cronometragem. Segue a tela na figura 65.

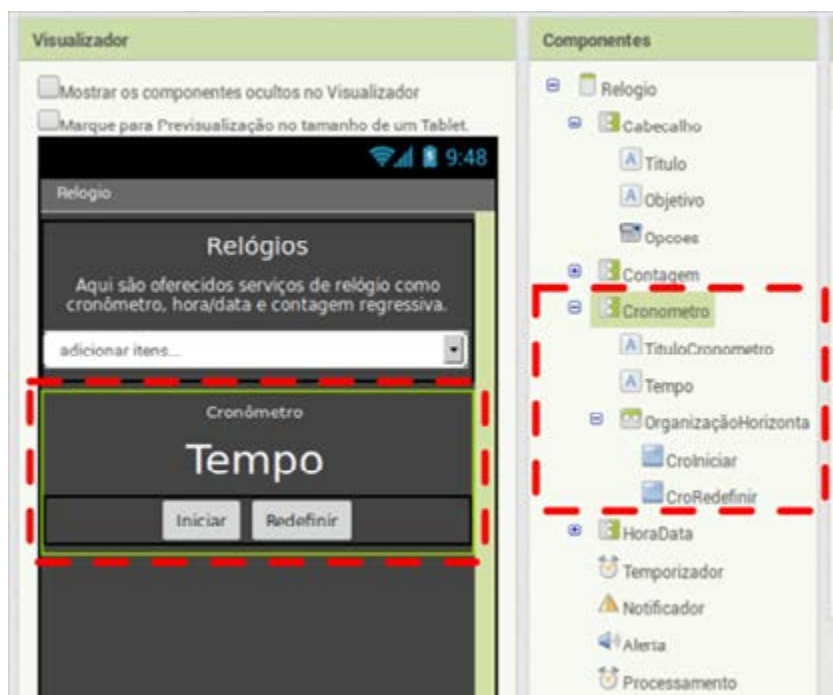


Figura 65: Serviço de cronômetro da tela de relógios.

(Fonte: App Inventor)

Evoluindo de nenhuma interação, passando por interação com botão, chegamos à interação com botões e um campo de texto com a

contagem regressiva. Diferente do cronômetro, antes de ser iniciada, uma contagem regressiva precisa de um tempo específico informado pelo usuário. Por isso, usaremos um campo de texto para conseguir esse dado dele.

Esse serviço é formado por uma legenda com seu título, um campo de texto onde o usuário inserirá o tempo de espera e, ao mesmo tempo, verá o valor ser diminuído e, além destes componentes, teremos também um botão para que o usuário informe ao aplicativo se quer iniciar ou parar a contagem. Segue a tela na figura 66.

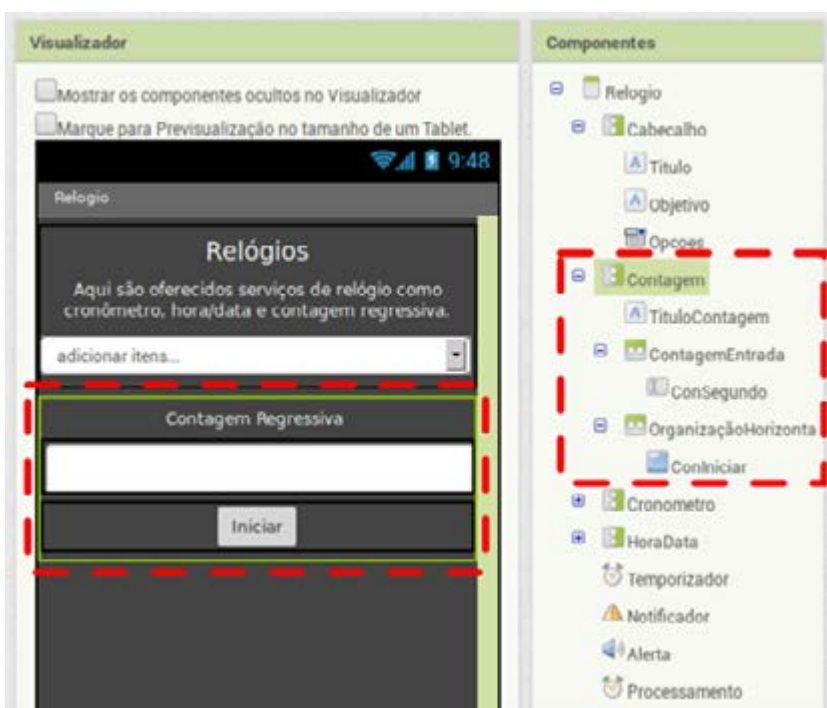


Figura 66: Serviço de contagem regressiva da tela de relógios.

(Fonte: App Inventor)

Agora que já temos as especificações, partamos para a parte mais detalhada dessa utilidade, a lógica e o uso dos blocos.

2.4.10.2. Editando os blocos do Relógio

Para iniciar, lembre-se que estamos fazendo uma ferramenta de atualização constante. Entretanto, perceba que usar uma estrutura de repetição infinita para manter um processamento contínuo de nossas funções relacionadas ao temporizador é uma péssima ideia. É como encher um filtro de água muito mais além da sua capacidade de vasão. A água começaria a esburrar para além do filtro, não sendo filtrada.

Estruturas de repetição infinitas causam problemas a partir do ponto em que elas tomam quase toda a atenção do processador de forma que ele não seja capaz de cuidar de outros aplicativos ou de si mesmo com a agilidade habitual. Um aplicativo que cai numa estrutura de repetição infinita pode causar travamentos ao smartphone em geral porque interfere no processamento de todos os outros aplicativos e até mesmo do sistema operacional do dispositivo.

Com isso, em mente, temos que achar um modo de fazer com que nosso código seja repetido, mas sem ser por meio de uma estrutura de repetição. Felizmente, podemos fazer isso por meio de um evento do temporizador! O temporizador é capaz de aceitar um valor temporal em milissegundos e causar eventos assim que esse tempo se esgota.

Além disso, esse disparo de evento pode ser único: acontecer apenas uma vez, ou contínuo: acontecer, e recontar o tempo indefinidas vezes para gerar os sucessivos eventos. Com isso em mente veja as seguintes propriedades do Temporizador na tabela 15.

Tabela 15: Propriedades do componente Temporizador.

Propriedade	Efeito
DisparosContínuos	Faz com que o temporizador conte o tempo de seu intervalo indefinidas vezes e, sempre que a contagem acaba, dispare um evento.
Ativado	Define se o temporizador deve continuar contando seu intervalo para gerar um evento, como um botão de liga/desliga.
Intervalo	Define o tempo (em milissegundos) que deve se passar entre um evento e outro do temporizador.

(Fonte: App Inventor)

Mas antes de lidar com os serviços dessa tela diretamente, precisamos nos preocupar em mostrar o serviço certo ao usuário por meio da escolha feita na caixa de combinação abordada no início deste tópico. Para isso, precisamos usar seu bloco de evento “DepoisDeSelecionar”. Perceba que o bloco de evento vem com um parâmetro chamado “seleção”, esse parâmetro terá como argumento um dos valores que definimos na propriedade de CadeiaDeElementos.

Tendo isso em mãos, temos que comparar dentro do escopo do bloco de evento da caixa de combinação qual foi a escolha do usuário, podemos fazer isso por meio de três blocos “SE’s” ou um bloco “SE” com uma adição de “SENÃO-SE” e um “SENÃO”, mas para manter a simplicidade, vamos apenas usar três “SE’s”.

Dentro de cada bloco SE faremos uma comparação entre um dos elementos que o usuário pode escolher e o argumento do parâmetro seleção, que é o que ele, de fato, escolheu. De acordo com a opção escolhida, alternaremos a visibilidade das Organizações Verticais que inserimos no aplicativo para agrupar os componentes de cada serviço, como é mostrado na figura 67.

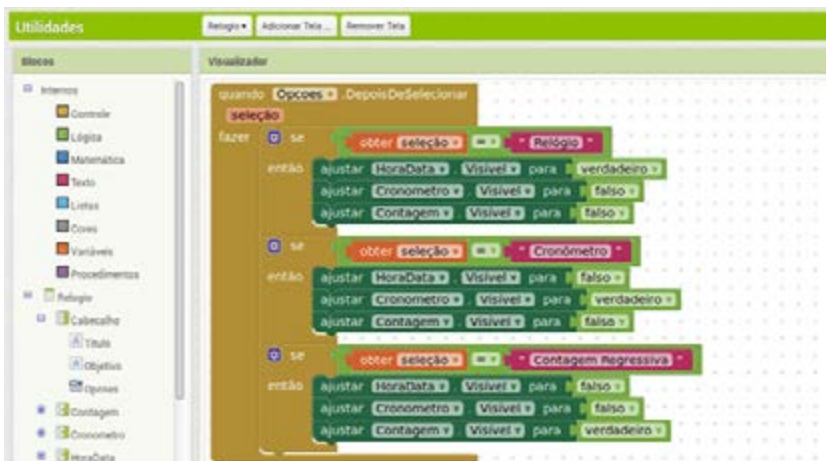


Figura 67: A opção selecionada no componente “ListaSuspensa” regula a visibilidade dos serviços.

(Fonte: App Inventor)

Depois que garantimos a aparição do serviço certo na tela, como manteremos todos os serviços sendo processados constantemente. Faremos isso por meio de um Temporizador que chamaremos de “Processamento”. O Processamento terá um intervalo de 5 centésimos de segundo (ou 50 milissegundos), mas lembre-se que, quanto menor o intervalo, mais lento o aplicativo ficará. Seus disparos serão contínuos, a cada vez que ele disparar um evento de fim de contagem de intervalo, chamaremos a função “processar”. Veja na figura 68 como ficará nosso código inicial.

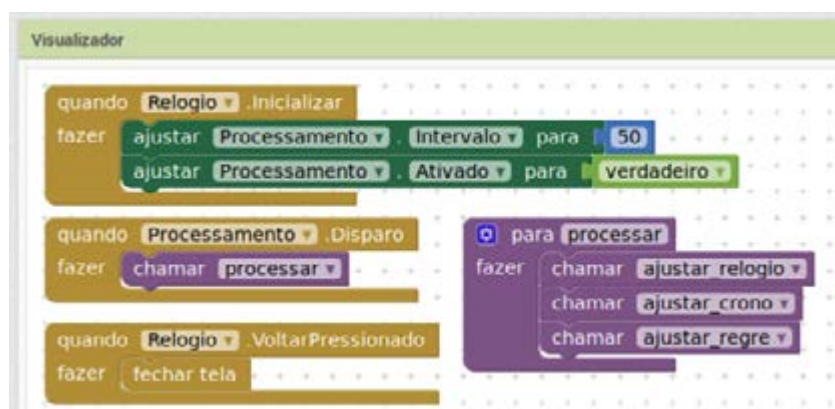


Figura 68: Usar um temporizador é uma boa forma de evitar estruturas de repetição infinitas.

(Fonte: App Inventor)

Por sua vez, a função “processar” chamará a função “ajustar_relogio” que tem como objetivo conseguir as horas e a data atuais do Android por meio do componente Temporizador e, então, atualizar a legenda onde tais informações aparecem. Entretanto, subdividiremos essas tarefas: a função “ajustar_relogio” só terá dois blocos principais de acesso ao atributo texto das legendas de hora e data, tendo como especificação de valor a chamada das funções “cria_relogio” e “cria_data”, respectivamente. Com isso, percebemos que estas últimas funções devem ser capazes de retornar um valor.

Ambas as funções são muito simples, dentro da função “cria_relogio” precisamos incluir um bloco de procedimento do temporizador,

o “FormatarHora”, esse bloco nos entrega um texto em formato de relógio pronto para exibição por meio da análise de um “instante” que discutiremos logo, o instante que precisamos é o instante que se refere ao agora e, não por coincidência, existe um bloco do temporizador que devolve exatamente esse instante, ele se chama “Agora”.

Prosseguindo para a função de “cria_data” as coisas ficam um pouco mais obscuras, embora o bloco que usaremos, o “FormatarData” também requira um instante (que em nosso caso também será o devolvido pelo bloco “Agora”), ele também requer uma especificação de formatação de data. Essa especificação é dada simplesmente por um texto, mas ele deve seguir algumas regras.

Segundo a documentação oficial do App Inventor a respeito do Temporizador (<http://ai2.appinventor.mit.edu/reference/components/sensors.html#Clock>), os “Métodos para converter um instante em texto” também estão disponíveis. Padrões aceitáveis são textos vazios, MM/dd/YYYY HH:mm:ss a, ou MMM d, yyyy HH:mm. Um texto vazio vai prover o formato padrão, que é ‘MMM d, yyyy HH:mm:ss a’ para FormatarDataHora ou ‘MMM d, yyyy’ para FormatarData.” (tradução livre).

Antes de encerrar a consideração, a página indica uma referência externa para o site da empresa Oracle onde estão arquivadas as documentações da linguagem Java. No link indicado (<https://docs.oracle.com/javase/7/docs/api/java/text/SimpleDateFormat.html>) pode ser encontrada uma tabela com todas as letras que formam todos os padrões de exibição de data e hora possíveis.

O esquema funciona da seguinte maneira: as letras e a quantidade de vezes que se repetem significam dados específicos, por exemplo, no caso das horas do dia: horas podem ser representadas de várias formas. Duas letras “H” maiúsculas significam a hora do dia indo de zero a 23 horas. Quatro letras “Y” minúsculas significam o ano, enquanto que apenas duas significam o ano em formato reduzido (apenas a dezena e a unidade).

Entretanto, precisamos entender um pouco melhor esse componente. Começando com os blocos, percebemos que podemos usar

muitos deles para diversas ações diferentes. Por enquanto focaremos em entender como um temporizador do App Inventor funciona.

2.4.10.2.1 O tempo é feito de Instantes

Um Temporizador do App Inventor trabalha predominantemente com a manipulação de instantes. Esse tipo de dado consiste em uma série de valores temporais que podem compreender segundos, minutos, horas, dias, meses, anos, dias da semana, nome do mês atual, milissegundos desde 1970 etc. Estes dados estão vinculados a um determinado ponto da linha do tempo.

Para acessar esses valores, o temporizador disponibiliza uma série de blocos de procedimentos voltados para atividades específicas envolvendo dados temporais. Veja na tabela 16 a função de cada procedimento.

Tabela 16: Blocos de procedimento do componente Temporizador.

Procedimento	Ação
Add (Adicionar)	Soma a quantidade pretendida da unidade de tempo indicada a um dado instante, retornando o instante com o novo tempo (podem ser usados anos, meses, semanas, dias, horas, minutos, segundos ou milissegundos (duração)).
Duração	Devolve quantos milissegundos se passaram entre dois instantes.
DurationTo[...] (DuraçãoPara)	Converte uma quantidade de milissegundos (duração) em um número específico da unidade de tempo especificada (há variantes deste procedimento para conversão de milissegundos em segundos, minutos, horas, dias e semanas, mas não para meses e anos por não serem unidades de tempo com medida fixa).
FormatarData, Formatar DataHora, FormatarHora	Pega uma certa hora e/ou data e coloca seus dados em formato texto. É preciso especificar um padrão via texto para que a data e/ou hora sejam mostradas no formato pretendido nos casos dos procedimentos "FormatarDataHora" e "FormatarData".

CriarInstante	Cria um instante a partir de uma data informada textualmente segundo o padrão definido pelo bloco: número do mês com dois dígitos (inclui zero à esquerda), barra, número do dia com dois dígitos (inclui zero à esquerda), barra, número do ano com quatro dígitos, espaço, hora do dia, minutos da hora e segundos do minuto separados por dois pontos (":") (os segundos são opcionais). Padrão: "MM/dd/aaaa hh:mm:ss" ou "MM/dd/aaaa hh:mm".
CriarInstanteAPartirDeMilissegundos	Cria um instante baseado numa quantidade de milissegundos a partir de 1970.
ObterMilissegundos	Obtém a quantidade de milissegundos passados até então desde 1970.
Segundo	Obtém o número do segundo de um instante.
Minuto	Obtém o número do minuto de um instante.
Hora	Obtém o número da hora de um instante.
DiaDoMês	Obtém o número do dia do mês de um instante.
DiaDaSemana	Obtém o número do dia da semana de um instante.
NomeDoDiaDaSemana	Obtém o nome do dia da semana baseado no número do dia do mês ou da semana de um instante.
Mês	Obtém o número do mês de um instante.
NomeDoMês	Obtém o nome do mês de um instante.
Ano	Obtém o número do ano de um instante.
Agora	Retorna o instante com todos os dados temporais do presente momento.
HoraDoSistema	Retorna o instante apontado pelo relógio interno do smartphone.

(Fonte: App Inventor)

Nos guiando por essas informações, precisamos entender que sempre que quisermos lidar com alguma informação temporal no App Inventor, teremos que lidar com instantes de forma direta ou indireta.

Poderíamos não depender dos procedimentos de formatar data e hora, mas isso nos exigiria mais esforço para reinventar a roda. Esse seria um caso em que usaríamos os acessos a dia, mês, ano, hora, minuto e segundo para formar o texto que mostrariamos em nossa

legenda. Poderíamos usar o bloco de junção de texto chamando cada procedimento para obter tais dados sob a aplicação do bloco “Agora” do temporizador para obter os dados temporais atuais.

Para finalizar esse serviço, veja como fica o código com os blocos finalizados na figura 69.

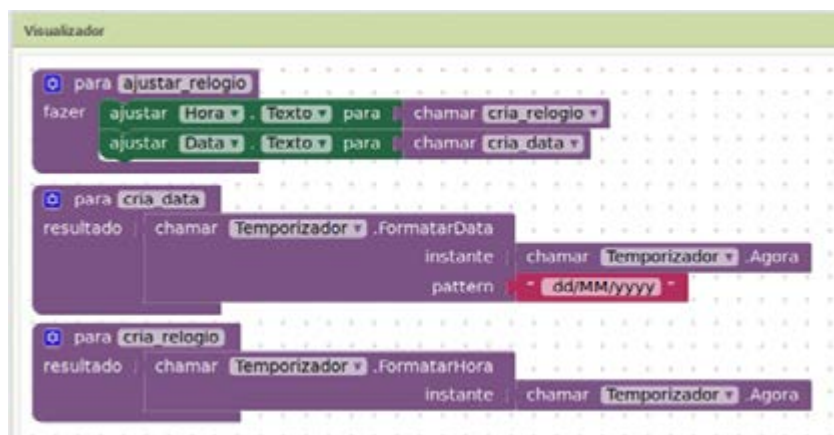


Figura 69: Com esses procedimentos podemos criar um relógio com data e hora em formato texto.

(Fonte: App Inventor)

2.4.10.2.2 Contagem regressiva para a interação

Contando os segundos para chegar ao zero, a contagem regressiva é um tanto mais complicada que o relógio. Dentre os procedimentos chamados pela nossa função “processar” está o procedimento “ajustar_regre” que cuida da contagem regressiva onde teremos nossa interação numérica com o usuário.

Esse procedimento tem um bloco SE que verifica se a variável global “regressiva” é verdadeira, isso define se a contagem regressiva será computada ou não. Mas esse não é o nosso caso, a variável só se torna verdadeira se o usuário pressionar o botão de iniciar contagem regressiva. Com isso, temos que focar no bloco que trata o clique deste botão. Veja na figura 70 o que acontece nesse evento.

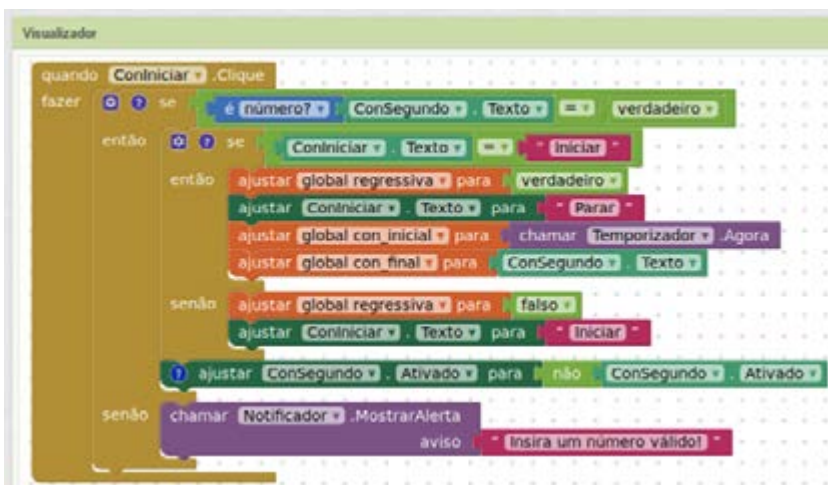


Figura 70: O botão “ConIniciar” dispara o evento que desbloqueia a contagem regressiva. (Fonte: App Inventor)

Analisando as informações da figura 70, podemos perceber que o botão só permite a mudança do valor da variável se houver um número válido dentro do campo de texto onde o usuário digita o número de segundos para a contagem (seguindo a lógica do primeiro bloco SE). A fim de diminuir a possibilidade de erros e simplificar as coisas, configuramos o campo de texto na sessão de design para aceitar apenas números, aceitando apenas o valor de segundos em nossa contagem.

Se não tivermos um número válido dentro do campo de texto, então o que precisamos fazer é usar um procedimento do componente Notificador para mandar uma mensagem ao usuário por meio do procedimento “MostrarAlerta” informando que ele precisa fornecer um número válido para que nosso serviço funcione corretamente.

Mas se tivermos um número válido dentro do campo de texto, então a primeira coisa que fazemos é verificar qual é o estado atual de nosso botão, porque ele servirá tanto para iniciar quanto para parar a contagem regressiva, no caso, fazemos isso por meio da verificação do texto do botão, se seu texto for “Iniciar”, isso significa que podemos

trocar o valor da variável global “regressiva” para verdadeiro e ativar a computação da contagem regressiva.

2.4.10.2.3 Desbloqueando o processamento da função

Em seguida, trocamos o texto do botão para “Parar” a fim mudar seu estado para executar outro código da próxima vez que for pressionado. Atribuímos um instante à variável global “con_inicial” para que possamos calcular quanto tempo a nossa contagem regressiva está contabilizando. Por fim, armazenamos o valor do texto do campo de texto (valor inserido pelo usuário) numa variável global chamada “con_final”. Assim que saímos do bloco SE que realiza a troca de estado do botão, trocamos o estado da propriedade “Ativado” do campo de texto a fim de permitir ou não a edição do texto (nesse caso, a proibimos).

Agora que a variável global “regressiva” está com o valor verdadeiro, toda vez que a função “processar” chamar a função “ajustar_regre” nossa contagem regressiva será processada porque seu código foi desbloqueado pela variável. Veja o código desta função na figura 71.

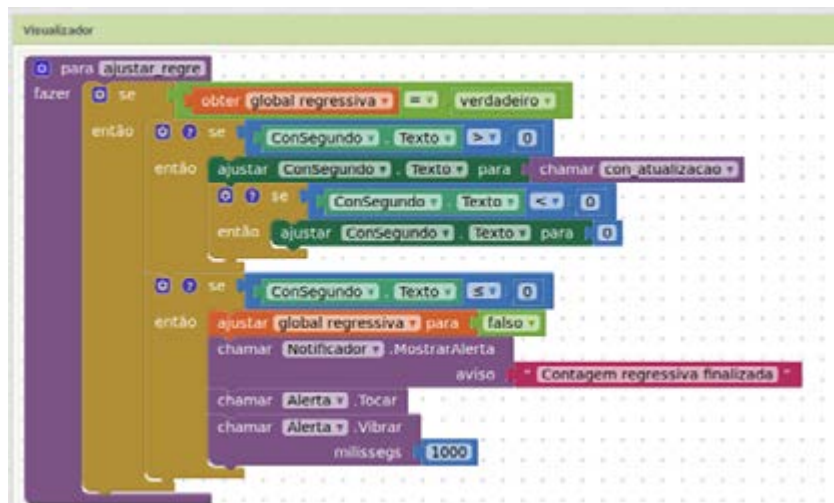


Figura 71: A função “ajustar_regre” realiza o processamento da contagem regressiva. (Fonte: App Inventor)

Dentro do SE que define o processamento da contagem, temos outros dois blocos SE. O primeiro deles trata das ações referentes ao tempo antes do fim da contagem quando o valor de tempo dentro do campo de texto ainda é maior que zero. Dentro desse bloco nós temos uma atualização do valor do campo de texto para o resultado da chamada da função “con_atualizacao” cujo objetivo é calcular quanto tempo se passou desde o último processamento da contagem regressiva e subtraí-lo do valor da variável “con_final” que tem o valor original do campo de texto.

Fazemos isso por meio daquele instante que gravamos na variável “con_inicial”. A função “con_atualizacao” repassa os valores para a função “tempo_passado” que usa um bloco de “Duração” do temporizador a fim de descobrir a diferença em milissegundos entre o instante inicial e o instante final, depois disso, ela divide por mil o resultado devolvido pelo “Duração” a fim de converter o valor para segundos, que é a unidade de medida de tempo usada pelo campo de texto.

De volta à função “con_atualizacao”, subtraímos o resultado que obtivemos do valor presente na “con_final” e retornamos esse valor à função “ajustar_regre” que coloca o novo valor no campo de texto, apenas esperando pelo próximo processamento. Veja essas funções extras na figura 72.

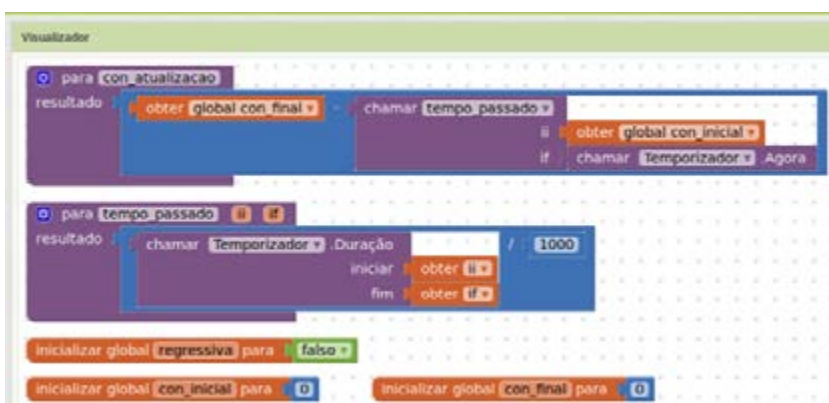


Figura 72: Estes são os procedimentos que auxiliam a função “ajustar_regre”.
(Fonte: App Inventor)

Assim que retornamos à nossa função principal deste serviço, podemos continuar nosso código. A próxima ação é verificar se o resultado presente no campo de texto ainda é maior que zero. Você pode estar se perguntando por que precisamos disso se o SE de ordem maior verifica exatamente o contrário. Acontece que mudamos o valor do campo de texto dentro deste SE e isso admite que o campo fique com um resultado negativo já que nossos laços baseados em temporizador não são tão constantes assim para conceder um resultado preciso.

2.4.10.2.4 Imprecisão do Disparo do Temporizador

É digno de nota que o temporizador do App Inventor não é perfeito. Isso é algo notável quando testamos a real duração de uma sequência de eventos disparados pelos fins de intervalos, o temporizador parece ser **mais lento do que deveria**¹⁶². Isso se deve a uma somatória de fatores. O primeiro deles é que nosso aplicativo é um software interpretado que perde desempenho por ter que ser traduzido em tempo real para código binário de máquina nativo.

O segundo fator se deve ao fato de a plataforma mobile ser, na maioria das vezes, menos potente e mais compacta do que de computadores de mesa ou desktops. Isso pode influenciar nosso aplicativo para o bem ou para o mal, smartphones mais modestos podem ter resultados piores do que smartphones com melhores especificações quanto às taxas de atualização do temporizador.

O terceiro fator se deve ao fato de estarmos executando nosso aplicativo em um ambiente multitarefa. Pelo fato de rodar em meio a outros aplicativos, o nosso aplicativo precisa competir com os outros pela atenção do processador que não é constante.

O quarto fator se deve ao fato de que a atualização dos valores de nossa função “processar” não é imediata, o próprio código demora certo tempo para ser executado e o disparo do temporizador de Proces-

¹⁶² Nessa situação, imagine que um Temporizador tem intervalos de 100 milissegundos. Cinco eventos de disparo deste temporizador, teoricamente, deveriam significar que exatamente 500 milissegundos se passaram, mas, na verdade, o tempo que se passou é sempre maior.

samento não é constante, esse aspecto leva justamente ao quinto fator: o evento de disparo não acontece exatamente de forma fixa sempre que o tempo de intervalo que definimos acaba.

Conforme foi reportado por vários usuários nos fóruns do App Inventor...

- <https://groups.google.com/forum/#!topic/mitappinventortest/J00bFHQ4hgg>
- <https://groups.google.com/forum/#!topic/mitappinventortest/wZFwp92bnL0>
- <https://groups.google.com/forum/#!topic/mitappinventortest/SqBsOzxFjGo>

... o componente de Temporizador do App Inventor é útil e trabalha constantemente, lida perfeitamente bem com dados temporais estáticos, mas **deixa a desejar quando se trata do evento de disparo**¹⁶³ e da contagem do tempo passado no intervalo.

Em nossa utilidade, inicialmente faríamos com que, tanto a contagem regressiva quanto o cronômetro, esperassem o evento de disparo para somar a quantidade de tempo do intervalo para contabilizar o tempo passado. Entretanto, esse método levaria a um erro acumulativo de contagem, a imprecisão do temporizador levaria ao retardamento da contagem, fazendo a contabilização do tempo passado ser mais lenta até que a diferença entre o tempo passado e o tempo contado fosse amplamente perceptível.

Como as soluções nos fóruns reportam, depender de intervalos pequenos para o Temporizador pode ser realmente uma medida desastrosa, em vez disso, é melhor recorrer aos procedimentos que lidam com durações ou milissegundos, ou mesmo usar a função “Agora” para calcular o tempo passado já que esses procedimentos se utilizam do relógio do sistema que é muito mais preciso que o evento de disparo.

¹⁶³ Se o evento de disparo não precisar executar exatamente nenhum código, então sua precisão é garantida.

2.4.10.2.5 Prosseguindo com o desenvolvimento

Somos capazes de informar um valor em segundos ao nosso aplicativo e podemos dar início à contagem regressiva, podemos checar o número de segundos sendo diminuído pouco a pouco pelo campo de texto conforme o tempo passa, mas você pode estar se perguntando como funcionaria o botão de parar.

Quando apertamos o botão de iniciar, o próprio botão muda seu estado e, conseqüentemente, o código que executa quando pressionado novamente. Retornando ao evento que trata do clique em nosso botão, temos que focar no bloco SE que compara se o estado do botão é o de “Iniciar”. Segundo a figura que já vimos sobre esse bloco de eventos, o código do botão quando ele se encontra no estado de “Parar” está na especificação “SENÃO” deste SE.

Ali dentro podemos perceber apenas duas instruções, bloquear o processamento da contagem regressiva por meio da troca de valor da variável “regressiva” (efeito de pausa) e a mudança do texto do botão para “Iniciar” (troca de estado). Simples assim, o botão de parar só precisa trancar a função “ajustar_regre”, todo resto é feito pelo resto do código.

Independente do estado do botão, a troca de estado de edição do campo de texto será realizada. Além disso, quando o usuário apertar o botão de iniciar novamente sem mudar valor, o campo de texto continuará com o valor onde a contagem foi pausada. Por sua vez, esse valor será o novo valor limite que inseriremos na variável “con_final”, é como começar uma nova contagem regressiva, mas com o último valor antes da pausa.

Por fim, quando a contagem termina, o campo de texto terá o valor zero dentro de si e o processamento será redirecionado para o segundo SE interno da função “ajustar_regre”. Dentro deste bloco, o que fazemos é simplesmente bloquear o processamento da própria função (já que a contagem acabou), avisar o usuário sobre o fim da contagem por meio de uma pequena notificação com o procedimento “MostrarAlerta” do componente Notificador e tocar um alerta sonoro usando um compo-

nente do tipo Som. Antes de terminarmos esse serviço de nossa utilidade, precisamos considerar dois componentes que serão úteis para nós.

2.4.10.2.6 O Componente Som e o Componente Tocador

Como esse livro terá um enfoque especial em ferramentas de acessibilidade, fazer uso de componentes que emitam informações sonoras é um ponto especial de nosso estudo (afetando pessoas que, por algum motivo, dependem mais da audição para entender o mundo à sua volta). Sendo assim, estudaremos esses dois componentes da categoria de mídia para que sejamos capazes de tirar proveito da capacidade sonora do smartphone.

Primeiro, precisamos conceituar estes dois elementos. Embora ambos tenham a capacidade de reproduzir um arquivo de áudio enviado ao App Inventor pelo quadro de mídia logo abaixo da árvore de componentes adicionados (na sessão de design), cada um tem uma melhor situação a que se aplica. O componente som é melhor para executar curtos avisos sonoros. O componente Tocador é melhor para tocar grandes fluxos de áudio como músicas, leituras etc. O quadro de mídia e a seleção de uma mídia para execução são mostrados na figura 73.



Figura 73: É possível enviar arquivo de áudio, imagem e texto para o App Inventor e usá-lo em componentes. Esses arquivos são empacotados junto à APK.

(Fonte: App Inventor)

O componente de Som tem apenas duas propriedades: Intervalo-Mínimo e Fonte. A primeira propriedade é onde definimos qual o tempo de silêncio do componente entre os avisos sonoros. Enquanto este tempo, especificado em milissegundos, não passar, todas as chamadas ao procedimento “Tocar” são ignoradas (isso evita que o alerta seja tocado constantemente).

É digno de nota que o componente Som possui um bloco de evento, mas este se tornou obsoleto. Se trata de um bloco cuja única função é detectar se um erro aconteceu ao tentar tocar um áudio. A própria dica de uso do bloco sugere o bloco de evento “OcorreuErro” presente no componente de Tela. Mas para além deste bloco de evento, podemos ver na tabela 17, quais são os procedimentos disponíveis para uso nesse componente.

Tabela 17: Procedimentos do componente Som.

Procedimento	Efeito
Tocar	Toca o arquivo de áudio fornecido por meio do atributo “Fonte”.
Pausar	Para momentaneamente a reprodução do arquivo de áudio especificado no atributo “Fonte”, até que o procedimento “Retomar” seja executado.
Retomar	Encerra a pausa na reprodução do som imposta pelo procedimento “Pausar”, reproduzindo o áudio imediatamente do ponto em que foi pausado.
Parar	Para definitivamente a reprodução do áudio. Não é possível retocá-lo a partir da posição de parada, o arquivo é tocado a partir do início.
Vibrar	Faz o smartphone vibrar durante um dado número de milissegundos.

(Fonte: App Inventor)

Agora que estudamos o componente de som, prossigamos para o componente Tocador. Começando com suas propriedades, o componente Tocador pode executar um arquivo de áudio da mesma forma que o Som, mas é mais bem aproveitado para situações em que é preciso reproduzir grandes fluxos de áudio, como músicas, por exemplo. Veja na tabela 18 quais são as propriedades deste componente.

Tabela 18: Propriedades do componente Tocador.

Propriedade	Objetivo
Repetir	Fazer com que o arquivo de áudio seja tocado vez após vez automaticamente.
TocarSomenteEmPrimeiroPlano	Quando desativada, faz com que o tocador continue reproduzindo o áudio mesmo que a tela onde ele se encontra não esteja sendo mostrada no momento.
Fonte	Especificar qual arquivo de áudio deve ser reproduzido.
Volume	Especificar qual é a amplitude que o som deve ser reproduzido.

(Fonte: App Inventor)

Mas apenas saber as propriedades não faz todo o truque de mágica acontecer, então prossigamos para os eventos que podem ser disparados por esse componente. Cada um deles e sua causa podem ser checados na tabela 19.

Tabela 19: Eventos lançados pelo componente Tocador.

Evento	Causa
Completo	Disparado sempre que o Tocador termina a reprodução do arquivo de áudio atual.
OutroTocadorIniciado	Disparado sempre que outro tocador começa a emitir áudio, útil para quando é necessário que apenas um Tocador possa reproduzir áudio.
PlayerError	Disparado sempre que acontece um erro ao tentar reproduzir um arquivo de áudio.

(Fonte: App Inventor)

Tendo em vista o conhecimento destes eventos, agora só precisamos tomar partido em saber quais são e para que servem os procedimentos do componente de Tocador. Diferentemente do que se pode esperar, tanto o Tocador quanto o Som são componentes com a

mesma funcionalidade, mas voltados para ocasiões diferentes, por isso os procedimentos de ambos são os mesmos.

Porém é importante notar que até agora dissemos o termo “arquivo de áudio” repetidas vezes até agora, isso traz à tona um tópico que precisamos discutir a respeito de arquivos e como isso afeta a forma como podemos usar estes componentes de Som e Tocador.

2.4.10.2.7 Formatos de Arquivo

Antes de enviar arquivos para o App Inventor, é preciso discutir um tópico relacionado especificamente a arquivos, e não ao desenvolvimento de aplicativos. Arquivos são formas de armazenar informação em computadores, mas isso é uma abordagem muito genérica, informações podem ser retratadas de inúmeras formas.

Para computadores, não faz sentido encarar uma melodia de instrumento musical como palavras de uma prosa, assim como, de forma simples, para nós também não faz. Arquivos de áudio são diferentes de arquivos de imagem, que são diferentes de arquivos de texto, que são diferentes de arquivos de vídeo, que são diferentes de arquivos de bancos de dados e assim sucessivamente.

Arquivos de um determinado tipo têm sua própria estrutura interna e forma de retratar os dados necessários, às vezes até mesmo usando de técnicas de **compressão**¹⁶⁴. Arquivos de imagem populares e que podem ser usados no App Inventor através do componente Imagem são:

- Portable Network Graphics (PNG) usado para imagens (geralmente com texto, ícones, elementos abstratos, ilustrações etc.)

¹⁶⁴ Comprimir ou compactar um arquivo significa dizer que os dados que precisam ser guardados são submetidos a um processo de análise e codificação para possam ser retratados com um número menor de dados, mas que seja equivalente ao volume de dados inicial depois de passar pelo processo reverso, a descompactação ou descompressão. Note que dissemos “equivalente”, alguns métodos de compressão e tipos de arquivo permitem a perda de dados a cada compressão, reduzindo bastante a quantidade de dados (é como tirar uma cópia da cópia da cópia... da cópia da informação original), enquanto que outros métodos e tipos de arquivos não permitem tal perda, mantendo a fidelidade ao conteúdo original, mas não sendo tão efetivos na redução de dados.

onde os dados visuais podem ser comprimidos, mas devem ser total e fielmente obtidos mesmo depois da descompressão.

- Joint Photographic Experts Group (JPEG/JPG) usado para imagens, geralmente fotografias, onde o volume de dados e quantidade de cores é muito grande. Este formato usa bastante compressão abrindo porta para uma significativa perda de dados em prol da economia de memória.
- Graphics Interchange Format (GIF) usado geralmente para imagens com animações, o GIF tem menos possibilidades de cores e costuma crescer muito em memória conforme a quantidade total de pixels aumenta. Embora imagens deste formato possam ser usadas no App Inventor e cheguem a ser mostradas no aplicativo rodando em Android, as animações geralmente não funcionam através do componente Imagem, conforme reportado várias vezes nos fóruns oficiais do App Inventor por alguns usuários (<https://groups.google.com/forum/#!topic/mitappinventortest/r5iKWCCe160>). É possível executar as animações por meio do componente NavegadorWeb.

Aproveitando esta discussão, o componente de Imagem é muito simples de se lidar, ele não possui blocos de eventos e nem de procedimentos, apenas de atributos. Por conseguinte, além de visibilidade, altura e largura (os quais já discutimos várias vezes), esses atributos são mostrados conforme a tabela 20.

Tabela 20: Propriedades do componente Imagem.

Propriedade	Objetivo
Imagem	Especifica qual arquivo de imagem deve ser exibido na tela do smartphone, este arquivo precisa ser enviado através do quadro de mídia.
RotationAngle	Define o quanto a imagem estará rotacionada (em graus) ao ser exibida na tela.

RedimensionarParaCaber	Na maioria das vezes uma imagem é maior ou menor que a tela do smartphone, nesse caso, essa opção serve para aumentar ou diminuir a imagem de forma que ela apareça na totalidade da tela em, pelo menos, uma das dimensões.
------------------------	--

(Fonte: App Inventor)

Seguindo com nosso tópico, arquivos de áudio também têm sua própria estrutura e forma de guardar informação. Enquanto imagens lidam com pixels, cores etc., arquivos de áudio lidam com ondas sonoras, canais de áudio, fluxo de som **stereo**¹⁶⁵ etc. Arquivos de áudio populares incluem:

- MPEG-1/2 Layer 3 (MP3) usado principalmente para codificação de músicas com boas taxas de economia de memória quando sob uso de compressão.
- Vorbis (OGG/OGA) muito usado na produção de jogos e softwares por conseguir realizar ótimas taxas de compressão sem perder muita qualidade e ainda ser um formato de arquivo livre.
- Microsoft Wave (WAV) usado para áudios gravados com fidelidade total, esses arquivos têm tamanhos gigantescos por não terem tipo algum de compressão, os dados sonoros estão presentes na íntegra, inclusive as frequências não detectáveis pela audição humana.

Existem milhares de tipos de arquivos e cada um deles é trabalhado por seus próprios softwares, alguns são mais populares, outros não, outros viraram padrões internacionais e são trabalhados por múltiplos softwares, ainda outros são quase desconhecidos e trabalhados apenas pelo software que definiu seu padrão de escrita e leitura.

¹⁶⁵ Dizer que um som é stereo significa dizer que ele pode ser tocado de formas diferentes por dois alto-falantes distintos ligados na mesma fonte do áudio.

Mas o que isso tem a ver com o App Inventor? Há dois pontos centrais que essa discussão chegará. O primeiro deles é que, por criar seus próprios aplicativos, você também terá o poder de escrever e ler seus próprios arquivos, assim como fizemos parcialmente na utilidade do bloco de notas. O arquivo que escrevemos era um simples arquivo de texto (TXT) conhecido quase que em todo mundo, capaz de armazenar alguns caracteres como símbolos, letras e números apenas.

De fato, muitos formatos de arquivo são derivados do formato TXT. Arquivos de códigos-fonte, arquivos **Collada**¹⁶⁶ de objetos 3D, arquivos de imagens vetoriais SVG e outros são todos arquivos de texto mas com **extensões**¹⁶⁷ diferentes.

Encare isso como um convite, derive seu próprio tipo de arquivo se os que já existem não suprem suas necessidades. Apenas como curiosidade, nem todos os arquivos são arquivos de texto, alguns são arquivos binários. Programas são arquivos binários, arquivos compactados também são, as APK's do Android também são, imagens baseadas em pixels e músicas também são. Estes arquivos são escritos, não passando textos simples para o arquivo, mas sim fluxos de valores lógicos que constroem o arquivo bit a bit diretamente em código binário.

Mas o ápice deste tópico se encontra na ideia de que, para ser capaz de abrir um arquivo e conseguir ler seu conteúdo, é preciso possuir um software que seja capaz de entender tal informação estruturada. Caso não possua, o usuário desperdiçará muito mais tempo e esforço tentando entender as informações por conta própria. Esse problema é um fato que ocorre na plataforma mobile, e é especialmente notável quando envolve arquivos de áudio.

166 Arquivos Collada (DAE, <https://www.khronos.org/collada/>) são arquivos de texto dotados de uma estrutura capaz de representar objetos tridimensionais (3D) em um computador. Imagens formadas de pixels possuem apenas duas dimensões: altura e largura. Objetos 3D também possuem essas dimensões, entretanto, acrescidas de uma terceira: comprimento. Enquanto uma imagem 2D poderia ser comparada com uma foto de uma maçã, um objeto 3D seria a própria maçã.

167 Todo arquivo possui um nome, para facilitar a identificação de seu formato, após o nome ele possui um ponto e uma sequência de caracteres que especifica seu formato. Pode haver arquivos sem extensão, mas isso não significa que ele não tenha um formato específico, além disso, mudar a extensão não converte um arquivo de um formato para outro.

Arquivos de áudio precisam de programas especiais chamados “**codecs**”¹⁶⁸ para serem lidos e executar os sons que eles contêm. Entretanto, por várias questões de patentes, economia de memória, previsão de baixa usabilidade etc., nem todos os codecs são incluídos nos sistemas operacionais. Isso acontece no Android porque a plataforma mobile é bastante limitada em sua esmagadora maioria (mas esse cenário pode mudar com o tempo).

Embora o App Inventor seja capaz de aceitar todos os formatos de áudio ou imagem, isso não significa que o Android seja e, consequentemente, o que o Android não aceita limita indiretamente a capacidade do App Inventor. Então se o Android não puder executar arquivos de áudio no formato XM, não adianta incluí-los no aplicativo esperando que eles sejam tocados.

Mesmo assim, a falta de codecs pode ser enfrentada pelos próprios aplicativos se eles incorporarem um codec dentro de si. Mas esse caso não acontece sempre, pensando nisso, a plataforma referencia em sua própria documentação uma página (<https://developer.android.com/guide/topics/media/media-formats>) do site oficial do Android presente no subdomínio dos desenvolvedores para ajudar você a discernir quais arquivos de multimídias você pode ou não usar!

2.4.10.2.8 Cronômetro como último serviço

Agora que já temos o relógio e a contagem regressiva prontos, só nos falta terminar o cronômetro para encerrarmos nosso primeiro aplicativo! O cronômetro tem por finalidade contar quanto tempo se passou desde um certo momento em que foi ativado. Em nosso caso, usaremos uma variável para guardar a quantidade de tempo no formato de segundos.

Começando pela parte do design, nosso cronômetro possuirá os seguintes elementos: duas legendas, uma para informar o título do

¹⁶⁸ Programas cuja função é codificar ou decodificar sinais de uma mídia para outra. No caso citado, se refere à decodificação de um arquivo de áudio em pulsos de eletricidade que modularão os movimentos do alto-falante que produzirá o som.

serviço e outra onde exibiremos o valor do tempo passado em horas, minutos, segundos e milésimos de segundo, assim como dois botões que garantirão ao usuário a capacidade de iniciar uma cronometragem, pausá-la, retomá-la e recomeçá-la.

Assim que terminamos os preparativos visuais e inclusão dos componentes, temos que lidar com seus blocos. Já possuímos a função “processar” que constantemente está executando a função “ajustar_crono”, exatamente a função responsável por contabilizar o tempo passado no cronômetro. Entretanto, assim como na contagem regressiva, o processamento dessa função está bloqueado, desta vez pela variável global “cronometro”.

Então precisamos saber quando desbloquear a função, isso se dará por meio do evento de clique do botão de iniciar. Para lidar com isso, temos que adicionar o bloco de evento do botão de início da cronometragem ao código. Dentro deste evento, seguimos a mesma lógica dos botões da contagem regressiva, com quase o mesmo esquema de trocas de estado. Veja como podemos fazer isso na figura 74.

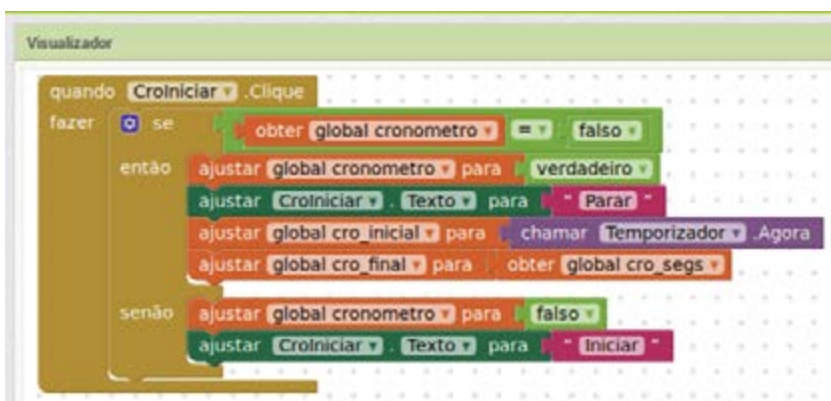


Figura 74: O botão “CroIniciar” desbloqueia o processamento da cronometragem.
(Fonte: App Inventor)

A primeira coisa que fazemos é checar se a cronometragem está ativada, se ela estiver, isso significa que nosso botão deve estar exibindo a palavra “Parar” e, quando for clicado, ele desativará o processo e trocará seu texto para “Iniciar”.

Agora, consideremos o caso em que a cronometragem está desativada. Quando o botão for pressionado, a primeira coisa que faremos é ativar a cronometragem. Depois, trocamos o estado de nosso botão, fazendo ele mostrar o texto “Parar”. Ajustamos o valor da variável global “cro_inicial” para o instante dado pelo procedimento “Agora” do temporizador e, por fim, ajustamos outra variável chamada “cro_final” para o valor de outra variável global chamada “cro_segs”, preservando a quantidade de tempo contada na última cronometragem (se essa for a primeira, o valor de “cro_segs” será zero).

Se o parágrafo anterior soou um tanto confuso é altamente recomendável que você cheque a tabela 21, nela explicamos quais são as variáveis envolvidas, seu objetivo, valor inicial e quando se dá sua mudança de valor.

Tabela 21: Variáveis controladoras do Serviço de Cronômetro.

Variável	Objetivo	Valor Inicial	Mudança de valor
cronometro	Bloqueia o processamento da função “ajustar_crono” quando falsa e desbloqueia quando verdadeira.	Falso	Toda vez que o botão de Iniciar/Parar é pressionado.
cro_segs	Armazena a quantidade total de tempo (em segundos) contabilizados pelos períodos em que a cronometragem esteve ativa.	Zero	Toda vez que a função “ajustar_crono” está desbloqueada e é processada.
cro_inicial	Armazena um instante de tempo (o momento em que a cronometragem inicia) usado para calcular o tempo passado entre a atualidade e o início da contagem.	Zero	Toda vez que o botão de Iniciar é pressionado.
cro_final	Armazena a quantidade de segundos acumulada pela última vez que a cronometragem foi ativada.	Zero	Toda vez que o botão de Iniciar é pressionado.

(Fonte: Do Autor)

Agora que desbloqueamos o processamento da cronometragem, podemos partir para o estudo da função “ajustar_crono”. Essa função, primeiramente, verifica se a cronometragem está bloqueada ou não, em nosso caso, agora não está. Sabendo disso, ela parte para o primeiro bloco dentro do bloco SE que realiza a checagem da variável “cronometro”. Veja como se dá esse procedimento na figura 75.

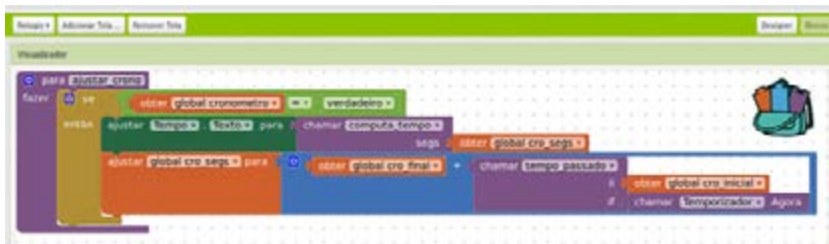


Figura 75: A função “ajustar_crono” computa quanto tempo se passou e informa o tempo passado ao usuário.

(Fonte: App Inventor)

A primeira instrução a ser executada é uma mudança de valor do atributo Texto da Legenda que estamos usando para mostrar o tempo do cronômetro. Mas perceba que não jogamos simplesmente a variável “cro_segs” dentro da especificação do atributo, desse jeito a legenda mostraria apenas o número de segundos passados.

A fim de converter o número de segundos em um tempo formatado em horas, minutos, segundos e milésimos, enviamos como especificação do atributo texto o resultado do processamento da função “computa_tempo” sobre o valor da variável “cro_segs”. Essa função é um tanto extensa, sua forma pode ser verificada por meio da figura 76.

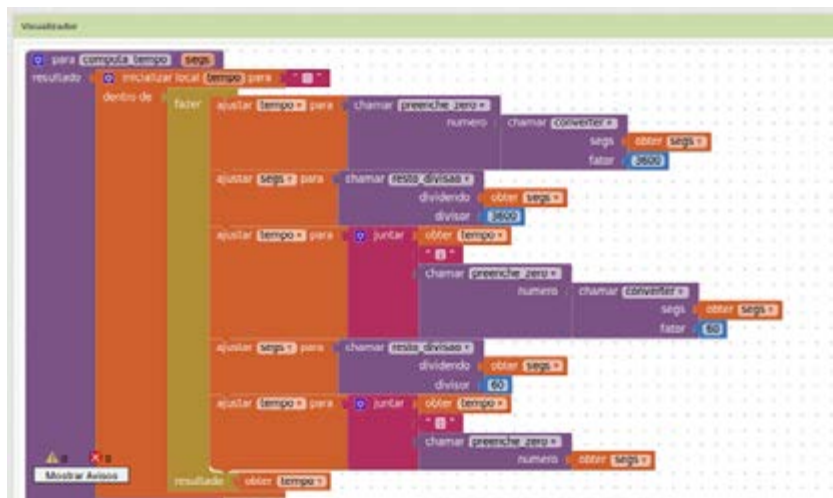


Figura 76: A função “computa_tempo” transforma os segundos da cronometragem em formato de horas textualmente.

(Fonte: App Inventor)

Dentro dessa função definimos um bloco de variável local que nomeamos como “tempo”, essa variável terá o tempo organizado em formato de cronômetro. Dentro do escopo da variável local, incluímos um bloco de especificação “Fazer” que pode ser encontrado na categoria de controle.

Fizemos isso porque funções capazes de dar um retorno inicialmente só funcionam com blocos de especificação, é por isso que temos um bloco SE adaptado para esse formato. Entretanto, sabemos que nem todos os blocos são assim, logo os desenvolvedores do App Inventor sentiram a necessidade de expandir a capacidade das funções com retorno por adicionar o bloco “Fazer”.

Este bloco tem o formato de um bloco de especificação, mas a sua única função é possibilitar a adição de blocos de instruções sequenciais a procedimentos que admitem retorno. Além disso, este bloco cobra do desenvolvedor que haja um retorno de dados ao seu fim, isso se dá porque não adianta criar uma função de retorno se ela não retornar valor algum a quem a chama.

Dentro do bloco “Fazer” começamos nossas instruções sequenciais. A primeira coisa que fazemos é ajustar o valor da variável texto para o resultado da função “preenche_zero” a partir do resultado da função “converter” com o uso do valor 3600 e de “segs” como argumentos. Veja na figura 77 como esse processo acontece.

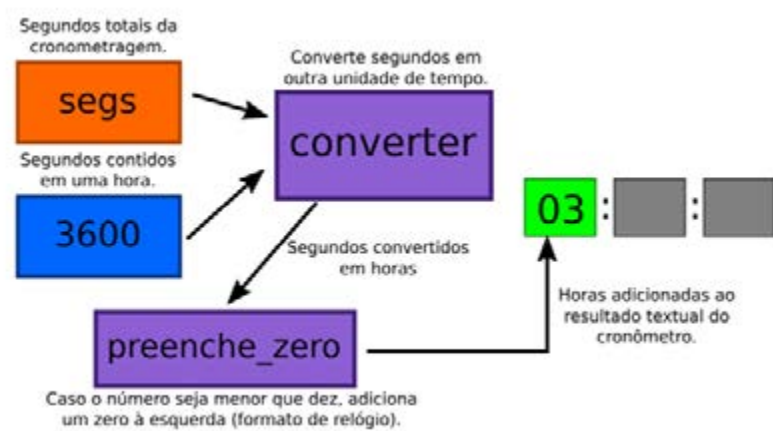


Figura 77: Esquematização da conversão de segundos em formato relógio.
(Fonte: Do Autor)

Depois disso, ajustamos o valor restante dos segundos para a quantidade restante da subtração do resultado de horas que obtivemos. Isso é feito passando o valor dos segundos e a constante 3600 para a função “resto_divisao” cujo objetivo é dividir o primeiro número pelo segundo, não aceitando números fracionários e retornando o resto da divisão (essa operação é feita pelo bloco matemático “Módulo de”). Logo, se dividirmos nossos segundos pela quantidade de segundos presentes em uma hora, essa função nos devolverá o quanto de segundos nos resta. Veja as funções “resto_divisao” e “converter” na figura 78.

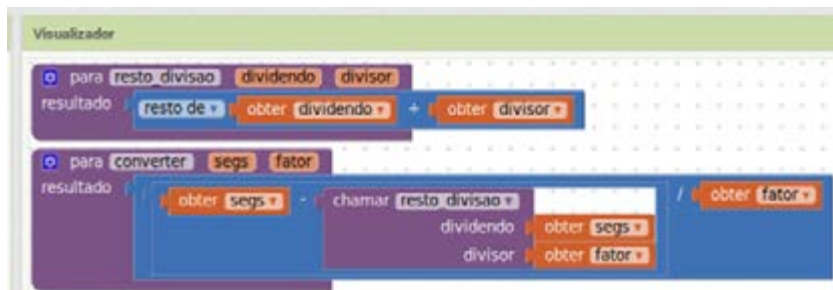


Figura 78: As funções “resto_divisao” e “converter” são auxiliadoras da conversão feita na “computa_tempo”.

(Fonte: App Inventor)

Em cima deste resultado, adicionamos ao valor da variável tempo um símbolo de dois pontos (“:”) para separar as horas dos minutos e, junto a este símbolo, o resultado da função de preencher o zero à esquerda, aplicada sobre o resultado da função “converter”, agora usando o número de segundos restantes da conversão para horas e a constante 60 (número de segundos em um minuto).

Por conseguinte, chamamos novamente a função “resto_divisao” para prover o valor de segundos restantes da conversão para minutos, a fim de completar nosso formato de relógio para o cronômetro. Por fim, adicionamos à variável local “tempo” o resultado da função “preenche_zero” aplicada diretamente ao valor de segundos que nos restou. No fim do bloco “Fazer”, retornamos o valor de tempo.

Note um detalhe sobre o bloco de definição de variável e o bloco Fazer, não poderíamos definir uma variável dentro dele se planejarmos retorná-la como resultado. O App Inventor consideraria que a parte onde o retorno acontece está fora do escopo de definição da variável e resultaria em erros. É importante prestar atenção a este detalhe.

Agora que terminamos de cobrir como nosso cronômetro funciona, só falta explicar o que acontece quando apertamos o botão de “Redefinir”. Veja na figura 79 qual é o código que executamos dentro de seu evento de clique.



Figura 79: O botão de redefinir retorna os dados da cronometragem ao estado inicial. (Fonte: App Inventor)

A primeira coisa a ser feita é bloquear o processamento da função “ajustar_crono”, em seguida, redefinimos o valor de “cro_segs” para zero a fim de que as próximas cronometragens comecem do zero. Depois disso, aproveitando que o valor de “cro_segs” se tornou zero, ajustamos o relógio textual do cronômetro para o resultado da função “computa_tempo”, causando um relógio zerado. Por fim, mudamos o estado do botão de Iniciar/Parar para Iniciar.

Com isso terminamos o último serviço e a última utilidade de nosso primeiro aplicativo!

2.5. Compartilhando um Aplicativo!

Quando você termina um aplicativo no App Inventor, é estendida a você a opção de compartilhar um link para download. Você pode estar se perguntando por que isso é necessário se a cada vez que o aplicativo é compilado, o App Inventor provê um código QR para download. Acontece que esse link é temporário e só dura algumas horas.

O código QR fornecido pelo App Inventor após a compilação do App possui um link de download encurtado para acesso rápido, mas imagine quantas pessoas estão compilando aplicativos a todo momento, as possibilidades de links curtos se esgotam rapidamente, por isso é um link temporário. Pensando nisso, os desenvolvedores do App Inventor criaram a possibilidade de compartilhar um App por um link permanente.

Na tela de projetos do App Inventor, você pode clicar na caixa de seleção do seu aplicativo a ser compartilhado e clicar no botão “Publicar na Galeria”. Veja isso acontecendo na figura 80.

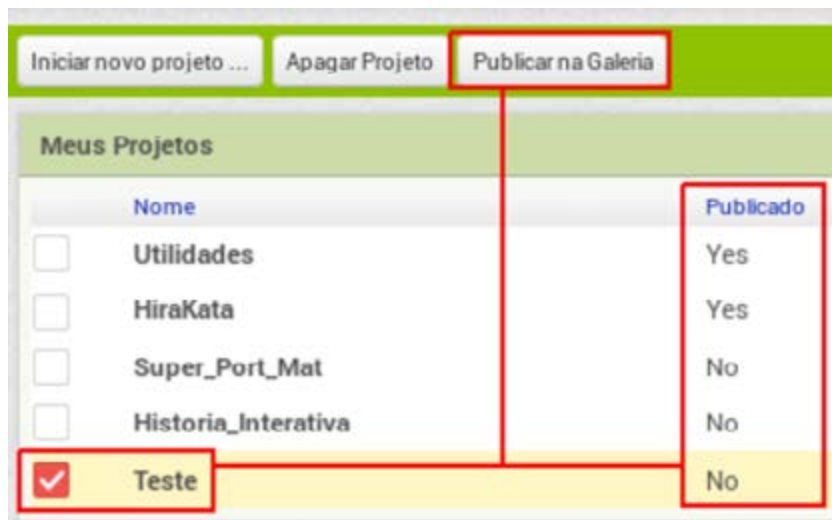


Figura 80: É possível publicar um App na galeria interna do App Inventor para que outras pessoas possam baixá-los, modificá-los e usá-los.

(Fonte: App Inventor)

Isso vai abrir um pequeno formulário para você inserir um ícone para o App e...

1. Dar um nome fantasia ao App (diferente do nome de projeto que tem restrições).
2. Prover um link de tutorial ou aula (em texto, áudio ou vídeo) para ensinar o uso do App.
3. Fornecer uma descrição sobre o App no geral.

Veja como a tela de cadastro se esquematiza na figura 81.



Figura 81: Para publicar um App na galeria do App Inventor, é aconselhável prover algumas informações sobre ele.

(Fonte: App Inventor)

Assim que o aplicativo for compartilhado, você pode procurá-lo por meio da galeria de aplicativos do App Inventor. Na página do aplicativo podem ser encontradas informações de curtidas, downloads, datas de criação e atualização, dados de autoria, aplicativos do mesmo autor...

São notáveis também três abas: Descrição (Description), Compartilhar (Share) e Reportar (Report). Respectivamente elas servem para:

1. Exibir a descrição do aplicativo que o desenvolvedor forneceu ao publicar.
2. Prover o link estendido e fixo de download do aplicativo para compartilhamento.
3. Denunciar o aplicativo por conteúdo sensível, ofensivo ou que, de alguma forma viole os termos de uso da plataforma.

Próximo ao ícone do aplicativo também é possível ver dois botões: “Abrir o App” e “Editar”. O primeiro serve para importar o projeto do aplicativo para sua tela de projetos e está visível para outros usuários da plataforma também. O segundo aparece apenas para o desenvolvedor e

somente o redireciona para a edição do projeto do referido aplicativo. Veja a página de um App na figura 82.



Figura 82: Página de um App publicado na galeria interna do App Inventor.
(Fonte: App Inventor)

2.6. Atividades de Fixação!

Abordamos bastante conteúdo nesse capítulo, veja se você conseguiu absorver o conteúdo com o questionário a seguir!

1. Por que definir um público-alvo é importante para desenvolver um aplicativo?
2. Primariamente, computadores lidam com problemas do mundo real por meio do processamento de informações que retratam tais problemas no mundo computacional. Essa afirmação é verdadeira ou falsa?
3. Como ter uma ideia/conceito de solução não tão vaga pode ser um bom começo no processo de desenvolvimento?
4. Como detalhar o problema enfrentado ajuda na formulação de uma solução?
5. É melhor tratar o problema como um todo de uma só vez ou subdividi-lo em partes menores? Por quê?

6. Por que fazer uso de técnicas de reutilização de código são vantajosas?
7. O que é um algoritmo e como criar um pode ajudar na construção de um programa ou aplicativo?
8. O que significam os princípios de manutenibilidade e escalabilidade? Por que eles são importantes?
9. O que são os componentes do App Inventor?
10. Qual é a principal missão da categoria “Interface de Usuário” do App Inventor?
11. Para que servem os componentes da categoria “Organização”?
12. Por que é interessante praticar o princípio da caixa-preta?
13. Como as propriedades dos componentes do App Inventor ajudam a deixar os aplicativos mais funcionais e personalizáveis?
14. Qual é a função principal de uma tela?
15. Um aplicativo feito no App Inventor não precisa ter tela alguma para funcionar. Essa afirmativa é verdadeira ou falsa?
16. Por que usar muitas telas no App Inventor é algo muito desaconselhado?
17. Qual é a função do componente Botão?
18. Para que é usado o componente de Legenda?
19. Com qual tipo de dado o componente Campo de Texto lida primariamente? Outros tipos de dados podem ser obtidos através deste?
20. Quais são as quatro opções de ajuste de tamanho para um componente com representação gráfica?
21. Qual é a função geral de blocos que possuem uma saída para encaixe à esquerda?
22. Blocos de eventos são disparados sempre que o usuário interage com um componente. Essa afirmação é falsa ou verdadeira? Explique o porquê.
23. Alguns blocos são capazes de delimitar um escopo sequencial onde outros blocos podem funcionar, cite três exemplos deles:

24. Alguns blocos possuem um encaixe de alça superior para serem executados, qual é a função primária destes blocos?
25. Quais são os tipos de blocos capazes de modificar propriedades dos componentes?
26. Quais opções formatam um texto como negrito ou itálico, por exemplo, numa legenda? Qual propriedade permite fazer os dois no mesmo texto?
27. Em que situação é preciso usar a organização horizontal?
28. Quando é recomendado usar componentes de organização capazes de realizar rolagem?
29. Alguns componentes vêm com procedimentos prontos para ações específicas deles. Cite três exemplos de procedimentos e descreva o que eles fazem:
30. Qual é o conceito de uma variável?
31. Por que usar variáveis, ou constantes (às vezes) é uma boa prática? Explique cada uma separadamente.
32. Qual elemento gráfico dos blocos retrata melhor a sequencialidade de instruções no App Inventor?
33. Porque Blocos SE, SENÃO-SE e SENÃO são de suma importância para um código que lida com cenários diferentes?
34. Qual seria o resultado da comparação lógica ((VERDADEIRO E FALSO) OU VERDADEIRO)? E da (VERDADEIRO E FALSO E VERDADEIRO)?
35. Como estruturas de repetição ajudam a “enxugar” nosso código e deixá-lo menor?
36. Por que é interessante usar funções/procedimentos para modularizar o código?
37. A fim de não depender completamente de variáveis globais, qual tipo de recebimento de dados funções/procedimentos podem usar e por que isso é aconselhado?
38. Alguns blocos possuem um pequeno ícone de engrenagem azul. Para que serve este bloco? Cite dois exemplos de blocos que possuem esse ícone e o que o uso dele causa nos blocos:

39. Dentre vários sistemas de formação de cor, qual é o usado pelo App Inventor e quais são suas cores primárias?
40. Como o App Inventor encara e trabalha com uma cor?
41. Defina um vetor:
42. Por que as listas são a evolução dos vetores?
43. Como o componente de deslizador é visualmente e como ele trabalha com números?
44. Qual é a principal diferença entre um Campo de Texto e um Campo de Senha?
45. Ao simular várias telas em uma só, qual configuração dos componentes pode ser intercambiada a fim de criar a simulação? Qual opção do visualizador evita que desenvolvedores fiquem trocando essa configuração o tempo todo?
46. Como o componente EscolheData fornece ao código a data escolhida pelo usuário?
47. Qual é o tipo de dado (que compreende outros dados) principal com o qual o Temporizador lida?
48. O que os procedimentos de duração do Temporizador fazem?
49. Por que corridas por dados são situações perigosas?
50. Como funciona o TinyDB em relação a registros e Namespaces?
51. Por que armazenar dados no cartão SD pode ser vantajoso e desvantajoso ao mesmo tempo? Qual componente é capaz de fazer isso?
52. O que são diretórios e caminhos?
53. Qual componente é capaz de mostrar uma pequena mensagem flutuante e temporária ao usuário?
54. Qual botão oferece uma lista de opções onde a selecionada se torna a opção mantida no botão? E como é possível saber qual foi a opção selecionada?
55. Por que usar estados é bom para reutilizar componentes no App Inventor?
56. O Android é compatível com todos os arquivos de áudio e imagens existentes?

57. Para executar sons no smartphone, quais são os dois componentes do App Inventor indicados para esse propósito? Qual é a diferença de propósito deles?
58. Qual é o problema apresentado pelo evento de Disparo do Temporizador do App Inventor?

2.7. Encerramento do capítulo

Sendo assim, concluímos nosso primeiro aplicativo feito no App Inventor. Aprendemos sobre componentes básicos mas que serão a base para impulsionar seu aprendizado no geral, agora você é capaz de criar aplicativos que conseguem fazer interação com o usuário, conseguir dados dele, computar esses dados e mostrar os devidos resultados, se aproveitando de técnicas de armazenamento ou não.

Para os próximos capítulos, este livro terá um foco especial numa área pouco discutida entre o público geral mas de suma importância para grande parcela dos usuários, as pessoas com deficiências ou diferenças motoras, visuais ou auditivas, a Acessibilidade.

Esperamos que tenha tirado proveito dos conceitos iniciais de desenvolvimento que este capítulo se preocupou em abordar. É recomendável que você treine fazendo aplicativos para você mesmo, solucionando problemas e situações de sua vida diária.

Os computadores são ferramentas digitais que existem para nos ajudar a lidar com informação. Se as suas situações-problema podem ser convertidas em informações que denotam alguns padrões, então os computadores podem ajudar você a entendê-las ou solucioná-las de alguma forma e, justamente, seu smartphone é um computador de bolso que cabe na palma da sua mão!

Então, não tenha receio de desenvolver aplicativos para uso pessoal de início. Posteriormente você poderá fazer aplicativos mais detalhados e compartilhá-los com as pessoas. O aplicativo que desenvolvemos pode ser baixado por meio de um link (<http://ai2.appinventor.mit.edu/?galleryId=5499516346826752>). Ele não aplica princípios de acessi-

bilidade. No final do próximo capítulo você pode encontrar uma versão desse app que aplica tais princípios.

Na figura 83 há um código QR com o link de download, atente para o fato de que será necessário autenticar na plataforma App Inventor com sua conta do Google.



Figura 83: Link para download do aplicativo de Utilidades.
(Fonte: Do Autor)

Sigamos nossos estudos e avancemos para o terceiro capítulo.

Capítulo 3: Segundo Aplicativo, agora com Foco em Acessibilidade

Agora que sabemos os componentes básicos e como manuseá-los, criaremos um aplicativo que pode ser aplicado numa situação real, visto que o primeiro possuía apenas utilidades genéricas para fins de teste e aprendizado. Como esse projeto será consideravelmente mais amplo e detalhado, saiba que, ao fim deste capítulo, há um link para este App pronto pelo qual você pode se guiar.

3.1. Estudo de Componentes

Você deve ter percebido que não usamos nem 40% dos componentes que o App Inventor disponibiliza. Entretanto, cobrimos todos os componentes de Organização e de **Interface do Usuário**¹⁶⁹. Ou... era o que pensamos.

Durante o desenvolvimento deste livro, o App Inventor continuou recebendo atualizações e, na atualização “nb176”, foi adicionado um componente novo à categoria de Interface de Usuário! Esse componente se chama “Switch” (do inglês, “interruptor”). É um componente cuja função é gravar um estado lógico, exatamente igual aos que vimos durante algumas condições de blocos SE.

Quando o usuário toca um Interruptor, ele é ativado, quando toca novamente, ele é desativado. Isso é familiar para você? Sim, isso mesmo,

¹⁶⁹ Com exceção do componente NavegadorWeb que é um tanto mais complexo.

o funcionamento é exatamente igual ao das caixas de checagem! Mas você pode se perguntar, por que existem dois componentes para realizar a mesma tarefa em um nível básico.

Essa é uma questão de design. Versões mais novas do SO Android agora usam por padrão interruptores para lidar com configurações lógicas. Muitos usuários também acham interruptores mais intuitivos e mais facilmente identificáveis do que caixas de checagem. Veja a diferença entre os dois componentes na figura 84.



Figura 84: Embora sejam diferentes visualmente, tanto as caixas de checagem quanto os interruptores têm o mesmo funcionamento.

(Fonte: Ubuntu, XFCE, Mozilla Firefox, Numix Project e W3Schools)

Mas a discussão vai para um pouco além disso. Algumas diretrizes de design do Google definem que tanto caixas de checagem quanto interruptores, mesmo que sirvam para a mesma coisa, existem para servir a visões diferentes!

Segundo o site Material.IO (<https://material.io/design/components/selection-controls.html>), os interruptores devem ser usados para modificar o estado de configurações individuais que não fazem necessariamente parte do mesmo contexto. Deve indicar algo que pode ser habilitado ou desabilitado assim que o interruptor trocar de estado.

Já as caixas de checagem devem ser usadas quando um usuário pode selecionar mais de um elemento em uma lista. Geralmente são usadas em situações em que o usuário precisa escolher vários itens de

uma lista, assim como escolher uma categoria inteira (que também é uma caixa de checagem por si só e ativa ou desativa todos os seus itens afiliados que, por sua vez, também são caixas de checagem).

Entretanto, note que ressaltamos as “versões mais novas”, isso se deu porque, se testarmos o componente de interruptor numa versão mais antiga do Android (que não os possuía), ele será automaticamente convertido para uma caixa de checagem, isso porque seu conceito ainda não existia na época em que a versão do SO foi desenvolvida. E isso nos traz um tópico de suma importância.

Embora o App Inventor disponibilize muitos componentes, pode ser que nem todos eles sejam capazes de se executados em todos os dispositivos usando o SO Android. Isso se deve ao fato de que alguns tablets, TV's, relógios de pulso inteligentes (...) ou mesmo **sistemas embarcados**¹⁷⁰ de veículos não têm a capacidade de executar os códigos de certos componentes. Deve ser muito desconfortável caminhar carregando uma SmartTV que conta os passos dados no exercício, e seria mais impressionante ainda saber que ela tem um sensor para esse tipo de coisa!

Mas mesmo que o funcionamento destes componentes não seja garantido em todos os dispositivos, é preciso expor pelo menos o que fazem e como podemos usá-los para os fins que poderemos vir a precisar.

3.2. Dentre as Atualizações, um Problema

Além de nos trazer o componente de interface “Interruptor”, a atualização “nb176” de Maio de 2019 trouxe um problema junto às adições de funcionalidades. Devido ao fato de a plataforma App Inventor ser utilizada por pessoas em vários países e haver usuários que

¹⁷⁰ Dizemos que um sistema é embarcado quando ele não provê um ambiente computacional para a resolução de problemas generalizados, mas sim para problemas específicos para onde é destinada toda a sua força e capacidade de processamento. Sistemas que controlam aviões são sistemas embarcados.

não leem ou entendem inglês, é necessário fazer esforços para traduzir a plataforma.

A interface da plataforma já foi traduzida em diversos idiomas, dentre eles, o português. Na atualização discutida, foram adicionadas as traduções para o público húngaro e para o público alemão. Entretanto isso revelou um problema que estava à espreita. A plataforma não tinha um idioma padrão para mostrar palavras no caso de alguma tradução estar incompleta.

Isso causou o seguinte problema, quando uma tradução não provia palavras para algum componente de interface da plataforma, o texto a ser mostrado era arbitrariamente escolhido em alguma outra tradução! No caso do App Inventor em português, era possível ver a maior parte da interface em nosso idioma, trechos em inglês e até partes em alemão! Veja a falha na figura 85.

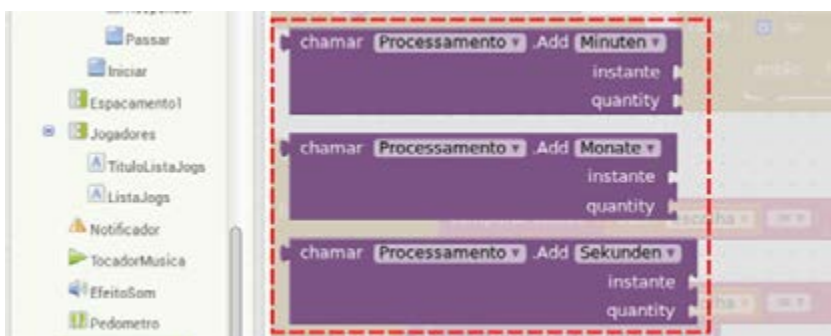


Figura 85: A falha no esquema de tradução da plataforma veio à tona na atualização “nb176”, App Inventor mostrando texto em português, inglês e alemão.

(Fonte: App Inventor)

É claro que os usuários não deixaram isso passar batido e logo notificaram os desenvolvedores, em relativamente pouco tempo, a atualização “nb177” foi publicada no dia 25 de Maio de 2019 e corrigiu os problemas citados. Todavia, traduções incompletas do App Inventor ainda fazem a plataforma recorrer ao inglês como língua base. Então ainda é possível encontrar trechos em português e inglês na interface, fique atento!

Para além disso, você pode estar se perguntando como saber quais são as mudanças que ocorrem entre uma versão e outra do App Inventor por meio das atualizações. Já discutimos isso no primeiro capítulo, mas é importante ressaltar já que o assunto é propício.

O App Inventor possui um registro de mudanças (também conhecido como “changelog”) que pode ser encontrado em seu site (<http://appinventor.mit.edu/ai2/ReleaseNotes.html>), lá constam todas as mudanças realizadas durante as atualizações.

Adicionalmente, se você encontrar um problema na plataforma, corra para os fóruns e verifique se alguém já alertou os desenvolvedores sobre o ocorrido. Caso ninguém tenha o feito, seja o primeiro a avisar! Você ajudará a melhorar a plataforma.

Voltando à essência de nossa discussão de aprendizado, é de suma importância para este capítulo discutir técnicas de como deixar nossos aplicativos mais acessíveis por meio do App Inventor e das tecnologias já oferecidas pela Google por meio do Android, nativamente. Então prossigamos para a ideia que guiará esse novo projeto.

3.3. Apresentação do Aplicativo a ser Desenvolvido

Manteremos um foco simples para o aplicativo deste capítulo, mas omitiremos um tanto mais os detalhes básicos explicados no capítulo anterior. Você perceberá que ainda faremos muito uso dos componentes básicos, isso acontece porque esses componentes são de interface de usuário. Em aplicativos que precisam mostrar informações na tela, os componentes de interface são muito usados.

3.3.1. Introdução

Boa parte das pessoas gosta de se divertir e descontraír de tempos em tempos. Se entreter é algo natural do ser humano. Desde as brincadeiras de rua como pique-esconde, passando por jogos simples e físicos de cartas ou tabuleiro até gincanas, jogos virtuais de nota Triplo-A ou gigantescos parques de diversão temáticos, fica claro que se divertir com algum tipo de jogo é comum em nossa rotina.

O mercado de jogos é algo que está sempre em voga. Jogos simples são feitos todos os anos em eventos chamados “JAM’s”¹⁷¹ e a grande indústria de videogames promove muitos lançamentos de grandes, médios e pequenos títulos para as mais diversas plataformas, dentre elas, a plataforma mobile.

3.3.2. Público-Alvo

Nosso aplicativo será um pequeno jogo educativo focado em perguntas simples de português e matemática. Sendo assim, uma ou várias pessoas podem jogar apenas para aprender ou para competir entre si e aprender, pouco a pouco, conceitos básicos destas duas áreas do saber.

Lembra-se que dissemos que este aplicativo aplicará conceitos de acessibilidade e será usado em uma situação real? O Instituto Federal de Educação, Ciência e Tecnologia do Espírito Santo (IFES) Campus Colatina fechou uma parceria com a Associação Colatinense de e para Pessoa Portadora de Deficiência Visual e, com isso, prometeu entregar um aplicativo que pudesse ser usado em atividades letivas. Consequentemente, tal aplicativo é o que desenvolveremos neste capítulo!

3.3.3. Ideia

Para esse capítulo construiremos um aplicativo em formato jogo de questionário. Esse jogo deve ser acessível, aplicando técnicas de alto-contraste e ampliação de textos. Além disso, continuaremos respeitando a limitação técnica do App Inventor quanto ao número de telas cujo funcionamento da plataforma é garantido.

O jogo terá dois modos, uma partida competitiva e outra de aprendizado, mudando ligeiramente alguns aspectos de funcionamento que observaremos com mais profundidade posteriormente. Além disso,

¹⁷¹ Um evento de tempo limitado onde os desenvolvedores precisam criar um jogo com ou sem tema definido e entregar o jogo para avaliação do público geral. Uma JAM muito conhecida é a Ludum Dare (<https://ldjam.com/>).

cada modo poderá lidar com perguntas da área de português ou matemática, o que nos dá um total de quatro cenários possíveis.

3.3.4. Definição da Situação

Precisamos de um aplicativo que seja capaz de realizar várias tarefas relacionadas às diferentes esferas:

- Configuração de partida;
- Manuseio de questões e respostas;
- Contabilização de respostas;
- Restrições de jogadas;
- Configurações de acessibilidade.

Para esquematizar melhor, subdividiremos nosso aplicativo em quatro telas, lembrando que algumas delas podem servir como mais de uma tela a fim de poupar processamento. Veja quais telas desempenharão determinadas funções de acordo com a tabela 22.

Tabela 22: Divisão do aplicativo de perguntas em telas e a função de cada uma.

Tela	Objetivo
Screen1	A tela principal é a responsável por filtrar qual tipo de ação o usuário deseja realizar, se ele quer saber do que se trata o aplicativo, ou se quer configurá-lo de alguma maneira específica, ou se só quer mesmo jogar.
Config	A tela de configurações onde o usuário pode definir qual é o tamanho base da fonte de todos os elementos textuais do aplicativo, ou a cor de fundo que implica no uso da técnica de alto-contraste, ou o número de jogadores que participarão de uma partida ou mesmo novas questões para outras partidas.
Resultados	Uma tela que serve, ao mesmo tempo, para mostrar as pontuações finais de cada jogador, declarando um ou mais vencedores, e também para criar novas perguntas/questões para novas partidas.
Matemática	A tela onde o jogo realmente ocorre, onde os jogadores têm suas pontuações, onde as perguntas são lançadas e onde o aplicativo pede as devidas respostas.

(Fonte: Do Autor)

Caso as conexões entre as telas tenham ficado pouco compreensíveis, veja na figura 86 como elas interagem entre si e como se dão as divisões de funcionalidades internamente.

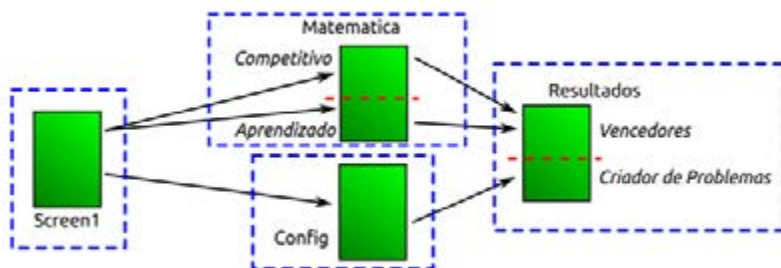


Figura 86: Conexões entre as telas reaproveitadas do aplicativo educacional proposto. (Fonte: Do Autor)

Note, porém, que os nomes das telas “Resultados” e “Matemática” não são totalmente condizentes com seus objetivos. Isso aconteceu porque quando o aplicativo estava sendo idealizado, a ideia do criador de novas questões e o fato de as perguntas de português e matemática poderem usar a mesma tela ainda não existia. Isso foi uma mudança de planejamento que aconteceu com o tempo sob demanda de necessidades de se economizar em número de telas.

Sendo assim, o aplicativo ficou mais enxuto e as telas ganharam uma maior concentração de componentes, tornando-as mais úteis. Entretanto, uma desvantagem que veio à tona depois de realizar esse processo é que o App Inventor **não fornece um meio oficial**¹⁷² de renomear telas. Então a melhor opção antes de dar um nome personalizado a uma tela é pensar bastante sobre o fato de que o nome dela terá que refletir suas funcionalidades mesmo após sofrer mudanças.

¹⁷² Todos os projetos do App Inventor podem ser baixados em forma de um arquivo compactado para o computador, localmente (arquivos do tipo “AIA”). Sendo assim, um usuário muito experiente poderia editar um projeto apenas modificando os conteúdos internos do arquivo e depois reenviando o arquivo para o App Inventor para ordenar a compilação do aplicativo. Por meio deste método, renomear telas é possível, mas é pouquíssimo aconselhado que usuários façam isso já que o arquivo pode ser facilmente corrompido.

Prosseguindo com nosso aplicativo, perceba que ele precisa ser capaz de obter perguntas definidas pelo usuário. Isso implica no fato de que é preciso obter essas questões de algum componente cujo recurso possa ser variável. O componente de arquivo é uma ótima opção para isso.

Podemos criar nosso próprio formato de arquivo armazenando dados de forma estruturada em um arquivo de texto puro (TXT). Esse arquivo será lido pelo aplicativo a cada início de partida e armazenará os dados obtidos do arquivo na memória principal do smartphone. Sendo assim, caso o usuário queira um novo conjunto de questões, é só trocar o arquivo ou seu conteúdo!

3.4. Mas antes de tudo, tem que ser acessível!

Agora que já temos uma ideia do que temos que fazer, comecemos a trabalhar no que não está relacionado à essência do aplicativo. Está tudo certo do jeito que está? Não, falta-nos algo muito importante! Antes de começarmos o desenvolvimento do nosso aplicativo, precisamos nos informar a respeito do que são e como aplicar técnicas de acessibilidade!

Para nos ajudar a compreender o que a acessibilidade no meio digital engloba, recorreremos a um especialista. Ele se chama **W3C**¹⁷³. Essa organização possui um escritório aqui no Brasil. Sendo assim, os profissionais envolvidos com as rotinas da organização em nosso país lançaram uma cartilha online sobre acessibilidade na Web, o conteúdo se destina a desenvolvedores, programadores, designers, estudantes etc., na esperança de que estes possam tirar proveito das diretrizes oficiais de acessibilidade lançadas pelo W3C.

¹⁷³ World Wide Web Consortium (<https://www.w3.org/>), é uma organização mundialmente conhecida. Foi fundada por Timothy John Berners-Lee, o fundador da World Wide Web (também conhecida como WWW ou W3), a rede mundial de troca de informações que surgiu a partir das conexões físicas mundiais lançadas pela internet. O W3C é muito atuante no meio internacional por publicar padrões de desenvolvimento em linguagens de marcação e programação como HTML e CSS para que a Web seja a ferramenta mais útil possível na tarefa de fornecer informações.

Segundo a “**Cartilha de Acessibilidade na Web**”¹⁷⁴, uma boa definição de acessibilidade no meio digital e, principalmente na Web, é: “Acessibilidade na web é a possibilidade e a condição de alcance, percepção, entendimento e interação para a utilização, a participação e a contribuição, em igualdade de oportunidades, com segurança e autonomia, em sítios e serviços disponíveis na web, por qualquer indivíduo, independentemente de sua capacidade motora, visual, auditiva, intelectual, cultural ou social, a qualquer momento, em qualquer local e em qualquer ambiente físico ou computacional e a partir de qualquer dispositivo de acesso.”.

Perceba que, segundo a definição usada na cartilha, o W3C Brasil conceitua que a acessibilidade não afeta apenas pessoas com deficiência. E isso é verdade! Tornar uma plataforma acessível (seja ela um programa de desktop, um aplicativo mobile ou uma página na web) significa assegurar ao seu usuário que, de alguma forma, ele vai conseguir acesso à informação que precisa, utilizando-se de todos os meios de comunicação e tecnologias possíveis e adequadas.

Você já se deparou com a estranha situação de navegar por um site (ou usando um aplicativo/programa) que não aplica as devidas técnicas de **design responsivo**¹⁷⁵, e as informações ou comandos que você precisava acessar ficaram, de alguma forma, fora do alcance da sua tela? Esse pode ser considerado um problema de acessibilidade! Você teve acesso à informação que precisava? Infelizmente não, então esse site, programa ou aplicativo não foi acessível a você.

Um exemplo prático dessa situação pode ser visto no uso de componentes de tela no App Inventor. Se você incluir muitos componentes gráficos em uma tela e não marcá-la como rolável em suas propriedades, provavelmente, em alguns smartphones, parte dos componentes ficará fora do alcance do usuário, inutilizando parte do seu aplicativo.

174 Esta cartilha pode ser encontrada no site do W3C Brasil (<http://www.w3c.br/pub/Materiais/PublicacoesW3C/cartilha-w3cbr-acessibilidade-web-fasciculo-I.html>).

175 Forma de organização da interface do usuário que se ajusta conforme o tamanho da tela, seja ela grande, média ou pequena.

Mas como se pode ver no cotidiano, no mundo de hoje, comunicação é algo extremamente importante, essencial. Tão importante quanto isso é garantir que essa comunicação chegue ao seu ouvinte, no nosso caso, usuário. Seja ele qual for e, independente de suas capacidades, talentos e dificuldades, temos que nos comunicar com nosso usuário para que ele tenha acesso ao conteúdo que estamos oferecendo.

Mas como pode um cego ter acesso ao componente de Legenda? Como pode um daltônico identificar as cores de nossos aplicativos? É aí que entra um grupo de ferramentas essenciais que nos ajudarão a resolver esses problemas, ou por enquanto, parte deles.

3.4.1. Tecnologias Assistivas

Assim como o nome diz, tecnologias assistivas estão presentes no meio digital para dar suporte às ações dos usuários que são/estão debilitados de alguma forma. Elas estão lá para prover auxílio, para dar assistência, ajudar na comunicação usuário-computador.

Lembra-se de quando discutimos sobre o fato de que, se não existissem sistemas operacionais, os programas e aplicativos criados teriam que ter uma versão especificamente feita para rodar no **PC**¹⁷⁶ pretendido? Em relação à acessibilidade o problema é quase o mesmo.

Pensando nisso, algumas empresas ou comunidades que criaram sistemas operacionais incluem ferramentas de acessibilidade no SO de forma nativa e pré-instalada. Se você usa Windows, já percebeu que existe uma ferramenta chamada “Lupa” cuja função é ampliar os conteúdos vistos na tela? Em alguns sistemas operacionais baseados em Linux, nas telas de bloqueio e autenticação de usuário é possível ver uma opção que troca o modo de exibição para alto-contraste. Veja um ambiente gráfico Linux aplicando alto-contraste na figura 87.

¹⁷⁶ PC é uma sigla para “Personal Computer” que, traduzido do inglês, significa “Computador Pessoal”. Geralmente se refere aos tradicionais computadores de mesa (desktops), ou aos computadores portáteis como notebooks/laptops.

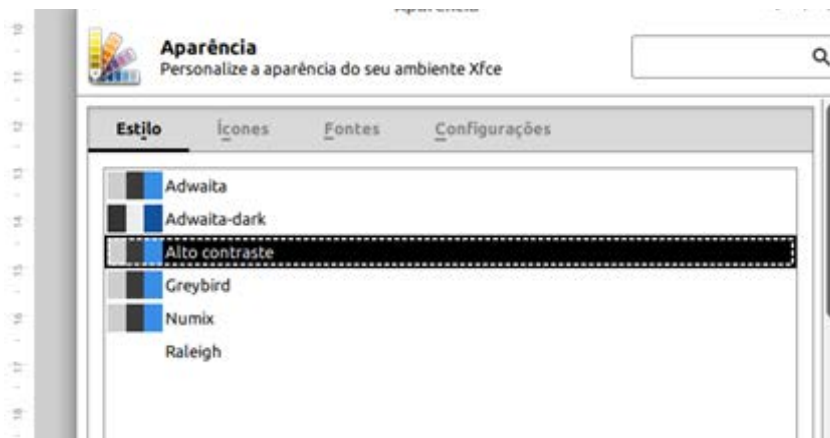


Figura 87: Ambiente gráfico XFCE aplicando técnicas de Alto-Contraste
(Fonte: Ubuntu, XFCE e LibreOffice)

Alguns outros sistemas operacionais possuem leitores de tela, programas capazes de reconhecer texto escrito em um aplicativo e ler o que está escrito para o usuário. Esse é o caso do Google TalkBack. Geralmente, esse aplicativo vem pré-instalado na maioria das versões do SO Android. Isso pode ajudar nossos usuários cegos a entender o que está acontecendo em nosso aplicativo.

Mas perceba que, a partir do momento que é ativado, o TalkBack afeta todo o SO. Mudando até um bocado da sua maneira de interagir com a interface gráfica. Ao tocar sobre um ícone de aplicativo, por exemplo, ele não será imediatamente aberto. O TalkBack focará no ícone e lerá o nome do aplicativo destacado na tela, ou dará uma breve explicação do que é aquele componente de interface. Para abrir o aplicativo seria necessário dar dois toques rapidamente. Veja esse funcionamento na figura 88.



Figura 88: Com um toque, o TalkBack seleciona um item e explica o que ele é, com dois toques, é possível interagir.

(Fonte: Android)

Essa forma de funcionamento pode ser bem diferente do que você está acostumado se você não tem problemas relacionados à visão, mas é de grande ajuda para pessoas que têm problemas relacionados à visão! Note que, para driblar problemas relacionados à visão em nosso aplicativo, estamos:

1. Facilitando a identificação de textos (com o subsistema de controle de tamanho de fonte e alto-contraste do aplicativo) para pessoas com visão parcial.
2. Lendo conteúdos em texto (por meio do Google TalkBack) para pessoas com visão nula.

O segundo ponto é importante para realizar uma reflexão. Informação está em todos os lugares e esse foi um dos tópicos que estamos discutindo desde o início deste livro. Porém, não há uma forma universal de se receber informação, o conteúdo está lá, a informação está difundida nele, seja ele um texto, vídeo, áudio ou figuras.

Todavia, pelo menos por enquanto, não existe uma forma universal de as pessoas receberem informação. O W3C, por meio da cartilha,

define que, no processo de interação de um usuário com a página Web existem 7 elementos envolvidos. Veja essa cadeia na figura 89.

1. Conteúdo, as informações que a página apresenta.
2. Navegadores, os programas que obtêm essas informações de servidores remotos via internet.
3. Tecnologias assistivas, que adaptam a informação do conteúdo apresentado para o formato de informação recebível pelo usuário.
4. O conhecimento do usuário em utilização da Web, especialmente necessária quando o usuário lida com problemas ou cenários não previstos pelas tecnologias assistivas, criadores/administradores de páginas ou navegadores.
5. Desenvolvedores, designers, codificadores, autores (...) e pessoas responsáveis por criar o conteúdo que tenta transmitir a informação desejada.
6. Ferramentas de autoria usadas para criar e páginas e websites.
7. Ferramentas de avaliação que checam a validade do código escrito para gerar as páginas visualmente e funcionamento no navegador do usuário.

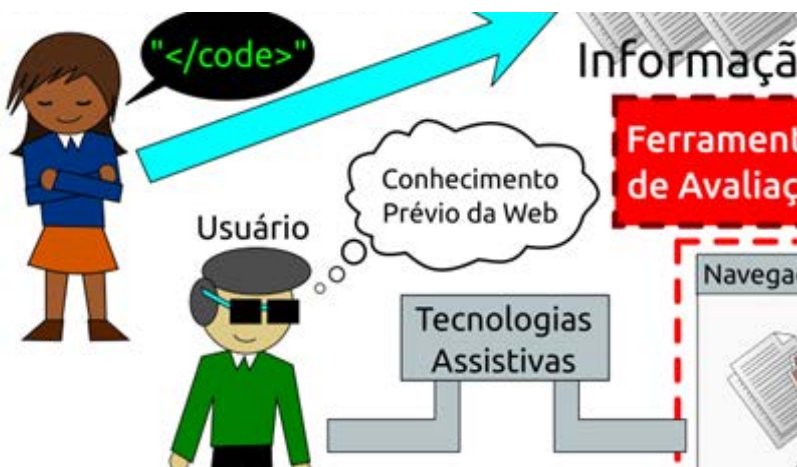


Figura 89: Cadeia de elementos envolvidos no acesso à Web e suas páginas, ilustrada a partir das definições do W3C Brasil.

(Fonte: Do Autor)

Disponibilizar um conteúdo em apenas uma forma pode ser um grande problema para pessoas que não são capazes de perceber este conteúdo da mesma forma que outras pessoas. Se formos transpor os elementos citados pela cartilha do W3C para nosso contexto, é possível perceber que podemos tirar bom proveito de vários conceitos apontados pelo documento.

Temos conteúdo a apresentar, esse conteúdo está embutido em nosso aplicativo que o disponibiliza ao usuário pela interface gráfica que, por sua vez, pode ser interpretada por tecnologias assistivas convertendo o conteúdo e suas informações para um formato compreensível ao usuário. Logo depois, como qualquer outra ferramenta digital, temos o conhecimento prévio do usuário, conforme preconiza a teoria da Aprendizagem Significativa de Ausubel (2003).

Ter esse conhecimento é de suma importância e, se o usuário não o tiver, temos que nos esforçar para que nosso aplicativo seja o mais intuitivo possível para que o aprendizado seja facilitado. A busca por subsunçores mais elementares, ou seja, essa busca pelo aspecto mais intuitivo da aplicação, tende a englobar/alcançar mais indivíduos, facilitando assim o processo de ensino aprendizagem e essa busca pela Aprendizagem Significativa.

É aí que entramos em cena, estamos criando aplicativos que servirão nossos usuários. É nosso papel fazer com que eles fiquem satisfeitos com o resultado apresentado com a ferramenta. Isso mesmo, o resultado! O aspecto mais frustrante de lidar com uma ferramenta para fazer um trabalho é exatamente não saber como usá-la! Bater cabeça para aprender que, para salvar uma configuração, seria necessário pressionar um botão quase escondido no fundo de uma tela pode ser muito frustrante para usuários leigos.

É por causa desses e de outros motivos que nós, os desenvolvedores, temos que fazer com que o usuário despenda o mínimo de tempo possível para aprender a usar nosso aplicativo! E é aí que entram as ferramentas de autoria, aquelas que nos ajudam a criar a solução que desejamos e embutir a informação na mesma. O App Inventor conta

como uma ferramenta de autoria porque criamos aplicativos nessa plataforma e eles podem carregar conteúdos em diversos formatos.

Por fim, temos as ferramentas de avaliação. Elas são cruciais se quisermos tornar nossos aplicativos acessíveis. Mas antes de comentarmos sobre elas, precisamos falar sobre uma dúvida que pode ter pairado sobre a cabeça de alguns leitores.

3.4.2. Reinventando a Roda?

Pode parecer um tanto quanto redundante a ideia de que precisamos adaptar nosso aplicativo já que existem as tecnologias assistivas que convertem o conteúdo no aplicativo para um outro formato que o usuário deseja. Entretanto, note que as ferramentas assistivas não são perfeitas. Não há como o Google TalkBack descrever minuciosamente uma imagem se não houver uma alternativa em texto ao seu conteúdo, de forma explícita ou implícita, que explique o que está na imagem.

Recorda-se de quando comentamos que o W3C publica padrões de diversas linguagens usadas no desenvolvimento Web que tornam as páginas da internet mais acessíveis? A linguagem HTML tem uma etiqueta específica para exibição de imagens, essa etiqueta (“”) possui um atributo (ou propriedade) chamada “alt” que especifica um texto alternativo que deve ser mostrado caso a imagem não possa ser vista, seja por conexão lenta com a internet, ou caso o usuário esteja usando uma tecnologia assistiva de leitura de tela (https://www.w3schools.com/TAGS/att_img_alt.asp).

Esse atributo “alt” deve ser usado de forma que o desenvolvedor insira a sua **descrição do que está sendo mostrado na imagem**¹⁷⁷. Entretanto, se recorrermos à janela de propriedades do componente imagem no App Inventor, veremos que não existe nada parecido.

Além disso, usar programas de Lupa podem tornar a utilização do aplicativo muito mais difícil pois elas apenas ampliam partes da tela, como se estivessem olhando-a mais de perto. Sendo assim, é necessário

¹⁷⁷ Ou uma descrição de onde a imagem levará o usuário ao ser clicada, caso seja um link.

ficar rolando/deslizando a tela e procurando os elementos desejados o tempo todo. Mesmo que torne o aplicativo parcialmente acessível, isso dificulta a visualização geral das informações.

Sites de instituições federais costumam aplicar técnicas de acessibilidade apesar de todas as tecnologias assistivas existentes. No site oficial do IFES Campus Colatina (<https://colatina.ifes.edu.br/>), é possível notar, por meio de uma barra de controle superior, que podem ser usados controles de tamanho de fonte, alto-contraste e até mesmo tradução do conteúdo para LIBRAS, a língua brasileira de sinais! Veja essa barra na figura 90.



Figura 90: Página Web do IFES demonstrando controles de acessibilidade na barra superior.

(Fonte: IFES)

Pelo menos do Android 6 em diante, há bastante opções de acessibilidade disponíveis, até mesmo opções relacionadas ao tratamento de cores para pessoas daltônicas, mas nem todas as pessoas possuem a **última versão do SO**¹⁷⁸.

¹⁷⁸ O principal motivo é dado pelo fato de que assim que o suporte para um modelo de smartphone acaba, o Android neste aparelho deixa de ser atualizado e, por motivos pessoais, econômicos ou outros, os usuários acabam por ficar com o aparelho desatualizado, tanto em hardware quanto em software. Além disso, alguns smartphones antigos podem ser mantidos por seus donos pelo fato de que aplicativos muito úteis a eles podem não ser compatíveis com a última versão do Android.

Em navegadores Web é possível usar controles de zoom nativos para aumentar o tamanho do conteúdo visto na tela, mas às vezes isso pode quebrar a organização do site se o nível de ampliação não conseguir ser tratado pelo design dele. Sabendo disso, o ponto é: não podemos depender das tecnologias assistivas em 100% dos casos, temos que fazer esforços próprios quando necessário.

Como nosso público-alvo se trata de pessoas com deficiências visuais focaremos em solucionar problemas de acessibilidade relacionados a essa área. Então voltemos a focar nas soluções que propusemos.

3.4.3. Acessibilidade no Android

Para ativar ferramentas de acessibilidade, é preciso abrir o painel de configurações do dispositivo executando Android e entrar no submenu de Acessibilidade. Dentre elas, é provável que o TalkBack estará incluso. Como nossas atividades para este capítulo se centram em prover um aplicativo acessível a pessoas cegas e como problemas na percepção visual de informação, focaremos nossos esforços em driblar estes tipos de dificuldades.

Mas existem diversas tecnologias assistivas para diversos tipos de dificuldade de percepção sensorial e interação com o ambiente. Se considerarmos que atividades relacionadas à audição, visão, fala e coordenação motora podem ou não ter dificuldades dentro destes quatro grupos, podemos ter um total de 16 perfis de usuário, mas essas são apenas algumas áreas, existem dificuldades presentes em muito mais campos envolvidos na formação do ser humano, sejam elas físicas, sensoriais, psicológicas etc.

É claro que, a partir do momento em que as dificuldades se fazem presentes em muitas modalidades ao mesmo tempo, o tratamento da acessibilidade tende a ficar muito mais rígido e técnico do que pode ser tratado no escopo deste livro. Não temos meios de, por exemplo, fornecer software e hardware especializado em controle do mouse

baseado no movimento dos olhos para o usuário, e sim, esse é um tipo de tecnologia assistiva!

Mas assim como nosso aplicativo é atrelado à gama de operações permitidas pelo SO, as tecnologias assistivas também estão submetidas a tais restrições. Em versões mais antigas do Android, pode ser mais difícil lidar com opções de acessibilidade pois essas foram incluídas ao AOSP gradativamente. No início as opções eram bastante escassas. Nesses casos, em que o TalkBack não vem configurado e pronto para utilização por padrão, será preciso seguir instruções específicas da plataforma utilizada.

3.4.4. Ferramentas de Avaliação

A fim de nos guiar no caminho do desenvolvimento de soluções melhores e padronizadas, existem ferramentas de avaliação que podem nos ajudar a perceber erros e práticas ruins na hora de converter nossas ideias em uma solução real e aplicável.

O W3C fornece ferramentas de validação de código que servem para verificar se o código escrito em uma determinada linguagem de programação, no caso da CSS (<http://jigsaw.w3.org/css-validator/>), ou marcação, no caso da HTML (<https://validator.w3.org/>). Mas para o nosso caso, precisaremos apenas de uma ferramenta de avaliação. Lembra-se que aplicaremos opções de alto-contraste no aplicativo, precisamos de uma ferramenta que cheque o contraste.

Você já percebeu como pode ser difícil ler letras pretas quando estão escritas sobre um fundo azul? O contrário também pode ser válido. Isso acontece porque o contraste entre as cores azul e preto não é muito alto visualmente (embora seja totalmente distinguível para computadores). Uma melhor escolha seria usar amarelo ou mesmo ciano em vez de azul. Veja na figura 91 como se dá a diferença de tons.



Figura 91: Embora tenham os mesmos valores para os computadores, visualmente, cores diferentes têm contrastes diferentes, mesmo em suas intensidades máximas.

(Fonte: Do Autor)

Na figura mostrada, apenas um par de cores não passa nos testes de contraste definidos pelas diretrizes de acessibilidade na web lançadas pelo W3C, esse par é o azul e preto. Como sempre, preto e branco serão os detentores do maior índice de contraste por serem exatos opostos no espectro de luz visível. O preto é a ausência de todas as cores enquanto que o branco é a presença de todas elas em intensidade máxima.

Para saber qual é o índice de contraste entre uma cor e outra, podemos usar uma ferramenta online (<https://webaim.org/resources/contrastchecker/>) disponibilizada pelo site **WebAIM**¹⁷⁹ chamada “Contrast Checker” (ou “Checador de Contraste”, em tradução livre).

A ferramenta pede duas cores (uma de fundo e uma de frente) e aplica as mesmas em três situações diferentes, uma para texto normal, uma para texto destacado e uma para indicar elementos de interface de usuário como botões e campos de texto.

Um cálculo é feito para determinar o nível de contraste estabelecido entre uma cor e outra por avaliar suas luminosidades relativas e, dependendo do nível de contraste, quantificar uma escala que represente o par. Essa escala possui dois números separados por dois pontos, o segundo número sempre será 1, já o primeiro pode variar de 1 a 21.

Se o par de cores conseguir determinados valores no primeiro número da escala, ele pode passar ou não nos testes de **WCAG**¹⁸⁰. Veja

¹⁷⁹ Um portal na internet dedicado a fornecer informações sobre acessibilidade na Web.

¹⁸⁰ Testes e requisitos de acessibilidade lançados pelo W3C. WCAG significa “Web Content Accessibility Guidelines” ou “Diretrizes de Acessibilidade de Conteúdo na Web”, em tradução livre.

o que acontece quando definimos as cores como azul e preto e checamos sua escala de contraste na figura 92.

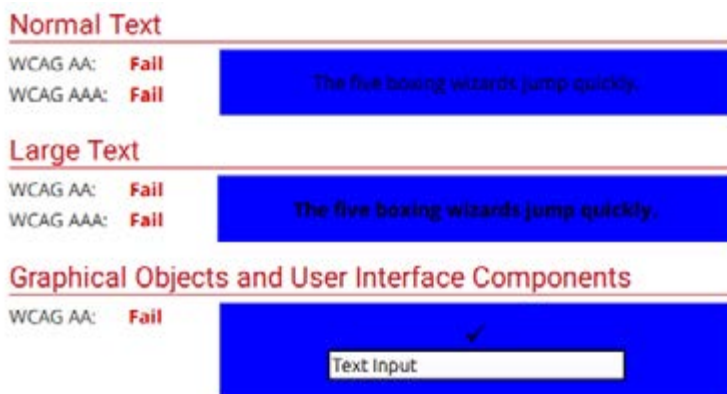


Figura 92: O par de cores Azul e Preto falhou em todos os testes WCAG por ter uma taxa de contraste insuficiente (2,44:1).

(Fonte: WebAIM)

Mesmo se trocarmos a cor do fundo com a cor de frente, a taxa de contraste será a mesma. Para não depender de cores específicas que até mesmo não podem ser identificadas por alguns usuários, recorreremos ao clássico par de preto e branco, talvez até cinza escuro e branco.

O par preto e branco é o detentor da maior taxa de contraste possível entre as cores por serem a ausência ou presença intensa de todas as cores. Isso faz com que este par passe em todos os testes WCAG, então apostaremos nisso para criar nosso aplicativo. Entretanto, precisamos nos atentar a um detalhe.

3.4.5. Usuários com visão não-debilitada

Nosso aplicativo também precisa ser utilizável por pessoas com visão não-debilitada, para isso, é necessário que nos atentemos a um detalhe. Segundo o site “uxmovement”, um portal dedicado a compartilhar informações sobre design de interfaces de usuários, sistemas e páginas Web, não se pode usar o preto absoluto 100% das vezes

(<https://uxmovement.com/content/why-you-should-never-use-pure-black-for-text-or-backgrounds/>).

Embora preto puro e branco intenso sejam muito bons para solucionar nossa dúvida de escolha quanto ao par de cores para o alto-contraste (conforme apontado no artigo), para usuários com visão não-debilitada, ler texto em branco com fundo em preto puro pode levar ao cansaço dos olhos. Por isso incluiremos uma forma de regular a cor de fundo do aplicativo para algum tom de cinza mais claro, ou tons de cinza mais escuro (ou preto) caso o alto-contraste esteja ativado.

3.5. Começando o Desenvolvimento!

Agora que já temos embasamento teórico do que é esperado que façamos neste aplicativo, passemos ao desenvolvimento em si. Para começar, criaremos um novo projeto no App Inventor. O nome deste aplicativo será “Super Port Mat” para fácil assimilação com o conteúdo (um jogo de perguntas e respostas de português e matemática). Como sempre, começaremos definindo nossa primeira tela, a Screen1.

Conforme apontado no início do capítulo, durante a apresentação do conceito do aplicativo, a Screen1 será responsável por mostrar o menu principal do aplicativo, fazendo uma triagem do que o usuário quer fazer no mesmo. Isso inclui: ver informações sobre o aplicativo, acessar a tela de configurações, indicar o início de uma partida do jogo (seja no modo competitivo ou no modo de aprendizado) ou apenas sair do aplicativo.

Antes de começar a mexer nos componentes que formarão a nossa tela e nos darão acesso às funcionalidades programáveis que formaremos posteriormente, é preciso configurá-la. Na parte de propriedades da Screen1 definimos a tela como rolável, porque incluiremos bastantes componentes de interface gráfica que ocuparão muito espaço na tela, assim como um dimensionamento responsivo para que a organização visual dos componentes não se desmantele quando o usuário mudar a orientação do dispositivo. O título da tela será “Menu Principal”, para que o usuário possa melhor se localizar no aplicativo.

Quanto à orientação padrão da tela, poderíamos definir como paisagem ou retrato, mas é melhor deixar isso a cargo do sensor sendo que o usuário pode controlar em qual forma de apresentação o aplicativo pode ser mais bem utilizado. Agora, na propriedade ícone, podemos definir o ícone que será a cara do aplicativo no menu de aplicativos do smartphone do usuário, para tal criamos uma logo para o App que pode ser vista na figura 93.



Figura 93: Logo do aplicativo Super Port Mat.

(Fonte: Do Autor)

Também usaremos um componente de imagem para mostrar esse mesmo ícone na Screen1, ao topo do menu principal. Já que o App Inventor não possui uma propriedade referente a texto alternativo de uma imagem, proveremos um componente de legenda logo abaixo deste ícone para que o usuário saiba que se trata de nosso aplicativo. A propriedade de nome do aplicativo será “Super Port Mat”, evitando que o App Inventor assuma que o nome do App tem que ser o mesmo do projeto criado na plataforma (com caracteres de **underscore**¹⁸¹ no lugar de espaços, números de versionamento etc.).

¹⁸¹ “Underscore” é um caractere (“_”), muitas vezes chamado erroneamente de “underline” que é muito usado na programação para substituir espaços. Underline, do inglês, significa “sublinhar”, é o ato de usar o caractere de underscore junto a outros caracteres para criar uma linha horizontal sob o que está escrito). A essa técnica de substituir espaços por underscores se dá o nome de “snake_case”, a mesma que estamos usando para nomear funções e variáveis neste livro.

Em seguida, usaremos 4 componentes de organização vertical. Cada um deles servirá para uma coisa:

1. Painel Inicial (“Cabecalho”): painel decorativo contendo o nome do App e seu respectivo ícone.
2. Botões de Menu (“MenuPrincipal”): conterá os botões de “Jogo Competitivo”, “Apenas Aprendizado”, “Configurações” e “Sobre”.
3. Informações sobre o App (“PainelSobre”): uma pequena descrição do aplicativo e um botão para fechar o painel que a contém.
4. Entrada no Jogo (“Confirmacao”): um painel que pode ser acessado pelos botões de jogo competitivo ou de aprendizado. Esse painel filtra qual é a área do saber que o usuário deseja abordar no jogo.

Confira todos os componentes integrantes destes painéis na figura 94.

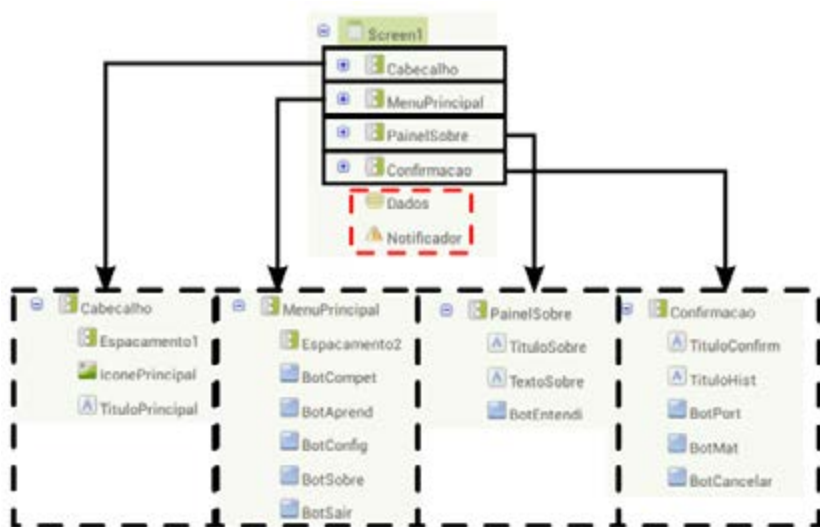


Figura 94: Painéis (Organizações Verticais) que formam os conjuntos de elementos da Screen1, em vermelho estão destacados o componente de notificação e o TinyDB.
(Fonte: App Inventor)

Mas apenas olhar a árvore de componentes adicionados na tela não nos dá uma ideia muito precisa dos aspectos visuais desta tela. Como nossas configurações de acessibilidade do aplicativo afetarão a parte visual, não alguns aspectos serão mudados em tempo de execução (mais apropriadamente, cores de fundo, tamanhos de texto e cores de texto). Entretanto, larguras, alturas e posicionamentos continuam intactos. Veja o aspecto visual conceitual do aplicativo na figura 95.



Figura 95: Aspecto visual conceitual do aplicativo Super Port Mat.

(Fonte: App Inventor)

Para explicar as conexões entre os componentes desta tela, usaremos um fluxograma simplificado. Segundo o site LucidChart (<https://www.lucidchart.com/pages/pt/o-que-e-um-fluxograma>), um fluxograma é uma representação gráfica de um algoritmo. Todo fluxograma tem um início e pode ou não ter fim. Ainda segundo LucidChart (<https://www.lucidchart.com/pages/pt/fluxograma-simbologia>), cada forma geométrica em um fluxograma tem um determinado significado, as principais são:

- Retângulo: um processo, uma ação a ser realizada.
- Forma oval: interruptor de terminal, denota o início ou fim de um algoritmo.

- Losango: tomada de decisão, uma condicional, assim como os comandos SE das linguagens de programação.

Veja na figura 96 o fluxograma que explica como os painéis e botões da Screen1 se conectam e podem levar o usuário para outras telas.

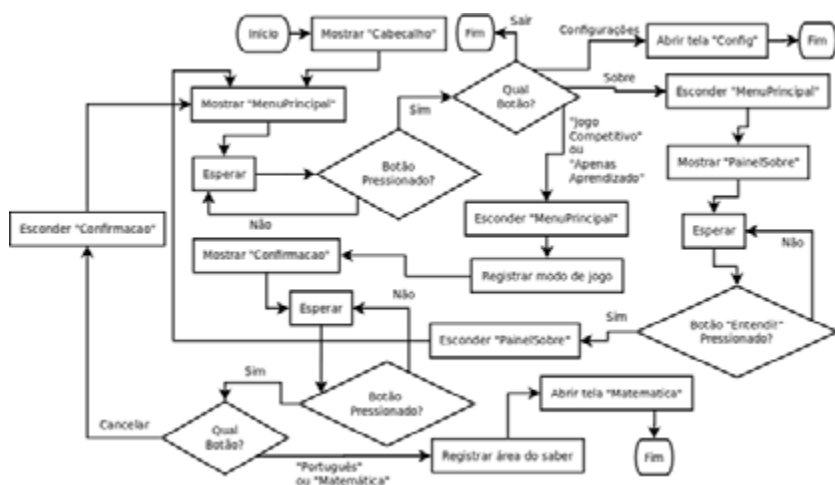


Figura 96: Fluxograma simplificado explicando as ligações entre os painéis e botões da tela Screen1 entre si mesmo e outras telas.

(Fonte: Do Autor)

Calma! Não fique assustado com todos esses símbolos, frases e setas! Pense que isso é um mapa e o lugar onde estamos agora é aquela forma meio arredondada na parte superior da figura. De fato um fluxograma é um mapa, perceba que as setas indicam os possíveis caminhos pelos quais podemos ou devemos trafegar.

Pode parecer um tanto quanto confuso, mas um fluxograma é, nada mais, nada menos, uma representação gráfica de uma sequência de passos. Note que tudo começa no terminal “Início” na parte superior da figura, a partir dali são executados dois conjuntos de instruções que estamos identificando genericamente por “Mostrar Cabecalho” (correspondente a assimilar o valor verdadeiro à propriedade “visível” deste componente) e “Mostrar MenuPrincipal”.

É claro que fluxogramas podem ficar muito maiores e complexos conforme um projeto cresce e mais componentes, elementos, tecnologias etc. são incorporados a ele. Mas para manter as coisas simples e introduzir este conceito, manteremos os procedimentos e funções do fluxograma numa alta abstração, para que fique mais simples de entender.

Perceba que, depois do passo “Mostrar MenuPrincipal”, executamos uma instrução chamada “Esperar”. Esse procedimento do fluxograma na verdade não é uma instrução real, usamos isso apenas para ilustrar o momento em que o usuário está apenas olhando para a tela ou ainda não interagiu com ela de alguma forma. Como nenhum botão está sendo pressionado, continuamos esperando uma interação do usuário.

Quando o usuário interagir com a tela pressionando algum dos botões no componente de organização que chamamos de “MenuPrincipal”, o fluxograma entra numa cadeia de duas condicionais, a primeira terá como resposta um “Sim” (pois o usuário pressionou um botão) que mudará o caminho do fluxo de processamento e nos levará à segunda condicional.

Na segunda condicional, fazemos uma triagem para saber qual botão foi pressionado, essa condicional representa os eventos que os botões lançam e são interceptados pelo nosso código. Caso o usuário tenha pressionado o botão “Sair”, apenas encerramos o aplicativo (perceba o fluxograma levando ao terminal “Fim”). Caso não tenha pressionado o botão de sair, ainda temos dois caminhos de fluxo de processamento para tratar.

O primeiro é o caso do botão “Sobre”, ao pressioná-lo, esconderemos o “MenuPrincipal” e mostraremos o componente “PainelSobre” com a breve descrição do aplicativo. Como esse painel só possui um botão com o qual o usuário pode interagir, entramos em um laço de espera de interação novamente. Enquanto o botão “Entendi!” não for pressionado, continuaremos esperando esta interação.

Assim que o usuário pressionar o botão, esconderemos o Painel-Sobre e retornaremos a um dos procedimentos iniciais do fluxograma, o “Mostrar MenuPrincipal”, recomeçando o ciclo.

Por outro lado, se o usuário tivesse pressionado o botão “Jogo Competitivo” ou o botão “Apenas Aprendizado”, teríamos seguido por um fluxo de processamento diferente: primeiro esconderíamos o MenuPrincipal, em seguida, **armazenaríamos qual foi o tipo de jogo escolhido com base no botão pressionado**¹⁸² e, posteriormente, mostraríamos o painel “Confirmacao” que possui três botões: português, matemática e cancelar.

A partir deste momento entramos em outro laço de espera por uma interação do usuário. Se ele não pressionar qualquer botão, continuamos esperando. Se pressionar, fazemos uma nova avaliação de qual botão foi pressionado. Caso o botão “Cancelar” tenha sido pressionado, então esconderemos o componente organização Confirmacao e mostraremos o MenuPrincipal, recomeçando o ciclo.

É digno de nota que não precisamos incluir um comando que mude a visibilidade do MenuPrincipal assim que Screen1 for aberta, podemos configurar a visibilidade dele pela própria sessão de design como visível em suas propriedades e deixar os outros painéis como invisíveis.

Veja na figura 97 como fica o código que trata tudo o que foi abordado nesse fluxograma.

182 Nesse ponto teríamos outra triagem de botão, mas simplificamos isso para diminuir o tamanho do gráfico do fluxograma.

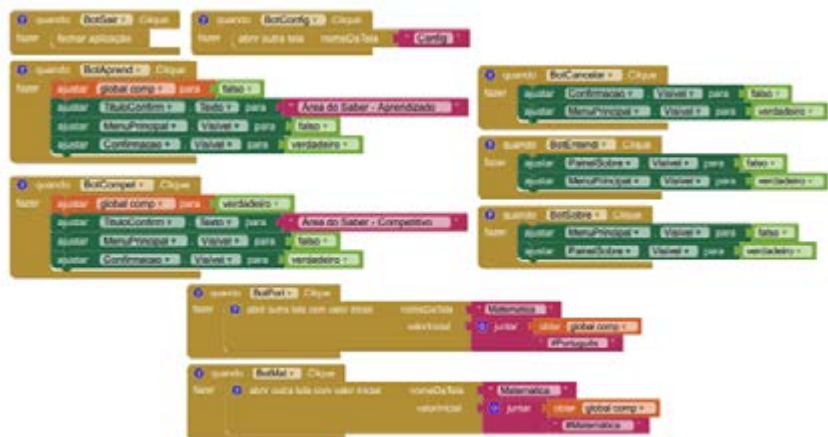


Figura 97: Código de alternância do atributo visível dos painéis e abertura/fechamento de telas de acordo com as entradas do usuário.

(Fonte: App Inventor)

Também adicionamos um mecanismo que evita o fechamento imediato do aplicativo caso o usuário pressione acidentalmente o botão de voltar do Android. Além disso, perceba na figura 98 que, perto desta parte do código, temos uma função que se chama “ao_mostrar_tela”.

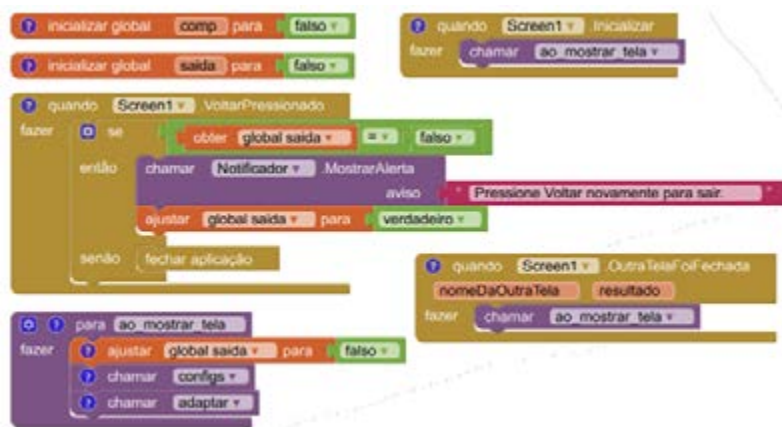


Figura 98: Código anti fechamento imediato e função “ao_mostrar_tela”.
(Fonte: App Inventor)

Essa função tem a única finalidade de assimilar o valor falso à variável global “saida” (para fazer valer o código que impede o fechamento imediato da tela) e de executar outras duas funções, uma chamada “configs” e outra chamada “adaptar”. Nossa tela Screen1 é dotada de um componente TinyDB que chamamos de “Dados”, dentro deste componente, uma de suas propriedades (namespace) está configurada como “CONF”, esse é o espaço de nomes que usaremos para guardar nossas configurações do aplicativo.

É aí que a função “configs” entra em ação. Seu objetivo é verificar se as variáveis que usamos para armazenar as configurações do aplicativo já foram devidamente criadas. Fazemos isso por meio da leitura de um rótulo chamado “inicializado”. Se o valor deste rótulo for falso, então realizamos todas as alocações de espaço para nossas variáveis e assimilamos valores padrão a elas.

Quando o aplicativo for executado pela primeira vez, nenhuma variável vai existir em qualquer espaço de nomes do nosso componente “Dados”, nem mesmo a “inicializado”. Portanto, ao fazer a leitura do rótulo, indicamos ao TinyDB que ele deve retornar o valor falso caso a variável não exista, o que é o nosso caso. Sendo assim, toda vez que o usuário abrir o App pela primeira vez ou limpar os dados dele nas configurações de aplicativo nativas do SO, executaremos essa função e criaremos as variáveis necessárias (ao fim criando o rótulo da variável “inicializado” com o valor verdadeiro).

Agora, caso o usuário já tenha aberto o aplicativo uma vez, nossas configurações estão gravadas, assim como a variável “inicializado” com o valor verdadeiro, isso indicará ao nosso App que não é preciso recriar tudo e economizaremos processamento, assim como evitaremos que o usuário tenha que reconfigurar o aplicativo porque teríamos sobrescrito todas as configurações. Verifique na figura 99 como é o código base da função “configs”.



Figura 99: Código base da função “configs” que inicializa variáveis de configurações no TinyDB.
(Fonte: App Inventor)

Para deixar explícito, verifique, na tabela 23, quais são as variáveis de configurações que estamos criando no TinyDB e quais valores padrão estamos assimilando a elas para que o aplicativo funcione da forma adequada.

Tabela 23: Variáveis usadas para configuração do aplicativo Super Port Mat.

Variável	Valor Padrão	Explicação
tam_texto	20	Tamanho base dos textos exibidos pelo aplicativo, mudar esse valor mudará todos (ou quase todos) os tamanhos de textos mostrados em componentes do App.
cor_texto	Cor Branca	Cor em que o texto do aplicativo será exibido.
cor_painel	Cor personalizada (32 de cada cor RGB)	Cor de fundo dos componentes que possuem uma propriedade “CorDeFundo” na sessão de design, comumente presente em botões e organizações verticais ou horizontais.
alto_con	Valor lógico verdadeiro	Indica ao aplicativo se a especificação de cor de fundo pode chegar ao preto absoluto ao mesmo tempo que proíbe tons de cinza muito claros.
jogadores	1	Número de jogadores que participarão de uma partida.

(Fonte: Do Autor)

Mas você também pode ter percebido que chamamos essas funções dentro da “ao_mostrar_tela”. Por que não as chamamos no próprio evento de Inicializar da tela? Para entender isso, é preciso explicar um conceito mais aprofundado de Pilhas que é aplicado na maioria das linguagens de programação.

3.5.1. Pilha de funções

Nosso programa é uma coletânea de funções onde as instruções são executadas sequencialmente e uma função pode chamar a outra. Mas você já percebeu que quando uma função chama a outra, ela só continua a ser executada quando a função chamada termina?

Vamos imaginar uma situação para exemplificar: pense que queremos comer bolo, mas não temos um bolo, primeiro precisamos fazê-lo. Assim que terminarmos de reunir os ingredientes, misturamos eles de forma sistemática e pronto, temos um bolo! Por fim, podemos comê-lo.

Se convertêssemos isso para código, teríamos uma função chamada “comer_bolo” que verificaria, primeiro, se temos um bolo. Como não temos, ela chama a função “fazer_bolo” que, por sua vez, chama a função “reunir_ingredientes” e depois prepara o bolo efetivamente.

Quando o bolo fica pronto, retornamos à função “comer_bolo” exatamente do ponto onde paramos e, agora que temos um bolo, podemos efetivamente comê-lo. Perceba que passamos por três níveis da pilha de funções. Lembra-se do conceito de pilha (lá do tópico de estruturas de dados no capítulo 2)? Os elementos que entram primeiro (a base da pilha) são os últimos a sair porque só podemos tirar elementos do topo dela.

Com funções isso também acontece! Quando chamamos uma função, a função chamada é empilhada acima da que a chamou, sendo assim, ambas as funções ocupam memória, mas a função que a chamou só volta a ser processada quando a que foi chamada termina.

Em alguns sistemas mais limitados em memória e processamento isso pode ser uma armadilha, porque chamar funções demais (uma

dentro da outra) pode esgotar rapidamente a memória RAM do dispositivo. Então, quando uma nova função for chamada, o sistema operacional não terá mais como alocar espaço e teremos um erro clássico no mundo da programação: o estouro de pilha. Pela falta de espaço na memória, o SO fica sem memória para seus próprios processos vitais do sistema.

Sendo assim, o estouro de pilha pode levar a comportamento indefinido, lentidão, travamento e até ao desligamento da máquina. Para ilustrar o que acontece, veja o estouro de pilha exemplificado na figura 100.

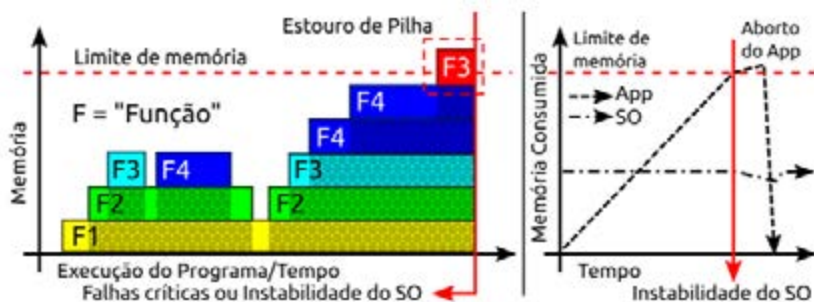


Figura 100: Funções são chamadas e empilhadas sucessivamente até que a memória disponível não consiga comportar todos os processos e o SO ao mesmo tempo, causando instabilidade.

(Fonte: App Inventor)

Essa é uma das razões pelas quais é preciso usar os recursos computacionais com sabedoria. Mas o ponto é, quando uma função chama outra, seu processamento é pausado e só é retomado quando as funções chamadas terminam de ser executadas (perceba a parte sombreada nos retângulos das funções na figura). Mesmo assim, elas continuam carregadas na memória com todos os seus dados guardados e congelados.

3.5.2. Pilha de Telas

No App Inventor, vimos que uma tela pode chamar a outra e isso funciona da mesma forma que a chamada de funções! Quando chamamos uma tela, a tela que chamou é congelada para que seu

processamento seja retomado assim que a tela chamada acabar seu processamento. Criamos uma função chamada “ao_mostrar_tela” porque o código dentro dela não será executado apenas pelo evento de Inicializar da tela Screen1.

Imagine a seguinte situação: você vai a um parque de diversões que só cobra o ingresso da entrada, os brinquedos são gratuitos uma vez dentro do parque. Você paga o ingresso uma vez e só voltará a pagá-lo quando, depois de ter ido embora, voltar no outro dia! Em nenhum momento um funcionário abordará você dizendo “Perdão pela intrusão, mas seu ingresso no parque tem que ser refeito porque você saiu de uma área temática e passou perto da entrada!”.

Quando abrimos o aplicativo, o evento Inicializar da Screen1 é executado. Depois disso, ele só será executado quando o aplicativo for aberto novamente (depois de fechado). O evento Inicializar não é disparado quando a tela aparece no visor do dispositivo, mas sim quando é alocada na memória! Quando saímos de uma tela para voltar à Screen1, ela já foi alocada, então o evento Inicializar não será disparado.

Durante toda a execução do aplicativo, o evento Inicializar da Screen1 só é executado uma vez. Mas precisamos que a função “adaptar” seja executada toda vez que a Screen1 aparecer na tela do dispositivo. Felizmente, há apenas duas situações em que uma tela aparece no visor: quando ela é alocada e exibida para uso e quando ela é mostrada após o fechamento de outra tela.

Se, enquanto estivermos na sessão de blocos e mecânica do App Inventor, clicarmos no componente da Screen1, podemos notar que há exatamente um evento chamado “OutraTelaFoiFechada”. Perceba também que este evento tem como parâmetros “nomeDaOutraTela” e “resultado”. Verifique na tabela 24 para quê serve cada um destes parâmetros.

Tabela 24: Parâmetros do evento “OutraTelaFoiFechada” componente Tela e seus objetivos.

Parâmetro	Objetivo
nomeDaOutraTela	Caso um código diferente precise ser executado dependendo da tela que foi fechada, o argumento desse parâmetro pode ser usado para distinguir qual tela foi fechada.
resultado	Caso a tela funcione como uma função e, ao término de seu processamento, retorne um valor, ele pode ser recebido pela função requisitante como argumento deste parâmetro.

(Fonte: App Inventor)

Como não faremos nenhum tratamento especial dependendo da tela fechada, apenas incluir a chamada da função “ao_mostrar_tela” no escopo do evento suprirá nossas necessidades. Agora chegou a hora de explicar o que acontece na função “adaptar”.

3.5.3. Acessibilidade agora mesmo

Durante o desenvolvimento deste aplicativo, você poderá perceber que repetiremos uma certa estrutura de códigos para cada tela. Essa estrutura será algo como, por exemplo, nossa “central de efetivação de configurações de acessibilidade”. É importante notar que só mudar uma variável não vai fazer todos os componentes mudarem suas propriedades automaticamente.

Por causa disso, teremos que botar a mão na massa e mudar as propriedades visuais de cada componente presente na tela manualmente. Parece um monte de trabalho, não é? Mas é mesmo! Embora haja um jeito semiautomático de percorrer componentes de uma tela e mudar suas propriedades, lidaremos com componentes de tipos diferentes (principalmente botões, organizações e legendas), o que tornará o processo tão trabalhoso quanto.

A função “adaptar” tem a única função de chamar outras 4 funções: “obter_config_local”, “adaptar_tam_textos”, “adaptar_cor_textos”, “adaptar_cor_paineis”. Além disso, a função “obter_config_local” depende de outra função chamada “obter_config” e de três variáveis

globais chamadas “tbt” (tamanho base do texto), “cbt” (cor base do texto) e “cbp” (cor base de **painel**¹⁸³).

As funções que adaptam cores podem usar os valores de suas variáveis globais diretamente. Já a função “adaptar_tam_textos” depende de três funções auxiliares: “titulo”, “subtitulo” e “texto”. Verifique na tabela 25 para quê cada um destes elementos serve.

Tabela 25: Conjunto de funções e variáveis usados para efetivar configurações de acessibilidade.

Variável	Objetivo
tbt	Guardar o tamanho base dos textos que serão mostrados no aplicativo.
cbt	Guardar a cor base dos textos que serão mostrados no aplicativo.
cbp	Guardar a cor base dos componentes que possuem a propriedade “CorDeFundo” no aplicativo.
Função	Objetivo
adaptar	Chamar as funções envolvidas na efetivação das configurações de acessibilidade.
obter_config	Mudar o espaço de nomes do componente TinyDB presente na tela para o espaço de nomes em que as configurações estão guardadas e retornar o valor de um rótulo informado pela função requisitante.
obter_config_local	Assimilar o valor dos rótulos “tam_texto”, “cor_texto” e “cor_painel” às variáveis “tbt”, “cbt” e “cbp” respectivamente por meio da função “obter_config”.
adaptar_cor_textos	Ajustar a cor de todos (ou quase todos) os componentes capazes de mostrar texto na tela.
adaptar_cor_paineis	Ajustar a cor de todos os componentes que possuem a propriedade “CorDeFundo”.
adaptar_tam_textos	Ajustar o tamanho do texto de todos (ou quase todos) os componentes capazes de mostrar texto na tela. Utiliza-se de três funções auxiliares para diferenciar texto comum de títulos e subtítulos.

¹⁸³ Nesse contexto de efetivação de configuração de acessibilidade, painel significará qualquer componente que tenha a propriedade “CorDeFundo”.

titulo	Devolver o tamanho base de texto com acréscimo de 50%
subtitulo	Devolver o tamanho base de texto com acréscimo de 25%
texto	Apenas devolver o tamanho base do texto.

(Fonte: Do Autor)

É digno de nota que poderíamos armazenar os valores multiplicadores do tamanho base no componente TinyDB, mas usaremos estas constantes para manter o código menos complexo e porque atende às nossas necessidades.

Prosseguindo, quando a função “adaptar” é chamada, a primeira função a ser executada é a “obter_config_local”. Sendo assim, a função ajusta o espaço de nomes do componente Dados (TinyDB) para “CONF”, onde estamos armazenando as nossas configurações. Em seguida, a função ajusta os valores das variáveis globais “tbt”, “cbt” e “cbp” respectivamente para o valor das chamadas da função “obter_config” com os parâmetros “tam_texto”, “cor_texto” e “cor_painel”. Confira os blocos na figura 101.

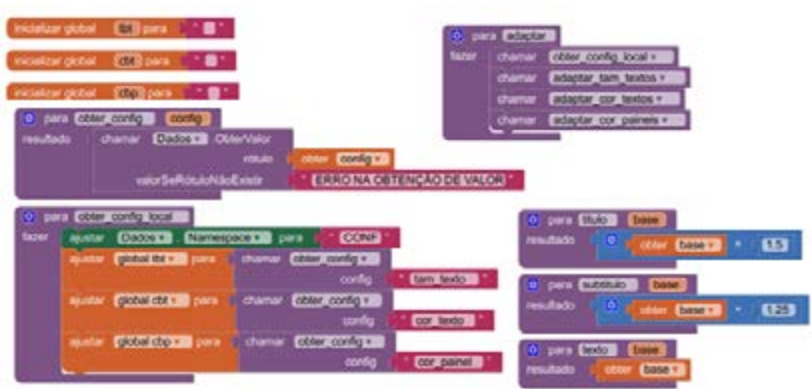


Figura 101: Funções auxiliares do processo de adaptação às configurações de acessibilidade.

(Fonte: App Inventor)

Perceba que a única coisa que a função “obter_config” faz é acessar o componente Dados e requisitar o rótulo passado a ela pelo parâmetro “config”. No mais, ela apenas retorna uma mensagem de erro padroni-

zada caso o rótulo não exista. Assim que as três variáveis globais têm seus valores supridos pelas respectivas funções, retornamos à função “adaptar”.

Em seguida, temos a chamada da função “adaptar_tam_textos”, e como dissemos, sua função é mudar o tamanho dos textos. Sendo assim, temos que reunir todos os blocos de modificação de atributos dos componentes capazes de mostrar texto na tela e que tenham um tamanho ajustável. Para cada um desses blocos, forneceremos o resultado das funções de modificação de tamanho de texto baseado no conteúdo do componente que estamos alterando. As três funções receberão o tamanho base mas, caso seja mais cômodo usar o tamanho base diretamente em vez da função “texto”, isso pode ser feito.

Depois de adaptar o tamanho do texto, é hora de mudar sua cor e a cor dos “painéis” (qualquer componente com o atributo “CorDe-Fundo”). Usaremos as funções restantes “adaptar_cor_textos” e “adaptar_cor_paineis”. A estrutura delas é exatamente a mesma que a da função “adaptar_tam_textos”, exceto pelo fato de que usaremos diretamente as variáveis globais que possuem as cores! Ao fim, veja como podem ficar nossas três funções de adaptação na figura 102.

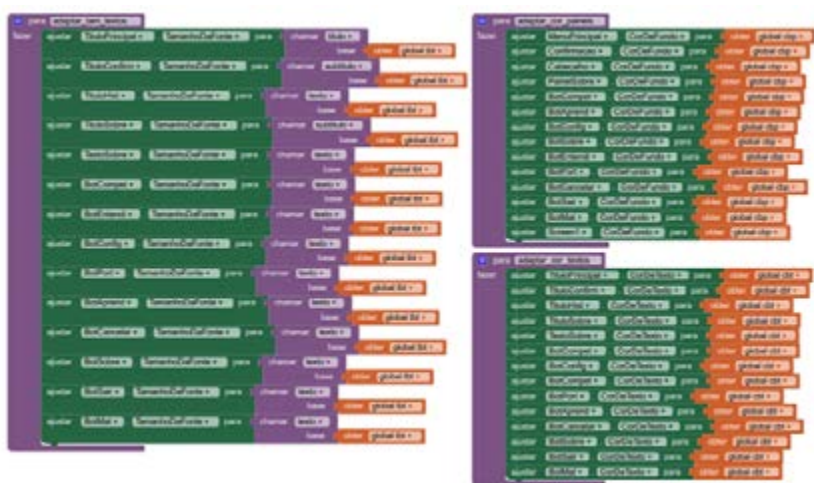


Figura 102: Funções de adaptação de tamanho de texto, cor de texto e cor de fundo. (Fonte: App Inventor)

Com isso, terminamos nossa tela de Menu Principal. Mas, antes de prosseguirmos para a tela de configuração, é interessante que transfi-ramos nossas funções e variáveis globais chamadas pela função “adaptar” para a mochila, já que reutilizaremos estes códigos. É digno de nota que as funções de adaptação de cor e tamanho devem ter apenas o bloco de procedimento transferido (não possuindo nenhum bloco em seu escopo) já que lidaremos com componentes de mesmo tipo mas de nomes diferentes. Verifique na figura 103 como ficará nosso aplicativo quando a função “adaptar” for executada.



Figura 103: Diferença entre o projeto de aplicativo no App Inventor e o aplicativo de fato, após a execução da função “adaptar”.

(Fonte: Android e App Inventor)

3.5.4. Tela de Configuração do Aplicativo

De forma simples e direta, a tela de configurações servirá para que o jogador possa: (1) criar arquivos de problemas (perguntas/respostas) para outras partidas, (2) ajustar o número de jogadores em uma partida, (3) mudar a cor de fundo do App (alto-contraste) e (4) ajustar o tamanho

do texto. Para além disso, é importantíssimo ressaltar que cada configuração deve ser salva e se tornar efetiva assim que é alterada!

3.5.4.1. Composição da Tela

Primeiro de tudo, temos que formar nossa tela pegando os componentes necessários, sendo assim, foquemos na sessão e design do App Inventor. Os componentes que utilizaremos são (além da tela):

- Organização Vertical;
- Legenda;
- Botão;
- EscolheLista;
- Deslizador;
- Caixa de Checagem;
- TinyDB;
- Temporizador;
- Notificador.

Verifique estes componentes e como eles estão arranjados na hierarquia da tela conforme o que foi ilustrado na figura 104.

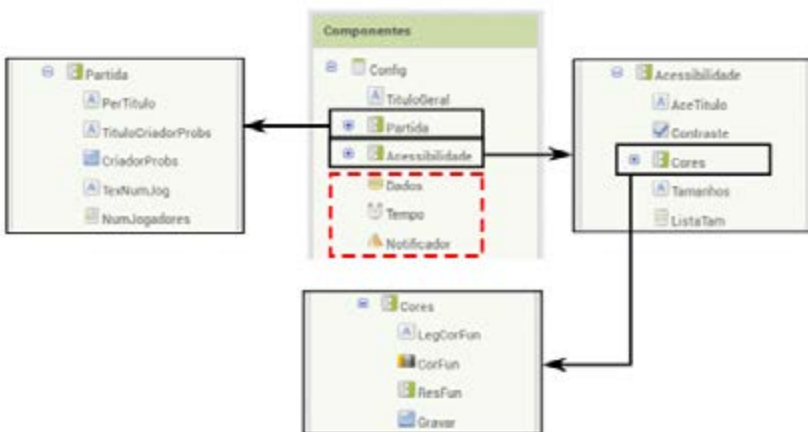


Figura 104: Componentes que formam a tela de configurações e o arranjo de painéis.
(Fonte: App Inventor)

Repare que também faremos uso da ideia de painéis nessa tela. As configurações do aplicativo podem pertencer a duas categorias: partida e acessibilidade. Utilizaremos a categoria de partida para abrigar as opções de definir número de jogadores da partida e para dar acesso à nossa tela onde ficará o criador de problemas para novas partidas.

Cada categoria terá um painel para si. Porém, usaremos um subpainel na categoria de acessibilidade para agrupar o controle de cor de fundo do aplicativo, já que para isso usaremos 4 componentes (todos com representação gráfica). Esse subpainel só deverá ser mostrado caso a caixa de checagem da opção alto-contraste estiver ativada.

Agora veja na figura 105 como fica a parte gráfica da tela de configurações do aplicativo. Cores de fundo/texto e tamanhos de fonte não importam muito já que, assim como na Screen1, essas configurações serão alteradas pela função “adaptar” quando requisitada.

(Fonte: App Inventor)

3.5.4.2. Partindo para o código-fonte

Falando nessa função, é hora de copiarmos todos os itens que deixamos na mochila e trazê-los para esta tela! Para fazer isso, é preciso clicar no ícone da mochila, no canto superior direito da área de edição de blocos. Perceba que um menu irá se abrir e lá estarão nossas funções criadas na Screen1, prontas para uso!

Para trazê-las da mochila para a edição de blocos, basta clicar e soltar o pedaço de código desejado, ou mesmo arrastar da mochila para a área como se fosse a lista de blocos que um componente oferece quando clicamos nele na lista de componentes presentes na tela ao lado esquerdo da sessão de blocos e mecânica. Verifique isso na figura 106.

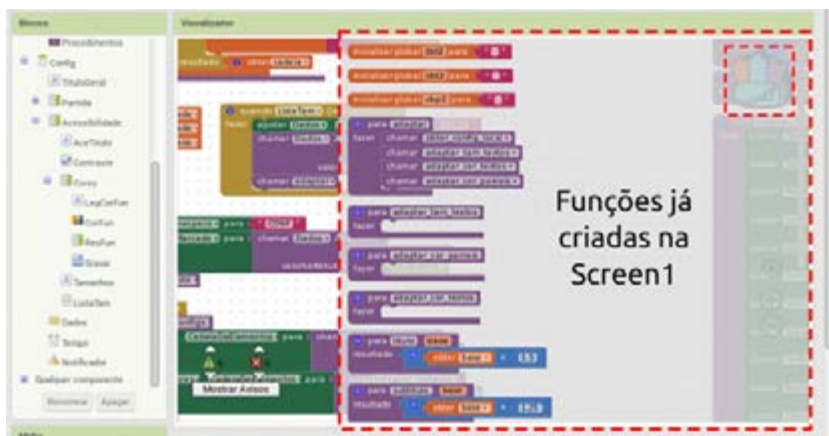


Figura 106: Funções de efetivação de configurações de acessibilidade já criada e guardadas na mochila.

(Fonte: App Inventor)

Quando todas as nossas funções de acessibilidade estiverem devidamente posicionadas no código e acessando todos os componentes necessários para fazer valer as configurações de acessibilidade, teremos completado aproximadamente um terço do código desta tela!

3.5.4.3. Inicialização da Tela

Agora que já garantimos a efetivação das configurações de acessibilidade, trabalhemos no código essencial da tela. Como de costume, começaremos pelo evento de Inicializar da tela Config. Nesse evento, a primeira coisa que precisamos fazer é executar a função “adaptar” e depois a “restaura_configs”. Você já se deparou com algum programa ou aplicativo que, ao abrir uma tela ou painel de configurações, todas as opções estavam com seus valores padrão independente de qualquer modificação feita?

Esse é um problema porque pode confundir usuários e fazer com que eles não tenham ideia de qual seja a configuração atual. A função “restaura_configs” resolve exatamente esse problema fazendo com que os componentes relacionados às opções tenham seus devidos estados e valores de acordo com as configurações vigentes.

Dentro da função “restaura_configs” temos apenas a chamada de outras funções que lidarão com a restauração de configurações, elas são chamadas “alto_con”, “num_jogadores” e “tam_texto”. Verifique na tabela 26 qual é o objetivo de cada uma dessas funções.

Tabela 26: Funções de restauração de configurações e seus objetivos.

Função	Objetivo
alto_con	Restaurar o estado da caixa de checagem da opção de alto-contraste de acordo com o que foi armazenado no TinyDB, assim como refletir a última cor de fundo escolhida para o aplicativo no deslizador correspondente.
num_jogadores	Informar o número de jogadores atualmente configurados para participar de uma partida.
tam_texto	Informar o tamanho base de texto configurado atualmente para ser usado.

(Fonte: Do Autor)

Algo comum a todo código que lidará com configurações é a modificação do espaço de nomes do componente Dados (TinyDB) para “CONF”, onde armazenamos nossas configurações. Prosseguindo, dentro da função “alto_con” temos uma alteração de atributo “Marcado” do componente “Contraste” (caixa de checagem responsável por ativar ou desativar o alto-contraste do aplicativo) para o valor que consta no rótulo da configuração correspondente em nosso armazenamento. Em seguida, chamamos a função “mudar_contraste”.

A função “mudar_contraste” serve para aplicar a cor de fundo armazenada no TinyDB ao deslizador, de forma que o deslizador e a prévia de cor retratem a cor que está sendo usada no momento (ou que está sendo escolhida pelo usuário). Sendo assim, ao entrar na função, primeiro verificamos qual é o estado do componente Contraste.

Se sua caixa de checagem estiver marcada ou não, mudamos os valores de máximo e mínimo do deslizador para concordar com o uso ou não do alto-contraste. No caso, usaremos o intervalo de zero a 32 para alto-contraste e de 32 a 80 para uma versão não alto-contraste. É importante notar que os dois intervalos permitem que a taxa de contraste possa ser ajustada de 21 a, no máximo, 8,06. Sendo assim,

qualquer tom de cinza de 1 a 80 (valores iguais para as três cores RGB) passa nos testes WCAG (lembrando que, para isso, a cor do texto deve ser branco).

Assim que mudarmos o intervalo, é preciso mudar a posição do indicador para que ele reflita a cor de fundo sendo usada. Para isso, chamamos a função “restaurar_posicao_deslizador”. Dentro de seu escopo, essa função compara se a configuração de alto-contraste armazenada é igual ao estado da caixa de checagem na tela (verdadeiro ou falso). Se for, então só temos que pegar a cor armazenada no TinyDB, reparti-la e conseguir o valor numérico de qualquer cor primária para aplicá-lo como posição do indicador. Bingo! O deslizador agora reflete a cor atual.

Agora, se o valor da configuração não condisser com o valor da caixa de checagem e se fizermos a mesma abordagem, corremos o risco de tentar colocar um valor fora do intervalo permitido no indicador. Sendo assim, para que o usuário saiba que houve uma mudança na abordagem, colocaremos o valor do indicador como simplesmente o valor mínimo do intervalo. Verifique como as funções “restaurar_posicao_deslizador” e “mudar_contraste” ficariam na figura 107.

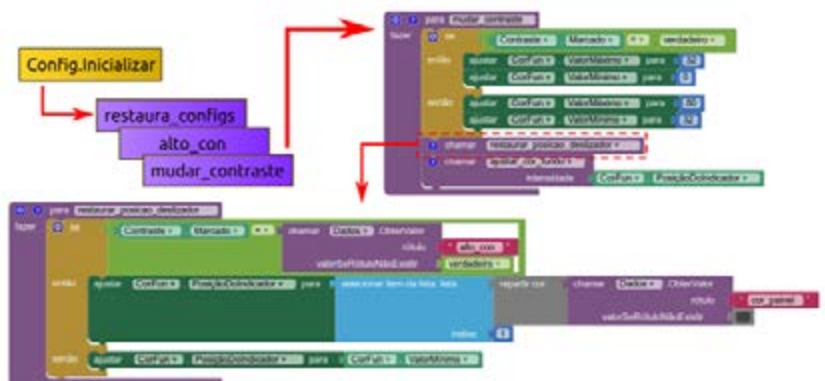


Figura 107: Caminho de chamadas de funções para fazer o deslizador e a prévia de cor refletirem a cor armazenada nas configurações.

(Fonte: App Inventor)

Perceba na figura que, antes de finalizarmos a função “mudar_contraste”, chamamos outra função de nome “ajustar_cor_fundo” com a posição do indicador como argumento. Essa função, apesar do nome, não mudará a cor de fundo dos componentes da mesma forma com que nossas funções de adaptação fazem, ela apenas mudará a cor de fundo do componente de organização que estamos usando logo abaixo do deslizador para exibir uma prévia da cor escolhida.

O que a “ajustar_cor_fundo” faz é apenas pegar a posição numérica do indicador, arredondá-la e generalizá-la para as três cores primárias RGB (colocando-as numa lista), criando a nova cor que será usada pelo componente de organização. Mas é preciso lembrar de chamar essa função a cada vez que o usuário alterar a posição do indicador no deslizador. Podemos fazer isso chamando a “ajustar_cor_fundo” assim que o evento “PosiçãoAlterada” do deslizador for disparado. Sendo assim, o usuário pode ver em tempo real qual cor está escolhendo para o fundo. Veja como essa função é e como ela se conecta com o evento na figura 108.



Figura 108: Função que mostra a prévia de cor de fundo escolhida pelo usuário e sua chamada em tempo real por meio de um evento.

(Fonte: App Inventor)

Agora que terminamos de mostrar a prévia de cor de fundo e configuramos corretamente os intervalos do deslizador de acordo com a caixa de checagem, as funções terminam sua execução e são desempilhadas, retornando à função “restaura_configs”. Temos que executar a segunda instrução, chamar a função “num_jogadores”.

A função “num_jogadores” tem como primeiro ato configurar o espaço de nome do componente Dados como “CONF”, depois, ela apenas muda o texto mostrado pela legenda que explica a funcionalidade de definir o número de jogadores mostrando quantos jogadores são permitidos na partida no presente momento. É uma função bem simples, verifique como ela é na figura 109.

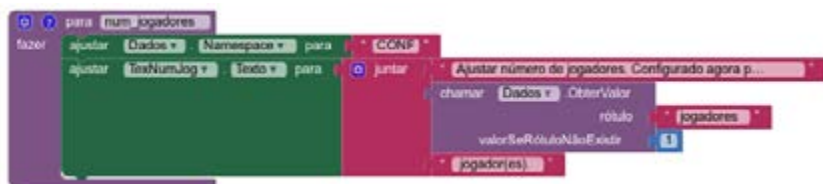


Figura 109: Função que atualiza a informação sobre o número de jogadores atualmente configurados para uma partida.

(Fonte: App Inventor)

Após isso, retornamos à função requisitante e executamos a função “tam_texto”. Sua tarefa é apenas mudar o texto da legenda que explica para quê serve o botão de mudar o tamanho base dos textos. Perceba na figura 110 que a estrutura é muito semelhante à da função “num_jogadores”.



Figura 110: Função que atualiza a informação sobre qual é o tamanho base de texto usado atualmente pelo aplicativo.

(Fonte: App Inventor)

3.5.4.4. Gravação de Configurações

Terminamos de restaurar as configurações para que o usuário possa se orientar, portanto, a função “restaura_configs” é desempilhada e retornamos ao evento de inicialização da tela. Lembre que estamos usando o componente EscolheLista para mostrar as opções de tamanho

de tela para o usuário. Esse componente possui uma propriedade chamada “CadeiaDeElementos” que funciona exatamente da mesma forma que a cadeia de elementos de um componente de lista suspensa!

O que temos que fazer é gerar um número de opções, cada uma contendo um número para cada uma das listas mostradas pelos componentes desejados. Para isso, estamos usando uma função chamada “listagem_numeros” que possui três parâmetros: valor_base, incremento e repeticoes. O que essa função faz é gerar uma cadeia de elementos aceitável pelos nossos componentes.

Ela recebe um valor inicial e soma o valor do incremento nele o tanto de vezes indicado pelo argumento no parâmetro repeticoes. Sendo assim, a cada adição de novo número, uma vírgula e um espaço são colocados antes do novo número para dar início a um novo item da lista no componente alvo. Internamente, a função usa mais duas variáveis: cadeia e contador.

A variável cadeia guarda o resultado final de todo o processamento da função, a lista completa com todas as opções separadas por uma vírgula e um espaço. A variável contador tem como objetivo acumular quantas adições de opções foram feitas para evitar que o laço continue infinitamente. Sendo assim, quando o valor na variável contador e no argumento do parâmetro repeticoes for igual, o laço acaba.

Dentro do laço principal da função, temos três operações. Primeiro, adicionamos o valor atual na lista (variável valor_base) preservando o valor inicial da variável cadeia (um texto vazio) e, ao fim do número, acrescentamos um texto com uma vírgula e um espaço (“,”). Segundo, incrementamos a variável valor_base com o argumento do parâmetro incremento para formar a próxima opção da lista. E terceiro, incrementamos a variável contador em uma unidade para contabilizar uma repetição.

Quando o laço terminar, precisamos adicionar o valor-base uma última vez, mas sem adicionar uma vírgula (para evitar que, ao fim da lista, haja uma opção em branco e isso venha a causar erros). Verifique na figura 111 como é a função “listagem_numeros” e como ela é

chamada pelo evento de inicialização da tela, assim como o resultado mostrado nos componentes EscolheLista.

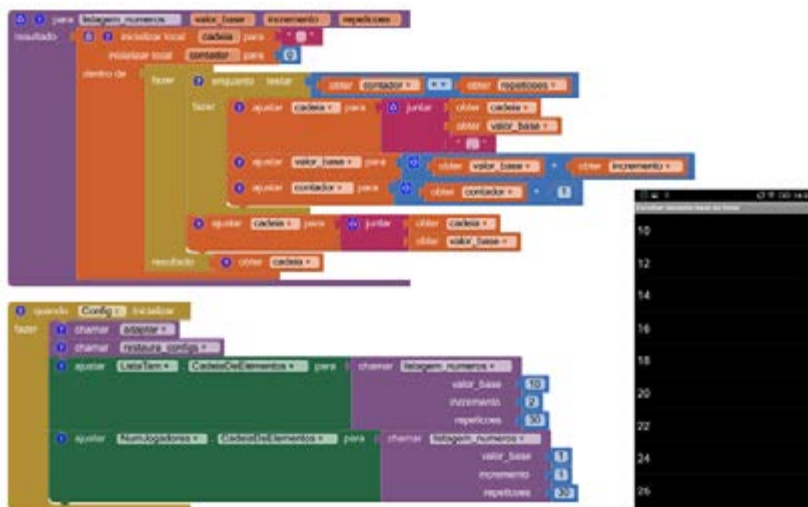


Figura 111: Criação de uma lista em texto para o componente EscolheLista e o resultado da listagem no aplicativo.

(Fonte: App Inventor e Android)

Como de costume, temos que adicionar um bloco para interceptar o evento de `VoltarPressionado`, fazendo com que a tela feche quando isso acontecer. Além disso, é preciso que, quando a caixa de checagem de alto-contraste for pressionada, chamemos a função “mudar_contraste” para adaptar o intervalo do deslizador que decidirá a nova cor de fundo do aplicativo.

Mas é preciso lembrar que o botão para abrir o criador de problemas de nosso aplicativo não precisa de nenhum processamento especial como nossas outras configurações. Então podemos apenas assimilar o comando de abrir outra tela ao evento de botão pressionado. Mas atenção, lembre-se que esta tela que abriremos é uma tela compartilhada, então teremos que abri-la com um valor inicial.

Para nossa conveniência, esse valor inicial será uma lista. O primeiro elemento desta lista deve indicar qual tipo de funcionalidade acessaremos na tela. Como estamos interessados no criador de

problemas, o primeiro (e único) elemento será um texto com o conteúdo “PROB”. Não precisamos passar nenhum elemento além desse já que o criador de problemas é uma parte independente do resto das funcionalidades do aplicativo.

Certo! Agora que a inicialização da tela terminou e já garantimos o funcionamento básico, temos que fazer com que o usuário possa editar de fato as configurações escolhendo o que é melhor segundo suas preferências. Começaremos com o componente `EscolheLista` que define o número de jogadores numa partida. Esse componente possui um evento chamado “`DepoisDeEscolher`”.

Dentro deste evento mudaremos o espaço de nomes do `TinyDB` para “`CONF`” e posteriormente armazenaremos o valor do atributo “`Seleção`” deste componente no rótulo “`jogadores`” em nossas configurações. Ao fim do tratamento do evento, apenas chamamos novamente a função “`num_jogadores`” para que a explicação da funcionalidade fique com o novo número de jogadores permitidos na partida.

Em seguida, trataremos da mudança do tamanho do texto. Da mesma forma usaremos o bloco de evento “`DepoisDeEscolher`” e armazenaremos o atributo “`Seleção`” do componente `ListaTam` que estamos usando para mostrar a lista com tamanhos base de texto disponíveis. Ao fim do código deste evento, precisamos não apenas chamar a função “`tam_fonte`” para mostrar o novo tamanho escolhido mas, também, a função “`adaptar`”. Sendo assim, a mudança de tamanho dos textos será imediata.

Por último e, para encerrar esta tela, temos o evento “`Clique`” do botão que grava a nova cor de fundo do aplicativo. Quando o evento for disparado, teremos que armazenar o estado da caixa de checagem (atributo “`Marcado`”) no rótulo “`alto_con`” e a cor de fundo da organização que estamos usando como prévia da cor no rótulo “`cor_painel`” escolhida. Assim como na mudança de texto, ao fim do evento, temos que executar a função “`adaptar`” para fazer valer a nova cor de fundo do aplicativo.

Note que não é preciso adaptar o valor do indicador no deslizador para refletir a nova configuração porque já tratamos isso na rotina de

inicialização da tela e o resto dos casos de alteração se dá quando o usuário manualmente altera a posição do indicador! Verifique na figura 112 como fica o código que estudamos nos últimos parágrafos.

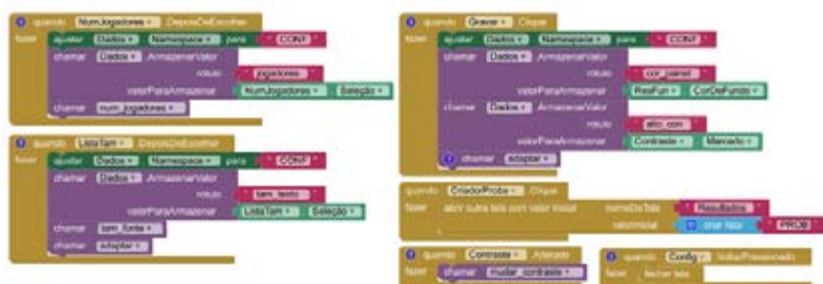


Figura 112: Gravação efetiva de novas configurações e funcionamento básico da tela.
(Fonte: App Inventor)

Sendo assim, concluímos a tela de configuração. Passemos para a tela de nome impreciso que tratará todo o jogo, a “Matemática”!

3.5.5. A Tela Principal do Jogo

Certo, agora, sendo honestos, aqui as coisas ficarão um tanto mais complexas. O código presente na tela onde o jogo realmente ocorre é o maior deste livro. Portanto, muita coisa será resumida e nem todo código será mostrado em figuras pelo fato de haver funções muito grandes. Então tentaremos usar explicações simples do que acontece em cada uma e como se dá o fluxo de processamento da tela.

3.5.5.1. Composição da Tela

Nossa tela é formada por uma grande quantidade de componentes, majoritariamente organizações, legendas e botões. Mas além destes componentes, precisamos de um TinyDB, um temporizador, um notificador e um componente de arquivo.

O componente de Temporizador servirá para que cronometremos as jogadas dos jogadores e possamos aplicar penalidades ou bonificações de tempo. O componente de TinyDB servirá para que possamos

adaptar a tela às configurações de acessibilidade definidas pelo usuário. Já o componente de Arquivo servirá de ponte de acesso entre nosso aplicativo e o armazenamento do smartphone para que possamos conseguir o arquivo com os problemas da partida.

Nossa tela de jogo é dividida em duas sessões principais: a “palco” e a “jogadores”. A sessão de jogadores é onde uma lista com todos os jogadores são mostrados com suas respectivas pontuações. Já a sessão de palco é onde o jogo acontece: onde o problema é mostrado, onde o usuário pode informar uma resposta, onde ele pode pressionar o botão de responder, onde ele pode ver informações de quem está jogando, qual é a área do saber, qual é o nível de dificuldade e tudo mais.

Dentro da sessão de jogadores, temos apenas dois componentes, duas legendas. Uma delas é um título informando que ali se encontra a listagem dos jogadores com suas pontuações. A segunda é a lista de pontuações dos jogadores.

Dentro da sessão de palco, temos três componentes: uma legenda (onde mostraremos quem joga no momento, a área do saber, nível de dificuldade e afins), um botão de iniciar (para que a chance de resposta de um jogador comece, seria ruim se o evento cronometrado começasse sem aviso) e, entre estes dois componentes, uma organização vertical chamada “PalcoProblema”.

Dentro desta organização vertical temos vários componentes. O primeiro deles é uma legenda onde será mostrado o tempo restante para que o jogador possa dar uma resposta ao problema que aparece na legenda subsequente. Depois disso, há um campo de texto (desativado, explicaremos posteriormente) onde o usuário escreverá sua resposta e ao fim, há dois botões, um de responder com o que foi digitado e outro de passar a vez.

Agora, é digno de nota que entre o campo de texto e o botão de responder há, novamente, um componente de organização vertical chamado “Recipiente”. Esse componente está aí por um propósito especial. Ele é composto por uma organização do tipo tabela e um botão de limpar resposta. Lembra-se que o campo de texto da resposta estará desativado durante todo o jogo? O componente de organização em

tabela foi configurado para exibir uma grade de 4 linhas para 3 colunas, resultando em 12 células.

Esse formato de 12 células parece familiar? Dentro desta organização, colocamos um teclado feito manualmente e especificamente para nosso aplicativo. Pensamos que apenas deixar o campo de texto ativado para o usuário digitar seria uma boa solução, entretanto, mesmo restringindo o comportamento do teclado para inserir apenas números no campo de texto, ainda assim números inválidos poderiam ser inseridos.

Com isso em mente, as primeiras nove células do componente de organização carregam, cada uma, um botão com um algarismo indo de 1 a 9. Em seguida, a décima célula carrega um botão para mudar o sinal do número (positivo/negativo), a décima primeira carrega o botão para inserção do zero e a décima segunda carrega o botão para inserção do ponto (infelizmente o padrão estado-unidense para números decimais é escrito com ponto em vez de vírgula).

Pode parecer um tanto quanto absurdo, mas isso nos trouxe algumas vantagens. Quando o jogo começava, era preciso explicitamente pressionar o campo de texto para que o teclado aparecesse desperdiçando uma parcela do tempo de resposta para tal ato. Embora o campo de texto tenha um procedimento interno para pedir foco ao SO Android, esse foco não habilita o teclado virtual automaticamente em alguns dispositivos. Além disso, a entrada de números inválidos pode ser evitada já que temos um teclado próprio cujo código temos de implementar.

Esses são todos os componentes que fazem parte de nossa tela. Verifique na figura 113 como cada componente se estrutura na árvore hierárquica.

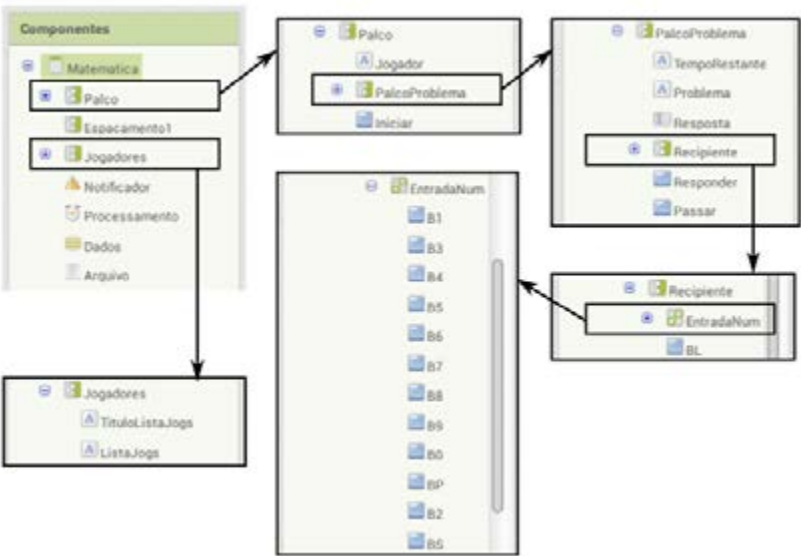


Figura 113: Componentes que formam a tela “Matematica”, onde o jogo ocorre.
(Fonte: App Inventor)

Agora veja, na figura 114, como a tela se parece no aspecto visual (no projeto do App Inventor já que essa não será a aparência final).

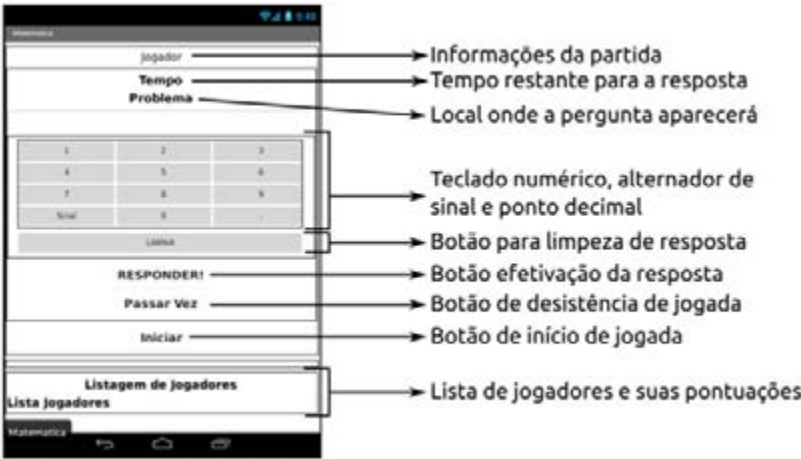


Figura 114: Aspecto visual da tela onde o jogo ocorrerá.
(Fonte: App Inventor)

3.5.5.2. A Ideia Geral

Não vamos esconder que o código para esta tela é imenso, mas antes que nos percamos nesta imensidão, vamos explicar qual é a ideia geral por trás de todo esse emaranhado de blocos. Dessa forma poderemos nos orientar melhor quanto ao que estamos fazendo e evitar o sentimento de que estamos perdidos.

A ideia geral para esta tela é que a tela seja iniciada de forma que permita armazenar valores de pontuação para cada jogador. Depois disso, com tudo configurado, a tela só repete o código necessário ao funcionamento do jogo. Conforme o jogo prossegue, os pontos são acumulados e, ao fim da partida, essas pontuações são repassadas à próxima tela para que vencedores sejam declarados.

Cada jogador tem três tentativas de resposta que são consumidas à medida que o jogador erra uma resposta. Quando o tempo de resposta de um jogador acaba ele é penalizado por isso perdendo pontos, passar a vez tem o mesmo efeito.

Os problemas mostrados na tela são extraídos de um arquivo presente no armazenamento ou no cartão SD do smartphone. Este arquivo está em formato de texto puro (TXT) mas estruturado de uma forma que seja possível separar dificuldades, problemas e elementos de problemas segundo símbolos especiais.

A cada vez que um jogador iniciar sua jogada, o jogo deve consultar seus dados sobre problemas, extrair o problema necessário de acordo com o nível de dificuldade vigente mostrar a pergunta na tela e esperar uma resposta. O jogo segue essa linha de perguntar e esperar resposta até que todos as dificuldades que foram informadas pelo arquivo **tenham acabado**¹⁸⁴. Quando chegar a este ponto, o processamento é repassado à próxima tela.

184 Para simplificar o código, estabelecemos uma condição para o término de um nível de dificuldade. Assim que todos os jogadores tiverem respondido a três perguntas cada, a dificuldade será incrementada (ou esse incremento será sugerido no modo de aprendizado). Quando o último problema de um nível for lançado mas os jogadores ainda não tiverem respondido três problemas cada, o aplicativo volta ao problema inicial do nível e segue esse ciclo até o nível terminar.

3.5.5.3. A Estrutura do Arquivo

Antes que possamos botar a mão na massa e interagir com os blocos em nosso aplicativo, é preciso primeiro explicar como nosso arquivo se estrutura. Tão importante quanto saber escrever o arquivo é saber lê-lo posteriormente. Nossa estrutura é definida de uma forma que possa ser facilmente desmembrada em diferentes níveis de objeto: dificuldade, problema e elemento de problema.

Dissemos que esses três objetos são separados por símbolos especiais, mas quais são esses símbolos? Bem, para separar dificuldades, estamos usando o símbolo de cerquilha (“#”), enquanto que, para separar problemas, estamos usando o símbolo de E comercial (“&”) e para separar elementos de problema, estamos usando o símbolo de cifrão (“\$”).

Por que um padrão assim é facilmente desmembrado por um aplicativo feito no App Inventor? Na categoria de blocos de texto há um bloco que reparte um texto em uma lista. Esse texto pode ser repartido de acordo com a ocorrência de certos caracteres. Então se usarmos esse bloco pedindo que o conteúdo do arquivo seja repartido a cada caractere de cerquilha, teremos uma lista com textos de cada nível de dificuldade!

Além disso, cada item desta lista terá os respectivos problemas e elementos de problemas em forma bruta. Para resolver isso, só precisamos usar o bloco de repartir novamente, mas desta vez usando o caractere de E comercial, para obtermos uma lista com os elementos de cada problema. Por fim, se usarmos o bloco mais uma vez, mas com o divisor de elementos. Teremos uma lista com os elementos de cada problema, o maior nível de **granularidade**¹⁸⁵ em nossa estrutura! Veja na figura 115 como é a estrutura de nosso arquivo.

¹⁸⁵ “Granularidade” é um termo comumente usado no contexto de desenvolvimento de sistemas para indicar o nível de detalhes envolvido em um esquema, processo, planejamento, e afins. Quanto maior o nível de granularidade, maior é a quantidade de detalhes envolvidos e vice-versa.

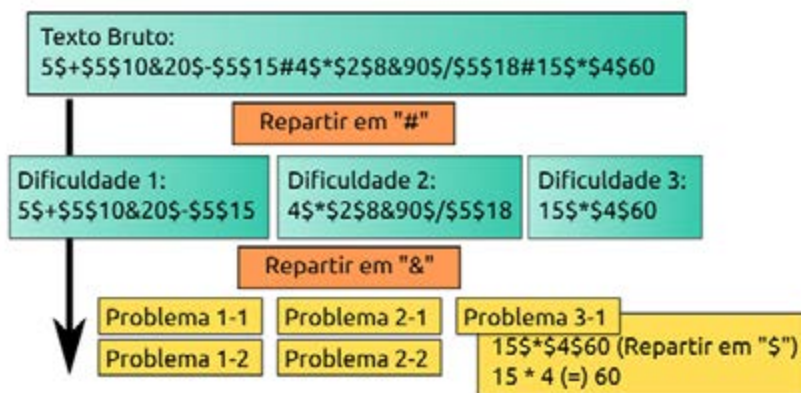


Figura 115: Estrutura do arquivo de texto (com os problemas do jogo) sendo desmembrada.

(Fonte: Do Autor)

3.5.5.4. Um Castelo de Blocos Sempre Começa por Unidades

Para começar, nosso código entra em ação por meio do evento Inicializar da tela Matematica. Esse evento trata de apenas uma parcela das operações de inicialização da tela visto que dependeremos de uma parte do código que pode ou não responder como desejado.

A primeira coisa a ser feita é executar a função “`configs_iniciais`”. Antes de explicarmos seu funcionamento, é preciso recordar que no Menu Principal de nosso aplicativo, na `Screen1` passamos um texto como valor inicial para esta tela. O primeiro elemento deste texto é o modo de jogo que pode ser verdadeiro ou falso e o segundo elemento é a área do saber que pode ser português ou matemática.

Esses dois elementos estão separados por uma cerquilha, então repartimos este texto em uma lista usando o caractere separador como fonte de repartição. Em seguida, assimilamos o primeiro elemento como valor da variável global “`comp`” e, como é um texto, temos que converter este texto para um valor lógico. Sendo assim, chamamos a função “`texto_para_logico`” dando o valor atual de `comp` como argumento e substituindo este valor pelo retorno desta função.

Em seguida, apenas assimilamos à variável global “area” o valor do segundo elemento da lista, a área do saber sobre a qual o jogo discorre. Tendo realizado estas ações, a função “configs_inicias” é finalizada e retornamos ao processamento do evento Inicializar. Antes de prosseguirmos, precisamos ressaltar que essa tela faz uso de muitas variáveis globais, então verifique na tabela 27 quais são essas variáveis, para quê estão sendo usadas e quais são seus valores iniciais.

Tabela 27: Variáveis globais da tela “Matematica”, seus objetivos e valores padrão.

Variável	Valor inicial	Objetivo
fim_de_jogo	Falso	Indica ao aplicativo que a partida acabou e é hora de declarar os vencedores.
tem_probs	Falso	Indica ao aplicativo se uma tentativa de extrair os problemas do arquivo já foi realizada.
p_num	Zero	Indica o número do problema a ser mostrado ao jogador na lista de problemas por dificuldade.
resp	Texto vazio	Contém a resposta dada pelo jogador para certo problema.
probs	Lista vazia	Uma lista que contém todos os problemas de uma certa dificuldade.
dif	Zero	Indica o nível de dificuldade a ser jogado na partida. É incrementado com o tempo.
jogadores	Zero	Indica o número de jogadores participantes de uma partida. Tem seu novo valor extraído do armazenamento TinyDB, onde o usuário especificou o número de jogadores.
jog_atual	Zero	Indica o número do jogador que joga na presente partida.
pont	Lista vazia	Lista de números onde cada número corresponde à pontuação de um jogador.
tentativas	3	Indicador de tentativas de resposta de um jogador. Cada jogador tem três tentativas. Uma tentativa é perdida a cada resposta errada.

tempo	Zero	Limite de tempo para cada pergunta (em segundos). O limite de tempo é definido de forma que, começando em 30 segundos, cada nível de dificuldade posterior, some 30 segundos ao limite.
pausa	Verdadeiro	Indica ao aplicativo se o cronômetro deve contabilizar o tempo passado.
fator_dif	Zero	Um contador genérico para adiar o incremento do nível de dificuldade. Após 3 perguntas feitas a cada jogador, o fator de dificuldade terá sido incrementado em três, isso define o aumento de dificuldade.
comp	Verdadeiro	Define se a partida será jogada em modo competitivo ou de aprendizado. O valor é recuperado a partir do valor inicial recebido da tela requisitante.
area	“Matemática”	Define qual área do saber será abordada na partida. O valor é recuperado a partir do valor inicial recebido da tela requisitante.
lim_dif	Zero	Indica qual é o valor limite de dificuldades, serve para julgar se a partida acabou ou não.
resp_mat_ponto	Falso	Usada pelo teclado numérico manual, indica se o símbolo de ponto já foi inserido no número da resposta ao problema de matemática.
cbt, cbp e tbt	Textos vazios	Usadas na adaptação da tela às configurações de acessibilidade.
tempo_inicial e tempo_final	Zero	Usadas pelo cronômetro e o temporizador para contabilizar com mais precisão o tempo de resposta passado.

(Fonte: Do Autor)

Veja na figura 116 como é o código da função “configs_iniciais”, como ela lida com o valor inicial da tela anterior e como ela se relaciona com a função “texto_para_logico” a fim de converter parte do valor inicial.

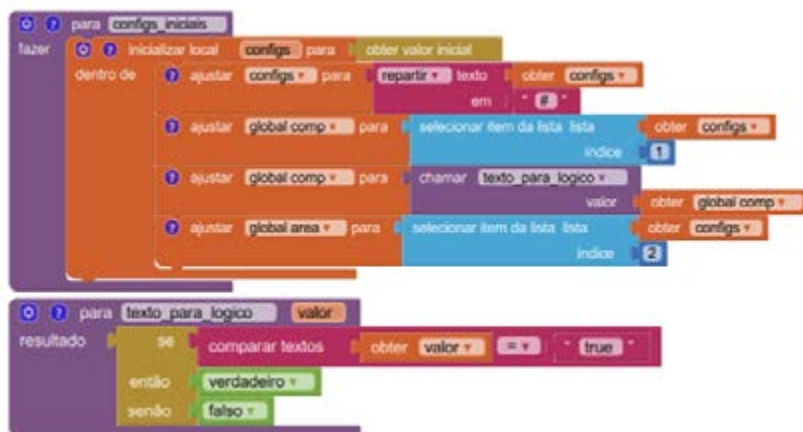


Figura 116: Funções “configs_iniciais” e “texto_para_logico” configurando a área do saber e o tipo de jogo da partida.

(Fonte: App Inventor)

Conforme já fizemos uma vez em nosso aplicativo, agora é a hora de pôr as mãos na mochila porque precisamos executar a função “adaptar” para fazer valer nossas configurações de acessibilidade. O procedimento é o mesmo: copiar o código da área de transferência do App Inventor (mochila) e incluí-lo em nosso código. Mas agora afetando os componentes que estão presentes nesta tela.

Repare que temos uma quantidade grande de elementos nesta tela. Afastando o nível de zoom do código, nossa cadeia de adaptação ficará enorme! Entretanto, é digno de nota que, na última tela, o número de componentes é muito maior mesmo que o código seja relativamente menor que a tela “Matematica”. Por causa disso é mais vantajoso usar outra forma de afetar todos os componentes desejados, mas isso fica para os próximos tópicos. Veja nossa cadeia de adaptação na figura 117, não se preocupe se não conseguir ler os blocos, é apenas para ilustrar como as funções cresceram.

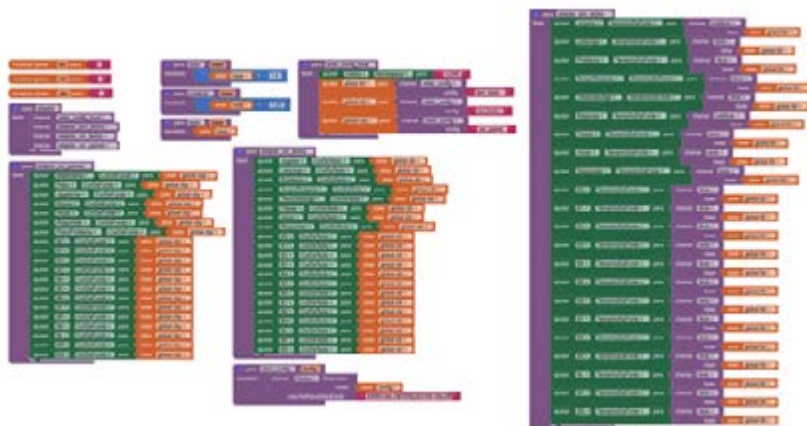


Figura 117: O alto número de componentes fez as funções de adaptação crescerem muito. (Fonte: App Inventor)

Assim que a função “adaptar” é concluída, **chamamos a função “obter_config”¹⁸⁶**, pedindo pelo valor associado ao rótulo “jogadores” (não se esqueça de configurar o espaço de nomes devidamente como “CONF”). Quando a função devolver o valor, armazenaremos o mesmo na variável global “jogadores”.

Quando tivermos o número de jogadores, chamaremos a função “criar_jogadores”. Esta função consiste em um laço que usa uma variável chamada “contador” (com valor inicial em zero) para criar a lista de pontuações dos jogadores, tendo como limite o número de jogadores que tratamos agora há pouco. Cada item da lista possui o valor zero já que o jogo está começando. A cada repetição o contador é incrementado em uma unidade prevenindo uma execução infinita.

De volta ao evento de Inicializar, executamos a última função tratada neste evento. Ela se chama “preparador_de_problema”. Em algum ponto pode soar engraçado que por falar em problema, parece que nosso aplicativo está querendo se meter em confusão ou causar.

¹⁸⁶ Não precisamos usar essa função apenas para a adaptação da tela, podemos usá-la para o resto do código já que sua estrutura modularizada permite um uso geral.

Gostaríamos que esse não fosse o caso, porque essa pilha de blocos é complicada!

Retornando ao tópico, a função “preparador_de_problema” foi feita para ser executada várias vezes mas com propósitos diferentes **dependendo do cenário**¹⁸⁷. Para fazer esse discernimento, usaremos uma variável de controle global chamada “tem_probs” cujo valor inicial é falso.

A preparador_de_problema verifica se o valor da variável é falso, se for, então significa que precisamos obter os problemas antes de prosseguirmos com o jogo. Sendo assim, o bloco SE é adentrado e a função avalia qual é a área do saber. Caso seja português, ela consulta o componente de arquivo pedindo a leitura do arquivo “prob_port.txt”. Caso seja matemática, ela faz a mesma coisa, mas requisitando o arquivo “prob_mat.txt”. Isso pode ser visto na figura 118.

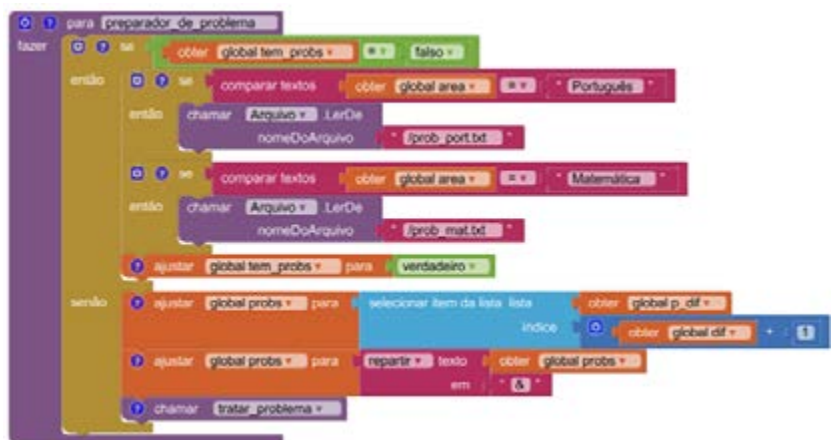


Figura 118: Função “preparador_de_problema” sendo executada de duas formas diferentes dependendo de uma variável de controle.

(Fonte: App Inventor)

187 Esse é o princípio básico da inteligência artificial (IA). Um sistema dotado de IA é capaz de tomar ações ou realizar procedimentos diferentes de acordo com o contexto com o qual está lidando. É claro que isso é bastante diferente dos conceitos de IA que vemos na cultura popular com robôs altamente inteligentes e capazes de agir quase como seres humanos, mas se algum dia isso se concretizar, poderemos ter certeza que esses robôs também serão formados, em algum momento, por mecanismos complexos formados por esses princípios simples que discernem qual é o contexto e que ações tomar perante ele.

Após essa triagem a função assume que seu papel foi feito e altera o valor da variável `tem_probs` para verdadeiro. Entretanto, aqui está o pulo do gato. Não há nenhum código adicional na função exceto a parte **SENÃO** do bloco **SE** em que estamos. Devido a isso, retornaremos ao evento de inicializar e as instruções se esgotarão. Nosso aplicativo ficará parado. Veja como o bloco Inicializar só cuidou de parte da inicialização da tela na figura 119.



Figura 119: Evento de inicialização da tela, este código cobre só uma parcela da inicialização.

(Fonte: App Inventor)

Mas espera aí! Dissemos que mandamos o componente de arquivo ler um arquivo certo? Sim, mas a leitura do arquivo é independente de nosso código. Ela funciona ao seu próprio passo, assim como a contagem de tempo de um temporizador. A única coisa que sabemos é que pedimos ao componente de arquivo que lesse um arquivo. Se o arquivo não existir, o aplicativo ficará esperando para sempre que o evento de leitura do arquivo aconteça já que seu código para de ser executado neste ponto.

Mas o que faremos se chegarmos a este ponto? Bem, na tela aparecerão os componentes que esperamos, legendas e o botão de iniciar, entretanto, alguns deles não mostrarão as informações que deveriam mostrar, por exemplo, a legenda no topo não mostrará o nível de dificuldade nem a área do saber já que não tocamos nela em momento algum no código, portanto este é um **sintoma**¹⁸⁸ do caso em que o arquivo de problemas não existe na localização onde precisamos dele!

188 Além disso, quando o jogador pressionar o botão de iniciar, inevitavelmente um erro acontece-

Mas prossigamos com o cenário em que o arquivo existe e está onde precisamos. Assim que a leitura do arquivo for finalizada, o componente de Arquivo disparará um evento chamado “Recebeu-Texto”. Antes de continuar, precisamos falar sobre um aspecto dos arquivos TXT que podem nos causar problemas. Em alguns sistemas operacionais, arquivos deste tipo podem ter incluído ao seu fim uma linha em branco.

Em nosso aplicativo, nossos mecanismos de tratamento de problemas podem considerar que este espaço em branco faz parte do último elemento de problema do último problema no último nível de dificuldade. Se isso acontecer em um problema de matemática, alguém nunca acertará a resposta de quanto é “ $2 + 2$ ”, porque por mais que a pessoa digite 4, isso é diferente de “4”¹⁸⁹.

Por causa disso, nossos arquivos deverão ser terminados com o símbolo de ponto e vírgula (“;”), assim que o conteúdo dos problemas chegar assim. Dessa forma, quando conseguirmos o texto bruto do arquivo, repartiremos ele primeiro por meio deste símbolo, separando o conteúdo da linha em branco. O primeiro elemento da lista resultante será o conteúdo que nos interessa, o segundo será a linha em branco que podemos descartar.

Quando este problema for tratado, a primeira coisa que devemos fazer é pegar o primeiro elemento da lista resultante da primeira repartição e reparti-lo nos símbolos de cerquilha. O resultado será uma lista que assimilaremos à variável global “p_dif” (ou “problemas por dificuldade”).

Em seguida, precisamos estabelecer qual é a dificuldade limite da partida, faremos isso assimilando à variável global “lim_dif” o valor retornado pelo bloco que avalia qual é o comprimento de uma lista por contar cada um dos seus itens. Forneceremos para esse bloco o valor da variável p_dif que, provavelmente, possui uma lista válida. Caso o conteúdo do arquivo não esteja estritamente no formato de arquivo que

rá e o aplicativo sugerirá o fechamento da tela para evitar problemas como procedimento padrão.

189 Leia-se como “texto 4-espaço”.

definimos, **erros acontecerão**¹⁹⁰ durante o processamento do jogo. Veja na figura 120 como o tratamento do evento ocorre.



Figura 120: Tratamento do erro de linha vazia e divisão do conteúdo bruto do arquivo. (Fonte: App Inventor)

Assim que estabelecemos o limite do nosso jogo como a última dificuldade fornecida pelo arquivo. É hora de executar a função “rodada”, essa função é o coração desta tela. Daqui em diante ela tomará conta do processamento a cada resposta de jogador. Embora o nome da função seja rodada, suas ações são voltadas à **jogada de cada pessoa individualmente**¹⁹¹. O escopo é formado basicamente de chamadas de outras funções e, ao fim, modificação de duas variáveis globais.

A primeira função chamada é “verificar_vitoria”, ela avalia se a dificuldade atual (variável dif com valor inicial em zero) é maior ou igual ao limite de dificuldade (variável lim_dif). Se for, então a partida acabou e chamamos a tela “Resultados”, a tela compartilhada com o criador de problemas. Entretanto, precisamos fazer isso fornecendo um valor inicial, assim como fizemos na tela de configurações. Sendo assim, formaremos uma lista com o primeiro item sendo um texto “PONT” e o segundo item sendo a lista de pontuações dos jogadores. Essa lista será o valor inicial da tela. Verifique o código envolvido na figura 121.

190 Em alguns casos, falhas de hardware ou, mais comumente, de software podem causar o rompimento de arquivos de forma que a leitura de dados não reflita exatamente o que está armazenado ou problemas na escrita causem danos à informação deixando-a desorganizada ou sem nexos no armazenamento. Pelo fato de computadores não lidarem com informações que não fazem sentido para eles, forçar isso causa erros no sistema, aplicativo etc.

191 Mesmo que seja executada a cada jogada individual a função também trata de procedimentos relacionados ao prosseguimento de uma rodada.



Figura 121: A função “rodada” é executada sempre que um jogador efetua sua resposta, a primeira coisa a ser feita é verificar se o jogo terminou.

(Fonte: App Inventor)

Mas se o fim do jogo não for o caso, apenas retornamos à função “rodada” e seguimos com sua execução. A próxima função a ser chamada é a “preparador_de_problema”. Lembra-se que ela foi feita para ser executada de forma diferente dependendo do cenário? Agora que temos os problemas, cairemos na parte SENÃO do bloco SE e a primeira coisa a ser feita é selecionar uma dificuldade da lista de problemas por dificuldade e conseguir os problemas brutos.

Podemos fazer isso somando uma unidade ao indicador de dificuldade atual e usando o resultado como índice para acesso à lista. O item obtido será armazenado na variável global “probs”, com isso, **jogamos seu conteúdo para repartição**¹⁹² de texto usando o caractere E comercial (“&”). Com isso teremos a lista de problemas da dificuldade atual.

Antes de finalizar essa função, há uma chamada de outra, a “tratar_problema”. Seu objetivo é mudar o valor da variável global “resp” para a resposta esperada de um problema dependendo da área do saber e do tipo de problema. Entretanto, antes de prosseguir, é necessário alertar: o escopo desta função é enorme, então tentaremos explicar da melhor forma possível já que mostrar o código na íntegra está fora de questão.

¹⁹² O resultado da repartição (uma lista com problemas da dificuldade separados em itens) será colocado na variável probs, substituindo o texto bruto usado na repartição.

Primeiro de tudo, em seu escopo há uma chamada da função “proximo_problema”, cujo objetivo é incrementar o valor da variável “p_num” em uma unidade. Caso o valor nessa variável seja maior que a quantidade de problemas disponíveis para o nível vigente de dificuldade, a variável recebe valor 1 e o ciclo recomeça, portanto, é possível que, dependendo do número de problemas, erros de resposta e jogadores, haja questões repetidas em uma partida. Verifique o código na figura 122.

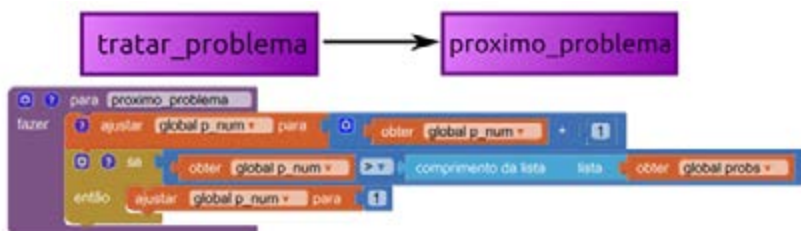


Figura 122: A função “proximo_problema” é responsável por mudar a exibição dos problemas aos quais os jogadores responderão.

(Fonte: App Inventor)

Retornando à `tratar_problema`, fazemos uma triagem com blocos SE para saber se a área do saber é Português ou Matemática. Caso seja português, temos três tipos de perguntas: quantas sílabas existem em uma palavra, qual é a sílaba tônica e qual é a sílaba acentuada da palavra. Para decidir qual tipo de questão será mostrada ao jogador, fazemos uso de uma variável local chamada “estilo”¹⁹³.

O valor da variável `estilo` é o resultado da chamada da função “decidir_estilo”. Na entrada desta função, temos a declaração de uma variável local chamada “possibilidades” que é inicializada como uma lista vazia. Dentro do escopo de declaração da variável, temos um bloco “fazer” para executar instruções sequenciais em uma função que exige blocos de especificação como retorno.

Dentro deste último, temos uma sequência de três blocos SE. Os três seguem a mesma estrutura: avaliam uma condição, se ela resultar

¹⁹³ Esta variável foi definida dentro do bloco SE para questões da área de português.

em verdadeiro, adicionamos o valor verdadeiro à lista de possibilidades, caso contrário, adicionamos o valor falso.

Antes de prosseguirmos, é preciso explicar que os problemas de português lidam com sílabas. Diferente dos problemas de matemática que seguem o padrão: 1º número, operador, 2º número e resposta, os problemas de português possuem apenas sílabas da palavra como elementos de problema.

Para além disso, o fato de a sílaba ser tônica ou acentuada é marcado por símbolos de controle: ponto de exclamação para sílabas acentuadas e ponto de interrogação para sílabas tônicas, podendo haver **ocorrência simultânea**¹⁹⁴.

Retornando à `decidir_estilo`, o primeiro bloco SE chama a função “`conseguir_palavra`”. Esta, por sua vez, retorna uma lista apenas com as sílabas da palavra do problema em vigência (**sem marcas especiais**¹⁹⁵ por usar consecutivamente a função “`remover_marca`” sobre o texto bruto do problema) e conta quantas sílabas a palavra tem. Se o número for maior do que zero, passamos no teste.

O segundo bloco SE testa (por meio da chamada “`num_acento`”) se há alguma sílaba acentuada na palavra. Se houver (no caso, se o retorno da função for um número maior do que zero) passamos no teste. E, por fim, da mesma forma, o terceiro bloco chama a função “`num_tonica`” que procura por uma sílaba tônica retornando um número maior que zero indicando sua posição na lista se ela existir. Veja como a função realiza os testes e uma chamada de função na figura 123.

194 Toda sílaba acentuada é uma sílaba tônica, mas nem toda sílaba tônica é acentuada. O til (“ã”) não é considerado acento, mas sim uma marca de nasalização. Da mesma forma, a crase (“ã”) não é um acento.

195 Dentro da função `conseguir_palavra`, chamamos uma outra função de nome “`remover_marca`” que possui um bloco de substituição de texto. Essa função possui dois parâmetros: texto e marca. No texto, provemos texto bruto do problema de onde queremos remover marcas. Na marca, fornecemos o caractere específico que queremos remover do texto fornecido. Sendo assim, o bloco de substituição precisa de três informações: o texto sobre o qual realizaremos as substituições (argumento no parâmetro texto), o segmento a ser buscado para substituição (argumento no parâmetro marca) e, por fim a substituição. Como queremos remover as marcas, apenas usaremos um bloco de texto vazio. O valor de retorno do bloco de substituição será o texto base com as substituições feitas.

Dentro da função “num_marca” (que é do tipo que retorna um valor assim como a “num_tonica” e a num_acento), temos a inicialização de duas variáveis locais chamadas “posicao” e “problema” com valores iniciais de 1 e a repartição por cifrão do resultado da função “problema_atual”, respectivamente.

Antes de prosseguirmos com essa função, temos que explicar o que a problema atual faz. Sendo um procedimento bem simples, a única coisa que a função “problema_atual” faz é: selecionar um item da lista na variável global probs (onde estão os problemas do nível de dificuldade atual) usando como índice o valor na variável global p_num (que indica qual problema está em vigência). Veja esse procedimento na figura 125.

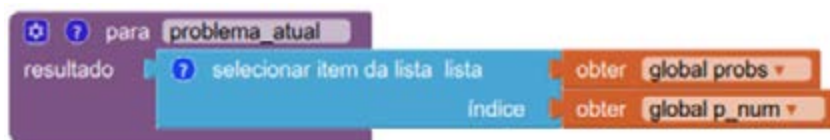


Figura 125: A função “problema_atual” tem como objetivo devolver o problema em vigência na jogada da partida.

(Fonte: App Inventor)

Assim que conseguirmos nossa lista bruta com elementos do problema atual, precisamos reparti-la por meio dos cifrões. A função num_marca já fez isso, então prosseguiremos com sua explicação. Após inicializar as variáveis locais, a função entra no bloco fazer e dá início a um laço. Enquanto o valor na variável posicao não for maior que o comprimento da lista na variável problema, executaremos o código dentro do laço.

Em seu interior, o código da repetição consiste em um bloco SE, um incremento da variável posição e, por fim, um último bloco SE. No primeiro bloco SE, verificamos se o item da lista em problema acessado pelo valor de posicao como índice possui a marca com o bloco de texto “contém”.

Como segundo argumento deste bloco informamos o valor do parâmetro “marca” da função num_marca que foi passado por suas

requisitantes. Se o caractere especial que estamos procurando não existir naquele elemento da lista. Apenas saímos do bloco SE, incrementamos e variável posicao e seguimos para o segundo SE.

Neste último verificamos se o valor em posicao é maior do que o comprimento da lista, se for, redefinimos o valor de posicao para zero e executamos um bloco de controle especial chamado “BREAK”. Mas por quê? Quando chegarmos ao fim da lista sem termos encontrado o que queríamos no primeiro bloco SE, cairemos no segundo bloco SE que redefine o valor de posicao para um índice inválido de acesso à lista.

Entretanto, se redefinirmos o valor sem executar o bloco BREAK, o laço será executado mais uma vez e tentará acessar a posição zero da lista, resultando em um erro. O bloco BREAK serve para “quebrar” uma repetição, sair imediatamente dela a partir do ponto em que é executado. E é exatamente o mesmo bloco que executamos se conseguirmos o que queremos no primeiro bloco SE!

Quando encontrarmos o caractere requisitado no item da lista indicado por posicao, não precisamos executar mais nada no laço, achamos o índice da lista que precisávamos! Por causa disso, executamos um BREAK e saímos da repetição retornando o valor em posicao. Da mesma forma, quando chegarmos ao fim da lista sem respostas, redefinimos o valor de posicao para um valor inválido que será tratado pelas requisitantes e devolvemos seu valor como retorno.

As funções num_acento e num_tonica retornarão os valores das buscas feitas por num_marca à função decidir_estilo e, agora que terminamos esta triagem, teremos uma lista com três valores lógicos na variável local possibilidades. Quando estivermos para enviar o retorno da função, chamaremos outra função de nome “aleatorio_verdadeiro” fornecendo a variável possibilidades como argumento ao parâmetro “lista”. Sendo assim, o resultado da função decidir_estilo será o resultado da aleatorio_verdadeiro. Veja como essas funções se relacionam na figura 126.



Figura 126: Relação entre as funções “decidir_estilo” e “aleatorio_verdadeiro”
(Fonte: App Inventor)

O objetivo da função `aleatorio_verdadeiro` é indicar um índice aleatório da lista de forma que o valor do item apontado pelo índice seja verdadeiro. Em linhas gerais, indicamos a esta função quais seriam as possíveis escolhas dela e ela tem que escolher aleatoriamente uma destas opções.

Se fôssemos ilustrar esse conceito seria como colocar três cartas de baralho em frente a uma pessoa com olhos vendados e embaralhar as cartas. A pessoa teria que escolher apenas uma carta que não poderia estar marcada com um X. Se ela escolhesse uma carta marcada, embaralharíamos as cartas e pediríamos que ela escolhesse novamente. O processo se repetiria até que uma carta limpa fosse escolhida.

Em nosso caso, o fato de embaralhar as cartas, significa que escolheremos um valor de índice aleatório, se, ao acessar aquele índice da lista, encontrarmos um valor verdadeiro, então devolveremos o índice daquele item (que está na variável escolhido). Analogamente, a marca do X nas cartas é correspondente aos valores falsos na lista.

Todavia, antes de começar a verificar se encontramos um valor verdadeiro num laço, precisamos verificar se a lista tem, pelo menos, um valor verdadeiro. Fazemos isso dentro de um bloco SE passando a lista no parâmetro `lista` da função `aleatorio_verdadeiro` como argumento da função `possui_verdadeiro`.

O que essa função faz é percorrer a lista por meio de uma variável local chamada `posicao` (com valor inicial em 1) e, se achar um valor verdadeiro na lista usando o valor em `posicao` como índice de acesso, a variável `resultado` terá seu valor definido como verdadeiro (sendo falso o valor de inicialização). Assim, garantimos que a função responda uma espécie de sim ou não: “Esta lista possui_verdadeiro? Resposta: Sim (Verdadeiro)/Não (Falso)”.

Independente de ter um ou vários, a função retornará o valor lógico verdadeiro se encontrar algum valor verdadeiro na lista, caso contrário, retornará falso. Veja como é seu escopo na íntegra por meio da figura 127.

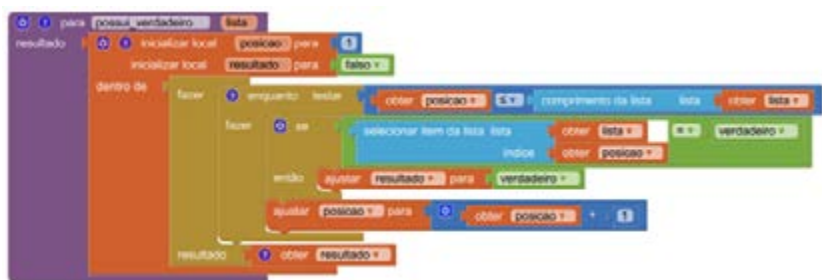


Figura 127: A função “`possui_verdadeiro`” verifica se há pelo menos um item na lista com valor verdadeiro com o intuito de evitar que a função “`aleatorio_verdadeiro`” continue executando infinitamente.

(Fonte: App Inventor)

Se a função `possui_verdadeiro` retornar valor falso, significando que não achou nenhum valor verdadeiro na lista, a função `aleatorio_verdadeiro` definirá que o valor da variável local escolhido é zero e retornará este valor. Mas, caso ocorra o contrário, então a função continuará acessando índices aleatórios até que o valor acessado na lista seja verdadeiro. Sendo assim, o índice encontrado será devolvido à função `decidir_estilo`.

Por sua vez, a função `decidir_estilo` retornará o número à função `tratar_problema`. A partir deste ponto, iniciamos um tratamento de erro. Se por algum motivo, o valor recebido for zero, então chamaremos o notificador pedindo que mostre o alerta na tela dizendo que um erro

muito grave aconteceu: nenhum estilo de pergunta pôde ser escolhido e fecharemos a tela.

Caso contrário, iniciaremos uma triagem com uma sequência de três SE's, todos eles seguirão um padrão: chamarão a função `formar_problema` passando como argumento um bloco de juntar texto, neste bloco de junção forneceremos um texto com a pergunta de acordo com o estilo escolhido, o resultado da função `conseguir_palavra` (para mostrar a palavra sem os símbolos especiais) e, por fim, uma interrogação para encerrar a pergunta a ser mostrada na tela.

Em cada um dos blocos SE, definimos o valor da variável global `resp` como sendo o resultado de uma chamada de função. No caso do estilo 1 (número de sílabas), `resp` assume o valor da função `num_silabas`, cujo código pode ser visto na próxima figura:

No caso dos estilos 2 e 3 (sílabas acentuada e sílaba tônica, respectivamente), definiremos o valor de `resp` como sendo a chamada das funções `num_acento` e `num_tonica`, respectivamente. Assim que terminarmos esses procedimentos de forma bem-sucedida, o primeiro bloco SE geral da função (que cuidou de aspectos relacionados a uma questão de português) terminará e teremos o segundo bloco SE que é relativamente menos complexo. Segue um esquema resumido do funcionamento da função `tratar_problema` na figura 128

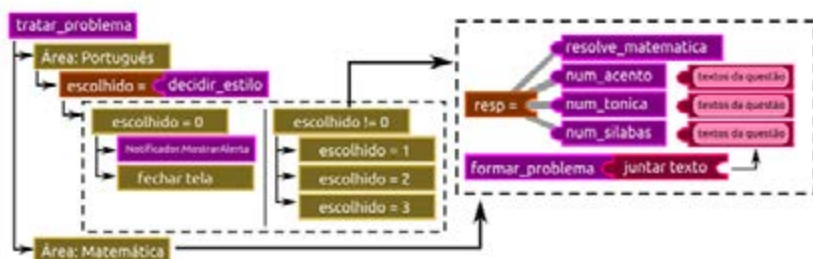


Figura 128: Esquema resumido da função “tratar_problema”.

(Fonte: Do Autor)

Se a área de conhecimento abordada pelo jogo for matemática, então entraremos neste bloco e a primeira coisa que faremos é definir o valor da variável global `resp` para o resultado da função `resolve_mate-`

matica. Esta função é responsável por obter o quarto elemento de problema do problema atual (a resposta já calculada do problema). Verifique o código na figura 129.



Figura 129: A função “*resolve_matematica*” consegue uma resposta calculada de antemão para o problema atual por acessar o quarto elemento de problema.

(Fonte: App Inventor)

Para finalizar a explicação da função *tratar_problema*, precisamos explicar o que a função *formar_problema* faz. Na verdade ela definitivamente não faz muita coisa. De forma precisa, ela só muda o atributo texto do componente Problema (legenda onde a pergunta do problema é mostrada ao jogador) com o argumento no parâmetro “*entrada*”.

Por fim, a função *tratar_problema* acabou e retornamos à requisitante que, por sua vez, é a função *preparador_de_problema* que consecutivamente também chega ao seu fim. Retornando a pilha de chamadas, voltamos à função rodada. Sendo assim, a próxima função a ser executada é a *tempo_por_dificuldade* que recebe o nível atual de dificuldade (variável global *dif*) como argumento de seu único parâmetro: *dificuldade*.

A única coisa que essa função faz é definir o valor da variável global *tempo* (o tempo limite que cada jogador tem para responder corretamente uma pergunta feita pelo jogo). A fórmula usada pela função diz

estabelece que o tempo de resposta é igual a 60 vezes a metade da soma em um do nível de dificuldade. Veja como é esta função na figura 130.



Figura 130: A função “tempo_por_dificuldade” define o tempo limite para um jogador responder corretamente um problema.

(Fonte: App Inventor)

Em seguida, assim que tivermos definido o tempo limite da jogada de cada um dos jogadores dependendo do nível de dificuldade atual da partida. A próxima função a ser chamada é a `informar_jogador_dif_area`. Essa é a função que mostra na tela qual é o jogador que joga no momento, qual é a área do saber abordada, qual é o nível de dificuldade atual e o limite de dificuldades assim como qual é o tipo da partida (competitivo/aprendizado).

A função faz isso por mudar o atributo texto da legenda Jogador que está no topo da tela. Segue na figura 131 como a função faz isso por meio de um bloco de junção de texto e acesso a várias variáveis e um acesso à função `tipo_partida` (também presente na figura).

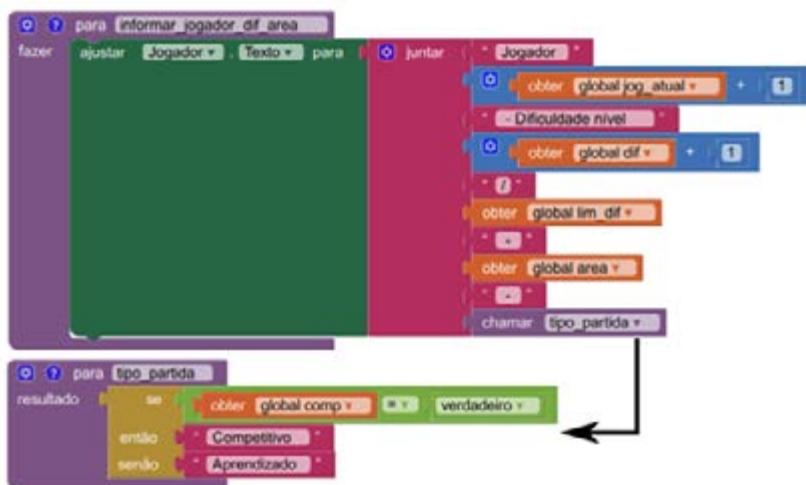


Figura 131: As funções “informar_jogador_dif_area” e “tipo_partida” proveem informações sobre a partida em uma legenda.

(Fonte: App Inventor)

Retornando à função rodada, a próxima função a ser executada é a `ajustar_placar`. Essa função conta com duas variáveis locais: `resultado` e `contador`. Como de praxe a variável `contador` começa com valor 1 porque será usada para percorrer uma lista, a de pontuações. A cada vez que o `contador` tem seu valor incrementado em um, adicionamos conteúdo à variável `resultado`.

O conteúdo adicionado consiste em: um texto de conteúdo “Jogador ” o valor atual do `contador` (para dar um número ao jogador), outro texto de conteúdo “:”, o valor da lista na variável global `pont` sob acesso do índice do `contador`, e um último texto de conteúdo “ pontos.
”.

Há um rótulo HTML de quebra de linha neste texto porque marcamos a opção `HTMLFormat` nas propriedades da legenda `ListaJogs` (sendo a que mostrará a lista de pontuações) na sessão de design, fazendo com que possamos pular uma linha e evitar que tudo seja escrito de uma vez em uma única linha. Por fim, mudamos o texto da respectiva legenda como se segue na 132.



Figura 132: A função “`ajustar_placar`” cria uma lista de jogadores com suas respectivas pontuações na legenda `ListaJogs`.

(Fonte: App Inventor)

Depois da função `ajustar_placar`, chamamos a função `bloquear_resposta`, seu objetivo é esconder o componente de `Organização PalcoProblema` (onde está o teclado numérico, botões e a questão do problema atual) e mostrar/ativar o botão `Iniciar` (isso se o jogo estiver

pausado). A função faz isso por, primeiro, definir como falsa a propriedade Visível de PalcoProblema, avaliar (por meio de um bloco SE) se a variável global fim_de_jogo possui um valor falso e, se possuir, definir como verdadeiras as propriedades Visível e Ativado do botão Iniciar.

Depois desta função, retornamos à rodada e executamos a função limpar_resposta. Essa função tem o simples objetivo de limpar o campo de texto onde fica a resposta digitada pelo jogador e redefinir o valor de uma variável global de controle, a resp_mat_ponto, para falso. Para fins de visualização, segue na figura 133 o código das duas últimas funções comentadas.



Figura 133: As funções “bloquear_resposta” e “limpar_resposta” são responsáveis por impedir uma nova resposta antes do tempo e limpar a resposta anterior, respectivamente. (Fonte: App Inventor)

Tendo realizado tudo isso, chegamos aos blocos finais da função rodada onde temos duas assimilações para as variáveis pausa e tentativas. A variável pausa recebe o valor verdadeiro para evitar erros com o cronômetro do jogo. Em seguida, redefinimos o valor em tentativas para seu padrão, sendo assim o jogador que se proporá a responder o jogo terá três chances assim como todos os outros. Com isso, a função rodada chega ao fim.

O que acontece agora? Vamos desempilhar! A função rodada foi chamada pelo evento de leitura do arquivo, então é para ele que voltaremos. Desta forma, seguimos para a função inicia_processamento. O que ela faz é configurar o Temporizador de nossa tela para disparar

eventos contínuos num intervalo de 50 milissegundos. É exatamente aqui que entra a diferença entre um jogo competitivo e um de aprendizado.

A função define o texto da legenda TempoRestante como sendo “Tempo Infinito!” e logo depois executa um bloco SE que avalia se o valor na variável global comp é verdadeiro (indicando uma partida competitiva). Se for, então o código cuidará disso adiante e o temporizador é ativado, senão, o bloco não tem seu escopo adentrado, o temporizador continua desativado e o texto continua intacto na legenda. Verifique esta função na figura 134.



Figura 134: A função “inicia_processamento” é onde acontece a diferença entre uma partida de aprendizado e uma competitiva.

(Fonte: App Inventor)

Assim que a `inicia_processamento` termina, terminamos também de inicializar nossa tela, tudo está pronto para que o jogo ocorra com suas perguntas e os jogadores possam se divertir durante a partida. Entretanto, mesmo que tenhamos preparado a partida no geral, apenas a primeira jogada terá ocorrência garantida.

3.5.5.5. Garantindo o Resto da Partida

Lembra-se que a função `bloquear_resposta` deixou o botão Iniciar à vista? Esse será nosso ponto de partida. Embora tenhamos que nos preocupar com as outras jogadas, primeiro precisamos nos preocupar com o que temos que fazer quando uma jogada estiver em realização.

Nessa linha de raciocínio, perceba que nossa tela está estagnada. Nada deu errado, mas nada foi efetivamente concluído em termos de

jogo. Esse é o momento em que esperamos pela entrada do usuário e, por causa disso, estamos dependendo de sua interação com o botão Iniciar. Então o próximo passo é programar o que acontece quando os usuários pressionam esse botão.

Nosso evento de botão será composto de três chamadas de funções: `verificar_erro_arquivo`, `des_pausar` e `desbloquear_resposta`. A primeira função se encarrega de verificar se não há nenhum nível de dificuldade para ser jogado (valor da global `lim_dif` é igual a zero). Se o componente de arquivo não puder ter encontrado ou lido o arquivo, o evento de leitura não será executado e tudo o que chamamos (in)diretamente dentro dele também não será.

Sendo assim, o valor na global `p_dif` não será alterado e continuará com seu valor inicial de lista vazia. Consequentemente, o código que define `lim_dif` como o comprimento da lista `p_dif` não será executado e `lim_dif` também ficará com seu valor inicial (zero). Por causa disso, saberemos que houve um erro na leitura do arquivo assim que o usuário pressionar o botão de iniciar (que deve estar visível na tela mesmo que não tenhamos executado a função `bloquear_resposta`).

Se o erro for constatado, recorreremos ao Notificador para que informe o usuário sobre um erro fatal e explique a relação com o arquivo. Em seguida, fecharemos a tela para prevenir futuros erros. Essa função só precisa ser executada apenas uma vez¹⁹⁶ durante a partida.

Se o erro não for constatado, apenas continuaremos executando as funções no evento de pressionamento do botão. A próxima da lista é a `des_pausar`. Essa função é composta por um bloco SE que avalia se a variável global `pausa` possui o valor verdadeiro. Se sim, significa que o jogo está saindo de uma pausa e, por conseguinte, teremos que redefinir os valores nas variáveis `tempo_inicial` e `tempo_final`. Por quê? Porque isso afeta a forma como nosso cronômetro trabalha.

¹⁹⁶ Como não poderíamos incluir esta chamada de função na parte anterior do código (pois esta depende da existência de um recurso externo ao aplicativo), fizemos sua chamada numa parte fixa do código. Entretanto, ela será executada em todas as jogadas, uma variável de controle poderia ser criada para evitar isso.

Quando executamos a função `inicia_processamento`, passamos por um bloco SE em que, se a partida for competitiva, o código ativaria o Temporizador. Em nosso caso, se a partida é competitiva, o Temporizador já está correndo contra o tempo. Porém, mesmo que ele esteja funcionando, quem contabiliza o tempo passado somos nós. Para isso vamos nos aproveitar de alguns códigos do primeiro aplicativo deste livro logo adiante.

Por enquanto apenas precisamos usar o bloco Agora de nosso Temporizador e associar seu retorno à global `tempo_inicial`, assim como associar o valor da variável global `tempo` (tempo para dar uma resposta) à global `tempo_final`. Com isso, concluímos o bloco SE da função `des_pausar` e agora a única coisa que precisamos fazer para encerrá-la é inverter o valor na variável global `pausa` com o bloco lógico NÃO.

Por fim, o evento de inicialização estará completo se executarmos a função “desbloquear_resposta”. Ela simplesmente torna visível a Organização PalcoProblema que já está configurada para mostrar o primeiro problema da partida ao primeiro jogador. Depois ela torna invisível e inativo o botão de Iniciar para prevenir erros. Por fim ela usa o procedimento do campo de texto resposta para pedir foco da tela.

Sendo assim completamos nosso início de jogada. Verifique na figura 135 como este trio de funções se liga com o evento de clique do botão iniciar.

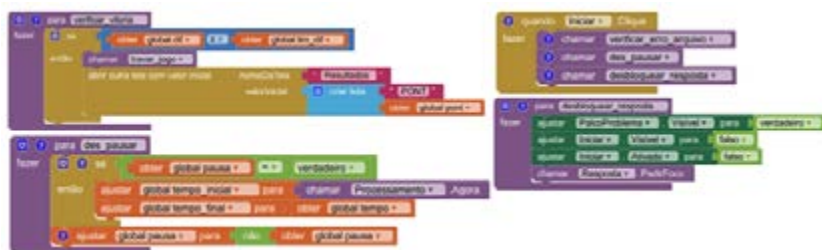


Figura 135: Para iniciar uma jogada, o jogador deve pressionar o botão Iniciar que acaba executando três funções preparatórias.

(Fonte: App Inventor)

É isso, agora é só deixar o jogador responder a pergunta que já está na tela. Certo? Errado! Temos que nos preocupar com mais algumas coisas: como o jogador saberá quanto tempo ainda tem disponível? E se o jogador quiser passar a vez? E se o tempo acabar sem uma resposta? E se o jogador responder corretamente dentro do tempo? Precisamos tomar conta de todas essas coisas. Começemos pelo tempo.

Lembra-se que deixamos o Temporizador ativo em nossa função `inicia_processamento`? No código desta função, também definimos que ele lançaria eventos constantes. Precisamos fazer algo para tratar estes eventos que estão sendo lançados o tempo todo!

Então, por causa disso, incluiremos no código um evento de disparo de nosso Temporizador (o qual chamamos de `Processamento`). Esse evento fará apenas uma coisa, chamar a função `atualizar_tempo`. Aqui começa nossa pilha. A função `atualizar_tempo` é responsável por apenas verificar se o jogo está pausado (valor verdadeiro na variável global `pausa`). Se estiver, atualizamos o valor da legenda `TempoRestante` com a junção de dois textos: “Tempo:” e o resultado da função `con_atualizacao`.

Por sua vez, a função `con_atualizacao` realiza um cálculo: o tempo restante de resposta do usuário é igual à do valor na variável `tempo_final` pelo valor resultado da função `tempo_passado`. Por sua vez, a função `tempo_passado` precisa de dois argumentos para funcionar: um instante inicial e um instante final. Ela usa o bloco de `Duração` entre dois instantes do Temporizador para conseguir a diferença em milissegundos.

A função `tempo_passado` já devolve a diferença em milissegundos convertida para segundos por meio de uma divisão por mil. Sendo assim, se fornecermos a variável **`tempo_inicial`**¹⁹⁷ como instante inicial e o procedimento `Agora` do Temporizador como instante final, teremos quantos segundos se passaram desde então.

¹⁹⁷ Assim que o jogador clica no botão de Iniciar, associamos um instante daquele momento à variável, sendo assim, o tempo que contaremos em segundos diz respeito ao tempo passado a partir do momento que o jogador decidiu responder a um problema.

Num último movimento, se retirarmos a quantidade de segundos passada da quantidade de segundos limite, teremos quantos segundos ainda sobraram para responder o problema, e esse é o retorno da função `con_atualizacao` para a `atualizar_tempo` que acaba de atualizar a legenda `TempoRestante`. Logo antes de desempilhar, esta função chama outra função chamada `checar_fim_tempo`. Mas por enquanto, veja na figura 136 como as funções citadas se relacionam.

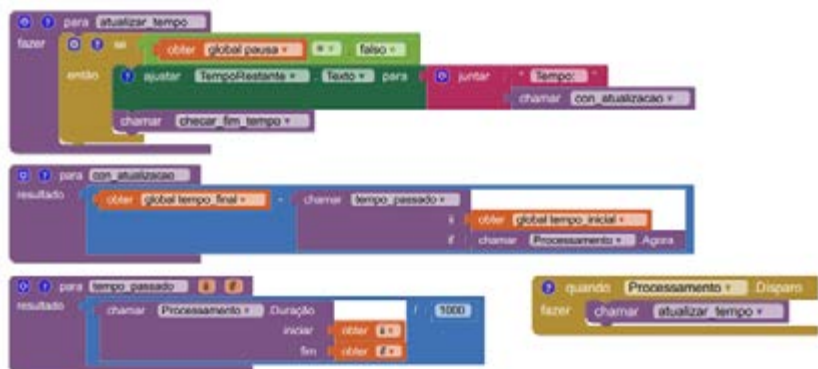


Figura 136: O tempo de uma resposta é contado pelo evento de disparo do temporizador que aciona as funções responsáveis por contabilizá-lo.

(Fonte: App Inventor)

A `checar_fim_tempo` faz exatamente o que o nome indica, checa se o tempo de resposta acabou. Quando ela é executada, a primeira coisa a ser feita é verificar se o resultado da função `con_atualizacao` é menor do que zero por meio de um bloco SE.

Se esse não for o caso, a função apenas acaba e devolve o controle para a `atualizar_tempo` (que também acaba e retorna ao evento que, por sua vez, também acaba). Se o tempo tiver acabado, chamamos o Notificador para que ele mostre um alerta informando que o tempo se esgotou e definimos o texto da legenda `TempoRestante` para “Tempo Esgotado!”. Redefinimos as tentativas para zero e chamamos a função `ganha_ou_perde` que será muito importante no jogo. Veja como se parece esta função na figura 137.



Figura 137: A função “chegar_fim_tempo” é responsável por verificar se o tempo de resposta de um problema acabou e disparar a aplicação de penalidades.
(Fonte: App Inventor)

Mas e se o jogador pular sua vez pressionando o botão Passar? Teremos que ter um evento para o clique deste botão! Mas o que ele fará? Simples, apenas definir o número de tentativas para zero e chamar a função `ganha_ou_perde`.

Afinal, o que esta função “`ganha_ou_perde`” faz?! Digamos que ela é o juiz da partida, ela é composta de dois blocos SE, um que verifica se o número de tentativas disponíveis para o jogador é maior do que zero, e um que verifica o oposto. Vamos abordar apenas o segundo caso, porque zeramos as tentativas do jogador tanto pelo botão de passar a vez quanto pelo fim do tempo.

No caso de falta de tentativas disponíveis, a `ganha_ou_perde` direcionará o fluxo de processamento para a função `decrementar_pontuacao` (como uma penalidade pelo desperdício de tentativas) fornecendo o número do jogador atual como argumento para o parâmetro `jogador`. Verifique o evento do botão Passar e a função `ganha_ou_perde` na figura 138.

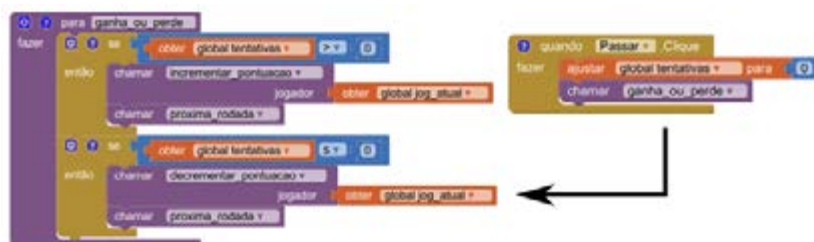


Figura 138: Algumas condições de jogo zeram o número de tentativas e chamam a função “ganha_ou_perde” para julgar tal efeito.

(Fonte: App Inventor)

A função `decrementar_pontuacao` possui uma variável local chamada `valor` que é inicializada em zero. Dentro de seu escopo, ela assume o valor da pontuação do jogador informado pelo único parâmetro da função. Sendo assim, seu valor é decrementado em trinta pontos assumindo que cada tentativa vale 10 pontos durante a aplicação de uma penalidade.

Por fim, temos uma substituição na lista de pontuações (há um bloco próprio para isso na categoria de blocos de lista) trocando a pontuação antiga pela nova decrementada. Verifique na figura 139 como é essa função (note que incrementamos o parâmetro `jogador` porque a variável global `jog_atual` permite o valor zero e isso pode causar erro no acesso à lista).



Figura 139: A função “`decrementar_pontuacao`” aplica uma penalidade de 30 pontos à pontuação de um jogador.

(Fonte: App Inventor)

Agora que tratamos dos casos em que o jogador perde pontos por passar a vez e por esgotamento de tempo, temos que tratar do caso em que o usuário responde corretamente. Mas como se dá uma resposta? Bem, as perguntas de sílabas de português pedirão o número de sílabas de uma palavra, o número da sílaba acentuada e o número da sílaba

tônica. Entretanto, antes de lidarmos com a entrada de dados do jogador, precisamos esclarecer algo que ficou para trás.

Durante a função `tratar_problema`, o segundo bloco SE (de primeiro nível) da função trata do que será feito se a área abordada for matemática. Embora todos os problemas de português recorram à função `conseguir_palavra` para formar uma questão que possa ser entendida pelo jogador (em vez de texto bruto codificado), problemas de matemática têm sempre a mesma pergunta no formato: “Qual o resultado da operação”, expressão matemática e “?”.

Para juntar a expressão matemática ao texto da pergunta, chamamos a função `conseguir_expressao`. Seu código é simples, porém extenso. O que esta função faz é conseguir o texto bruto do problema de matemática, subdividi-lo em elementos de problema e juntar cada elemento em um texto que será devolvido à requisitante.

Sendo assim, o primeiro elemento (número) é mantido, assim como o terceiro. O último elemento é descartado porque se trata da resposta. O trabalho se dá no processamento do operador. Os símbolos básicos (asterisco e barra) não representam muito bem as operações matemáticas às quais correspondem, então fazemos trocas de todos os operadores matemáticos **ASCII para seus correspondentes Unicode**¹⁹⁸. Desta forma, “+ - * /” viram “+ − × ÷”

Agora, retornando ao momento em que o jogador for responder a pergunta, ele terá que interagir com o teclado numérico. Entretanto, perceba que nosso teclado foi feito manualmente. Precisaremos tratar as interações com cada botão. Calma! A maioria deles segue um padrão, os botões numéricos funcionam da mesma forma: o evento de clique

198 ASCII e Unicode são dois conjuntos de caracteres (também chamados de “charsets”) que são algo como grandes alfabetos com todos os caracteres que um computador pode representar. ASCII é mais usado em computadores americanos sendo bastante simples e limitado apenas a 256 símbolos especiais, sejam eles gráficos (mostrados na tela) ou de controle (sem representação gráfica, ajudando na lógica dos computadores). O Unicode é um grande e extensível conjunto de caracteres que se tornou muito popular nos últimos anos. Pelo fato de ser extensível, ele já conta com um gigantesco acervo de caracteres, para citar um exemplo, toda a biblioteca de emojis (caracteres em forma de pessoas, bandeiras de países objetos etc.) comumente usados em aplicativos de mensagens instantâneas.

chama a função `adc_sim_resp` (adicionar símbolo à resposta) com seu respectivo número associado. Veja como isso funciona na figura 140.

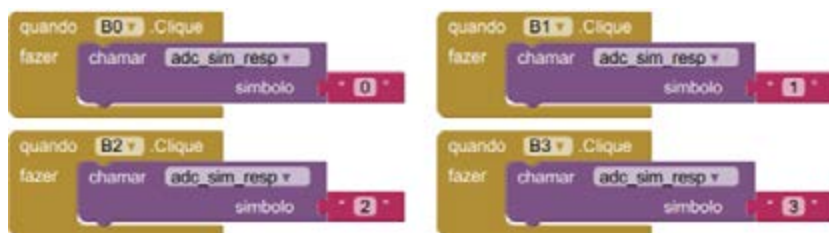


Figura 140: Os botões do teclado numérico precisam ser programados manualmente.
(Fonte: App Inventor)

O que a função `adc_sim_resp` faz é adicionar um caractere ao fim do texto no campo de texto da resposta. Entretanto além dos dez botões numéricos um outro botão é capaz de adicionar um símbolo na resposta: o botão de ponto. Esse é um caso especial, o botão de ponto pode ser pressionado várias vezes mas só pode fazer efeito uma vez. É para isso que serve a variável global `resp_mat_ponto`.

Quando pressionamos o botão de ponto, enviamos o caractere de ponto para a função `adc_sim_resp`. Dentro de seu escopo, a primeira coisa que a função faz é avaliar se o caractere recebido é um ponto. Se sim, entramos em seu escopo e encaramos um segundo bloco SE.

Essa segunda avaliação é responsável por, justamente, verificar se o valor em `resp_mat_ponto` é falso. Se for, podemos adicionar o ponto à resposta (não esquecendo de definir o valor de `resp_mat_ponto` como verdadeiro), senão, apenas deixamos o código continuar. Como o resto da função é a parte `SENÃO` (onde adicionamos qualquer caractere enviado por saber que ele não é um ponto) do único SE de maior nível, então a função acaba por aqui. Veja na figura 141 como o evento do botão de ponto se liga com a `adc_sim_ponto`.

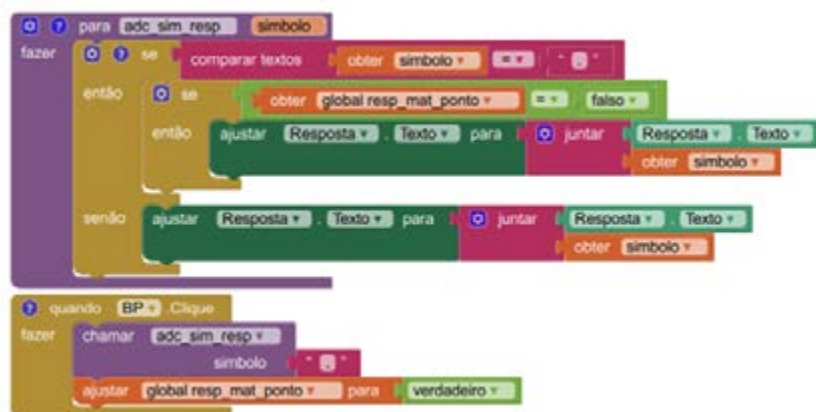


Figura 141: O botão de ponto do teclado numérico precisa recorrer a um tratamento especial da função “adc_sim_resp” para evitar números com múltiplos pontos.
(Fonte: App Inventor)

Caso o jogador tenha errado a digitação da resposta, fornecemos um botão para limpar a resposta. Logo, assim como a função rodada fez uma chamada para a função limpar_resposta, o evento deste botão também fará. Para finalizar o teclado numérico, resta apenas um botão, o de mudança de sinal (respostas negativas são admitidas).

Neste caso, quando o botão de sinal for clicado, seu evento chamará a função alternar_sinal_resposta. Dentro desta função temos um bloco SE com extensão SENÃO. Primeiro, avaliamos (com o bloco de texto contém) se há um sinal de menos no texto do campo de texto da resposta. Se houver, usamos um bloco de substituição assim como fizemos na função remover_marca para remover o sinal negativo.

Caso contrário, quando entrarmos no SENÃO, apenas adicionaremos um sinal de menos ao início do texto. Sendo assim a função é concluída, veja na figura 142 como ela e o botão de sinal se relacionam em código assim como o botão de limpar resposta fica com o código enxuto pela modularização.



Figura 142: O botão de limpeza apenas chama a função “limpar_resposta” e o de sinal chama a “alternar_sinal_resposta” que adiciona ou remove o sinal de menos da resposta. (Fonte: App Inventor)

Assim que o jogador digitar sua resposta, ele poderá pressionar o botão de responder. Esse botão terá um evento de clique em que executaremos o seguinte código: um bloco SE verifica se o conteúdo no campo de texto é igual ao número na variável global resp (onde armazenamos uma resposta pré-processada). No caso da resposta estar certa, chamamos a função ganha_ou_perde sem mexer na quantidade de tentativas.

Sendo assim, quando entrarmos na função ganha_ou_perde, cairemos no primeiro caso de julgamento. Mas explicaremos o que acontece nesse primeiro bloco SE posteriormente. Caso o jogador tenha errado em sua resposta, cairemos na parte SENÃO do SE no evento de clique do botão.

Como o jogador errou a resposta, precisamos fazer com que ele perca uma tentativa (apenas se o jogo for competitivo, indicando o uso de um bloco SE para checar a global comp). Logo depois, temos que verificar se as tentativas do jogador acabaram, se isso tiver acontecido, então chamamos a função ganha_ou_perde, fazendo o jogador ser penalizado pelas tentativas perdidas. Ao fim, chamamos o notificador para informar o jogador que ele perdeu uma tentativa. Veja o código deste evento na figura 143.

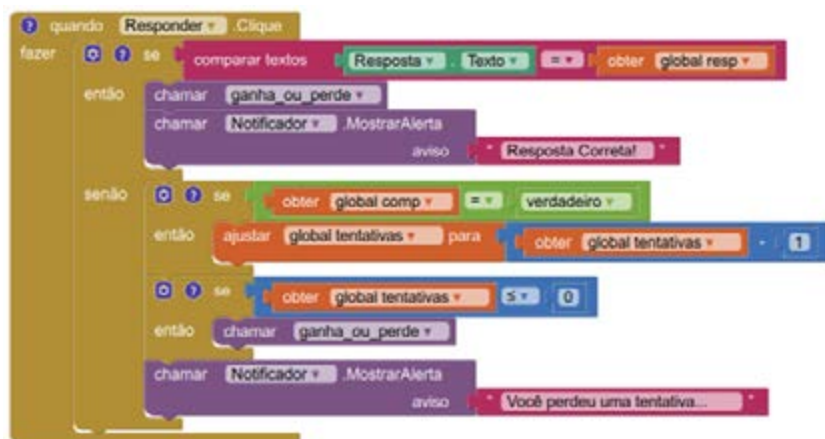


Figura 143: Quando o jogador responde uma pergunta, diferentes ações são tomadas quando ela está certa ou errada.

(Fonte: App Inventor)

Agora, no caso do jogador ter conseguido responder a pergunta corretamente, a função `ganha_ou_perde` cai em seu primeiro bloco SE (uma tentativa com uma resposta correta não é decrementada). Desta forma, chamamos a função `incrementar_pontuacao`. Essa função funciona quase exatamente da mesma forma que a função `decrementar_pontuacao`, a única coisa a ser modificada é o cálculo da pontuação.

Para isso a nova pontuação é incrementada com um ganho base de 20 pontos e são aplicados dois bônus: bônus por tempo restante e bônus por tentativas restantes. Para isso chamamos 2 funções auxiliares chamadas de `bonus_tempo` e `bonus_tentativa`.

Na `bonus_tentativa`, cada tentativa vale 5 pontos, retornando o valor da multiplicação desses 5 pontos pelo valor na variável `global tentativas`. Já a função `bonus_tempo` é um pouco mais complexa, como o modo de aprendizado não contabiliza bônus por tempo, temos que fazer um bloco SE que verifique a variável `global comp`. Se seu valor for falso, apenas retornamos o valor zero. Mas se esse não for o caso, retornamos o resultado de um cálculo.

O cálculo do bônus por tempo consiste em pegar o tempo restante (consulta feita à função `con_atualizacao`) em segundos e arredondá-lo. Cada segundo equivale a um quarto de um ponto (0,25). Verifique na figura 144 como as funções `bonus_tempo`, `bonus_tentativa` e `incrementar_pontuacao` se relacionam e se parecem.



Figura 144: A função “`incrementar_pontuacao`” recorre a outras duas funções para calcular uma pontuação de resposta correta: “`bonus_tempo`” e “`bonus_tentativa`”. (Fonte: App Inventor)

Agora que o jogador conseguiu os pontos por sua resposta correta, retornamos nossa atenção à função `ganha_ou_perde` que, ao final dos dois casos, chama a função `proxima_rodada`. Essa função é responsável por executar ações ao fim de cada jogada. Quando entramos em seu escopo, a primeira coisa que executamos é um incremento do número na variável global `jog_atual` para que o próximo jogador possa jogar. Entretanto, se este número for maior que o número de jogadores (global `jogadores`) em que a partida foi configurada, temos que tomar algumas precauções. O primeiro deles é retornar a variável `jog_atual` para zero para que os primeiros jogadores voltem a jogar (fechando o ciclo de jogadas).

Além disso, aumentamos em um o valor na global `fator_dif` para aproximar dos jogadores o aumento do nível dificuldade. Porém, se, depois do incremento, o valor em `fator_dif` for maior ou igual a 3 (todos os jogadores responderam 3 perguntas cada), então se o jogo for competitivo, chamamos a função `incrementar_dificuldade` (responsável por

simplesmente incrementar o valor em dif e zerar o valor em fator_dif). Veja como fica o escopo dessas funções na figura 145.

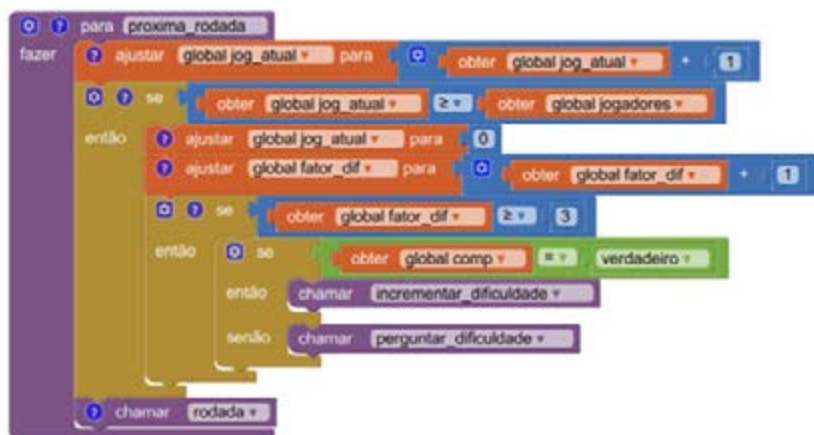


Figura 145: A função “proxima_rodada” garante que novos problemas possam ser respondidos pelos próximos jogadores até que o jogo acabe.

(Fonte: App Inventor)

Entretanto, se a partida for de apenas aprendizado, executamos outra função: `perguntar_dificuldade`. Essa função apenas chama o procedimento do Notificador que mostra uma pequena janela na tela perguntando se os jogadores querem aumentar o nível de dificuldade, tendo dois botões: Aumentar e Manter (a janela não é cancelável). Quando um dos botões for pressionado, o evento `DepoisDeEscolher` do Notificador será lançado.

Neste evento, o único parâmetro presente se chama `escolha`, possuindo o texto do botão pressionado. Dentro do escopo, teremos três blocos SE, um para o caso onde `escolha` tem valor “Aumentar”, outro para valor “Manter” e outro para valor “Sim” (este último explicaremos depois).

No caso de manter a dificuldade como está, apenas zeramos a variável `fator_dif`. No caso de aumentar a dificuldade, apenas chamamos a função `incrementar_dificuldade`. Veja como o código da função `perguntar_dificuldade` e do evento do notificador ficaram na figura 146.

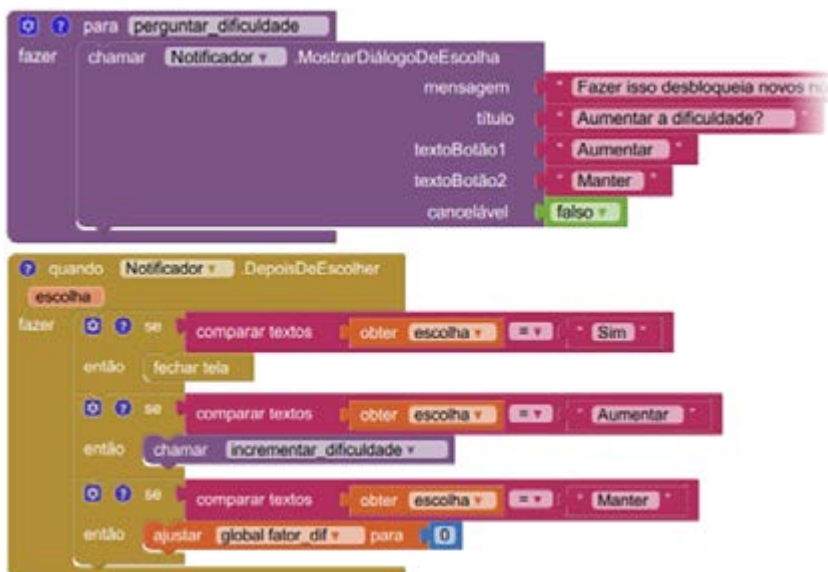


Figura 146: Em uma partida de aprendizado, o aumento de dificuldade não é automático, precisamos perguntar por meio do notificador.

(Fonte: App Inventor)

Retornando para a função `proxima_rodada`, depois que os jogadores decidirem se querem ou não aumentar a dificuldade (ou esta ser incrementada automaticamente ou não), fazemos uma chamada à função `rodada` que prepara tudo novamente para uma nova jogada. Assim garantimos um ciclo de jogadas até que o jogo termine. Quando isso acontecer, a função `verificar_vitoria` finalmente entrará no escopo do bloco SE.

Quando entrarmos no escopo, a fim de evitar erros, executaremos a função `travar_jogo` que retornará o contador de dificuldade para zero fazendo a partida voltar ao início e definir o valor na global `fim_de_jogo` como sendo verdadeiro. Isso afetará o comportamento da função `bloquear_resposta` fazendo com que o botão de Iniciar não seja mais mostrado.

Quando retornarmos à `verificar_vitoria`, prepararemos o valor inicial da tela Resultados criando uma lista com os seguintes valores:

um texto “PONT” e a lista que guardamos na global pont. Após feito isso, chamamos esta tela e deixamos que ela cuide do resto.

Entretanto, antes de deixarmos esta tela, precisamos cuidar de alguns detalhes adicionais. Caso um dos jogadores pressione o botão de voltar do smartphone, executaremos uma função chamada confirmar_saida. Essa função chama o notificador assim como a perguntar_dificuldade, porém perguntando se o usuário realmente deseja sair da partida e os efeitos de se fazer isso. Veja como o código se parece na figura 147.



Figura 147: Precisamos impedir que os usuários saiam da partida acidentalmente. (Fonte: App Inventor)

A resposta Sim a esta janela entrará em um dos casos do evento DepoisDeEscolher que já abordamos, sendo assim, fecharemos a tela já que está foi a escolha dos jogadores. Não precisamos tomar nenhuma providência caso o jogador pressione o botão de Não, então este caso passa batido pelos blocos SE do evento e a janela apenas é fechada.

Por fim, a última coisa que precisamos fazer nesta tela é garantir que ela seja fechada imediatamente após a Resultados ser fechada (porque a partida acabou e temos que retornar ao menu principal). Logo, utilizaremos o bloco de evento OutraTelaFoiFechada comparando se o argumento no parâmetro nomeDaOutraTela é igual a “Resultados”. Se for, fechamos a tela. Verifique como fica o código na figura 148.



Figura 148: Quando a tela Resultados for fechada, precisaremos fechar a tela Matematica imediatamente depois, pois a partida acabou.

(Fonte: App Inventor)

Com isso finalizamos nossa tela onde o jogo ocorre, agora passaremos para a tela onde os resultados da partida são mostrados e onde teremos nosso criador de problemas para que os jogadores possam criar seus próprios arquivos com problemas personalizados e quantos níveis de dificuldade quiserem.

3.5.6. A tela de Resultados e Criação de Problemas

O código desta tela é grande, mas não tanto quanto o da tela Matematica. Mesmo assim, a quantidade de componentes que utilizaremos para lidar com o Criador de Problemas é imensa. Por esse motivo, estudaremos essa tela pela divisão de funcionalidades: primeiro estudaremos a parte de resultados, depois estudaremos o criador de problemas.

3.5.6.1. O Exibidor de Resultados

Quando a tela Matemática chega ao fim, ela manda uma lista com uma chave e uma lista de pontuações para a tela Resultados, essa chave é o texto “PONT”. Sendo assim, a tela deve processar sua parte correspondente ao exibidor de resultados e não o criador de problemas.

Como subdividimos a tela em organizações, podemos habilitar a visibilidade da organização correspondente à fração da tela que queremos. Desta forma, o que será exibido na tela é um conjunto de componentes bem simples que pode ser verificado visualmente na figura 149.



Figura 149: Aparência do exibidor de resultados.

(Fonte: App Inventor)

Agora precisamos dar uma olhada nos componentes que formam essa tela e como eles são arranjados. Primeiro, no aspecto mais geral possível, a tela é formada por duas organizações verticais. Uma chamada “Finalizacao” e outra chamada “CriadorProbs”. Os demais componentes de nível geral são os de costume: TinyDB, Arquivo, Temporizador e Notificador.

Dentro da organização de Finalizacao, temos os seguintes componentes:

1. Uma legenda com o texto de congratulações.
2. Uma legenda onde os jogadores vencedores serão declarados.
3. Uma legenda com a pontuação mais alta do jogo.
4. Uma legenda com uma mensagem mais detalhada de parabenização.
5. Uma organização vertical usada como separador visual.
6. Uma legenda com um título para indicar uma listagem.
7. Uma legenda com os jogadores e suas respectivas pontuações na partida.
8. Uma organização vertical usada como separador visual (novamente).

9. Um botão para fechar a tela.

A árvore de componentes inteira pode ser vista na figura 150.

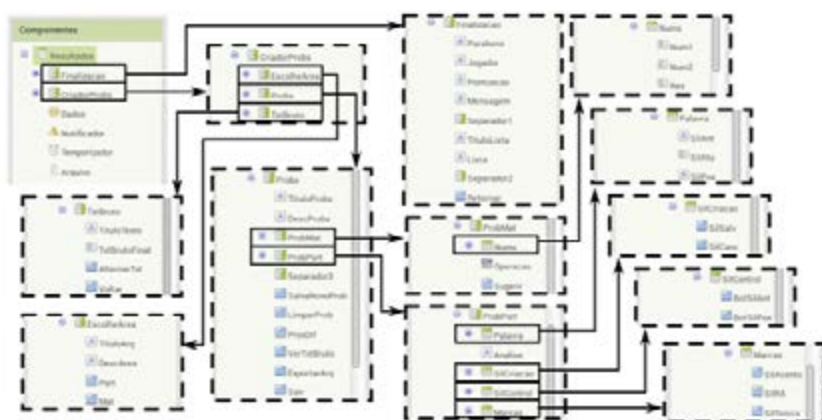


Figura 150: Grande árvore de componentes da tela Resultados.

(Fonte: App Inventor)

Não se assuste! A quantidade de componentes é grande, mas abordaremos por partes, começaremos pela organização vertical Finalizacao.

3.5.6.1.1 Blocos

Quando entrarmos na tela, cairemos no evento de Inicializar da tela. Dentro deste evento, temos a chamada da função adaptar como a primeira coisa a ser executada. Entretanto, as funções de adaptação serão um pouco diferentes devido ao grande número de componentes a serem adaptados. Criaremos algumas variáveis globais para que o manejo de componentes fique mais fácil quando realizarmos a nova abordagem da adaptação. Verifique na tabela 28 como se chamarão e para que servirão tais variáveis.

Tabela 28: Variáveis globais que conterão listas com componentes para adaptação às configurações de acessibilidade.

Variável	Objetivo
legendas	Todos os componentes do tipo legenda.
titulos	Componentes do tipo legenda que serão adaptados para tamanho de título.
caixasdetexto	Todos os componentes do tipo caixa de texto.
botoes	Todos os componentes do tipo botão.
orgsh	Todos os componentes do tipo organização horizontal.
orgsv	Todos os componentes do tipo organização vertical.

(Fonte: Do Autor)

Sendo assim, cada uma das variáveis globais criadas receberá uma lista e cada elemento desta lista será um bloco que represente os componentes designados. Quando formos executar as funções de adaptação, usaremos dois novos blocos: o “Para cada item na lista” e o “Ajustar [componente].[propriedade]”.

Entretanto, onde podemos encontrar estes blocos? Se olharmos na lista de categorias de blocos, o Para Cada Item na Lista está nos blocos de Controle enquanto que o Ajustar Componente Propriedade está no final da lista de categorias, nos blocos que podem ser executados para “Qualquer Componente”.

3.5.6.1.2 Qualquer Componente

Antes de prosseguirmos, precisamos esclarecer algo, os blocos do tipo “Qualquer Componente” não aceitam, literalmente, qualquer componente. Eles esperam componentes do mesmo tipo. Se um bloco de qualquer componente orientado a legendas receber uma caixa/campo de texto para tratar, isso resultará em erros. Por causa disso, em nossa tela estamos separando cada um dos componentes em variáveis distintas que carregarão uma lista de todos os componentes de mesmo tipo.

3.5.6.1.3 O Bloco Componente

Lembre de quando estávamos comentando sobre blocos de acesso a atributos, naquele tópico falamos que não poderíamos usar o bloco de atributo que só possui o nome do componente. Entretanto, este bloco será útil agora. Não queremos mudar propriedades individuais dos componentes como viemos fazendo até agora, mas sim mudar de vários, de uma só vez, sem depender de tratamento individual.

O conteúdo das listas das variáveis globais que criamos serão estes blocos, os que contêm apenas o nome dos componentes. Por quê? Porque quando chamarmos os blocos de qualquer componente para agir sobre as listas seria como estar numa fila.

Os componentes se organizariam e o bloco de qualquer componente chamaria o nome do primeiro. Quando o componente chamado fosse atendido, o bloco informaria qual propriedade mudar e para qual valor mudar. O componente faria isso e seu atendimento teria acabado. Depois, quando o segundo componente fosse chamado, o bloco faria a mesma coisa, e assim sucessivamente até todos os componentes terem sido atendidos.

Mas como percorreríamos esta lista de forma simples. Há todo este tempo viemos usando contadores e laços ENQUANTO para percorrer uma lista a primeira até a última posição. Entretanto, existe um bloco criado especificamente para percorrer listas. Ele se chama “Para Cada Item na Lista”.

Como esse bloco funciona? Primeiro, fornecemos uma lista para que o bloco possa percorrer, a partir daí, só precisamos criar o código dentro de seu escopo para lidar com as tarefas repetitivas. Mas como fazemos o acesso à lista pelo índice? Simples, não fazemos! O bloco possui um parâmetro chamado item (que pode ser renomeado). O argumento neste parâmetro muda a cada ciclo de repetição.

Sendo assim, quando o bloco começa sua execução, o valor em item é o valor do primeiro item da lista. Quando o primeiro ciclo acaba, o valor em item se torna o valor do segundo item, depois, do terceiro e assim por diante. Desta forma, só precisamos fazer acesso a este parâ-

metro para acessar os dados na lista, porque o acesso já está automatizado.

3.5.6.1.4 As funções de adaptação

Agora que temos as listas e as funções de adaptação, só precisamos fazer os laços de lista com os blocos de qualquer componente. Na função `adaptar_cor_paineis` teríamos a adaptação da cor de fundo da tela e depois três laços de lista fornecendo, respectivamente, os botões, as organizações horizontais e as organizações verticais. Consequentemente, os blocos de qualquer componente que utilizaríamos seriam os de qualquer botão, qualquer organização horizontal e qualquer organização vertical (todos mudando a cor de fundo).

Já a função de adaptação de cor dos textos funcionaria da mesma forma, porém recebendo os botões e as legendas. Por fim, a adaptação dos tamanhos de textos seria um pouco diferente, ela receberia legendas, botões e caixas de texto. Entretanto, as legendas foram subdivididas em duas variáveis: legendas e títulos. Sendo assim, primeiro aplicamos o tamanho base a todas as legendas, depois aplicamos outro tamanho de texto para as de títulos. Veja um exemplo na figura 151.



Figura 151: Usando blocos de Qualquer Componente, as funções de adaptação ficam enxutas.

(Fonte: App Inventor)

As outras funções de adaptação continuam as mesmas, intactas, então podemos prosseguir com o código de nossa tela voltando para o evento Inicializar.

3.5.6.1.5 A Identificação de Tarefa

Depois de adaptarmos a aparência da tela, precisamos receber o valor da tela requisitante e discernir qual é o papel que ela deseja que esta tela desempenhe. Faremos isso por meio de uma variável global chamada `val_ini` (que receberá o valor inicial) e a passaremos para um bloco SE: o primeiro elemento do valor inicial é um texto onde está escrito “PONT”? Se sim, sabemos que temos que executar o código referente ao exibidor de resultados. Então vamos nessa!

3.5.6.1.6 Trabalhando em Anunciar Vencedores

Quando entramos no bloco SE, precisamos transferir a lista do valor inicial (segundo item) para a variável global `pontuacoes`. A partir daí, mudamos o título da tela para “Resultados” e fazemos a organização Finalizacao ficar visível (as principais organizações devem estar invisíveis no início da tela). Tendo feito isso, chamamos a função `listar_jogadores` passando a variável global `pontuacoes` como seu argumento.

A função `listar_jogadores` funciona basicamente da mesma forma que a função `ajustar_placar` da tela matemática, exceto pelo ponto em que o texto muda ligeiramente para “Jogador [Número]: Conseguiu [Pontos] pontos!
”. A função percorre a lista adicionando uma linha dessa para cada jogador. No caso, percorremos esta lista da forma convencional porque precisamos usar o contador para identificar os jogadores. Verifique seu escopo na figura 152.

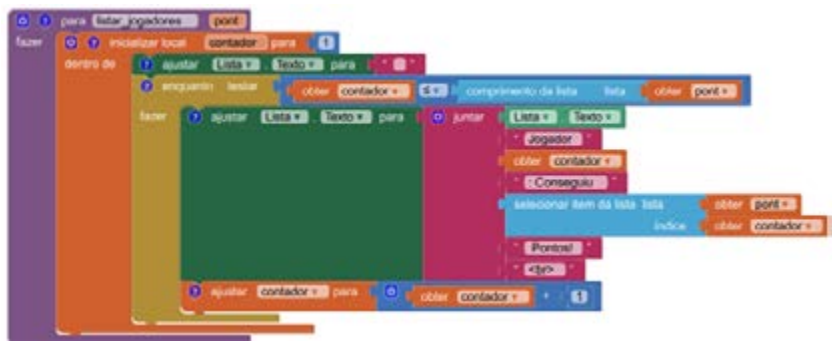


Figura 152: A função “listar_jogadores” cria uma lista textual de jogadores e suas pontuações.
(Fonte: App Inventor)

Com isso, a função é finalizada e retornamos ao evento de Inicializar. Agora temos que chamar a função anunciar_vencedor passando o mesmo parâmetro de pontuacoes. Essa função tem quatro variáveis locais, veja na tabela 29 para quê serve cada uma destas variáveis.

Tabela 29: Variáveis locais da função “anunciar_vencedor”.

Variável	Objetivo
contador	Contador genérico usado para percorrer a lista de pontuações.
p_venc	Pontuação mais alta da partida encontrada pela chamada da função valor_extremo.
n_venc	Número de jogadores que alcançaram a pontuação máxima (vencedores).
temp	Variável de acesso temporário a uma pontuação específica.

(Fonte: Do Autor)

Dentro do escopo da função, limpamos o texto da legenda Jogador para que possamos escrever uma nova sentença depois. Então entramos em um laço ENQUANTO, percorrendo a lista até seu fim usando um contador. Armazenamos o valor da pontuação atual da lista na variável

temp e, dentro de um bloco SE, comparamos se aquele valor é igual ao valor em p_venc (a maior pontuação dos jogadores).

O valor na variável p_venc é o resultado da chamada da função valor_extremo. O que essa função faz é achar o menor ou o maior valor em uma lista. Seu escopo é dotado de três variáveis locais: valor (o resultado da busca por um valor extremo), contador (usado para percorrer a lista) e temp (usado para armazenar temporariamente um valor da lista).

Enquanto isso, a função tem dois parâmetros: lista (recebe a lista em que a busca será realizada) e maior (um valor lógico que indica que deve ser procurado o maior valor, quando verdadeiro). A primeira coisa a ser realizada na função é entrar em um laço ENQUANTO para que a lista seja percorrida até seu fim pelo contador. Então, pegamos o valor atual da lista apontado por contador e armazenamos em temp.

A partir daqui começam as avaliações. No bloco SE subsequente, avaliamos se este é o primeiro elemento da lista por ver se o contador é igual a 1, se for, definimos o valor da variável valor como sendo o mesmo em temp, senão, caímos na extensão SENÃO do bloco e nos deparamos com outro SE. Neste último, verificamos se o argumento de maior é verdadeiro.

Seja o argumento verdadeiro ou não, tanto no SE quanto no SENÃO, nos depararemos com um bloco SE. O primeiro verifica se o valor em temp é maior do que o em valor e repassa o valor de temp para a variável valor se a condição for satisfeita. O segundo faz a mesma coisa, porém, apenas se temp for menor do que valor. Por fim, saindo dessa cadeia de blocos SE, voltamos ao ENQUANTO e incrementamos o contador para continuar percorrendo a lista. Veja esta função na figura 153.

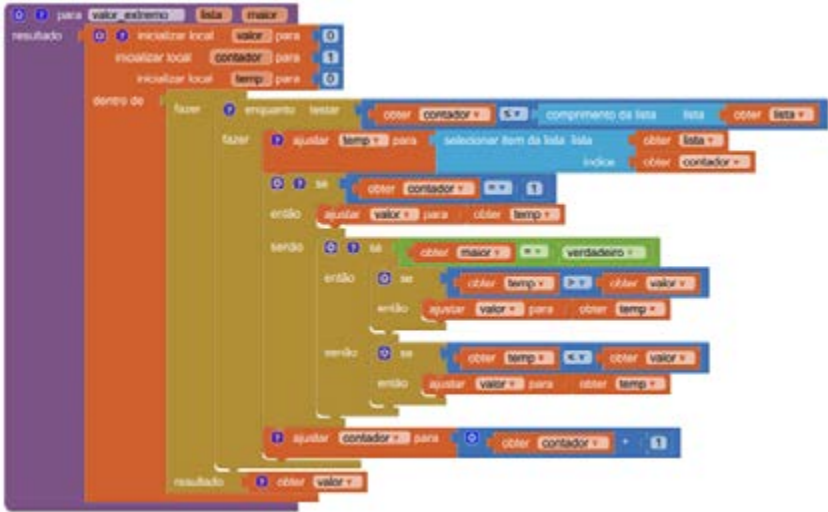


Figura 153: A função “valor_extremo” procura o menor ou maior valor em uma lista.
(Fonte: App Inventor)

Retornando para a função `anunciar_vencedor`, continuamos executando nosso laço ENQUANTO que verifica quais jogadores alcançaram a pontuação máxima da partida e os declara vencedores na legenda. A cada novo vencedor detectado, incrementamos o valor em `n_venc` (número de vencedores) e o contador para continuar percorrendo a lista. Veja o laço ENQUANTO desta função na figura 154.



Figura 154: A função “anunciar_vencedor” procura vencedores por meio de um laço.
(Fonte: App Inventor)

Por fim, ao sair do laço, nos deparamos com um bloco SE que verifica se o valor em `n_venc` é maior do que 1 (significando mais de um vencedor). Se for, então anunciamos na própria legenda que tivemos um empate, senão, declaramos apenas um vencedor. Antes de finalizar a função adicionamos a pontuação mais alta à legenda `Pontuacao`. Veja este bloco SE na próxima figura:

Com isso finalizamos a exibição dos resultados de uma partida, mas e agora? Ainda falta o Criador de Problemas! Então continue firme na leitura e vamos para o próximo tópico

3.5.6.2. O Criador de Problemas

O criador de problemas é a parte mais complexa desta tela, idealizamos ele para que o usuário não precisasse editar o arquivo de problemas manualmente. Sendo assim, o criador de problemas seria intuitivo e fácil de usar não requerendo o conhecimento técnico do usuário. Por conseguinte, pela grande quantidade de componentes envolvidos no criador de problemas, abordaremos ele por partes.

3.5.6.2.1 Opção de Área

Antes de começarmos a editar o arquivo, temos que saber qual área do saber o arquivo está sendo criado para servir problemas sobre. Então, o primeiro componente de organização vertical a ser mostrado quando entramos na tela de Resultados por causa do Criador de Problemas é o componente `EscolheArea`. Veja como fica seu aspecto visual na figura 155.

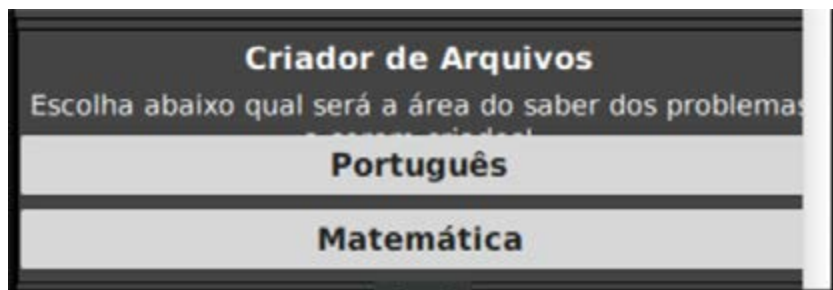


Figura 155: Antes de criar problemas, um usuário precisa definir a área deles.
(Fonte: App Inventor)

3.5.6.2.2 Criador de Problemas Matemáticos

Precisamos estabelecer um padrão de interface que seja intuitivo para que o usuário possa informar os valores matemáticos que comporão a expressão. Com isso, fornecemos 3 campos de texto onde são fornecidos, respectivamente, o primeiro número, o segundo número e uma resposta que pode ser pré-calculada pelo aplicativo. Além disso, há uma caixa suspensa para escolher o tipo de operação aritmética usada. Veja como fica o criador de problemas matemáticos na figura 156.

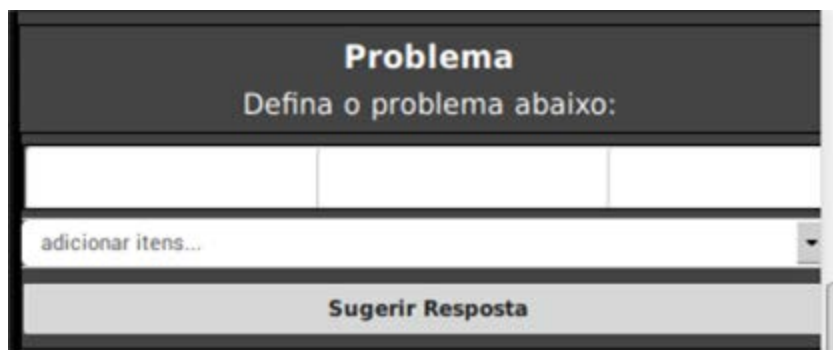


Figura 156: O criador de problemas de matemática espera dois números, uma operação e pode calcular um resultado.

(Fonte: App Inventor)

3.5.6.2.3 Criador de Problemas de Português

Quando o usuário decidir criar problemas de português, teremos uma interface ligeiramente diferente, composta de um campo de texto central e duas legendas laterais, o usuário poderá escrever na ordem silábica que preferir. A sílaba editada aparecerá na caixa de texto à medida que as anteriores aparecerão na legenda esquerda e as posteriores na legenda direita. Verifique na figura 157 a aparência do criador de problemas de português.



Figura 157: Problemas de português podem ter uma sílaba editada por vez.
(Fonte: App Inventor)

3.5.6.2.4 Botões de Ação

Independente da área que tenhamos escolhido para criar os problemas, alguns botões continuarão os mesmos a aparecer na tela, esses são nossos botões de ação, verifique na figura 158 como arranjamos ele de forma que fiquem apresentáveis.



Figura 158: Independente da área, algumas ações feitas pelo usuário continuam as mesmas.

(Fonte: App Inventor)

3.5.6.2.5 Visualizador de Texto Bruto

Por último, temos um visualizador de texto bruto que terá o texto que será escrito no arquivo quando o usuário decidir exportar o conteúdo criado. Não permitiremos uma edição manual de primeira, porém o usuário poderá desbloquear a edição se quiser editar manualmente o arquivo. Veja como o visualizador de texto bruto é na figura 159.

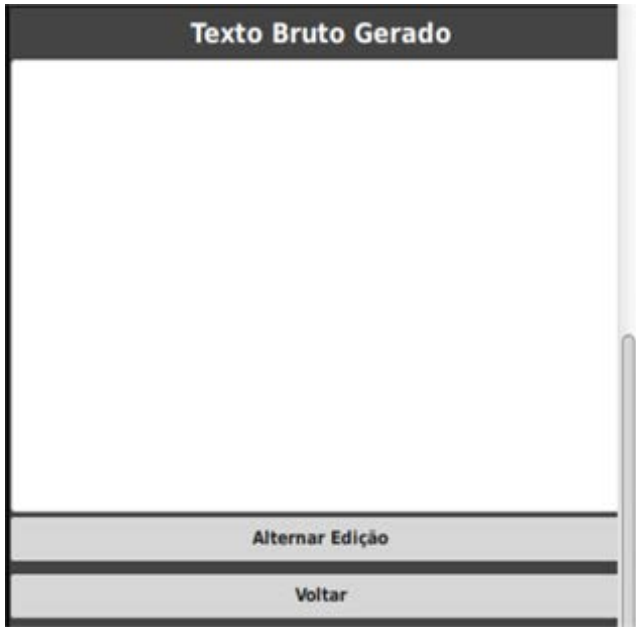


Figura 159: O visualizador de texto bruto pode permitir uma edição manual do conteúdo do arquivo.

(Fonte: App Inventor)

3.5.6.2.6 Código-fonte

Agora que temos uma noção do que cada conjunto de componentes fará, precisamos fazer o código que tratará tudo isso. Primeiro explicaremos as variáveis globais envolvidas nas operações. Veja na tabela 30 para qual finalidade cada uma delas é usada.

Tabela 30: Variáveis globais do Criador de Problemas e seus objetivos.

Variável	Valor Inicial	Objetivo
prob_inicial	Verdadeiro	Informa ao aplicativo se o problema que o usuário está fazendo não é precedido por um divisor de problema (“&”).
nome_arq	Texto vazio	Nome do arquivo a ser escrito o conteúdo gerado.
area	Texto vazio	Área do saber abordada pelo arquivo sendo feito.
prob_port	Lista com um item de texto vazio	Uma lista com cada uma das sílabas da palavra escolhida pelo usuário para ser usada como um problema.
sílaba	1	Indica a sílaba que está sendo editada no momento.
acento	Zero	Indica a posição da sílaba acentuada na palavra.
tonica	Zero	Indica a posição da sílaba tônica na palavra.

(Fonte: Do Autor)

Agora que sabemos para quê servem as variáveis globais, podemos começar a abordar a funcionalidade do código. A diferença fundamental entre a tela de Resultados ser aberta para mostrar pontuações e vencedores ou o criador de problemas acontece no evento de inicialização da tela. É nesse código que avaliamos se o primeiro elemento da lista repassada é um texto “PROB”. Se esse for o caso, então mudamos o título da tela para “Criador de Problemas” e deixamos visíveis as organizações CriadorProbs e a EscolheArea.

Sendo assim, nosso usuário estará vendo dois botões em sua frente, cada um dizendo uma das duas possíveis áreas de abordagem do arquivo. Sendo assim, se o usuário pressionar qualquer um destes botões, as coisas que temos que fazer são as mesmas, mas mudando valores.

Primeiro, temos que definir o valor da variável area como sendo a área escolhida pelo usuário, “Português” ou “Matemática”. Segundo, a variável nome_arq assume nomes diferentes de acordo com a área: “prob_port.txt” (se o usuário escolher português) ou “prob_mat.txt” (se o usuário escolher matemática).

Depois de modificar as variáveis, temos que mostrar os respectivos criadores de problemas, então tornamos visível a organização ProbMat e invisível a organização ProbPort (no caso da escolha por matemática)

e vice-versa. No fim do evento dos dois botões, chamaremos a função `entrar_difs`. Mas antes de abordarmos seu conteúdo, veja como ficam os códigos dos eventos dos botões na figura 160.



Figura 160: Os botões de escolha de área têm comportamentos semelhantes.
(Fonte: App Inventor)

O funcionamento da função `entrar_difs` é muito simples, ela apenas esconde a organização `EscolheArea`, muda a legenda `TituloProbs` para que o título confira com a área escolhida e tornamos visível a organização `Probs` que é onde ficam os criadores de problemas específicos das áreas do saber que abordamos. Veja na figura 161 como essa função se parece.



Figura 161: A função “`entrar_difs`” mostra organizações e modifica um título dependendo da área escolhida.

(Fonte: App Inventor)

Agora que as funções já fizeram seus papéis, ficamos à espera da interação do usuário, então abordaremos o código via eventos. Os primeiros eventos que abordaremos são os dos botões de ação (a partir daqui nos referiremos às ações dos botões como vinculadas ao evento `Clique`). Um deles é muito simples, o de `Sair`. Sua única função é fechar a tela.

Em seguida, temos o botão `VerTxtBruto` (ver texto bruto), responsável por esconder a organização `Probs` e mostrar a organização

TxtBruto (ao mesmo tempo que desativa a edição da caixa de texto para o texto bruto do arquivo).

Depois disso, temos o botão AlternarTxt que, quando clicado, torna editável a caixa de texto que citamos. Entretanto, se o usuário não quiser editar o texto bruto, ele pode querer voltar aos criadores de problemas que, agora, estão ocultos. Para isso, o botão Voltar trata de esconder a organização TxtBruto e mostrar a Probs.

O botão ExportarArq é responsável por concretizar todas as ações do criador de problemas por apenas adicionar um símbolo de ponto e vírgula ao fim do texto bruto na caixa de texto e jogar todo o seu conteúdo dentro de um arquivo cujo nome foi indicado na variável nome_arq (assim como notificar o usuário da exportação).

Já o botão LimparProb executa duas funções: limpar_problema_mat e limpar_problema_port. A primeira é simples por apenas limpar o texto das caixas de texto Num1, Num2 e Res (do criador de problemas matemáticos). Já a segunda é responsável por retornar as variáveis globais acento, tonica e silabas a seus valores originais (zero, zero e um respectivamente), além disso, ela precisa chamar a função limpar_lista fornecendo a variável prob_port como argumento. Veja o código dos botões abordados na figura 162.

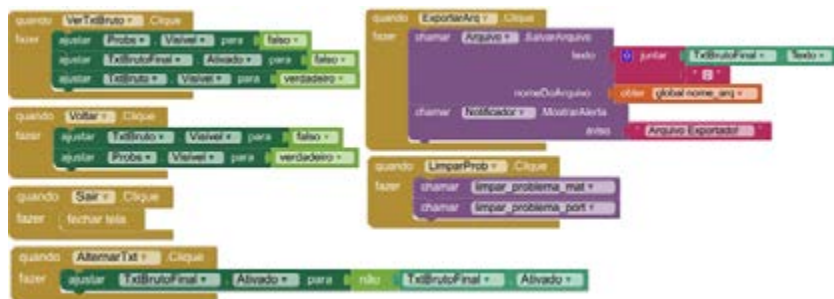


Figura 162: Alguns botões da tela proveem funcionalidades independentes da área do saber escolhida.

(Fonte: App Inventor)

Agora veja como o botão LimparProb se relaciona com as funções de limpeza na figura 163.

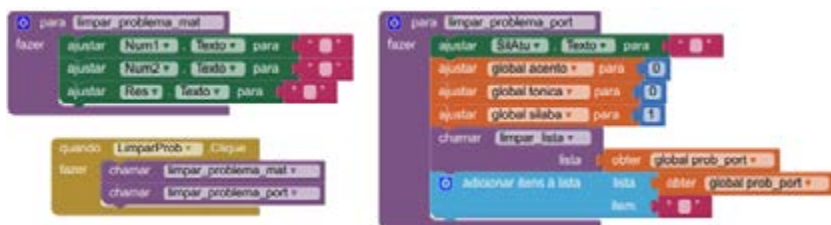


Figura 163: As funções de limpeza poupam esforço do usuário.

(Fonte: App Inventor)

A função `limpar_lista` funciona de uma forma bem simples: enquanto a lista recebida como argumento tiver, pelo menos, 1 item, o item inicial deve ser removido. Sendo assim, todos eles serão removidos. Ao finalizar essa função, retornamos à requisitante que adiciona um elemento com texto vazio a fim de evitar erros. Feito isso, a função acaba, veja seu código na figura 164.



Figura 164: A função “`limpar_lista`” remove o primeiro elemento de uma lista até que não sobre nada.

(Fonte: App Inventor)

Quando o usuário decidir que quer salvar o problema que acabou de elaborar, ele pressionará o botão `SalvaNovoProb`. Esse botão primeiro verifica qual é a área do arquivo de problemas e executa as funções `salvar_prob_port` ou `salvar_prob_mat` dependendo da área. Veja o código na figura 165.



Figura 165: O botão de salvar problema executa métodos de salvamento diferentes dependendo da área escolhida

(Fonte: App Inventor)

A função `salvar_prob_port` verifica se a palavra tem alguma sílaba tônica (variável `global tonica` maior do que zero), se sim, então chamamos a função `empacotar_problema`, senão, informamos um erro via notificador (a palavra é inválida).

Já a função `salvar_prob_mat` verifica se o resultado da função `problema_mat_eh_valido` é verdadeiro. Se não for, informamos um erro via notificador (os números informados são inválidos). Mas se o retorno for verdadeiro, entramos em outro bloco SE que verifica se o retorno da função `resposta_eh_valida` é verdadeiro, se não for, informamos que a resposta é inválida, caso contrário, chamamos a função `empacotar_problema`. Veja as duas funções de salvar na figura 166.

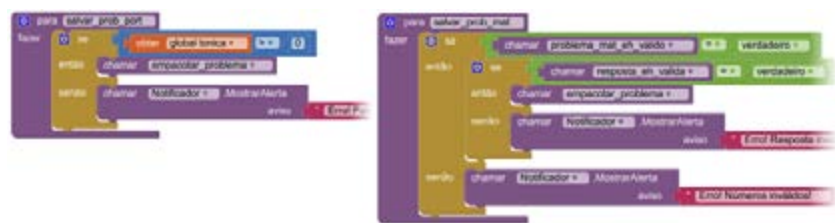


Figura 166: Funções de salvamento de um problema verificando erros antes de salvá-lo.

(Fonte: App Inventor)

As funções `resposta_mat_eh_valida` e `resposta_eh_valida` funcionam da seguinte maneira: a primeira verifica se os textos nas caixas de texto, onde são digitados os dois números da expressão, são realmente números. Caso sim, retornamos verdadeiro, caso não, retor-

namos falso. A segunda função verifica se o campo de texto Res (onde fica a resposta da expressão) é um número válido, se sim, retornamos verdadeiro, senão, falso. Veja essas funções na figura 167.



Figura 167: Antes de salvar um problema matemático, é preciso checar se ele é válido. (Fonte: App Inventor)

A função empacotar_problema é muito importante, o que ela faz é definir uma variável local chamada pacote (inicializada com um texto vazio). A partir daí, temos dois blocos SE para verificarmos qual é a área do saber sendo usada. Se for português, chamamos a função empacotar_port, se for matemática, chamamos a função empacotar_mat. Veja sua primeira parte na figura 168.



Figura 168: A função “empacotar_problema” prepara um problema para ser incluído no texto bruto final do arquivo.

(Fonte: App Inventor)

A função empacotar_mat é bem simples, ela gera um texto que é uma junção respectiva do texto em Num1, o resultado da função conseguir_operador, e os textos em Num2 e Res (campos de texto) mas colocando símbolos de cifrão entre eles para que o padrão do arquivo quanto a elementos de problema sejam satisfeitos. Veja como ela efetua a junção na figura 169.



Figura 169: A função “empacotar_mat” junta elementos de problema para formar um texto bruto.

(Fonte: App Inventor)

A função conseguir_operador apenas analisa, por meio de uma série de blocos SE qual foi a operação aritmética escolhida pelo usuário por meio da lista suspensa e retorna seu correspondente como caractere ASCII. Veja como ela faz isso na figura 170.

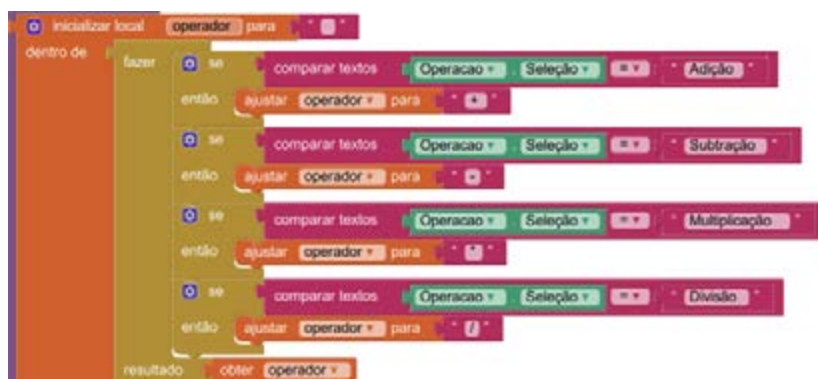


Figura 170: A função “conseguir_operador” converte a opção de uma lista suspensa em um caractere ASCII.

(Fonte: App Inventor)

Já a função empacotar_port é mais complexa. Ela define duas variáveis locais chamadas contador (inicializado em 1) e prob_empacotado (inicializado em um texto vazio). Sendo assim, definimos o valor de

prob_empacotado como sendo o primeiro item da lista e incrementamos o contador. Por que precisamos fazer isso? Porque agora entramos no laço ENQUANTO que adicionará as novas sílabas ao texto precedidas de um cifrão (por meio do uso da função finalizar_silaba). Veja seu código na figura 171.

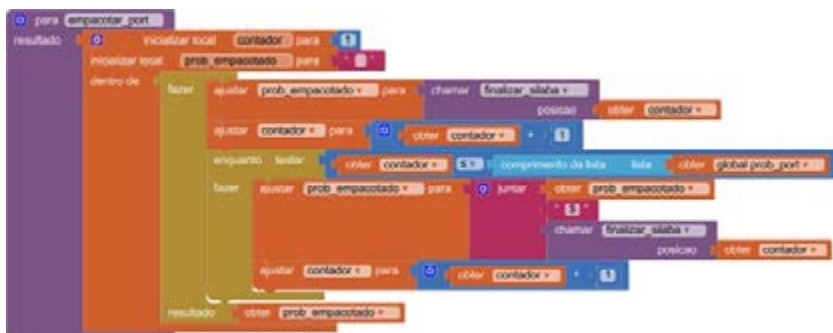


Figura 171: Idealmente, em nossas estruturas, nenhum caractere de elemento, problema e/ou dificuldade deve ficar imediatamente após ou antes de outro.
(Fonte: App Inventor)

A função finalizar_silaba é responsável por receber o número da sílaba a ser finalizada e verificar se esta sílaba é acentuada e/ou tônica e aplicar as devidas marcas especiais ao seu texto, devolvendo-o.

Se entrássemos de vez no laço ENQUANTO, o aplicativo entenderia que há uma sílaba vazia no início do problema. O primeiro item da lista resultado da repartição (feita pelo caractere cifrão) de um texto começando em cifrão seria um texto vazio. Por isso tratamos este erro não precedendo o primeiro elemento de tal símbolo. Veja o escopo da função finalizar_silaba conforme a figura 172.

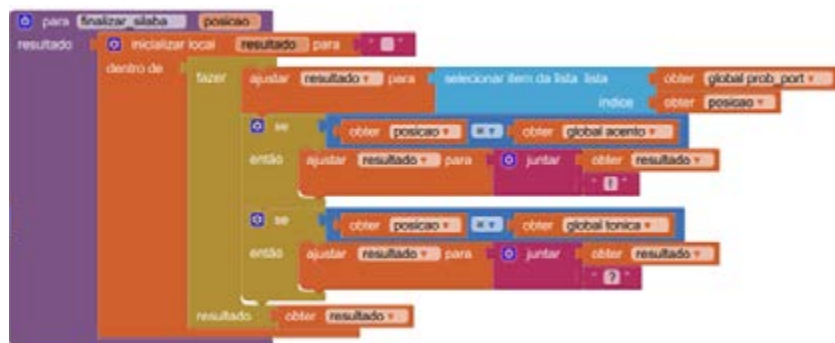


Figura 172: A função “finalizar_silaba” adiciona marcas especiais a sílabas acentuadas/tônicas.

(Fonte: App Inventor)

Feito isso, agora só precisamos percorrer a lista até o fim adicionando um cifrão antes dos próximos itens da lista. Por fim, o que a função faz é devolver o valor em `prob_empacotado`, onde estivemos armazenando o texto bruto gerado pelo laço ENQUANTO.

Tanto o resultado da função `empacotar_port` quanto o da `empacotar_matéjogadoparaavariávelpacotenafunçãoempacotar_problema`. Dando continuidade ao escopo da função, entramos em um bloco SE para ver se o comprimento do texto é maior do que zero (para evitar que erros ocorridos se propaguem). Se não for, informamos, via notificador, que houve um erro durante o empacotamento do problema. Mas, se esse não for o caso, apenas prosseguimos.

A partir daí, nos deparamos com um novo bloco SE que verifica se a variável `prob_inicial` tem valor verdadeiro. Se sim, então apenas ajustamos o valor de `prob_inicial` para falso, senão, adicionamos o separador de problemas (“&”) ao início do texto na variável `pacote`. Feito isso, fora do bloco SE, adicionamos o problema empacotado ao fim do texto na caixa de texto `TxtBrutoFinal`. Chamamos as funções de limpeza de problemas e o usuário estará apto a elaborar o problema seguinte, por fim, notificamos que o problema foi salvo com sucesso. Veja o caso de sucesso e falha da função na figura 173.

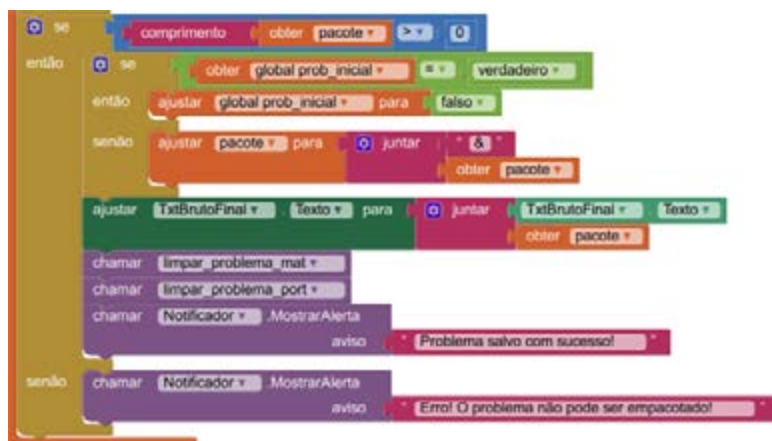


Figura 173: Alguns tratamentos especiais precisam ser feitos para que problemas não aconteçam quando o aplicativo ler o arquivo.

(Fonte: App Inventor)

Mas o que precisamos fazer agora? Voltar nossa atenção aos eventos novamente. O que o criador de problemas matemáticos pode lançar de eventos? O usuário pode pressionar o botão Sugerir. Esse botão executa a função `sugerir_resposta` que, primeiro, verifica se o problema é válido por consultar a função `problema_mat_eh_valido`, se sim, os valores em `Num1` e `Num2` são substituídos por seus correspondentes arredondados e o texto em `Res` se torna o resultado da função `calcular` tendo como argumentos os dois números nas caixas de texto. É importante notar que, se o problema não for válido, então teremos que notificar mais um erro via notificador. Veja a relação do botão com a função na figura 174.

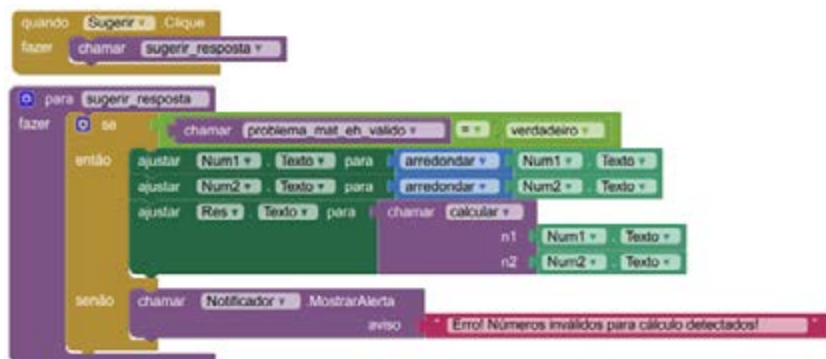


Figura 174: O aplicativo é capaz de sugerir uma resposta a um problema matemático. (Fonte: App Inventor)

A função calcular possui dois parâmetros: $n1$ e $n2$. Dentro de seu escopo, é definida uma variável local chamada resultado, a partir daí, começamos um conjunto de 4 blocos SE seguidos para avaliar qual foi a escolha de operação do usuário na lista suspensa Operacao. Cada operação é executada com os blocos simples do App Inventor, exceto a função de divisão.

Números fracionários com muitas casas decimais podem ser difíceis de se calcular de cabeça, então limitaremos a precisão para duas casas decimais. Infelizmente, o App Inventor não tem um bloco para isso, então fizemos uma função chamada divisao. Veja o trecho do código onde a função divisao é chamada pela calcular na figura 175.



Figura 175: Trecho da função calcular onde a operação de divisão deve ter tratamento especial.

(Fonte: App Inventor)

A função *divisao* recebe dois argumentos nos parâmetros *dividendo* e *divisor*. Antes de qualquer coisa, temos que verificar se o valor no parâmetro *divisor* não é igual a zero, pois isso causaria erros. Se for, devolvemos o texto “ERRO!”.

Se o divisor não for zero, começamos o processo: primeiro, dividimos o *dividendo* pelo *divisor*. Depois, o resultado da primeira operação é multiplicado por 100 e arredondado. Sendo assim, estamos preservando as duas primeiras casas decimais na casa das dezenas e das unidades. Agora, para transformá-las em casas decimais novamente, só precisamos dividir o valor por 100 novamente. Com isso a função *divisao* entrega um valor arredondado até a segunda casa decimal. Quando retornarmos à função *calcular*, devolvemos o valor na variável *resultado*. Assim, retornamos à função *sugerir_resposta* e esta se encerra. Veja a função *divisao* na figura 176.



Figura 176: A função “*divisao*” divide um número por outro arredondando o resultado até a segunda casa decimal.

(Fonte: App Inventor)

Mas quando o usuário quiser avançar o nível de dificuldade, o que ele faz? O botão de ação *ProxDif* faz isso. Em seu evento, ele verifica se a variável *prob_inicial* é verdadeira. Se for, isso significa que não há nenhum problema registrado para aquela dificuldade, não podemos deixar uma dificuldade vazia no arquivo, por causa disso, informamos um erro. Se esse não for o caso, adicionamos um símbolo de cerquilha ao fim do texto bruto na devida caixa de texto, notificamos o incremento via notificador e redefinimos a variável *prob_inicial* como verdadeira. Veja o código na figura 177.



Figura 177: O botão ProxDif toma alguns cuidados antes de criar um novo nível de dificuldade.

(Fonte: App Inventor)

Agora, quando o usuário for editar um problema de português, como ele interagirá com a interface? À primeira vista, ele digitará uma sílaba no campo de texto e clicará no botão de SilSalv (componentes com Sil como prefixo estão relacionados à sílaba). Esse botão desencadeia uma série de ações.

A primeira ação é verificar se a quantidade de letras usadas para formar a sílaba é maior do que zero. Se não for, informamos um erro. Se for uma sílaba válida, entramos em outro bloco SE que verifica se o indicador de sílaba (variável global *silaba*) é menor do que 1 (a sílaba deve ser inserida no início da palavra). Se esse for o caso, usamos o bloco de inserção em posição específica informando a lista, a posição desejada (1) e o item a ser adicionado (no caso, o conteúdo da caixa de texto *SilAtu*). Depois chamamos a função *movimentar* com -1 como argumento.

Caso o valor no indicador de sílaba atual seja maior que o comprimento da lista, então temos que adicionar a sílaba ao fim da lista. Agora, se o número estiver dentro do intervalo que vai desde o início até o fim da lista, faremos uma substituição de itens com o novo valor da sílaba sob o acesso do valor em *silaba*. Veja o escopo do evento *Clique* do botão *SilSalv* na figura 178.



Figura 178: O botão SilSalv tem que lidar cuidadosamente com vários cenários para prevenir erros.

(Fonte: App Inventor)

Quando terminarmos esse bloco SE, voltaremos ao escopo geral da função e as últimas coisas que ela faz é executar a função `atualizar_analise`. O que essa função faz é mudar o texto da legenda Analise para informar ao usuário qual é a sílaba tônica, qual é a sílaba acentuada e quantas sílabas a palavra informada possui. Seu escopo pode ser visto na figura 179.

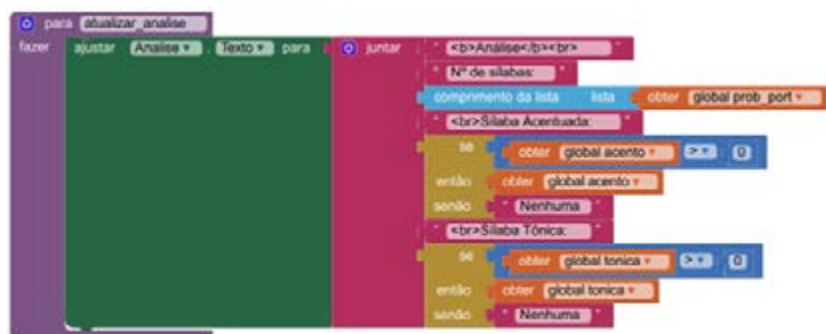


Figura 179: O usuário precisa saber se alguma sílaba já recebeu marcas especiais.

(Fonte: App Inventor)

Depois disso, a função a ser executada é a atualizar_botoes_movimentacao. Essa função verifica se o valor no indicador de sílaba é menor que 1 ou maior que o limite da lista e bloqueia os botões de movimentação dependendo destes fatores. Verifique as restrições na figura 180.



Figura 180: O usuário não precisa editar sílabas sequencialmente, mas a movimentação desregulada do ponteiro de sílaba pode causar erros.

(Fonte: App Inventor)

Mas o que são esses botões de movimentação? No fluxo de nossa interface, o usuário pode passar de uma sílaba para outra, seja da esquerda pra direita ou vice-versa. Isso é feito por meio dos botões de movimentação. Temos dois deles: BotSilAnt (sílabas anterior) e BotSilPos (sílabas posterior). Veja seus eventos na figura 181.



Figura 181: Os botões de movimentação permitem uma edição não linear de uma palavra.

(Fonte: App Inventor)

O que esses botões fazem é chamar a função movimentar fornecendo o valor -1 para retroceder uma sílaba ou 1 para avançar. A função movimentar soma o valor no indicador de sílaba ao valor fornecido como argumento, diminuindo ou aumentando seu valor. Depois de mudar o indicador, ela chama a função atualizar_botoes_movimentacao (para impedir que o usuário acesse posições da lista que causem

erros), a função atualizar_palavra e a função atualizar_analise. Veja como é a função atualizar_palavra na figura 182.

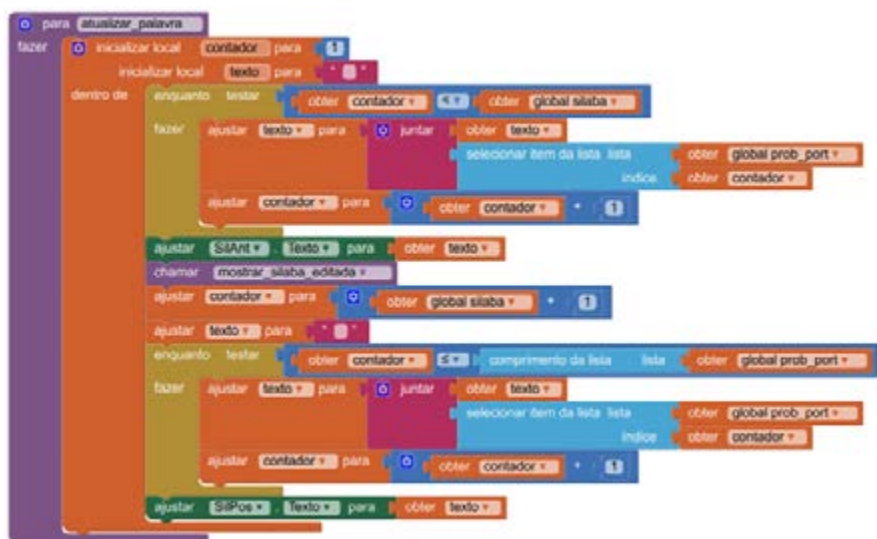


Figura 182: A fim de tornar a escrita das sílabas, apenas a atual será mostrada na caixa de texto enquanto as anteriores e posteriores serão mostradas em legendas laterais. (Fonte: App Inventor)

A função atualizar_palavra é um tanto complexa, tendo duas variáveis locais (texto inicializado em texto vazio e contador inicializado em 1), ela entra em um laço ENQUANTO e percorre a lista da palavra até onde o indicador de sílaba atual se encontra, juntando os itens na variável texto. Quando chegar ao fim do laço, a função muda o texto da legenda SilAnt para o conteúdo em texto.

A partir daí a função chama a função mostrar_sílaba_editada. Esta função tem por objetivo verificar se o indicador sílaba está dentro do intervalo de posições permitidas de acesso à lista em prob_port. Se sim, o texto na caixa de texto SilAtu deve ser a sílaba apontada pela variável indicadora. Senão, a caixa deve ficar em branco. Verifique seu escopo na figura 183.



Figura 183: Antes de mostrar uma sílaba que está sendo editada, esta função toma algumas precauções.

(Fonte: App Inventor)

Ao retornar à função `atualizar_palavra`, temos um incremento da variável contador (para pular a sílaba atual), redefinimos o valor em texto para texto vazio e depois temos outro laço ENQUANTO que faz a mesma coisa que o primeiro, mas desta vez, indo até o fim da lista para mostrar as sílabas posteriores na legenda `SilPos`.

Tendo abordado essas funções, veremos mais quatro botões e seus objetivos: `SilCanc`, `SilRA`, `SilAcento`, `SilTonica`. O botão `SilCanc` é responsável por cancelar uma sílaba que está sendo escrita fora dos limites da lista. O código executado consiste em um bloco SE que verifica se o indicador de sílaba atual está antes do início da lista, se estiver, é preciso mover de volta para o início. Senão, se estiver depois do fim dela, é preciso mover de volta para o fim. Veja seu código na figura 184.



Figura 184: Se o usuário desistir de digitar uma sílaba o botão `SilCanc` se encarrega de eliminá-la.

(Fonte: App Inventor)

O botão `SilRA` serve para zerar a variável global acento de forma que a marca de acento não seja incluída em determinada sílaba. O botão `SilAcento` assimila o valor em sílaba para a variável acento (assim como para a tônica já que toda sílaba acentuada é tônica). O botão `SilTonica` faz o mesmo que o botão `SilAcento`, porém sem intervir no valor da

variável acento. Todos estes três botões concluem suas ações com uma chamada à função `atualizar_analise`. Veja seus eventos na figura 185.

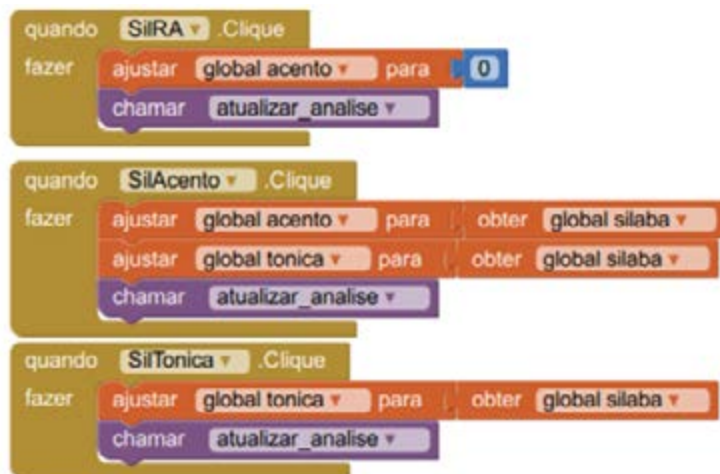


Figura 185: O trio de botões apresentado lida com a adição ou remoção das marcas especiais de uma palavra.

(Fonte: App Inventor)

Com isso, terminamos a criação de nosso segundo aplicativo, agora partiremos para a publicação dele!

3.6. Publicação do Aplicativo na Galeria App Inventor

O App está pronto para ser publicado, só precisamos ir até a sessão de projetos, marcá-lo e pressionar o botão de publicação na galeria. Com isso, teremos que preencher o formulário sobre dados do App, ícone, tutoriais, descrição, informações de autoria e finalizar a publicação para deixá-lo disponível aos usuários da plataforma para download e estudo!

Você pode baixar este aplicativo por meio do seguinte link (<http://ai2.appinventor.mit.edu/?galleryId=6194607099084800>), ou por meio do código QR mostrado figura 186! Note que também há na figura um link (<http://ai2.appinventor.mit.edu/?galleryId=4667719780270080>) e

um código QR para o aplicativo de Utilidades adaptado às configurações de acessibilidade que usamos nesse aplicativo:



Figura 186: Códigos QR para download do App Super Port Mat e do App Utilidades Acessíveis.

(Fonte: Do Autor)

3.7. Atividades de Fixação!

Agora que terminamos de passar por essa jornada para construir um aplicativo acessível e que seja capaz de executar um jogo baseado em um recurso externo, responda as próximas questões para fixar o conteúdo!

1. Por que é bom que aplicativos e sites sejam acessíveis?
2. Quais técnicas de acessibilidade utilizamos no aplicativo para torná-lo acessível?
3. Por que usamos preto e branco para cores de fundo e texto de nosso aplicativo respectivamente?
4. Como os testes WCAG podem nos ajudar a criar aplicativos acessíveis?

5. Por que, pelo menos por enquanto, o uso de imagens no App Inventor deve ser evitado na criação de aplicativos acessíveis?
6. Por que tivemos que criar uma função para ser executada toda vez que uma tela é mostrada para o usuário?
7. O que significa o valor inicial de uma tela?
8. Para quê serve um bloco BREAK?
9. Por que executamos a função adaptar e todas as suas funções associadas?
10. Como a mochila foi útil para replicar as configurações de acessibilidade?
11. Por que implementamos um teclado numérico manual na tela Matemática?
12. Por que, em algum momento, nosso aplicativo poderia vir a ficar estagnado durante uma partida (causando erros)?
13. Como lidamos com a escolha de um estilo de pergunta para a área de Português?
14. Por que foi necessário converter caracteres ASCII para Unicode no momento de montar uma expressão matemática na pergunta de um problema?
15. Qual é o princípio de funcionamento dos botões numéricos do teclado que fizemos?
16. O que fizemos para remover certos caracteres de textos brutos (por exemplo, na função remover_marca)?
17. O que o bloco de texto “Contém” faz e o que ele retorna?
18. Como o notificador lida com os botões apertados na janela resultante da chamada de seu procedimento MostrarDiálogoDeEscolha?
19. Qual seria um efeito colateral da ausência do arquivo de perguntas no decorrer da partida?
20. Qual é a diferença entre as funções incrementar_pontuacao e decrementar_pontuacao?
21. O que aconteceu com as funções de adaptação de cor de texto, tamanho de texto e cor de fundo na tela Matemática?
22. Por que foi preciso travar o jogo assim que a partida terminou?

23. Qual seria uma maneira de evitar que a função `verificar_erro_arquivo` fosse executada toda vez que o botão iniciar fosse executado?
24. Como garantimos o ciclo de jogadas na partida (não apenas a primeira jogada pode ser feita)?
25. Quais seriam algumas diferenças entre uma partida de aprendizado e uma competitiva?
26. O que faz com que o nível de dificuldade de uma partida seja incrementado?
27. Qual código do aplicativo anterior foi reaproveitado nesse aplicativo?
28. Como o botão e a função de alternar sinal da resposta fazem a alternância de sinal?
29. Por que precisamos criar a variável de controle `resp_mat_ponto` que foi preciso criar a variável de controle `resp_mat_ponto`?
30. Que bloco foi fundamental para criar listas a partir de textos brutos com certos padrões na tela Matemática?
31. Por que precisamos executar várias vezes a função adaptar várias vezes na tela de configurações?
32. De que formas o criador de problemas permite que o usuário escreva o arquivo?
33. Em quais três casos o jogador é penalizado pela função `ganha_ou_perde`?
34. Como podemos deixar mais enxuto o processo de adaptação na tela Resultados?
35. Como funcionam os blocos orientados para “Qualquer componente”?
36. Como a tela Resultados é subdividida em funcionalidade?
37. Como a função `anunciar_vencedor`, detecta os jogadores com maiores pontuações?
38. Por que tentamos evitar a edição manual de um arquivo de problemas?
39. No criador de problemas, como funciona a previsão de resposta de um problema de divisão?

Capítulo 4: Alcançando o Mundo Inteiro!

Estamos chegando ao fim de nossa jornada! Agora focaremos nossos esforços em publicar nossos aplicativos na Google Play Store, a loja oficial de aplicativos da Google para o SO Android! Mas antes de continuarmos é preciso atentar para alguns detalhes.

4.1. Alcançar o Mundo tem um Custo

A loja de aplicativos pode ser acessada gratuitamente por qualquer pessoa que possua um computador com acesso à internet ou mesmo um smartphone com o aplicativo da loja. Entretanto ela é gratuita para usuários comuns. Para se tornar um desenvolvedor de uma das lojas de aplicativos mais famosas do mundo há um custo.

4.1.1. Considerações Iniciais

A Google cobra uma taxa de 25 dólares para que uma pessoa adquira um perfil de desenvolvedor em sua loja. Isso pode ser feito com uma conta Google assim como fizemos no acesso à plataforma App Inventor. Primeiro, é preciso entrar na sua conta e se dirigir para o site da Google Play, porém não a parte onde é possível ver as publicações da loja, e sim a parte reservada a desenvolvedores, a Google Play Console (<https://play.google.com/console>). Veja como a página se parece na figura 187.

(

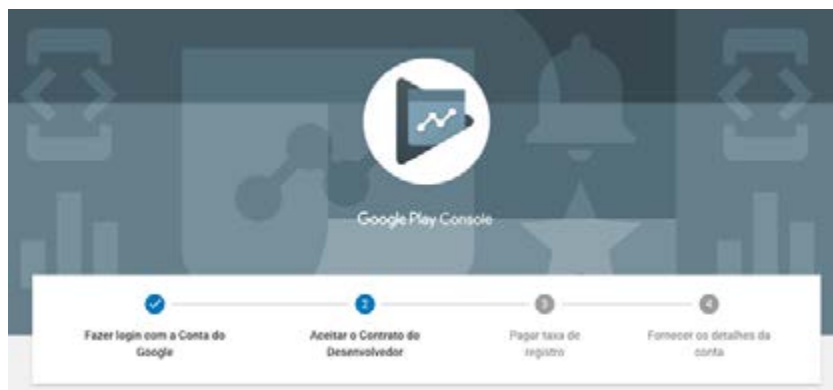


Figura 187: A Google Play Store tem uma sessão destinada para desenvolvedores chamada Console.

Fonte: Google Play Console)

Assim que entrar na página, será pedido que o contrato de desenvolvedor (referenciado na própria página) seja aceito pelo usuário e que ele se declare maior de idade. Além disso é preciso ficar atento às informações que a página oferece sobre comercialização de Apps na plataforma já que, em alguns países, usuários não podem baixar aplicativos gratuitos ou pagos (ou fazer compras por meio deles), além de mostrar a moeda usada no país e a faixa de preços que pode ser praticada na loja de acordo com cada país. Na figura 188 está representada uma fração da página com informações que o usuário deve considerar antes de adquirir seu perfil de desenvolvedor.



Figura 188: A página de aquisição de perfil de desenvolvedor apresenta várias informações úteis para quem deseja publicar aplicativos.

(Fonte: Google Play Console)

Assim que o botão de “Continuar Pagamento” for pressionado, uma pequena janela popup será mostrada pedindo alguns dados. Dentre os dados pedidos estão: endereço do usuário, dados do cartão de crédito, produto sendo pago (registro de uma conta de desenvolvedor) e o valor que está sendo cobrado por tal compra. Após preencher todos os esses dados, é preciso confirmar o pagamento à empresa por meio da operadora do cartão. Verifique a janela popup na figura 189.



Figura 189: Informações de pagamento para aquisição de direitos de desenvolvedor. (Fonte: Google Play Console)

Se for necessário executar etapas adicionais, a plataforma terá as informações indicadas para lidar com elas, mas geralmente este é o procedimento simples. Após confirmado o pagamento, o acesso à Google Play Console é imediatamente liberado e o desenvolvedor se deparará com algumas opções para publicação, comercialização, gerenciamento e observação do aplicativo.

4.1.2. Cadastrando um Aplicativo na Loja

Assim que entrarmos no Console, temos a tela inicial onde podem ser vistos os aplicativos já publicados. Para introduzir este ambiente, mostraremos apenas a publicação do aplicativo Utilidades. Então, para começar a publicação de um novo aplicativo, precisamos clicar no botão azul de “Criar App”. Verifique a página inicial do Console na figura 190.

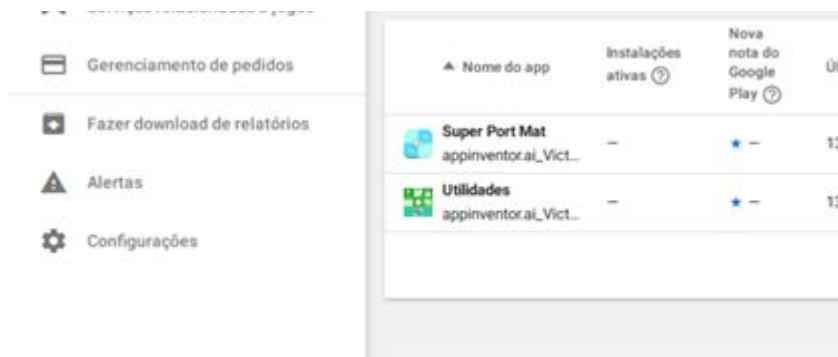


Figura 190: A tela inicial da Google Play Console mostra os aplicativos publicados, é preciso clicar no botão “Criar App” para publicar um App.

(Fonte: App Inventor)

Quando clicarmos no botão, uma janela popup aparecerá perguntando qual é o idioma base/padrão do App e qual será seu **título/nome**¹⁹⁹ (o nome pelo qual as pessoas poderão pesquisar o App na loja).

Quando terminarmos de preencher estes dois dados básicos, seremos transferidos para uma sessão específica da publicação do App. Há novas categorias de ferramentas e formulários que precisam ser preenchidos para que os padrões de qualidade sejam alcançados. Daremos uma olhada nas principais categorias entretanto, ressaltaremos um ponto de vantagem em ter investido 25 dólares numa conta

¹⁹⁹ É importante lembrar que há um limite de 50 caracteres para a escrita do título do App. Não se preocupe, esse título pode ser mudado depois.

de desenvolvedor: pode-se publicar **quantos aplicativos forem desejados**²⁰⁰!

4.2. Fornecendo Informações do App

A primeira sessão de publicação de um App que destacaremos é a “Detalhes do App” que terá todos os dados mais básicos que estão constantemente e mais facilmente visíveis na loja, confira na tabela 31 quais são esses dados e o que se espera deles no formulário:

Tabela 31: A sessão de publicação “Detalhes do App” possui um formulário com diversos itens que caracterizarão o App na loja.

Campo do Formulário	Dados Desejados
Título	O título do App pelo qual os usuários poderão pesquisá-lo na loja. Limitado a 50 caracteres.
Breve Descrição	Uma simples e rápida descrição do App limitada a 80 caracteres.
Descrição Completa	Uma descrição profunda e detalhada do que o App envolve, pode fazer, suas características, qualidades etc. O texto é limitado a 4000 caracteres e a página sugere leituras complementares para evitar erros comuns na hora de informar os metadados do App.
Ícone de Alta Resolução	Uma imagem PNG (com transparência) de 512 pixels em largura e altura. A imagem pode ser um quadrado se transparência. A própria Google Play se encarrega de moldar o ícone para que tenha bordas arredondadas e sombra.
Capturas de Tela	De duas a oito capturas de tela do App para cada tipo de dispositivo (smartphone, tablet, TV com Android e Wear OS). As imagens devem ser do formato JPG/JPEG ou PNG (sem transparência) com 320 pixels como medida mínima de cada lado da imagem e 3840 pixels como medida máxima.

200 Segundo a documentação do Suporte Google (<https://support.google.com/googleplay/android-developer/answer/113469?hl=en>), não há restrições quanto ao número de aplicativos que um desenvolvedor pode publicar, mas há um limite sobre o tamanho virtual dos arquivos APK enviados para a loja. Apps focados em Android 2.2 e versões mais antigas são restringidas a APK's com tamanhos até 50MB, já Apps focados em Android 2.3 e versões mais novas são restringidos a até 100MB. A documentação ainda faz uma ressalva sobre o fato de que os usuários precisam usar o App da Google Play com versão igual ou maior a 5.2 para conseguir baixar APK's de tamanho próximo a 100MB.

Gráfico de Recursos	Uma imagem PNG de 1024 por 500 pixels JPG/JPEG ou PNG (sem transparência) que será usada como miniatura de prévia para algum recurso de vídeo que possa vir a ser adicionado para a promoção do App.
Vídeo Promocional	Um link de um vídeo promocional sobre o aplicativo hospedado pelo serviço de exibição de vídeos do Google, YouTube.
Tipos de App	É preciso enquadrar o App em um tipo como sendo um simples App ou um Jogo.
Categoria	Uma série de categorias em que o App pode se encaixar, existem dois conjuntos diferentes dependendo do tipo do App.
Tags	Tags são rótulos que ajudam as pessoas a encontrarem o App em meio ao grande acervo da loja aumentando a chance do resultado aparecer quando certos textos são usados na pesquisa.
Site	Um site que fale sobre o aplicativo e tenha mais informações sobre ele.
E-Mail	Um endereço de e-mail que pode ser usado para contatar o desenvolvedor do App, não precisa ser o endereço de e-mail GMail da conta Google.
Telefone	Um telefone que pode ser usado para contatar o desenvolvedor, não precisa ser o telefone associado à conta Google.
Política de Privacidade	Um link para um site que explique a política de privacidade usada pelo App no que diz respeito à utilização/manejo de dados pessoais do usuário e das informações armazenadas em seu smartphone. Embora obrigatório, esse dado pode ser omitido temporariamente por acionar uma caixa de seleção logo abaixo do campo de texto onde a URL deve ser inserida.

(Fonte: Google Play Console)

Sobre os erros comuns que podem ser cometidos durante a criação de metadados do aplicativo numa descrição completa, a empresa Google disponibiliza uma leitura complementar sobre o que pode ser feito para evitar problemas e melhorar a divulgação do App (<https://play.google.com/intl/pt-BR/about/storelisting-promotional/metadata/>).

A propósito, caso o conceito de transparência de uma imagem tenha ficado obscuro, esse é um conceito da edição de imagens. Alguns formatos de imagens digitais são capazes de representar transparência. A maioria das ilustrações deste livro foram feitas no formato PNG (que suporta transparência). Os trechos de código em blocos foram cortados

de um download de todos os blocos de todas as telas em imagens, em seguida foi colocado um retângulo branco como fundo de todas as imagens para que os recortes dos blocos não ficassem evidentes.

Imagens são o resultado final de uma edição em camadas. Camadas são como chapas de vidro do mesmo tamanho posicionadas umas sobre as outras. Para dar um exemplo, um artista desenha uma pessoa na chapa superior e o céu na do fundo. Na chapa do meio ele desenha a paisagem. Os desenhos estão, de fato, separados, mas quando as chapas fossem empilhadas e o artista as visse de cima, a imagem estaria completa, como um desenho só. As partes das chapas que não estão cobertas por tinta possibilitam a visão das camadas inferiores. Veja um exemplo na figura 191.



Figura 191: Esquema mostrando como uma imagem pode ser composta de camadas com transparência.

(Fonte: Do Autor)

Continuando nossa discussão sobre a publicação de nosso aplicativo na Google Play, boa parte dos campos de dados citados na tabela são obrigatórios. Agora que destacamos a principal categoria de formulários para publicação de um App, cobriremos as funcionalidades de outras categorias principais, verifique na tabela 32 o que cada uma dessas categorias aborda.

Tabela 32: Categorias principais de formulários para publicação de um App.

Categoria	Abordagem
Painel	Informações úteis para quem está lançando aplicativos pela primeira vez. Assim como artigos informativos e tutoriais.
Versões de Apps	A categoria responsável por receber nosso arquivo APK com o aplicativo em si. É por essa categoria que o aplicativo será publicado.
Catálogo de Dispositivos	Nem todos os dispositivos podem executar aplicativos feitos no App Inventor por causa de certos componentes (desde que eles tenham sido incluídos no App). Os dispositivos que não são capazes de executar o App podem ser excluídos do catálogo por incompatibilidade.
Classificação de Conteúdo	Essa página se encarrega de aplicar questionários para limitar o público-alvo por faixa etária segundo os temas abordados pelo App. Ela fornece diversos selos de classificação indicativa de vários países nos quais um App pode ser enquadrado.
Conteúdo do App	Uma série de formulários para conseguir informações gerais do App.
Preço e Distribuição	A categoria onde os aplicativos podem ser definidos como gratuitos ou pagos, esclarecer se fazem uso de anúncios e em quais países o App será disponibilizado.

(Fonte: Google Play Console)

4.3. Enviando um pacote APK

Para ser capaz de enviar um pacote APK, é preciso entrar na categoria Versões de Apps. Lá pode ser observado que há várias categorias em que um App pode ser lançado, elas podem ser Alpha, Beta e Produção. O que esses termos significam? Alpha significa que um aplicativo ou programa foi liberado para uso e teste de um grupo seletivo e reservado de pessoas.

Enquanto isso, Beta é um aplicativo que passou por muitas melhorias mas ainda está em fase de testes. Porém o público de aplicativos Beta é maior do que o Alpha, para intensificar a busca por erros e encontrá-los mais rápido.

Já a Produção é o estágio onde a maioria dos erros foram solucionados e o App já está pronto para uso, mesmo que não seja perfeito, os erros não acontecem com uma frequência grande o suficiente para estragar toda a experiência de usuário.

Em nosso caso, a categoria que nos interessa é a de produção. Lançaremos os aplicativos para que fiquem disponíveis ao público geral. Portanto, na categoria de Produção, há um botão de Gerenciar, clicando nele, somos redirecionados para uma página onde seremos convidados a clicar no botão Criar uma Versão.

Quando este botão for clicado, a plataforma pedirá se a empresa Google pode assinar os Apps publicados a fim de que os smartphones Android saibam que a procedência do aplicativo é de uma fonte confiável. Continuaremos com essa opção ativa por clicar no botão Continuar.

Agora chega a parte em que somos convidados a enviar o pacote APK de nosso aplicativo. Após enviado, a própria plataforma já interpreta o conteúdo do pacote e dá o espaço em disco gasto, o tipo do arquivo, o código da versão e nos convida a nomear a versão (um nome/número automático pode ser dado).

Em seguida, temos que informar o que foi incluído nesta versão no quadro “O que há de novo?”. Aqui podemos informar funcionalidades que antes não existiam no App e foram adicionadas nesta versão, entretanto, como é nossa versão inicial, colocaremos apenas que se trata da primeira versão do App.

Sendo assim, temos que salvar o rascunho. Ainda não podemos publicar porque temos que visitar outras categorias antes de encaminhar o App à Google. Como, por exemplo, a categoria Classificação de Conteúdo.

Essa categoria diz respeito ao conteúdo e temáticas abordados pelo App. Dependendo do que pode se encontrar de conteúdo no App, ele deve ser restrito a públicos-alvo de certas faixas etárias. Nessa categoria são aplicados questionários que concedem classificações indicativas ao aplicativo.

Ao entrar nela, somos apresentados a uma justificativa e explicação da existência desta página. Teremos que clicar no botão de Continuar. Depois disso, somos convidados a informar alguns dados como e-mail para contato e a selecionar uma categoria em que nosso App se encaixa. Em nosso caso, escolheremos a opção relacionada a conteúdo educa-

tivo. O questionário aborda questões como violência, sexualidade, linguagem e outros. Todas as perguntas têm que ser respondidas.

Assim que respondermos todas as questões, teremos que salvar o questionário e um botão aparecerá para Calcular a Classificação. Clicando nele, teremos selos de classificação indicativa de vários países em que nosso App foi enquadrado. Para concluir o processo temos que clicar no botão Aplicar Classificação.

Agora foquemos na categoria que pode levar nosso App para o mundo: Preço e Distribuição. Nesta categoria, definimos se nosso aplicativo será pago ou gratuito. Em nosso caso ele será gratuito por se tratar de um App educacional, além disso, podemos ver nessa página uma lista de vários países onde nosso App poderia ficar disponível para Download.

Por padrão todos os países estão desabilitados para que possamos escolher a dedo os países em que vamos disponibilizar o App. Em nosso caso, escolheremos apenas o Brasil. Além disso, precisamos responder se nosso App contém anúncios, propagandas, essa pergunta é obrigatória. Por fim, temos que nos certificar de que nosso App está de acordo com as diretrizes de conteúdo do Android (a plataforma provê links com artigos úteis quanto ao tema) e se estamos de acordo com as leis de exportação dos Estados Unidos da América.

Quando finalizarmos, novamente teremos que salvar o rascunho do App. Falta pouco para o lançamento!

Depois desta categoria, outra que devemos destacar é a Conteúdo do App. Nesta categoria somos convidados a Iniciar um questionário. A primeira etapa diz respeito às faixas etárias às quais o uso é permitido. Como decidimos não adicionar uma política de privacidade na categoria Detalhes do App, neste questionário não podemos permitir o uso do App por crianças abaixo de 13 anos.

Indo para a próxima etapa, como já passamos pela fase de Detalhes do App e pela declaração de uso ou não de anúncios, duas etapas são puladas. Na quarta etapa somos perguntados se o App pode atrair acidentalmente a atenção de crianças. E, na última etapa, temos que

revisar e confirmar nossas respostas (ou voltar e corrigir, se necessário).

Com isso, voltamos à etapa inicial, a categoria Versões de Apps. Agora que passamos por todos estes questionários e formulários, podemos ver que, na faixa de produção, há um aviso informando que temos uma versão de Produção ainda não lançada.

Quando clicarmos no botão Editar Versão, podemos descer até o fim da página para iniciar uma revisão por clicar no botão Revisar. Após observar e corrigir o que estiver errado, finalmente, poderemos enviar nosso aplicativo à Google clicando no botão Iniciar Lançamento Para Produção (confirmando o popup que aparecerá perguntando se temos certeza de tal ato).

4.4. O Mundo é Nosso!

Finalmente enviamos nosso aplicativo à Google, nossa jornada está concluída. A empresa se responsabilizará por analisar nosso App e, se tudo ocorrer bem, ele estará na loja, disponível para download, em pouco tempo.

Foi um prazer compartilhar todo o conhecimento que conseguimos absorver da plataforma Google Play Console e da MIT App Inventor com você. Agradecemos grandemente por se dedicar a aprender algo novo e lançar suas ideias ao mundo. Desejamos a você um ótimo caminho e muito sucesso na estrada de criação de aplicativos!

4.5. Atividades de Fixação

1. Como é possível enviar aplicativos para a loja de aplicativos do Google?
2. Qual é a diferença entre um aplicativo Alpha, Beta e Produção?
3. Após responder várias perguntas sobre o conteúdo e o teor do aplicativo, o que o Google fornece em termos de certificação?
4. Em nível geográfico é possível distribuir o aplicativo pela loja baseado em quais parâmetros?

Gabarito de atividades

Aqui são apresentadas todas as respostas dos questionários de fixação, use isso para comparar suas próprias respostas e sempre se lembre, gabaritos assim são para ajudar no seu aprendizado, não para se ter como referência sempre! Tente responder as questões com seriedade e deixe a curiosidade consumir você um pouco!

Capítulo 1

1. Máquinas capazes de executar instruções e computar dados.
2. O código binário composto apenas por dois caracteres, zero e um, significando ausência e presença de energia, respectivamente.
3. Processador, memória principal ou primária, memória secundária e barramentos de conexão.
4. Um sistema que cria uma abstração do sistema de hardware em software para que outros programas sejam executados no computador quase que totalmente independentes das peças que possui.
5. Um conjunto de instruções que um computador pode ler e executar para realizar determinadas ações e manipulação de dados.
6. A proteção de hardware criada pela abstração do sistema operacional sobre a máquina.
7. Linguagens escritas e formais (entendíveis por humanos) mais diversificadas do que o código binário que podem ser convertidas em código binário para que os computadores possam entender o que humanos querem que ele faça.
8. Estruturar instruções e juntá-las de forma sistemática a fim de criar um programa funcional.
9. Não, quanto maior a abstração de uma linguagem, mais próxima ela fica da linguagem humana, quanto menor a abstração, mais próxima fica do código binário.

10. Linguagens compiladas geram programas otimizados para a plataforma em que serão executados e ganham em desempenho por isso. Linguagens interpretadas geram programas que podem ser executados independente da plataforma sacrificando parte do desempenho pela adição de um interpretador no processo de execução.
11. Por meio da rede mundial de computadores que conecta bilhões de dispositivos ao redor do mundo todos os dias, a internet.
12. Eles têm mais capacidade de processamento e realmente podem ser encarados como um minicomputador de bolso.
13. A plataforma App Inventor, sua filosofia é que qualquer pessoa é capaz de criar aplicativos que podem impactar o mundo de alguma forma.
14. Entendendo o problema e identificando pontos dele que podem ser subdivididos e resolvidos individualmente pela técnica de dividir para conquistar.
15. Por meio de esquemas de versionamento.
16. O App Inventor surgiu de uma ideia parecida com a lançada pela plataforma e linguagem de programação Scratch, porém voltada para a plataforma móvel com o objetivo de ajudar as pessoas a criarem aplicativos de forma fácil e divertida para o sistema operacional Android de dispositivos móveis, sendo que os aplicativos podem ser disponibilizados na loja oficial do SO, a Google Play.
17. Criação, alteração, publicação e exclusão de projetos de aplicativos.
18. Edição do aspecto visual e de componentes do aplicativo.
19. Edição da lógica e da mecânica, integração dos componentes e manipulação de dados do aplicativo.

Capítulo 2

1. Ter um público-alvo ajuda a saber quais funcionalidades do aplicativo devem ser priorizadas e quais problemas ou necessidades o aplicativo deve resolver ou suprir.
2. Verdadeira.
3. Quanto mais se tiver uma ideia de solução menos perdido o desenvolvedor se sentirá em relação ao projeto.
4. O nível de detalhes sabidos sobre o problema ajuda a entender quais técnicas ou tecnologias podem ser usadas, e como usá-las, na resolução.
5. Subdividir. Resolver pequenos problemas que, unidos, resolvem o todo, torna todo o processo de desenvolvimento mais fácil.
6. Porque evitam o esforço de “reinventar a roda” usando estruturas de código que já foram feitas pela primeira vez e depois são apenas reutilizadas, não refeitas.
7. Algoritmos definem uma sequência de passos finita e não ambígua para a resolução de um problema, eles podem ser usados como um mapa para o desenvolvedor saber o que precisa codificar em seu aplicativo.
8. Manutenibilidade é a capacidade do sistema de ser submetido a manutenções e consertos, Escalabilidade é a capacidade do sistema de se manter operante e funcional mesmo em situações de uso crescente e frequente. Um sistema com melhores escalabilidade e manutenibilidade podem ser úteis ao seu público-alvo por consideravelmente mais tempo.
9. São objetos do Android já prontos que podem ser incluídos em um aplicativo para gerar as funcionalidades dele.
10. Prover componentes que façam interação direta com o usuário a fim de conseguir e mostrar informações.
11. Para agrupar e posicionar os componentes exibidos na tela do aplicativo.

12. Esconder o funcionamento de algo que já funciona comprovadamente diminui parte da complexidade do sistema.
13. As propriedades regulam partes das características dos componentes, seja por seu funcionamento ou por sua aparência.
14. Comportar outros componentes dentro de si.
15. Falsa. Aplicativos feitos em Java não precisam de telas, mas os feitos no App Inventor precisam de, pelo menos, uma para existir.
16. Porque um número grande de telas consome muita capacidade de processamento, tanto do navegador do desenvolvedor, quanto da plataforma App Inventor, quanto do sistema Android.
17. Prover um componente clicável que dispara um evento quando pressionado.
18. Para exibir um texto na tela do smartphone.
19. Dados do tipo texto. Sim, o App Inventor pode considerar números e valores lógicos como texto e vice-versa dependendo do caso.
20. Modo Automático, Modo de Preencher Principal, por Pontos/Pixels e por Porcentagem.
21. Agir como uma especificação de blocos que os exigem, principalmente para retornar um valor.
22. Falsa, os eventos não estão diretamente ligados ao usuário em todos os casos. Eles podem acontecer pelo uso de um componente por parte de outro componente, ou serem disparados pelo próprio código.
23. O bloco SE, o bloco Fazer e o bloco de definição de variável.
24. Executar uma instrução sequencial: dependendo do bloco, o processamento do código acaba nele ou segue para a próxima instrução.
25. Blocos de Atributos.
26. FonteNegrito e FonteItálico, para unir as duas formatações, é preciso combiná-las ou usar a opção HTMLFormat.

27. Quando é preciso que os componentes sejam arranjados visualmente um ao lado do outro.
28. Quando a quantidade de componentes que precisa ser mostrada na tela ao mesmo tempo excede a capacidade física da tela e é preciso usar a técnica de rolagem.
29. OcultarTeclado do componente Caixa de Texto, ObterValor do componente TinyDB e Agora do componente Temporizador.
30. É um endereço na memória identificado por um nome capaz de guardar um dado de determinado tipo.
31. Variáveis podem guardar valores que precisam ser usados em mais de uma linha de código sendo capazes de enxugar o código. Usar valores em sua forma direta pode levar a erros e custam mais tempo para conserto, usando constantes, se o valor estiver errado, só é preciso alterar o valor da constante.
32. A alça inferior e o encaixe para ela na parte superior dos blocos de instruções.
33. Porque eles são capazes de comparar valores e executar certos códigos dependendo do resultado da comparação.
34. Verdadeiro, Falso.
35. Elas concentram apenas uma vez, dentro de si, um código que precisaria ser repetido diversas vezes seguindo um padrão.
36. Por centralizar o código nas mesmas funções e procedimentos, implicando em reutilização e facilitando manutenções.
37. Uso de parâmetros que recebem dados como argumentos.
38. Para modificar a forma de alguns blocos de forma a estender sua funcionalidade. O bloco de junção de texto e o bloco de soma de números.
39. RGB. Vermelho, Verde e Azul.
40. Uma cor no App Inventor é uma lista onde o primeiro item é a intensidade da cor vermelha, o segundo é a intensidade da cor verde e o terceiro é a intensidade da cor azul.
41. Uma variável especial que aponta para diversos endereços de memória diferentes que podem ser acessados dependendo do

valor de um índice numérico que é usado junto ao nome do vetor.

42. Porque seu tamanho é maleável e podem ou não ser indexadas ou conter tipos diferentes de dados.
43. Visualmente os deslizadores são barras com um pequeno indicador clicável e arrastável onde a posição indica um determinado número numa escala entre um número máximo e mínimo indicados pelo desenvolvedor.
44. Ambos têm a mesma função de receber dados textuais, porém o campo de senha formata todos os caracteres digitados como pequenas bolinhas para que não seja possível discernir o que foi digitado.
45. A configuração de Visibilidade. Acima do visualizador há a opção “Mostrar os componentes ocultos no Visualizador” que mostra todos os componentes do aplicativo independente de suas configurações de visibilidade.
46. Por meio de seus atributos de “Dia”, “Mês” e “Ano”.
47. Instantes, eles lidam com anos, meses, semanas, dias, horas, minutos, segundos, milissegundos etc.
48. Calculam a duração entre dois instantes, devolvendo o resultado em milissegundos ou convertem milissegundos em outras unidades de tempo.
49. Porque o código pode agir de forma indefinida já que uma estrutura de código usa em suas instruções um valor que supostamente não deveria mas foi colocado num armazenamento que é compartilhado com outra estrutura de código.
50. Registros são como variáveis, guardando um dado sobre um identificador de registros. Identificadores são agrupados por meio dos Namespaces.
51. Porque o cartão SD pode armazenar mais dados sem consumir a memória interna do smartphone que é usada primariamente para instalar aplicativos. A desvantagem é que ele pode ser removido a qualquer momento pelo usuário ou o smartphone

pode não ter um cartão. O componente que lida com esse tipo de armazenamento é o de Arquivo.

52. Diretórios são popularmente conhecidos como pastas e são arquivos especiais usados para conter e agrupar outros arquivos dentro do sistema. Caminhos são uma sucessão linear, estruturada e hierárquica de nomes de diretórios que o sistema deve percorrer para encontrar um arquivo.
53. O componente Notificador.
54. O componente ListaSuspensa, após selecionada a opção, o componente lança um evento chamado “DepoisDeSelecionar” cujo parâmetro recebe o texto da opção selecionada como argumento para ser usado no código.
55. Usar estados ajuda a reaproveitar e reutilizar componentes no App Inventor e poupa tempo de processamento do Android, economizando recursos computacionais.
56. Não, a documentação oficial do App Inventor aponta para uma página no site oficial do Android para uma lista de formatos de arquivos multimídia reconhecidos pelo Android.
57. Os componentes de Som e Tocador, o Som é indicado para uso de pequenos efeitos sonoros, o Tocador é para grandes fluxos de áudio, como músicas.
58. Sua contagem de intervalos não é exata pelo fato do próprio código necessitar de tempo para ser executado. Os intervalos sempre demoram mais do que o tempo configurado.

Capítulo 3

1. Porque as pessoas terão a garantia de que receberão a informação que precisam e poderão compreendê-la quase que independente de suas condições sensoriais.
2. Variação de tamanho de texto e seleção de cores para padrões de alto-contraste
3. Porque pessoas daltônicas podem ter problemas em distinguir cores, então recorreremos ao preto e branco já que representam

a ausência e a presença de todas as cores em intensidades extremas do espectro visível de cores.

4. Eles podem ajudar por lançar diretrizes para adaptação de interfaces de forma que possam ser entendidas por pessoas, independente dos dispositivos em que estão sendo mostradas, e permitindo acesso de tecnologias assistivas para converter a informação presente em um formato entendível pelo usuário.
5. Porque o componente de imagem não tem algum atributo que possibilite uma descrição em áudio do que está retratado nela.
6. Porque o evento de Inicializar só é disparado quando a tela é alocada na memória, para atingir a condição que precisamos, tanto o evento de Inicializar quanto o de OutraTelaFoiFechada devem apontar para a mesma função. Assim a função será executada sempre que a tela aparecer para o usuário.
7. São dados passados por uma tela anterior que chamou a atual.
8. Para interromper imediatamente a estrutura de repetição imediatamente associada a ele e continuar o código subsequente.
9. Porque mudar valores de tamanho de texto ou cor de fundo em variáveis não concretizam as configurações de acessibilidade, elas precisam ser aplicadas a todos os componentes que são mostrados na tela. As funções de adaptação fazem isso.
10. O código de funções relacionadas à adaptação das telas para as configurações de acessibilidade pôde ser copiado para a mochila a fim de ser colado em outras telas com facilidade. A partir daí o código é personalizado para atender à tela em específico.
11. Porque ter que esperar pelo teclado do SO pode tomar indesejadamente tempo de resposta de um jogador e ainda não ajudar na prevenção de erros.
12. Se o arquivo de problemas não existisse no armazenamento ou no cartão SD, o evento de leitura do arquivo não aconteceria e a partida nunca chegaria a estar preparada para deixar o jogo começar, resultando em erros.

13. A função `tratar_problema` chama a `decidir_estilo` que analisa quais tipos de perguntas podem ser feitas em uma palavra específica. Após esta análise a função `aleatorio_verdadeiro` escolhe um dos estilos possíveis.
14. Porque os caracteres básicos ASCII podem não representar muito bem uma operação matemática (como asterisco e barra).
15. O botão chama a função `adc_sim_resposta` passando seu número associado como argumento e a função se encarrega de adicionar o dígito na resposta.
16. Usamos o bloco de substituição de texto para agir em um texto específico, procurando pelo caractere que necessitávamos e substituindo ele por um texto vazio.
17. Verifica se um trecho de texto específico existe dentro de um outro texto, se existir, retorna o valor lógico verdadeiro, senão, falso.
18. O texto do botão é usado como argumento do parâmetro `escolha` do evento `DepoisDeEscolher` do Notificador, onde deve ser tratado.
19. As informações sobre a partida não aparecerão na legenda ao topo da tela `Matematica`.
20. A fórmula para calcular o novo valor da pontuação de um certo jogador.
21. Devido à quantidade muito maior de componentes de interface do usuário, o escopo das funções cresceu ao mesmo ritmo.
22. Porque erros poderiam acontecer ao acessar um nível de dificuldade inexistente ou alguma pontuação poderia ser prejudicada pelo fato do usuário pressionar o botão de Iniciar mesmo com o fim da partida.
23. Criar uma variável de controle (assim como a `resp_mat_ponto`) para que ela fosse executada somente uma vez.
24. A função `proxima_rodada` chama a função `rodada` que se encarrega de preparar todos os detalhes necessários para possibilitar uma nova jogada. Sendo que a `proxima_rodada` é sempre executada quando uma jogada acaba (bem ou mal).

25. O tempo de resposta é infinito e não é contabilizado como bônus de pontuação, além disso, as tentativas são infinitas.
26. O nível é incrementado sempre que cada jogador responder, no mínimo, 3 perguntas.
27. O código para lidar com o tempo contabilizado pelo Temporizador.
28. Eles verificam se um sinal de menos já existe no texto do campo de texto da resposta, se houver, ele é removido, senão, ele é acrescentado ao início do texto.
29. Para evitar que um determinado número tenha múltiplas partes decimais.
30. O bloco de tratamento de texto Repartir.
31. Isso é preciso para que o usuário possa ver as suas mudanças de configurações fazendo efeito.
32. De forma assistiva usando os controles exibidos na tela ou de forma manual por editar diretamente o texto que será escrito no arquivo.
33. Pelo desperdício genuíno de três tentativas, pelo ato de passar a vez e pelo fim do tempo disponível para responder um problema.
34. Por reunir todos os componentes de interface gráfica em listas e depois usar blocos orientados a “Qualquer componente” para fazer a adaptação de uma só vez.
35. Eles se baseiam no tipo do componente em vez de um componente específico. Sendo assim, o uso de listas para afetar vários componentes de um mesmo tipo se torna uma ideia atrativa. Qualquer propriedade de um determinado tipo de componente pode ser acessada por esses blocos.
36. Parte é responsável por mostrar os vencedores de uma partida do jogo enquanto a outra parte é responsável por possibilitar o uso do criador de problemas do aplicativo.
37. Primeiro, ela acha o valor mais alto dentre as pontuações na lista repassada a ela. Depois ela verifica quais jogadores têm aquela pontuação e os declara vencedores.

38. Porque qualquer erro de digitação que fuja do padrão estabelecido pela estrutura de nosso arquivo pode causar erros no processamento de uma partida executando sobre tal arquivo.
39. O resultado é calculado normalmente, então é multiplicado por 100, arredondado e depois dividido por 100 novamente a fim de arredondar o valor para duas casas decimais.

Capítulo 4

1. Por meio da ferramenta Google Play Console, uma pessoa com uma conta Google, assim que pagar a taxa de 25 dólares, pode fazer envio de aplicativos para a loja.
2. Um app Alpha possui muitos problemas e é liberado seletivamente para teste. Um app Beta conta com várias correções, mas ainda está em fase de testes. Um app de Produção tem quase todos os problemas solucionados e apresenta uma experiência de usuário estável.
3. Uma série de classificações indicativas aceitas internacionalmente para prevenir que o conteúdo do app seja acessado por pessoas em faixas etárias não permitidas.
4. Levando-se em consideração os países aceitos, as faixas etárias permitidas e os dispositivos compatíveis com o App.

Referências

AI2. ai2.appinventor.mit.edu, 2019. Disponível em: http://ai2.appinventor.mit.edu/?locale=pt_BR. Acesso em: 27 fev. 2019.

ALGOL. algol.dcc.ufla.br, 2019. Disponível em: <http://algol.dcc.ufla.br/~bruno/wxlogo/docs/oqueue.html>. Acesso em: 27 mar. 2019.

ANDROID, Developers. android-developers.googleblog.com/, 2019. Disponível em: <https://android-developers.googleblog.com/2017/06/ending-support-for-android-market-on.html>. Acesso em: 7 mar. 2019.

ANDROID. www.android.com/, 2019. Disponível em: <https://www.android.com/>. Acesso em: 27 fev. 2019.

APP INVENTOR, About. appinventor.mit.edu/, 2019. Disponível em: <http://appinventor.mit.edu/explore/blogs/karen/2017/08/about.html>. Acesso em: 19 mar. 2019.

APP INVENTOR, Blog. appinventor.mit.edu, 2019. Disponível em: <http://appinventor.mit.edu/explore/blogs/hal/2015/07/mit-app.html>. Acesso em: 25 mar. 2019.

APP INVENTOR, Components. ai2.appinventor.mit.edu, 2019. Disponível em: <http://ai2.appinventor.mit.edu/reference/components/userinterface.html>. Acesso em: 20 abr. 2019.

APP INVENTOR, Development.appinventor.mit.edu/, 2019. Disponível em: <http://appinventor.mit.edu/explore/content/development-starts-mit.html>. Acesso em: 20 mar. 2019.

APP INVENTOR, Images and sounds. appinventor.mit.edu, 2019. Disponível em: <https://appinventor.mit.edu/explore/ai2/concepts/images-and-sounds.html>. Acesso em: 6 abr. 2019.

APP INVENTOR, Manyscreens. ai2.appinventor.mit.edu/ , 2019. Disponível em: <http://ai2.appinventor.mit.edu/reference/other/manyscreens.html>. Acesso em: 28 abr. 2019.

APP INVENTOR, Sensors. ai2.appinventor.mit.edu, 2019. Disponível em: <http://ai2.appinventor.mit.edu/reference/components/sensors.html>. Acesso em: 9 abr. 2019.

APP INVENTOR, Setup Emulator. appinventor.mit.edu, 2019. Disponível em: <http://appinventor.mit.edu/explore/ai2/setup-emulator.html>. Acesso em: 21 jul. 2019.

APP INVENTOR, Storage. ai2.appinventor.mit.edu/ , 2019. Disponível em: <http://ai2.appinventor.mit.edu/reference/components/storage.html>. Acesso em: 2 abr. 2019.

APP INVENTOR. appinventor.mit.edu, 2019. Disponível em: <http://appinventor.mit.edu/explore/about-us.html>. Acesso em: 16 mar. 2019.

APPINV. appinv.us, 2019. Disponível em: <http://appinv.us/xnot1>. Acesso em: 14 mar. 2019.

APPINVENTOR. [www.appinventor.mit.edu](http://appinventor.mit.edu), 2019. Disponível em: <http://appinventor.mit.edu/ai2/ReleaseNotes.html>. Acesso em: 25 jan. 2019.

AUSUBEL, D. P. Aquisição e retenção de conhecimentos: uma perspectiva cognitiva. Tradução de: Lígia TEOPISTO. 1. ed. Lisboa: Paralelo, 2003.

CANALTECH. canaltech.com.br, 2019. Disponível em: <https://canaltech.com.br/mercado/mercado-de-aplicativos-bate-recorde-de-downloads-e-receita-no-fim-de-2017-107384/>. Acesso em: 11 fev. 2019.

CELES FILHO, Waldemar. Introdução à estrutura de dados: com técnicas de programação em C. 11. Reimpressão. Rio de Janeiro: Elsevier, 2004.

CODE. code.org/, 2019. Disponível em: <https://code.org/>. Acesso em: 13 mar. 2019.

COLATINA, Ifes. <https://colatina.ifes.edu.br/>, 2019. Disponível em: <https://colatina.ifes.edu.br/>. Acesso em: 15 maio 2019.

DEVELOPER, Media formats. developer.android.com, 2019. Disponível em: <https://developer.android.com/guide/topics/media/media-formats>. Acesso em: 11 abr. 2019.

DEVELOPER. developer.android.com/, 2019. Disponível em: <https://developer.android.com/>. Acesso em: 2 mar. 2019.

DEVMEDIA. www.devmedia.com.br, 2019. Disponível em: <https://www.devmedia.com.br/os-4-pilares-da-programacao-orientada-a-objetos/9264>. Acesso em: 12 jul. 2019.

GOOGLE PLAY. play.google.com/, 2019. Disponível em: <https://play.google.com/>. Acesso em: 9 mar. 2019.

GOOGLE PLAY. play.google.com/, 2019. Disponível em: <https://play.google.com/console>. Acesso em: 28 jun. 2019.

GOOGLE, Forúns. <https://groups.google.com/> , 2019. Disponível em: <https://groups.google.com/forum/#!forum/mitappinventortest>. Acesso em: 20 mar. 2019.

GOOGLE, Support. support.google.com , 2019. Disponível em: <https://support.google.com/googleplay/android-developer/answer/113469?hl=en>. Acesso em: 21 jun. 2019.

GOOGLE. www.google.com, 2019. Disponível em: <https://www.google.com>. Acesso em: 14 jan. 2019.

GS. gs.statcounter.com, 2019. Disponível em: <http://gs.statcounter.com/os-market-share>. Acesso em: 10 fev. 2019.

HARDWARE. www.hardware.com.br, 2019. Disponível em: <https://www.hardware.com.br/guias/historia-informatica/eniac.html>. Acesso em: 25 fev. 2019.

INFOESCOLA. www.infoescola.com, 2019. Disponível em: <https://www.infoescola.com/matematica/conectivos-logicos/>. Acesso em: 22 abr. 2019.

JIGSAW. jigsaw.w3.org, 2019. Disponível em: <http://jigsaw.w3.org/css-validator/>. Acesso em: 15 jun. 2019.

KHRONOS. www.khronos.org, 2019. Disponível em: <https://www.khronos.org/collada/>. Acesso em: 19 jul. 2019.

LDJAM. ldjam.com, 2019. Disponível em: <https://ldjam.com/>. Acesso em: 3 fev. 2019.

LUCIDCHART. www.lucidchart.com/, 2019. Disponível em: <https://www.lucidchart.com/pages/pt/o-que-e-um-fluxograma>. Acesso em: 25 jun. 2019.

MATERIAL, Selection controls. material.io, 2019. Disponível em: <https://material.io/design/components/selection-controls.html>. Acesso em: 13 maio 2019.

MINHA VIDA. www.minhavidacom.br, 2019. Disponível em: <https://www.minhavidacom.br/alimentacao/tudo-sobre/32159-imc>. Acesso em: 18 abr. 2019.

ORACLE, Simple Date Format. docs.oracle.com, 2019. Disponível em: <https://docs.oracle.com/javase/7/docs/api/java/text/SimpleDateFormat.html>. Acesso em: 9 abr. 2019.

PORTAL DA EDUCAÇÃO. www.portaleducacao.com.br, 2019. Disponível em: <https://www.portaleducacao.com.br/conteudo/artigos/informatica/afinal-o-que-e-um-computador/41539>. Acesso em: 16 fev. 2019.

SCRATCH. scratch.mit.edu/, 2019. Disponível em: <https://scratch.mit.edu/>. Acesso em: 21 mar. 2019.

SEMVER. semver.org/, 2019. Disponível em: <https://semver.org/>. Acesso em: 18 mar. 2019.

SIGNIFICADOS. www.significados.com.br, 2019. Disponível em: <https://www.significados.com.br/software/>. Acesso em: 21 fev. 2019.

SOURCE. source.android.com/ 2019. Disponível em: <https://source.android.com/>. Acesso em: 4 mar. 2019.

UNIVERSIDADE DA TECNOLOGIA. universidadedatecnologia.com.br, 2019. Disponível em: <https://universidadedatecnologia.com.br/o-que-e-linguagem-de-programacao/>. Acesso em: 27 fev. 2019.

UXMOVEMENT. [uxmovement.com](https://uxmovement.com/content/why-you-should-never-use-pure-black-for-text-or-backgrounds/), 2019. Disponível em: <https://uxmovement.com/content/why-you-should-never-use-pure-black-for-text-or-backgrounds/>. Acesso em: 22 jun. 2019.

VALIDADOR, W3. validator.w3.org/, 2019. Disponível em: <https://validator.w3.org/>. Acesso em: 18 jun. 2019.

W3. www.w3.org, 2019. Disponível em: <https://www.w3.org/>. Acesso em: 5 fev. 2019.

W3C. Cartilha. [www.w3c.br/](http://www.w3c.br/pub/Materiais/PublicacoesW3C/cartilha-w3cbr-acessibilidade-web-fasciculo-I.html) , 2019. Disponível em: <http://www.w3c.br/pub/Materiais/PublicacoesW3C/cartilha-w3cbr-acessibilidade-web-fasciculo-I.html>. Acesso em: 15 maio 2019.

W3SCHOOLS. www.w3schools.com, 2019. Disponível em: https://www.w3schools.com/TAGS/att_img_alt.asp. Acesso em: 19 maio 2019.

WEBAIM. [webaim.org/](https://webaim.org/resources/contrastchecker/) , 2019. Disponível em: <https://webaim.org/resources/contrastchecker/>. Acesso em: 20 jun. 2019.

WIKI. [hen.scratch-wiki.info](https://en.scratch-wiki.info/wiki/Scratch), 2019. Disponível em: <https://en.scratch-wiki.info/wiki/Scratch>. Acesso em: 18 mar. 2019.

YOUTUBE. <https://www.youtube.com/>, 2019. Disponível em: <https://www.youtube.com/>. Acesso em: 19 jul. 2019.

